CS (Degree) Paper-II — Concepts, Importance Ratings & Answered Questions

NPSC CTSE 2026 · Computer Science & Engineering (Degree) · Descriptive Paper-II (100 marks, 3 hours) · Monday 12 Oct 2026

How to use this PDF

Tonight: read Part C (concepts) by importance. Do every **9–10** first, then the **7–8s**. Skim **6 and below**.
Tomorrow morning: read Part D2 (predicted questions with answers), then Part D1 (the real 2025 paper, answered).
In the hall: the paper gives choice (any **8 of 10** × 5, **3 of 5** × 10, **2 of 4** × 15). Pick the questions where you can **draw a diagram**, because diagrams earn marks quickly.

The importance scale (on every concept)

Rating	Meaning
10	**Sure to come.** An unasked syllabus phrase or a core numerical. Learn it cold.
9	**Very likely** to appear as a question or part of one
8	**Likely.** A good Section A / B candidate.
7	**Good chance.** It may also come as an MCQ.
6 · 5	**Moderate / possible.** Read once.
4 – 1	**Background only.** Mostly supports other answers.

How the predictions were made

1. **The 2025 Paper-II copies the syllabus wording almost word for word.** Examples: "knowledge-level vs symbol-level learning", "building a knowledge base", "phases of website development", "cost estimation methods", "query processing". So **syllabus phrases not yet asked** are the strongest 2026 candidates.
2. **The 2025 subject spread will most likely repeat.** It was:

Section	2025 subjects (one question each)
A (10 × 5)	OS ×2 · SE ×2 · DBMS ×2 · Networks ×2 · Web ×2
B (5 × 10)	OS · Networks · Web · AI · ML
C (4 × 15)	OS · SE · DBMS · AI

3. Topics asked in 2025 can come back reworded, e.g. "memory management" (2025) → "virtual memory and page replacement" (2026). They get 7–8, not 10.

4. Online research: I found no NPSC CTSE Paper-II papers other than 2025 online. A 2015 and 2017 CTSE CS (Degree) paper exists in an index, but it can't be downloaded from here. General state-PSC and UGC-NET Paper-II material repeats the same core: OS (scheduling, deadlock, paging, page replacement, disk scheduling), DBMS (normalisation, SQL, transactions) and networking (OSI/TCP-IP, IP addressing) (GfG UGC-NET notes, CSS Paper-II syllabus, GfG OS questions). For AI, EBL vs inductive learning is a standard exam contrast (GfG EBL).

★ Sure-to-come list for 2026 (by subject)

Subject	Strongest 2026 candidates (unasked in 2025)	Rating
Operating Systems	Scheduling numerical (FCFS/SJF/Priority, Gantt chart) · Deadlock (conditions, Banker's) · Page replacement (FIFO/LRU/Optimal) · Process states + PCB · Evolution/generations of OS · I/O & disk scheduling · File system management · Security & protection	9–10
Software Engineering	Software testing strategies · Quality assurance · Process models (waterfall / spiral / prototype) · Requirement analysis & SRS · Concept of a system · Project scheduling (Gantt/PERT)	8–10
DBMS	Data abstraction (3 levels) · Data independence · Distributed databases · Security & integrity · DDL/DML with SQL · Types of attributes · Data models / network model	8–9
Networks	IP addressing & classes (+ subnetting) · Networking devices · Transmission media · OSI vs TCP/IP · Internet routing · LAN/MAN/WAN	8–10
Web Technologies	HTML basic tags, lists, hyperlinks, images, font · Passing information between pages (GET/POST/session/cookie) · Adding PHP to HTML · PHP errors · Installing PHP	8–9
AI	Inductive learning · Explanation-based learning · Pattern recognition (structured / symbolic description, object identification, speech recognition) · LISP & PROLOG	9
Machine Learning	Ensemble methods (bagging / boosting / random forest) · K-means & kernel K-means · Distance measures	9–10

Contents: Part C — Concepts (§1 OS · §2 SE · §3 DBMS · §4 Networks · §5 Web · §6 AI · §7 ML) · Part D2 — Predicted 2026 questions with answers · Part D1 — Real 2025 Paper-II answered.

Part C — Concepts

1. Operating Systems

Syllabus phrase (Paper-II): *Introduction to Operating system, Evolution, Generations, Components & Services, Process Management, Scheduling algorithms, Memory management, I/O management, File system Management, Security & Protection.* 2025 asked: components (Q1, 5 m), logical vs physical address (Q2, 5 m), Round Robin (Q11, 10 m), memory management incl. paging & segmentation (Q16, 15 m).

1.1 Introduction — what an Operating System is

Importance 7/10 · good chance · opening phrase of the syllabus; usually the first part of any OS answer

In simple words: the OS is the "manager" program that sits between you and the hardware. It hands out CPU time, memory and devices to programs, and gives you an easy way to use the machine.

Definition (Silberschatz): An operating system is a program that manages the computer hardware, provides a basis for application programs and acts as an intermediary between the user and the computer hardware.

Two views of an OS:

- **Resource allocator** — decides who gets the CPU, memory, I/O devices, files, and resolves conflicting requests fairly and efficiently.
- **Control program** — controls execution of user programs to prevent errors and improper use of the computer.

Goals: **convenience** (easy to use) and **efficiency** (good use of hardware). The **kernel** is the one program running at all times; everything else is a system program or an application.

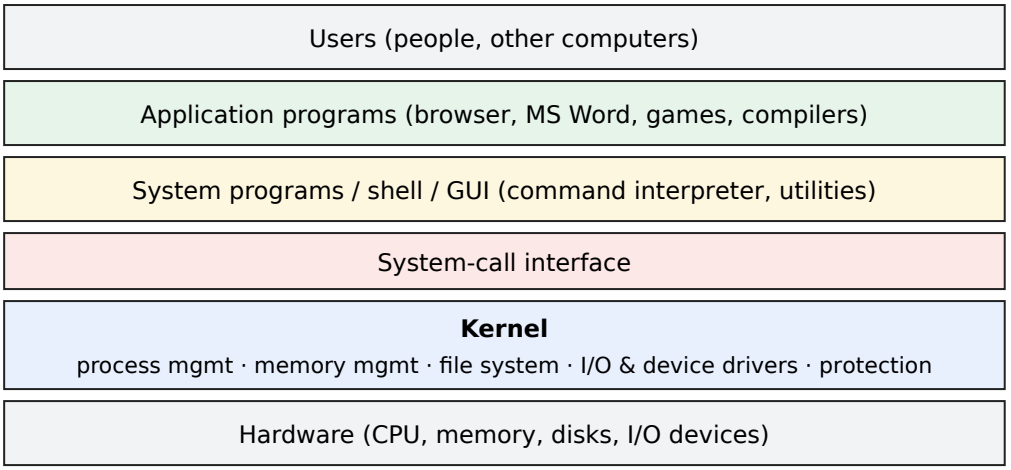


Fig: Layered view of a computer system — the OS (kernel + system programs) sits between users/applications and hardware

Functions of an OS (list for a 5-mark answer): process management · memory management · file management · I/O / device management · secondary-storage management · protection & security · networking · command interpretation / user interface · error detection · accounting.

Exam tip: Start every OS long answer with this definition + the two views; it earns the "introduction" marks in 2 lines.

1.2 Evolution and Generations of Operating Systems

Importance 8/10 · likely · "Evolution, Generations" is an unasked syllabus phrase — strong 2026 Sec A/B candidate

In simple words: early computers had no OS — a person loaded each program by hand. Over time the OS took over more jobs: first batching jobs, then running several at once, then letting many users share one machine, then networking many machines.

Generation	Period / hardware	OS stage	Key idea
1st	1945–55, vacuum tubes, plugboards	**No OS** (serial processing)	Programmer books machine time, loads program by switches / punched cards; huge CPU idle time during set-up
2nd	1955–65, transistors	**Simple batch systems**	Similar jobs grouped into batches; a **resident monitor** loads jobs one after another (e.g. IBM FMS, IBSYS); card readers, magnetic tape; JCL
3rd	1965–80, integrated circuits	**Multiprogramming, time-sharing, spooling**	Several jobs in memory; CPU switches when a job waits for I/O. Time-sharing (CTSS, MULTICS) gives each terminal user short CPU slices; UNIX born (1969)
4th	1980–present, LSI/VLSI, PCs	**Personal-computer, GUI, network & distributed OS**	MS-DOS, Windows, Mac OS, Linux; network OS (Novell), distributed OS
5th (often added)	1990s–present, mobile/cloud	**Mobile, embedded, cloud/virtualised OS**	Android, iOS, hypervisors, real-time embedded OS

The Paper-II(B) syllabus lists the same evolution as **Sequential → Batch → Multiprogramming → Multiprocessing → Real-time → Multitasking**, so be ready to describe it in that order too.

Exam tip: Draw the 5-row table above; mention the **problem each stage solved** (idle CPU → batching → I/O overlap → interactive response).

1.3 Types of Operating Systems

Importance 8/10 · likely · ties directly to "Evolution"; frequent MCQ and 5-mark source

Type	In simple words	Key points	Example
Batch	Jobs with similar needs collected and run in a group, no user interaction	Operator sorts jobs; long turnaround; CPU often idle during I/O	IBM 7094 IBSYS, payroll runs
Multiprogramming	Keep several jobs in memory; when one waits for I/O, run another	Increases **CPU utilisation**; needs memory mgmt + job scheduling	IBM OS/360
Time-sharing / multitasking	Logical extension of multiprogramming: CPU switches so fast that every user feels they own the machine	Short time slices (RR), response time < 1 s, interactive	UNIX, Linux, Windows
Multiprocessing	Two or more CPUs share memory and bus	Increased throughput, economy of scale, reliability; symmetric (SMP) vs asymmetric	Modern multicore OS
Real-time (RTOS)	Must respond within a fixed deadline	**Hard RT** — missing deadline = failure (airbag, missile); **Soft RT** — late is tolerable (video streaming)	VxWorks, QNX, FreeRTOS
Distributed	Many independent computers appear to the user as **one single system**	Resource sharing, computation speed-up, reliability, communication	Amoeba, LOCUS
Network OS	Each computer has its own OS; users are **aware** of the other machines and log in / share files explicitly	File and printer sharing over LAN, server-centric	Novell NetWare, Windows Server

Network OS vs Distributed OS: in a NOS the user knows resources are remote (e.g. maps `\\server\share`); in a distributed OS remote resources are accessed transparently as if local.

Exam tip: Multiprogramming maximises **CPU utilisation**; time-sharing minimises **response time** — a classic MCQ pair.

1.4 OS Components and Services

Importance 8/10 · likely · asked 2025 Q1 (components); "services" half not yet asked — likely in a different form

In simple words: *components* are the internal departments of the OS (who does what inside); *services* are what the OS offers to users and programs (what you get from it).

System components (Silberschatz):

Component	Responsibility
Process management	Create/delete processes, suspend/resume, synchronisation, communication, deadlock handling
Main-memory management	Track which parts of memory are used and by whom, allocate/free space, decide which process to load
File management	Create/delete files and directories, map files to secondary storage, backup
I/O system management	Buffering, caching, spooling, device-driver interface
Secondary-storage management	Free-space management, storage allocation, disk scheduling
Networking	Communication between machines (protocols, distributed access)
Protection system	Control access of processes/users to resources
Command interpreter (shell)	Reads and executes user commands (CLI or GUI)

OS services:

- **For the user:** user interface (CLI, GUI, batch), program execution, I/O operations, file-system manipulation, communication (shared memory or message passing), error detection.
- **For efficient system operation:** resource allocation, accounting (who used how much), protection and security.

Exam tip: For "components" write the table; for "services" split into "user-helpful" and "system-efficiency" groups — examiners like the classification.

1.5 System Calls

Importance 7/10 · good chance · part of "services"; common MCQ (fork, exec, wait)

In simple words: a system call is the doorway through which a user program asks the kernel to do something privileged (read a file, create a process). The CPU switches from **user mode** to **kernel mode** via a trap.

Category	Examples (UNIX)	Examples (Windows)
Process control	`fork()`, `exec()`, `exit()`, `wait()`	`CreateProcess()`, `ExitProcess()`
File management	`open()`, `read()`, `write()`, `close()`	`CreateFile()`, `ReadFile()`
Device management	`ioctl()`, `read()`, `write()`	`SetConsoleMode()`
Information maintenance	`getpid()`, `alarm()`, `sleep()`	`GetCurrentProcessID()`
Communication	`pipe()`, `shmget()`, `mmap()`	`CreatePipe()`
Protection	`chmod()`, `umask()`, `chown()`	`SetFileSecurity()`

Parameter passing: in registers, in a block/table in memory (address in a register), or pushed on the stack.

Exam tip: `fork()` returns **0 to the child** and the **child's PID to the parent**; n calls to `fork()` create 2^n processes in total ($2^n - 1$ new).

1.6 Process Concept, Process States and PCB

Importance 9/10 · very likely · core of "Process Management"; state diagram is a favourite 5/10-mark item

In simple words: a **program** is a passive file on disk; a **process** is that program *in execution* — with its own program counter, registers, stack, data and heap.

Process in memory: **text** (code) · **data** (globals) · **heap** (dynamic) · **stack** (function calls, locals).

Five states (Silberschatz): New (being created) · **Ready** (waiting for CPU) · **Running** (instructions executing) · **Waiting/Blocked** (waiting for I/O or event) · **Terminated** (finished).

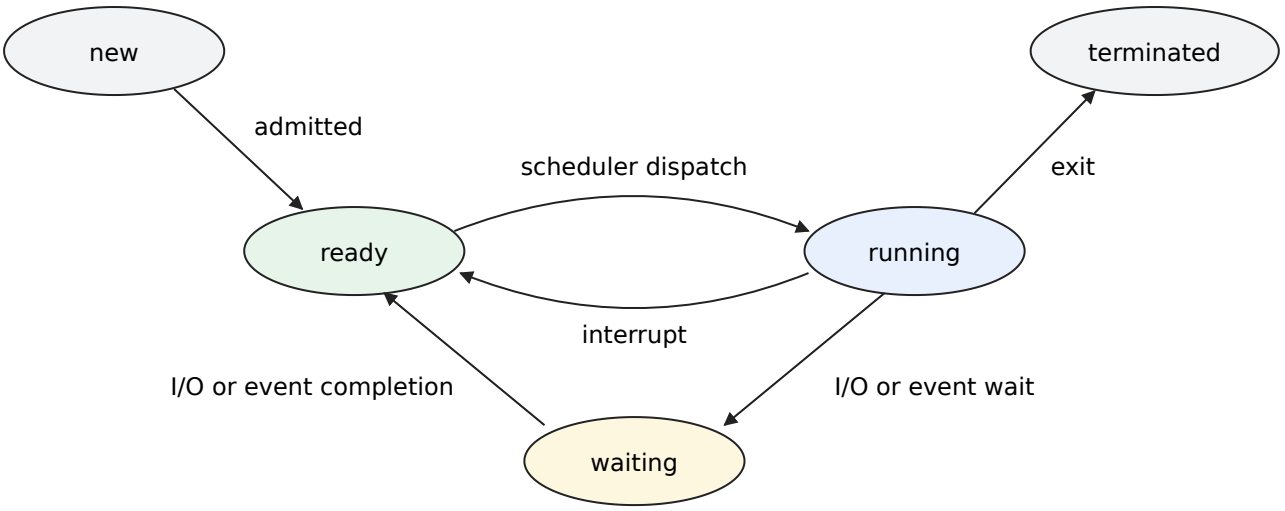


Fig: Five-state process transition diagram (Silberschatz)

Process Control Block (PCB / task control block): the kernel data structure that represents one process. It is what gets saved and restored on a context switch.

pointer (to next PCB in queue)	
process state	new / ready / running / waiting / terminated
process number (PID)	
program counter	address of next instruction to execute
CPU registers	accumulators, index regs, stack pointer, PSW
memory limits / page table	memory-management info (base/limit, page tables)
list of open files	I/O status info: devices allocated, open files
. . .	CPU-scheduling info (priority, queue ptrs), accounting info

Fig: Process Control Block (PCB) fields

Scheduling queues & schedulers: job queue → ready queue → device queues. **Long-term scheduler** (job scheduler) picks jobs from disk into memory, controls degree of multiprogramming; **short-term scheduler** (CPU scheduler) picks the next ready process, runs very often; **medium-term scheduler** swaps processes out/in.

Exam tip: Draw the state diagram with all **six labelled transitions**; a missing label (e.g. "interrupt") loses marks.

1.7 Context Switch

Importance 7/10 · good chance · short-note / MCQ favourite

In simple words: switching the CPU from one process to another — save the old one's PCB, load the new one's PCB.

- Triggered by interrupt, system call, time-slice expiry, I/O wait.
- **Pure overhead**: no useful work during the switch; time depends on hardware (number of registers, special instructions, multiple register sets).
- Too small a time quantum in RR ⇒ too many context switches.

1.8 Threads

Importance 7/10 · good chance · part of process management; process-vs-thread table is a regular 5-marker

In simple words: a thread is a "lightweight process" — a single path of execution inside a process. Threads of the same process share code, data and open files but each has its own **thread ID, program counter, register set and stack**.

Process	Thread
Heavyweight; own address space	Lightweight; shares address space of its process
Creation and context switch costly	Cheap to create and switch
Communication via IPC (pipes, shared memory)	Communicate directly through shared data
One process crashing does not affect others	One thread crashing can kill the whole process

Benefits: responsiveness, resource sharing, economy, scalability on multicore. **User-level threads** (managed by a library, kernel unaware) vs **kernel-level threads** (managed by OS). Multithreading models: **many-to-one, one-to-one, many-to-many**.

1.9 CPU Scheduling — Criteria and Algorithms

Importance 10/10 · sure to come · scheduling numericals come every year (RR asked 2025 Q11); FCFS/SJF/SRTF/Priority equally likely now

In simple words: when several processes are ready, the CPU scheduler decides who runs next. Different rules (algorithms) give different waiting times.

Criteria: maximise **CPU utilisation** and **throughput** (processes completed per unit time); minimise **turnaround time**, **waiting time** and **response time**.

Formulas: Turnaround Time (TAT) = Completion Time − Arrival Time · Waiting Time (WT) = TAT − Burst Time · Response Time = first time on CPU − Arrival Time.

Preemptive vs non-preemptive: non-preemptive — once the CPU is given, the process keeps it until it ends or blocks; preemptive — the OS can take the CPU away (on arrival of a better process or a timer interrupt).

Algorithm	Rule	Preemptive?	Pros	Cons
FCFS	Run in order of arrival (FIFO queue)	No	Simple, fair in order	**Convoy effect** — short jobs wait behind a long one
SJF	Pick smallest next CPU burst	No	**Optimal** — minimum average WT	Burst length must be predicted; starvation of long jobs
SRTF	Preemptive SJF: run job with shortest *remaining* time	Yes	Even lower average WT	Starvation; many switches
Priority	Highest priority first (smaller number = higher priority in Silberschatz)	Either	Important jobs first	**Starvation** → solved by **aging**
Round Robin	FCFS + fixed **time quantum** q; preempt when q expires	Yes	Fair, good response time, no starvation	Large q → FCFS; very small q → context-switch overhead
Multilevel queue	Ready queue split into fixed queues (system, interactive, batch), each with own algorithm; fixed priority between queues	Yes	Separates job classes	Lower queues may starve; process cannot move
Multilevel feedback queue	Like above but processes **move** between queues by CPU usage (CPU-bound sinks, aged ones rise)	Yes	Most flexible, adapts	Most complex to tune

Multiprocessor scheduling (Paper-II(B) phrase): asymmetric (one master CPU schedules) vs symmetric (each CPU self-schedules); **processor affinity** (keep a process on the same CPU for cache warmth); **load balancing** (push / pull migration).

Worked numerical (verified)

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	1
P4	3	2

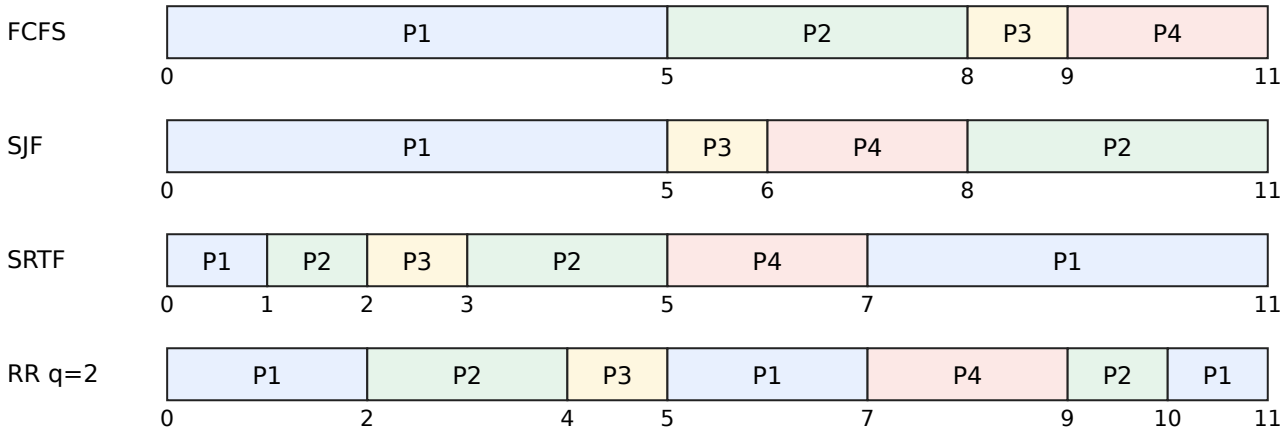


Fig: Gantt charts for the same 4 processes under FCFS, SJF (non-preemptive), SRTF and Round Robin (q = 2)

Full working — SRTF (at every arrival, compare remaining times):

- t=0: only P1 (rem 5) → run P1. t=1: P2 arrives (3) < P1 rem 4 → preempt, run P2.
- t=2: P3 arrives (1) < P2 rem 2 → run P3; P3 finishes at 3.
- t=3: P4 arrives (2); P2 rem 2, P4 2, P1 4 → tie P2/P4, earlier arrival P2 runs → finishes at 5.

- t=5: P4 (2) < P1 (4) → P4 finishes at 7; then P1 finishes at 11.

Process	AT	BT	CT	TAT = CT−AT	WT = TAT−BT
P1	0	5	11	11	6
P2	1	3	5	4	1
P3	2	1	3	1	0
P4	3	2	7	4	2
Average				**20/4 = 5.00**	**9/4 = 2.25**

Round Robin (q=2) queue trace (new arrivals join before the preempted process): t=0 P1 → t=2 queue [P2, P3, P1] → t=4 [P3, P1, P4, P2] → P3 done at 5 → P1 to 7 → P4 done at 9 → P2 done at 10 → P1 done at 11.

Algorithm	CT (P1, P2, P3, P4)	Avg TAT	Avg WT
FCFS	5, 8, 9, 11	27/4 = 6.75	16/4 = 4.00
SJF (non-preemptive)	5, 11, 6, 8	24/4 = 6.00	13/4 = 3.25
SRTF	11, 5, 3, 7	20/4 = 5.00	9/4 = 2.25
RR (q = 2)	11, 10, 5, 9	29/4 = 7.25	18/4 = 4.50

Check: SRTF gives the lowest average waiting time, as theory predicts (SJF/SRTF optimal). All values recomputed by a simulation script.

Exam tip: Always (1) draw the Gantt chart with times underneath, (2) give the CT/TAT/WT table, (3) state averages, (4) write one line of comparison. State your tie-breaking rule (e.g. FCFS on ties).

1.10 Process Synchronisation

Importance 7/10 · good chance · inside "Process Management"; critical-section conditions and semaphores are standard 5-markers

In simple words: when processes share data, their instructions can interleave badly and give wrong results (a **race condition**). Synchronisation makes sure only one at a time touches the shared data.

Critical section = the code segment that accesses shared data. A solution must satisfy:

1. **Mutual exclusion** — at most one process in its critical section.
2. **Progress** — if none is inside, the choice of who enters next cannot be postponed indefinitely (only those wanting to enter take part).
3. **Bounded waiting** — a limit on how many times others can enter before a waiting process gets its turn.

Tools: Peterson's solution (2 processes, software), hardware instructions (`TestAndSet`, `CompareAndSwap`), **mutex locks**, **semaphores**, monitors.

Semaphore (Dijkstra): an integer S accessed only through two atomic operations.

```
wait(S)   { while (S <= 0) ; /* busy wait */  S--; }   // P / down
signal(S) { S++; }                                     // V / up
```

Counting semaphore (any non-negative value — counts resource instances) vs **binary semaphore** (0/1, acts like a mutex lock). Blocking implementations put the waiting process in a queue instead of busy-waiting.

Classic problems (one line each):

- **Bounded-buffer (producer–consumer):** semaphores `mutex=1`, `empty=n`, `full=0`; producer waits on empty, consumer on full.
- **Readers–writers:** many readers may read together; a writer needs exclusive access (`rw_mutex`, `mutex`, `read_count`).

- **Dining philosophers:** 5 philosophers, 5 chopsticks; naive solution can deadlock; fixes — allow max 4 at table, pick both chopsticks atomically, or odd/even ordering.

1.11 Deadlock

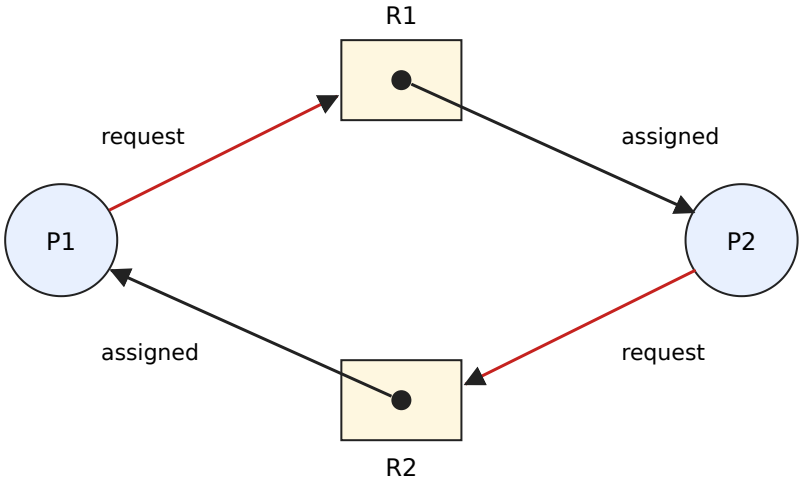
Importance 9/10 · very likely · syllabus "Process Management"/Paper-I(B) "Deadlock"; not asked in 2025 Paper-II — strong 10/15-mark candidate

In simple words: a deadlock is a standstill where each process in a group holds something and waits for something held by another process in the same group, so nobody can ever proceed.

Four necessary (Coffman) conditions — all must hold simultaneously:

Condition	Meaning
Mutual exclusion	At least one resource is non-sharable
Hold and wait	A process holds at least one resource while waiting for others
No preemption	Resources are released only voluntarily
Circular wait	P0 waits for P1, P1 for P2, …, Pn for P0

Resource-Allocation Graph (RAG): circles = processes, rectangles = resource types (dots = instances). **Request edge** P → R; **assignment edge** R → P. No cycle ⇒ no deadlock. Cycle ⇒ deadlock **if every resource has a single instance**; with multiple instances a cycle means *possible* deadlock.



Cycle P1 → R1 → P2 → R2 → P1 with single-instance resources ⇒ DEADLOCK

Fig: Resource-allocation graph with a cycle — P1 holds R2 and wants R1; P2 holds R1 and wants R2

Methods for handling deadlock:

1. **Prevention** — make one Coffman condition impossible: share resources where possible (mutual exclusion — rarely possible); request all resources at once (hold & wait); take resources away from a waiting process (no preemption); impose a **total ordering** of resource types and request in increasing order (circular wait — most practical).
2. **Avoidance** — OS uses advance knowledge of maximum needs and grants a request only if the system stays in a **safe state**. Single instance: RAG with claim edges; multiple instances: **Banker's algorithm**.
3. **Detection and recovery** — let deadlock happen, detect with a wait-for graph (single instance) or detection algorithm (multiple), then recover by **terminating processes** (all, or one at a time) or **resource preemption** (select victim, rollback, avoid starvation).
4. **Ignore** (ostrich algorithm) — used by UNIX/Windows; reboot if it happens.

Safe state: there is at least one order (safe sequence) in which every process can get its maximum need and finish. Unsafe ≠ deadlocked, but may lead to deadlock.

Banker's algorithm — worked example (verified)

5 processes, 3 resource types A, B, C with totals A=10, B=5, C=7. Available = (3, 3, 2). Need = Max − Allocation.

Process	Allocation (A B C)	Max (A B C)	Need (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Safety check (Work starts = Available = 3 3 2; pick any process with Need ≤ Work, then Work += its Allocation):

Step	Process chosen	Need ≤ Work?	New Work
1	P1	1 2 2 ≤ 3 3 2 ✔	3 3 2 + 2 0 0 = **5 3 2**
2	P3	0 1 1 ≤ 5 3 2 ✔	+ 2 1 1 = **7 4 3**
3	P0	7 4 3 ≤ 7 4 3 ✔	+ 0 1 0 = **7 5 3**
4	P2	6 0 0 ≤ 7 5 3 ✔	+ 3 0 2 = **10 5 5**
5	P4	4 3 1 ≤ 10 5 5 ✔	+ 0 0 2 = **10 5 7** (= totals ✔)

Safe sequence **⟨P1, P3, P0, P2, P4⟩** ⇒ system is in a **safe state**. (Silberschatz lists ⟨P1, P3, P4, P2, P0⟩ — more than one safe sequence can exist; any valid one earns full marks.)

Exam tip: Always show the Need matrix and the Work vector after each step; final Work must equal the total resources — that is your self-check.

1.12 Memory Management — Basics, Logical vs Physical Address

Importance 7/10 · good chance · logical vs physical asked 2025 Q2; still needed as the opening of any memory answer

In simple words: a program thinks it lives at addresses 0, 1, 2 … (logical); the hardware actually puts it somewhere else in RAM (physical). A hardware unit translates on the fly.

Logical (virtual) address	Physical address
Generated by the **CPU**	Seen by the **memory unit** (loaded into MAR)
Set of all = logical address space	Set of all = physical address space
User program only ever sees logical addresses	Never seen directly by the user program
Same as physical in compile-time/load-time binding	Differs from logical in **execution-time binding**

MMU (Memory-Management Unit): hardware that maps logical → physical at run time. Simplest form: **relocation (base) register** — physical = logical + base; with a **limit register** to check logical < limit (protection).

Address binding: compile time (absolute code) · load time (relocatable code) · execution time (needs MMU; allows moving process during execution).

Also know: **dynamic loading** (routine loaded only when called), **dynamic linking** (shared libraries, stubs), **swapping** (move whole process to backing store and back).

1.13 Contiguous Allocation and Fragmentation

Importance 8/10 · likely · "fragmentation" appears in two syllabi; first technique in any memory 15-marker

In simple words: each process is given one single continuous block of RAM.

- **Fixed (static) partitioning** — memory pre-divided into partitions; one process per partition; causes **internal fragmentation**.
- **Variable (dynamic) partitioning** — partition made exactly the size of the process; holes appear over time; causes **external fragmentation**.
- **Placement strategies: First fit** (first hole big enough — fast), **Best fit** (smallest hole big enough — leaves tiny holes), **Worst fit** (largest hole). First fit and best fit are better than worst fit in time and storage use.

Internal fragmentation	External fragmentation
Wasted space **inside** an allocated block	Enough total free space exists but **not contiguous**
Occurs in fixed partitions and paging (last page)	Occurs in variable partitions and segmentation
Fix: smaller / variable-size blocks	Fix: **compaction**, paging, segmentation with paging

Exam tip: 50-percent rule (first fit): for N allocated blocks, about 0.5 N blocks are lost to fragmentation — one-third of memory may be unusable.

1.14 Paging

Importance 9/10 · very likely · asked inside 2025 Q16; paging numericals and address translation are very likely again

In simple words: chop the program into equal-size **pages** and RAM into equal-size **frames**; any page can go into any free frame. A page table records where each page went. No external fragmentation.

- Logical address = **page number (p)** + **page offset (d)**. With page size 2^n and logical address space 2^m: p = high (m − n) bits, d = low n bits.
- **Page table** (one per process) maps p → frame number f. Physical address = f × page size + d.
- **Page-table base register (PTBR)** points to the page table; each access would need 2 memory references, so a **TLB** (translation look-aside buffer, associative cache) holds recent mappings.
- **Effective access time** = h × (TLB time + mem) + (1 − h) × (TLB time + 2 × mem).
- Protection: valid–invalid bit, read/write bits per entry. Shared pages: reentrant code shared between processes.
- Large page tables: hierarchical (multi-level) paging, hashed page tables, inverted page table.

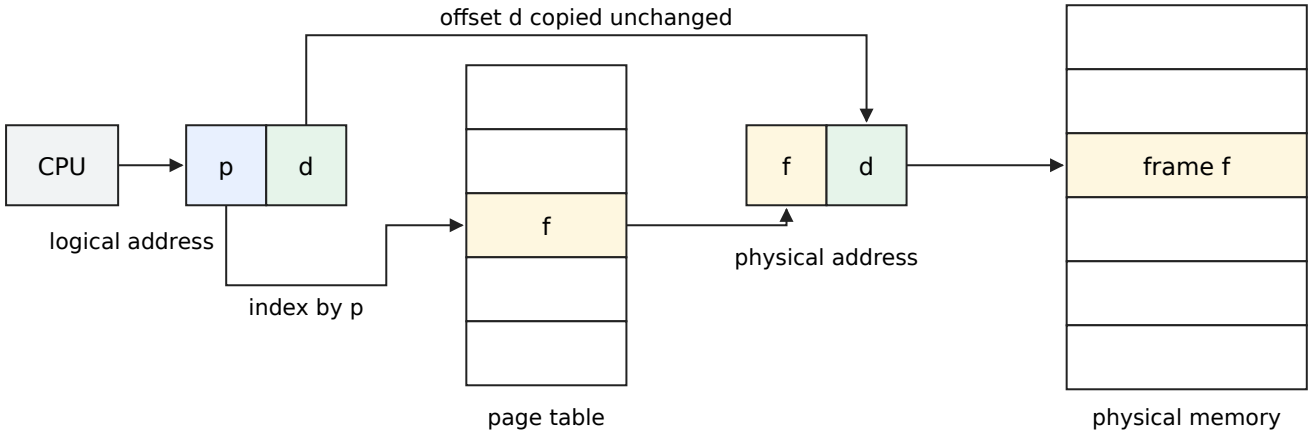


Fig: Paging hardware — page number p indexes the page table to get frame f; offset d is appended unchanged

Mini-numerical: page size 1 KB (1024 B), logical address 2500, page table entry for page 2 = frame 5. p = ⌊2500/1024⌋ = 2, d = 2500 − 2048 = 452. Physical = 5 × 1024 + 452 = **5572**.

1.15 Segmentation, and Paging vs Segmentation

Importance 9/10 · very likely · second half of 2025 Q16; comparison table is likely as a 5-marker

In simple words: segmentation divides a program the way the programmer sees it — main code, functions, stack, arrays, symbol table — into variable-size **segments**.

- Logical address = ⟨**segment number s**, **offset d**⟩.
- **Segment table**: each entry has **base** (start in physical memory) and **limit** (length). If d < limit → physical = base + d; else **trap** (addressing error).
- Supports protection and sharing per logical unit; suffers external fragmentation. Modern systems (Intel x86) use **segmentation with paging**.

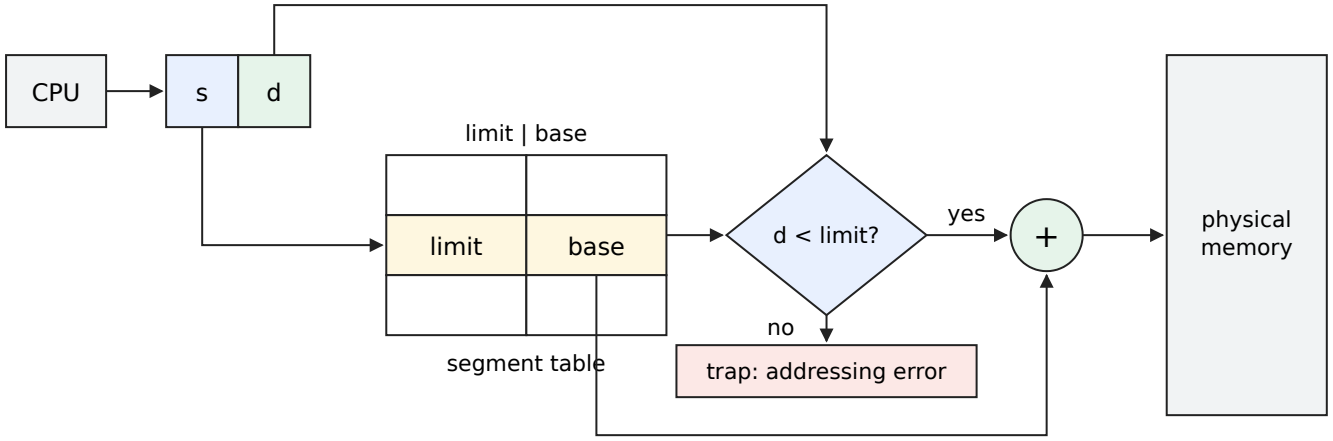


Fig: Segmentation hardware — offset is checked against the limit, then added to the base

Point	Paging	Segmentation
Unit	Fixed-size pages/frames	Variable-size segments
View	Physical (invisible to programmer)	Logical (programmer's view)
Address	⟨page no., offset⟩	⟨segment no., offset⟩
Table entry	Frame number	Base + limit
Fragmentation	Internal (last page)	External
Sharing/protection	Per page (awkward)	Per logical unit (natural)
Size decided by	Hardware	Compiler / programmer

1.16 Virtual Memory, Demand Paging and Page Faults

Importance 9/10 · very likely · syllabus "Virtual memory" (Paper-I(B)) + most likely follow-up to 2025 Q16

In simple words: virtual memory lets a program run even if only part of it is in RAM. Pages are brought in from disk only when needed, so programs can be bigger than physical memory and more programs fit at once.

Demand paging: a lazy swapper (**pager**) loads a page only when it is referenced. Each page-table entry has a **valid–invalid bit** (v = in memory, i = not in memory / illegal).

Page-fault handling steps:

1. Process references a page whose entry is marked **invalid** → hardware **trap** to OS.

2. OS checks an internal table: invalid reference → abort; valid but not in memory → page it in.
3. Find a **free frame** (if none, run page replacement).
4. Schedule a disk read to bring the page into the frame.
5. Update the page table (set valid bit) and internal tables.
6. **Restart the instruction** that caused the fault.

Effective access time: EAT = (1 − p) × memory access + p × page-fault service time (p = fault rate). Example: 200 ns memory, 8 ms fault service, p = 0.001 → EAT = 0.999 × 200 + 0.001 × 8,000,000 ≈ 8,200 ns — a 40× slowdown, so p must be tiny.

Benefits: larger programs than RAM, higher multiprogramming, less I/O to load/swap, shared libraries, faster process creation (copy-on-write).

1.17 Page Replacement — FIFO, LRU, Optimal, Belady's Anomaly

Importance 9/10 · very likely · page-fault counting numericals are the most likely 10-mark memory question in 2026

In simple words: when a page fault occurs and no frame is free, the OS must throw out ("victimise") one resident page. Different rules decide which.

Algorithm	Victim	Notes
FIFO	Page loaded earliest	Simple queue; suffers **Belady's anomaly**
Optimal (OPT/MIN)	Page not used for the **longest time in future**	Lowest possible fault rate; not implementable (needs future) — used as benchmark
LRU	Page **not used for the longest time in the past**	Good approximation of OPT; implement with counters or a stack; stack algorithm ⇒ no Belady
LRU approximations	Reference bit, **second-chance (clock)**	Cheap hardware support
LFU / MFU	Least / most frequently used	Counting-based, less common

Worked example (verified)

Reference string: **7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1** · 3 frames · all frames initially empty.

FIFO (frames shown after each fault; "–" = hit):

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
F1	7	7	7	2	–	2	2	4	4	4	0	–	–	0	0	–	–	7	7	7
F2		0	0	0	–	3	3	3	2	2	2	–	–	1	1	–	–	1	0	0
F3			1	1	–	1	0	0	0	3	3	–	–	3	2	–	–	2	2	1
Fault	F	F	F	F		F	F	F	F	F	F			F	F			F	F	F

FIFO faults = **15**. **LRU** faults = **12**. **Optimal** faults = **9** (3 compulsory + 6 replacements). Hits: FIFO 5, LRU 8, OPT 11. All three counts were checked with a simulation.

How OPT decides (first replacement): at ref 2, frames {7, 0, 1}; next uses — 0 at position 5, 1 at position 14, 7 at position 18 → evict **7** (used farthest in future).

Belady's anomaly: for FIFO, *more frames can give more faults*. Classic string **1 2 3 4 1 2 5 1 2 3 4 5**: FIFO with 3 frames = **9 faults**, with 4 frames = **10 faults** (verified). OPT and LRU are **stack algorithms** and never show the anomaly.

Exam tip: Draw the frame table column by column and mark F under each fault; write "Hit ratio = hits/references" at the end for bonus clarity.

1.18 Thrashing

Importance 7/10 · good chance · natural extension of virtual memory; short note

In simple words: thrashing is when the system spends more time swapping pages in and out than actually executing — CPU utilisation collapses.

- Cause: degree of multiprogramming too high → each process has fewer frames than its active set → constant page faults → CPU idle → OS (seeing low utilisation) admits *more* processes → worse.
- **Locality model:** a process moves from locality to locality; it needs enough frames for its current locality.
- Remedies: **working-set model** (keep the set of pages referenced in the last Δ references; if Σ WSS > total frames, suspend a process), **page-fault-frequency (PFF)** control (give frames if fault rate above upper bound, take away if below lower bound), local replacement, reduce multiprogramming.

1.19 I/O Management — Hardware, Polling, Interrupts, DMA

Importance 8/10 · likely · "I/O management" is an unasked syllabus phrase — strong Sec A/B candidate

In simple words: the OS hides the messy details of thousands of different devices behind a uniform interface, and decides how data moves between device and memory.

I/O hardware: device ↔ **port** ↔ **bus** (PCIe, USB, SATA) ↔ **device controller** (has status, control, data-in, data-out registers) ↔ CPU. Access via **port-mapped I/O** (special `in`/`out` instructions) or **memory-mapped I/O** (device registers appear as memory addresses).

Technique	How it works	Pros / cons
Programmed I/O (polling)	CPU repeatedly reads the device's busy/status bit until ready	Simple; wastes CPU cycles (busy waiting)
Interrupt-driven I/O	Device raises an **interrupt** line when ready; CPU saves state, runs the **interrupt handler** (via interrupt vector), then resumes	CPU free while waiting; overhead per byte for large transfers
DMA (Direct Memory Access)	CPU gives the **DMA controller** source, destination, count; controller transfers the whole block directly to memory and interrupts **once** at the end	Best for bulk transfers (disk); uses **cycle stealing** on the bus

Kernel I/O subsystem services:

- **Buffering** — memory area holding data in transit; copes with speed mismatch (modem vs disk), transfer-size mismatch, and copy semantics. Single, double, circular buffers.
- **Caching** — keeps a *copy* of data in faster storage for reuse (a buffer may be the only copy; a cache is always a copy).
- **Spooling** (Simultaneous Peripheral Operations On-Line) — buffer holding output for a device that cannot interleave streams, e.g. printer: each job's output goes to a disk file, printed one at a time.
- Also: I/O scheduling, device reservation, error handling, I/O protection (all I/O instructions privileged).

Device classes: **block** (disk — `read`, `write`, `seek`), **character** (keyboard — `get`, `put`), **network** (sockets). Blocking vs non-blocking vs asynchronous I/O.

1.20 Disk Structure and Disk Scheduling

Importance 8/10 · likely · "Mass Storage Structure" + Paper-I(B) "I/O scheduling"; head-movement numericals are easy 10 marks

In simple words: a hard disk has spinning **platters**; data lies on concentric **tracks** split into **sectors**; the same track on all platters forms a **cylinder**. The read/write head must move (seek) to the right cylinder, so the order in which we serve requests matters.

- **Access time = seek time + rotational latency + transfer time.** Seek time dominates → scheduling minimises total head movement.
- Disk is addressed as a 1-D array of **logical blocks** mapped onto sectors.
- Disk management: **low-level formatting** (sectors with header/ECC), **partitioning**, **logical formatting** (file system), **boot block** (bootstrap loader), bad-block handling. **RAID** levels 0 (striping), 1 (mirroring), 5 (block striping + distributed parity), etc. for performance/reliability.

Algorithm	Rule	Remark
FCFS	Serve in arrival order	Fair, but wild swings
SSTF	Serve request with **shortest seek** from current head	Like SJF; may starve far requests
SCAN (elevator)	Move in one direction to the **end of disk**, serving requests, then reverse	Uniform-ish wait
C-SCAN	Move one way to the end, **jump back** to the start without serving, continue same direction	More uniform wait time
LOOK / C-LOOK	Like SCAN / C-SCAN but go only as far as the **last request**, not the disk end	Practical versions

Worked example (verified)

Cylinders 0–199, head at **53**, queue: **98, 183, 37, 122, 14, 124, 65, 67**.

Algorithm	Service order	Total head movement
FCFS	53→98→183→37→122→14→124→65→67	45+85+146+85+108+110+59+2 = **640**
SSTF	53→65→67→37→14→98→122→124→183	12+2+30+23+84+24+2+59 = **236**
SCAN (towards 0)	53→37→14→**0**→65→67→98→122→124→183	53 + 183 = **236**
C-SCAN (towards 199)	53→65→…→183→**199**→**0**→14→37	146 + 199 + 37 = **382** (counting the return jump)
LOOK (towards 199)	53→65→67→98→122→124→183→37→14	130 + 169 = **299**

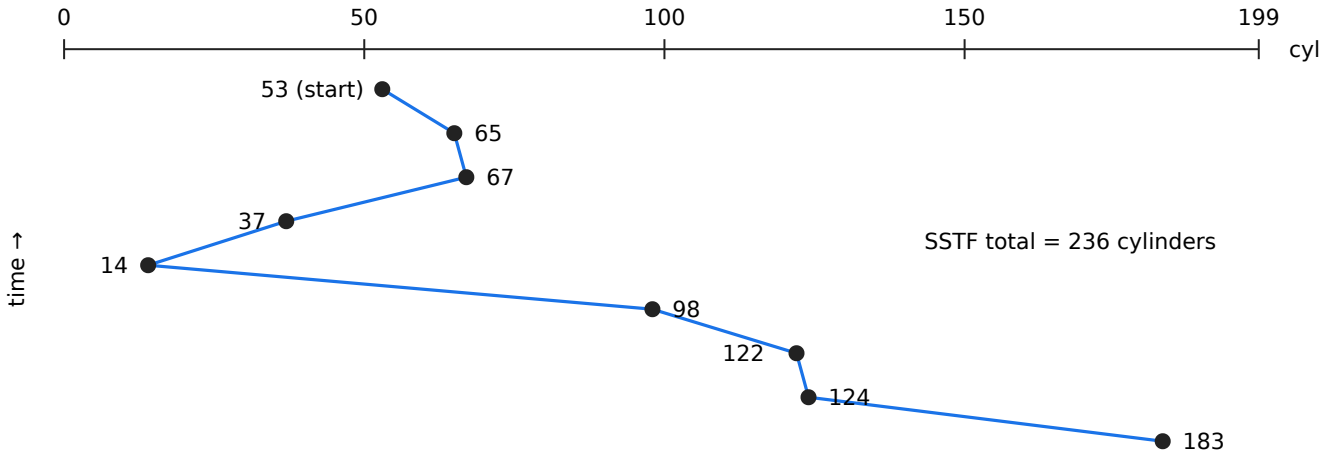


Fig: SSTF head movement for queue 98, 183, 37, 122, 14, 124, 65, 67 with head at 53

Exam tip: State the direction assumption for SCAN/C-SCAN/LOOK explicitly — the answer changes with it (SCAN towards 199 here: 53→199 = 146, then 199→14 = 185, total **331**; C-LOOK towards 199 = 130 + 169 + 23 = **322**). Always mention whether the C-SCAN return seek is counted.

1.21 File System Management

Importance 8/10 · likely · "File system Management" is an unasked syllabus phrase — strong 10-mark candidate

In simple words: a file is a named collection of related information stored on secondary storage. The file system decides how files are named, organised in directories, stored on disk blocks and protected.

File attributes: name · identifier (unique number) · type · location (pointer to device/blocks) · size · protection (read/write/execute rights) · time, date and user identification. Kept in the **directory entry / File Control Block (inode in UNIX)**.

File operations: create, open, read, write, reposition (seek), delete, truncate, close; OS keeps an **open-file table** (file pointer, open count, location, access rights).

Access methods:

Method	How	Example
Sequential	Records processed in order, one after another (read next / write next / rewind)	Tape, log files, compilers reading source
Direct (relative)	Jump to any block number n directly (read n / write n)	Databases, airline reservation
Indexed (index-sequential)	An index of keys → block pointers; search index then access block	ISAM, large record files

Directory structures:

- **Single-level** — one directory for all users; naming conflicts.
- **Two-level** — separate User File Directory (UFD) per user under a Master File Directory (MFD); isolates users, no grouping.
- **Tree-structured** — arbitrary depth, current directory, absolute/relative path names (UNIX, Windows).
- **Acyclic-graph** — allows shared subdirectories/files via **links**; no cycles.
- **General graph** — cycles allowed; needs garbage collection / cycle detection.

Allocation methods:

Method	Idea	Pros	Cons
Contiguous	File occupies consecutive blocks; directory stores start + length	Fast sequential and direct access; minimal seeks	External fragmentation; file growth hard
Linked	Each block holds a pointer to the next; directory stores first and last block	No external fragmentation; easy growth	Sequential access only; pointer space; one broken pointer loses the rest. Variant: **FAT** (pointers kept in a table)
Indexed	All pointers kept in one **index block**; directory points to index block	Direct access, no external fragmentation	Index-block overhead; large files need linked/multilevel index or **combined scheme** (UNIX inode: 12 direct + single, double, triple indirect)

Free-space management:

- **Bit vector (bitmap)** — 1 bit per block (1 = free in Silberschatz); easy to find contiguous free blocks; needs memory.
- **Linked list** — free blocks linked together; no waste but slow traversal.
- **Grouping** — first free block stores addresses of n free blocks.
- **Counting** — store (first free block, count of contiguous free blocks).

Exam tip: For a 10-mark "file system management" answer use the order: file concept → attributes → access methods → directory → allocation → free space, with one table each.

1.22 Security and Protection

Importance 8/10 · likely · "Security & Protection" is unasked in 2025 Paper-II; appears in two other syllabi too

In simple words: **protection** is the *internal* mechanism that controls which process/user may access which resource; **security** defends the system against *external and internal attacks* (it needs protection plus authentication, encryption, etc.).

Goals of security (CIA): Confidentiality (no unauthorised reading), **Integrity** (no unauthorised modification), **Availability** (no denial of service). Often add authentication and non-repudiation.

Program threats:

Threat	What it is
Trojan horse	Program that appears useful but secretly misuses the rights of the user who runs it (e.g. fake login screen)
Trap door / back door	Hidden entry point left by the programmer bypassing normal security
Logic bomb	Code that triggers malicious action when a condition occurs (date, event)
Virus	Code fragment that **attaches itself to a host program** and spreads when the host runs; needs human action (boot-sector, file, macro, polymorphic, stealth)
Worm	**Standalone** program that replicates itself over the network **without** a host program (e.g. Morris worm, 1988)

System/network threats: denial of service, port scanning, spoofing, man-in-the-middle.

User authentication: something the user **knows** (password, PIN — use salting + hashing, one-time passwords), **has** (smart card, token, OTP device), **is** (biometrics — fingerprint, iris). Multi-factor = two or more of these.

Protection — domain and access matrix: a **domain** is a set of ⟨object, rights⟩ pairs (e.g. ⟨file F1, {read, write}⟩). The **access matrix**: rows = domains (users/processes), columns = objects, cell = set of rights.

	F1	F2	Printer
D1	read		
D2			print
D3	read, write	execute	

Access Control List (ACL)	Capability list
Matrix stored **by column** — per object	Matrix stored **by row** — per domain
Object keeps list of ⟨domain, rights⟩	Domain keeps list of ⟨object, rights⟩ (unforgeable tickets)
Easy to see/revoke who can access an object	Easy to see what a domain can do; revocation harder
UNIX rwx bits, Windows NTFS ACLs	Hydra, CAP, Linux POSIX capabilities

Principle of **least privilege**: give each program/user only the rights it needs.

Exam tip: virus vs worm vs Trojan is a top MCQ — **worm needs no host**, **virus needs a host**, **Trojan does not self-replicate**.

2. Software Engineering

Syllabus phrase (Paper-II §2): *Concept of systems, Software development process models, software project planning and management; cost estimation methods, scheduling, requirement analysis, software testing strategies, Quality Assurance.* 2025 asked SDLC phases (5), WBS (5) and cost estimation methods (15). Everything else in the phrase is still unasked.

2.1 Concept of Systems

Importance 8/10 · likely · first phrase of the syllabus, unasked in 2025 — classic 5-mark "define system and its characteristics"

A **system** is a set of interrelated components that work together inside a boundary to achieve a common goal by taking inputs, processing them and producing outputs. A college, a payroll program and the human body are all systems. Software engineering treats software as one part of a larger *computer-based system* (hardware + software + people + procedures + data).

Elements / characteristics of a system

Element	Meaning	Payroll example
Components (sub-systems)	Parts that make up the system	Attendance module, salary module, printer
Boundary	Line separating the system from everything outside it	Only the HR department's processing
Environment	Everything outside the boundary that affects it	Tax laws, bank, employees
Interfaces	Points where the system meets the environment or other sub-systems	Bank transfer file, login screen
Input	Data/energy entering the system	Attendance, pay rates
Processing	Transformation of input to output	Salary calculation
Output	Results produced	Payslips, bank advice
Control & Feedback	Monitoring output and adjusting input/processing	Audit reports, error correction
Objective	Common goal all parts serve	Pay correct salary on time

Other characteristics: **organisation** (structure/order), **interaction** (components work with each other), **interdependence** (one part depends on another), **integration** (parts work as a whole), **central objective**.

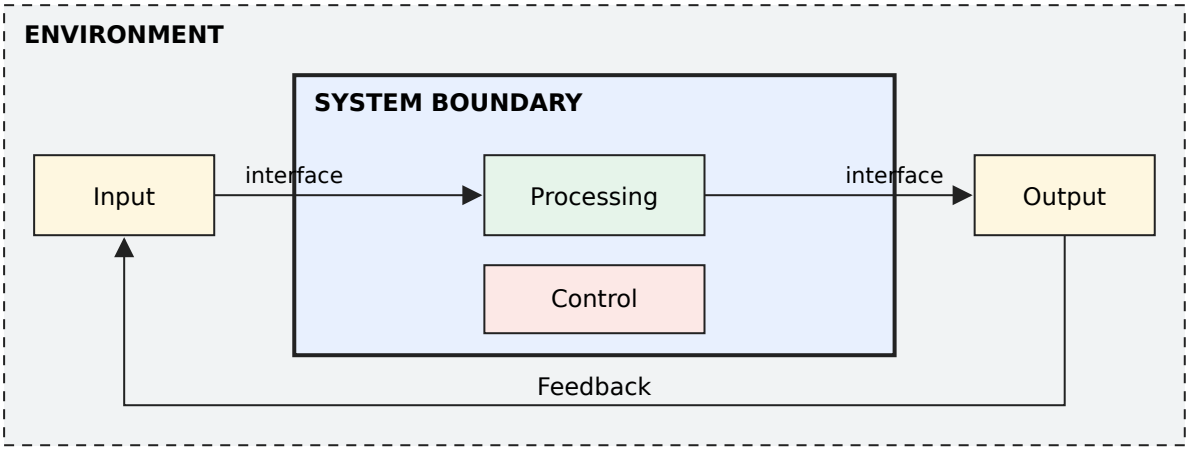


Fig: General model of a system — input, processing, output, control and feedback inside a boundary

Types of systems

Pair	Type 1	Type 2
Open vs Closed	**Open**: interacts with its environment (takes input, gives output) — e.g. business system, any real software	**Closed**: isolated from environment (theoretical) — e.g. a sealed chemical reaction
Physical vs Abstract	**Physical**: tangible — computers, files, people	**Abstract**: conceptual — models, formulas, data-flow diagrams
Deterministic vs Probabilistic	Behaviour fully predictable (a program)	Behaviour predictable only with probability (inventory demand)
Natural vs Man-made	River system	Information system
Permanent vs Temporary	Long-lived (banking)	Short-lived (an election cell)

Exam tip: For a 5-mark answer, give the definition, draw the input–process–output box with boundary and feedback, and list at least six elements. Mention "a software system is an *open, man-made, physical* system".

2.2 Software, its Characteristics and the Software Crisis

Importance 6/10 · moderate · background to "process models"; frequent MCQ ("software does not wear out")

Software = computer programs + associated documentation + configuration data (Pressman/Sommerville). Software engineering is "the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software" (IEEE).

Characteristics of software (Pressman):

- Software is **developed/engineered, not manufactured** — cost is in engineering, not in production copies.
- Software **does not wear out** but it **deteriorates** due to changes. Hardware follows the "bath-tub" failure curve; software's idealised curve is flat, but the actual curve has spikes after each change.
- Most software is still **custom-built**, although component reuse is increasing.
- It is intangible, complex and easy to change (which makes it easy to break).

Software crisis (late 1960s, NATO conference 1968): projects ran over budget and time, software was unreliable, hard to maintain, did not meet requirements, and some was never delivered. Causes: growing complexity, no systematic method, poor estimation, changing requirements. Software engineering as a discipline emerged in response.

2.3 Software Development Life Cycle (SDLC) — Phases

Importance 7/10 · good chance · asked as 5-mark in 2025; may return as part of a model question

The **SDLC** is the sequence of activities through which software is conceived, built, delivered and maintained.

#	Phase	What happens	Main output
1	Feasibility study / Planning	Is the project technically, economically, operationally viable?	Feasibility report
2	Requirement analysis & specification	Gather and document what the system must do	SRS document
3	Design	High-level (architecture, modules) and low-level (algorithms, data structures)	Design document (DFDs, UML)
4	Coding / Implementation	Translate design into programs	Source code, unit-tested modules
5	Testing	Integration, system and acceptance testing	Test reports, defect fixes
6	Deployment	Install at user site, data conversion, training	Working system
7	Maintenance	Corrective, adaptive, perfective, preventive changes	Updated versions

Exam tip: In the "Waterfall" answer these same phases become the boxes of the diagram — learn the list once and reuse it.

2.4 Software Development Process Models

Importance 9/10 · very likely · syllabus names "process models"; waterfall/prototype/spiral are evergreen 10-markers and MCQs

A **process model** (life-cycle model) is an abstract representation of the order in which SDLC activities are carried out.

(a) Waterfall (classical linear sequential) model — Royce 1970

Each phase is completed fully and signed off before the next begins; output of one phase is input to the next. Feedback to earlier phases is possible only in the *iterative waterfall* variant.

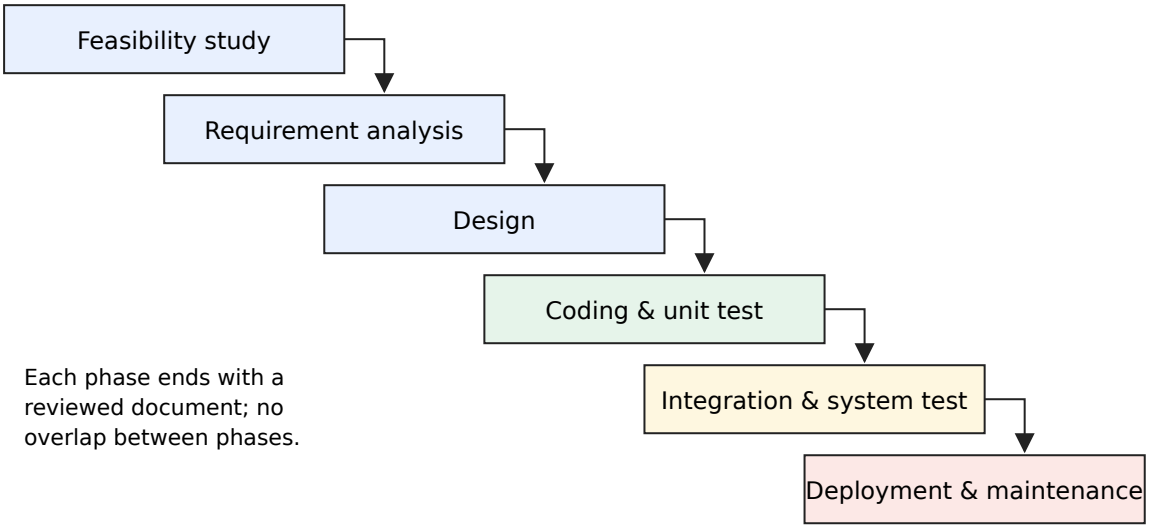


Fig: Waterfall model — strictly sequential phases cascading downwards

- **Advantages:** simple, easy to manage, well-documented, good for small projects with stable, well-understood requirements.
- **Disadvantages:** no working software until late; requirement changes are expensive; errors found late; customer sees product only at the end; high risk for large projects.

(b) Incremental model

The product is built and delivered in **increments**; each increment is a mini-waterfall that adds functionality. The first increment is the core product. Gives early partial delivery and spreads risk; needs good up-front architecture.

(c) Prototyping model

A quick, throwaway (or evolutionary) **prototype** is built from partial requirements, shown to the customer, refined using feedback, and repeated until requirements are clear; then the real system is engineered.
Steps: requirement gathering → quick design → build prototype → customer evaluation → refine → engineer product.

- Use when requirements are **unclear** or the UI is critical.
- Risks: customer may treat prototype as final product; poor design choices kept "for now"; extra cost.

(d) Spiral model — Boehm 1986

A **risk-driven** meta-model combining prototyping's iteration with waterfall's control. Each loop of the spiral is one phase and passes through four quadrants. The radius shows cumulative cost; the angle shows progress.

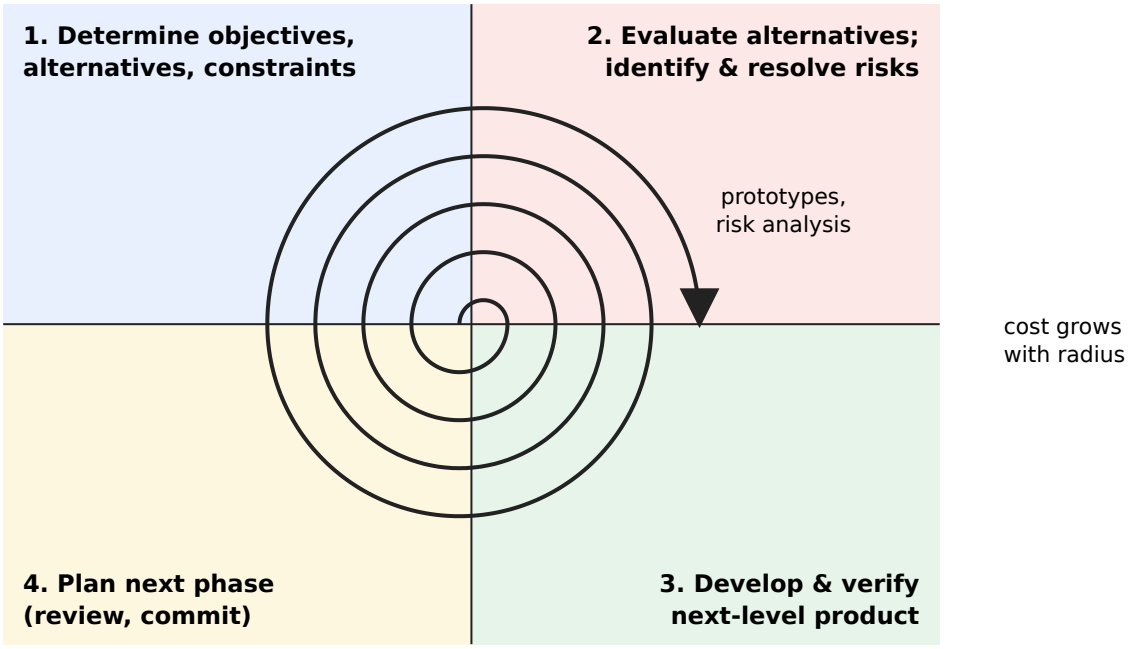


Fig: Boehm's spiral model — every loop passes through the four quadrants; risk analysis is done in each loop

- **Advantages:** explicit risk analysis; suits large, high-risk projects; changes absorbed in later loops; early prototypes.
- **Disadvantages:** costly; needs risk-assessment expertise; hard to define milestones; overkill for small projects.
- Spiral is called a **meta-model** because it can contain waterfall, prototyping and incremental models.

(e) RAD (Rapid Application Development)

Very short cycle (60–90 days) using component reuse, CASE tools and parallel teams. Phases: business modelling → data modelling → process modelling → application generation → testing & turnover. Needs well-modularised requirements and skilled developers; not suitable for high technical risk.

(f) V-model (Verification & Validation model)

An extension of waterfall in which **every development phase on the left has a matching test phase on the right**. Test plans are written during the corresponding development phase.

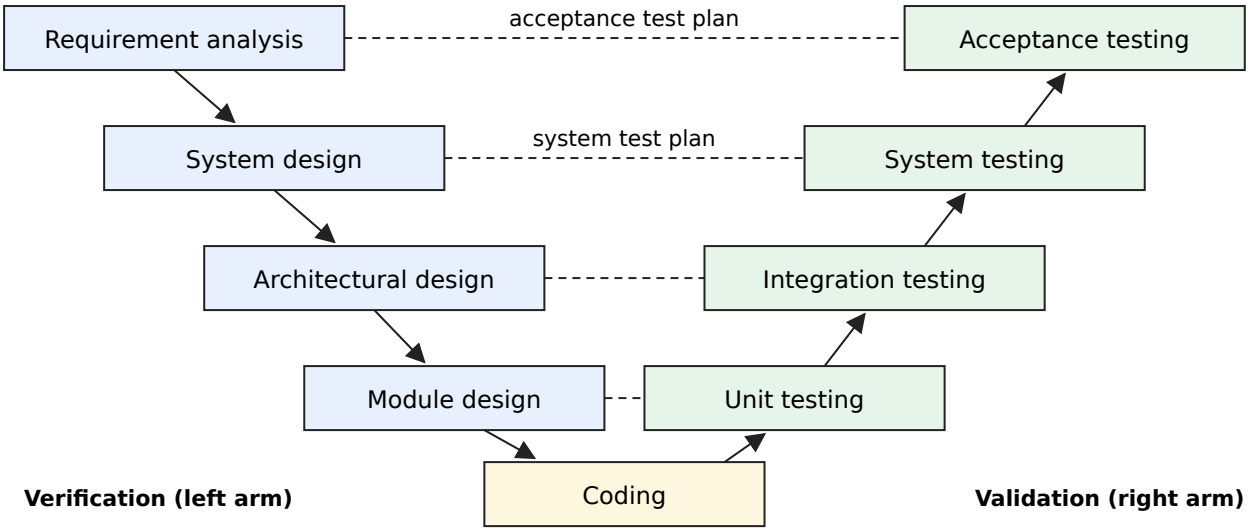


Fig: V-model — each design phase is paired (dashed) with the test level that checks it

(g) Agile and Scrum

Agile (Agile Manifesto, 2001) values *individuals and interactions* over processes and tools, *working software* over comprehensive documentation, *customer collaboration* over contract negotiation, *responding to change* over following a plan. Delivery is in short iterations with continuous customer feedback.

Scrum (an agile framework): roles — **Product Owner** (owns product backlog), **Scrum Master** (facilitator, removes impediments), **Development Team**. Artefacts — product backlog, sprint backlog, increment. Events — sprint (1–4 weeks), sprint planning, daily stand-up (15 min), sprint review, sprint retrospective. Other agile methods: XP (pair programming, TDD), Kanban.

Comparison of process models

Feature	Waterfall	Prototyping	Incremental	Spiral	V-model	Agile
Requirements	Fixed, clear	Unclear	Mostly known	Evolving	Fixed, clear	Changing
Flow	Linear	Iterative	Linear in increments	Iterative, risk-driven	Linear + paired testing	Iterative short sprints
Risk handling	Poor	Medium	Medium	Excellent	Medium	Good
Customer involvement	Start and end	High	Per increment	Each loop	Low	Continuous
Working software	At end	Prototype early	Early partial	Per loop	At end	Every sprint
Cost of change	High	Low	Medium	Medium	High	Low
Best for	Small, stable projects	UI-heavy, unclear needs	Early market release	Large, high-risk	Safety-critical	Fast-changing business

Exam tip: For "compare waterfall and spiral" draw both figures and give 5–6 rows of the table. The words "risk analysis" and "meta-model" earn marks for spiral; "no going back" and "documentation-driven" for waterfall.

2.5 Software Project Planning and Management

Importance 7/10 · good chance · syllabus phrase; 2025 asked only the WBS sub-part

Software project management is planning, organising, staffing, monitoring and controlling a software project so that it is delivered **on time, within budget and with required quality**. The triple constraint is **scope – time – cost** (quality in the middle).

Project planning activities (in order):

1. Define scope and objectives (what is in/out).
2. Estimate size (LOC, function points).
3. Estimate effort and cost (COCOMO, expert judgement).
4. Schedule — WBS, task network, Gantt/PERT.
5. Allocate resources (staff, hardware, tools).
6. Risk planning.
7. Prepare Software Project Management Plan (SPMP); then monitor and control.

Project manager's role: planning and estimation, scheduling, staffing and team building, monitoring progress against plan, risk management, communication with customer and management, configuration/change control, reporting.

4 P's of project management (Pressman): People, Product, Process, Project.

Work Breakdown Structure (WBS)

Importance 6/10 · moderate · asked in 2025 (5 marks); repeat less likely but quick to answer

A **WBS** is a hierarchical, deliverable-oriented decomposition of the total project work into smaller, manageable pieces. The lowest level items are **work packages** that can be estimated, scheduled, assigned to one person/team and tracked.

Purpose: defines full scope (100% rule — the children add up to exactly the parent); basis for estimation, scheduling, resource allocation and cost control; assigns clear responsibility; helps progress tracking.

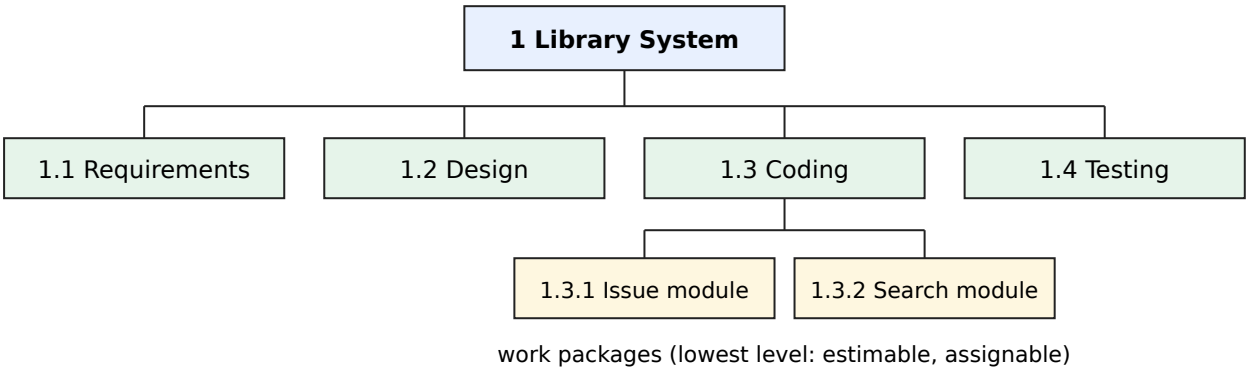


Fig: A small WBS — project decomposed into phases, then into work packages

Exam tip: Define WBS, state the 100% rule, draw a 3-level tree, list 4–5 purposes. That is a full 5-mark answer.

2.6 Cost Estimation Methods

Importance 8/10 · likely · asked as 15-marker in 2025; COCOMO numerical can come in Sec B/C or MCQ

Software cost estimation predicts the **effort** (person-months), **time** (months) and **cost** needed to develop software. Size (KLOC or function points) is usually estimated first, then effort.

Classification of methods

Method	How it works	Pros	Cons
Expert judgement	One or more experienced people estimate from experience	Quick, uses domain knowledge	Subjective, biased
Delphi technique	Experts estimate anonymously; coordinator circulates a summary; repeated rounds until consensus (Wideband Delphi adds group discussion)	Reduces individual bias and dominance	Time-consuming
Estimation by analogy	Compare with similar completed projects and adjust	Based on real data	Needs a history database; no two projects identical
Top-down	Estimate total cost from global properties, then split among components	Fast; system-level costs included	Low-level technical problems missed
Bottom-up	Estimate each component (WBS work package) and sum	Detailed, accurate	Slow; integration costs missed
Parkinson's law / Price-to-win	Cost = available resources / what wins contract	—	Not real estimation; often unrealistic
Algorithmic (parametric)	Mathematical model: Effort = f(size, cost drivers) — COCOMO, Putnam (SLIM), Function Points	Objective, repeatable	Needs calibration; size must be estimated first

COCOMO (Constructive Cost Model, Boehm 1981) — Basic COCOMO

Three project classes:

- **Organic:** small team, familiar environment, well-understood problem (e.g. payroll, up to ~50 KLOC).
- **Semi-detached:** medium team, mix of experienced and new staff (e.g. compiler, DBMS).
- **Embedded:** tight hardware/software/operational constraints (e.g. avionics, ATM, real-time).

Formulas: **Effort E = a × (KLOC)^b person-months**, **Development time D = c × (E)^d months**, **Average staff = E / D**, **Productivity = LOC / E**.

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Worked example (verified with python3): Estimate effort and time for a 32 KLOC project in all three modes.

Mode	E = a(32)^b (PM)	D = 2.5(E)^d (months)	Staff = E/D	Productivity (LOC/PM)
Organic	2.4 × 32^1.05 = **91.33**	2.5 × 91.33^0.38 = **13.90**	6.57 ≈ 7 persons	350.4
Semi-detached	3.0 × 32^1.12 = **145.51**	2.5 × 145.51^0.35 = **14.29**	10.18 ≈ 10 persons	219.9
Embedded	3.6 × 32^1.20 = **230.40**	2.5 × 230.40^0.32 = **14.25**	16.16 ≈ 16 persons	138.9

Observation: development time barely changes with mode (~14 months), but effort and team size grow sharply from organic to embedded.

Intermediate COCOMO multiplies basic effort by an **EAF (Effort Adjustment Factor)** = product of 15 cost drivers (product, hardware, personnel, project attributes). **Detailed COCOMO** applies drivers phase-wise. **COCOMO II** (1995/2000) has Application Composition, Early Design and Post-Architecture models.

Function Point (FP) analysis — Albrecht 1979 (briefly)

Measures size by **functionality delivered**, independent of language. Count five components, weight them (simple/average/complex), sum to **UFP**, then adjust:
FP = UFP × [0.65 + 0.01 × ΣFi], where Fi are 14 general system characteristics each rated 0–5 (so the factor ranges 0.65–1.35).

Component	Average weight
External Inputs (EI)	4
External Outputs (EO)	5
External Inquiries (EQ)	4
Internal Logical Files (ILF)	10
External Interface Files (EIF)	7

Example (verified): EI=10, EO=8, EQ=5, ILF=4, EIF=2, all average → UFP = 40+40+20+40+14 = **154**; ΣFi = 42 → factor 1.07 → **FP = 164.78**.

Exam tip (for "which method is most reliable?"): No single method is reliable alone. Say: algorithmic models (COCOMO) are the most *objective and repeatable* once calibrated with organisational history; best practice is to use **two or more methods (e.g. COCOMO + Delphi/analogy) and reconcile** them, and re-estimate as the project progresses.

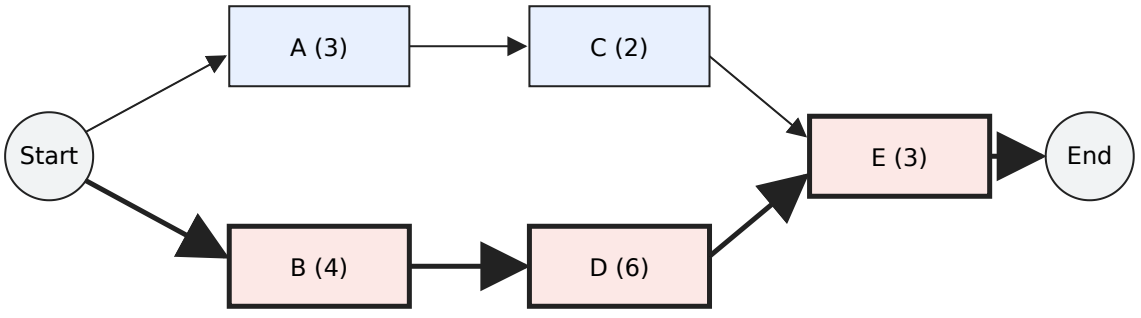
2.7 Project Scheduling (Gantt, PERT/CPM)

Importance 8/10 · likely · "scheduling" is a syllabus word not yet asked; easy 10-marker with a network numerical

Scheduling fixes start/end dates for every task, their dependencies and resources. Steps: identify activities (from WBS) → find dependencies → estimate durations → draw activity network → find critical path → allocate resources → draw Gantt chart.

- **Gantt chart:** bar chart — tasks on the y-axis, time on the x-axis; each bar shows the task's start and duration. Good for tracking; does not show dependencies clearly.
- **PERT (Program Evaluation & Review Technique):** network of tasks with *probabilistic* times. Expected time **te = (o + 4m + p) / 6** (o = optimistic, m = most likely, p = pessimistic).
- **CPM (Critical Path Method):** network with *deterministic* times. The **critical path** is the longest path through the network; its length is the minimum project duration. Tasks on it have **zero slack (float)**; any delay on them delays the project.

Worked CPM example (verified with python3). Activities: A=3 (start), B=4 (start), C=2 (after A), D=6 (after B), E=3 (after C and D).



Critical path (bold): Start → B → D → E → End = 4 + 6 + 3 = 13

Fig: Activity-on-node network; bold boxes and arrows form the critical path

Activity	Duration	ES	EF	LS	LF	Slack (LS−ES)
A	3	0	3	5	8	5
B	4	0	4	0	4	**0**
C	2	3	5	8	10	5
D	6	4	10	4	10	**0**
E	3	10	13	10	13	**0**

Paths: A–C–E = 8, B–D–E = 13 → **critical path B–D–E, project duration 13 units**. Forward pass: ES = max EF of predecessors, EF = ES + duration. Backward pass: LF = min LS of successors, LS = LF − duration.

PERT vs CPM: PERT is event-oriented, probabilistic (3 time estimates), used for R&D/new projects; CPM is activity-oriented, deterministic (1 estimate), also handles time–cost trade-off (crashing).

2.8 Requirement Analysis and SRS

Importance 9/10 · very likely · syllabus word "requirement analysis" unasked; SRS characteristics is a favourite 5/10-marker

Requirement analysis is the process of discovering, analysing, documenting and validating what the customer needs from the system. Steps: **feasibility study → elicitation → analysis (modelling: DFD, ER, use cases) → specification (SRS) → validation → management of changes.**

Feasibility study — is the project worth doing?

Type	Question
Technical	Can it be built with available technology and skills?
Economic	Do benefits exceed costs (cost–benefit analysis)?
Operational	Will users accept and use it?
Schedule	Can it be done in the required time?
Legal	Any legal/regulatory obstacles?

Functional vs non-functional requirements

Functional requirements	Non-functional requirements
What the system must do — services, functions	*How well* the system does it — qualities/constraints
e.g. "User can search a book by title"	e.g. "Search returns within 2 s", "99.9% uptime"
Verified by functional testing	Verified by performance, security, usability testing
Documented as use cases, features	Performance, reliability, security, usability, portability, maintainability

(Domain requirements come from the application domain, e.g. banking regulations.)

Elicitation techniques: interviews (structured/unstructured), questionnaires, observation, document study, brainstorming, **JAD (Joint Application Development)** workshops, prototyping, use-case/scenario analysis.

SRS (Software Requirements Specification) — the official statement of what developers must implement; a contract between customer and developer (IEEE 830 / ISO/IEC/IEEE 29148). Contents: introduction (purpose, scope, definitions), overall description, functional requirements, non-functional requirements, external interfaces, constraints.

Characteristics of a good SRS (IEEE 830): correct, unambiguous, complete, consistent, ranked for importance/stability, verifiable, modifiable, traceable (+ understandable). It states *what*, not *how*.

Exam tip: Memorise the eight IEEE 830 qualities as "**C-U-C-C-R-V-M-T**" and give a one-line meaning of each.

2.9 Software Testing Strategies

Importance 10/10 · sure to come · biggest unasked phrase in SE; expect a 10/15-marker on levels + black/white box

Testing is executing a program with the intent of finding errors. A testing **strategy** gives the order and levels of testing from small units to the whole system. "Testing can show the presence of bugs, never their absence" (Dijkstra).

Verification vs Validation (Boehm)

Verification	Validation
"Are we building the product **right**?"	"Are we building the **right** product?"
Checks conformance to specification/design	Checks it meets user's real needs
Static: reviews, walkthroughs, inspections	Dynamic: executing the software
Done during each phase	Done on the built product

Levels of testing

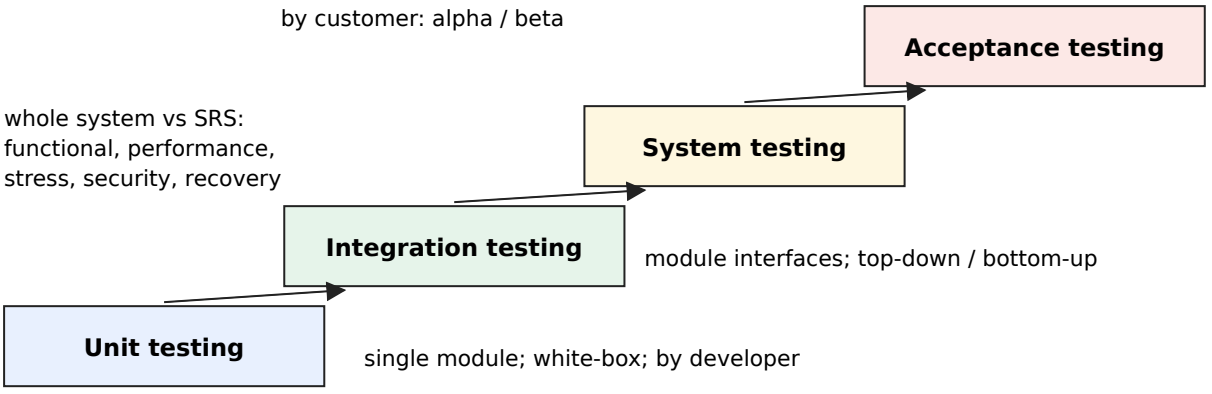


Fig: Testing levels — from the smallest unit up to customer acceptance

1. **Unit testing** — each module tested in isolation using **drivers** (call the module) and **stubs** (stand in for modules it calls). Mostly white-box.
2. **Integration testing** — modules combined and interfaces tested.
3. **Big-bang:** all at once — hard to locate faults.
4. **Top-down:** start from main module, add lower modules; uses **stubs**; early skeleton of system; lower-level testing delayed.
5. **Bottom-up:** start with lowest modules, build clusters upward; uses **drivers**; no early working skeleton.
6. **Sandwich (hybrid):** top-down for upper layers + bottom-up for lower layers simultaneously, meeting at a middle target layer.
7. **System testing** — complete integrated system against SRS: functional, **performance, stress (beyond limits), load, volume, security, recovery, usability, compatibility** testing.
8. **Acceptance testing** — by the customer to decide whether to accept.
9. **Alpha testing:** by users at the **developer's site**, in a controlled environment with developers present.
10. **Beta testing:** by real users at **their own sites**, developer absent; they report problems.
11. **Regression testing** — re-running previously passed tests after a change to ensure nothing that worked is now broken. Done at every level after bug fixes and in maintenance.

Black-box vs White-box testing

Black-box (functional/behavioural)	White-box (structural/glass-box)
Based on specification; internal code not seen	Based on internal code/logic
Tests *what* the program does	Tests *how* it does it
Techniques: equivalence partitioning, boundary value analysis, cause–effect graph, decision table	Techniques: statement, branch (decision), condition, path coverage; basis-path testing; loop testing
Done by testers; higher levels	Done by developers; unit level
Finds missing functions, interface errors	Finds logic errors, dead code, unreachable paths

Equivalence partitioning: divide input domain into classes that the program should treat the same; test one value per class. For input "age 18–60": valid class {18..60}, invalid {<18}, invalid {>60} → 3 tests (e.g. 30, 10, 70).
Boundary value analysis (BVA): errors cluster at edges, so test at min, min+1, nominal, max−1, max (and min−1, max+1 for robustness). For 18–60: 17, 18, 19, 59, 60, 61.

Coverage criteria (white-box), weakest to strongest: statement coverage (every statement run once) < branch/decision coverage (every branch true and false) < path coverage (every independent path). Branch coverage implies statement coverage.

Cyclomatic complexity (McCabe, 1976)

Measures the number of **linearly independent paths** in a program = minimum number of test cases for basis-path testing. For a control-flow graph with E edges, N nodes, P connected components:
V(G) = E − N + 2P = (number of predicate/decision nodes) + 1 = number of regions of the planar graph.

```
int f(int n) {
    int sum = 0, i = 1;          /* node 1 */
    while (i <= n) {             /* node 2 (decision) */
        if (i % 2 == 0)          /* node 3 (decision) */
            sum += i;            /* node 4 */
        else
            sum -= i;            /* node 5 */
        i++;                     /* node 6 */
    }
    return sum;                  /* node 7 */
}
```

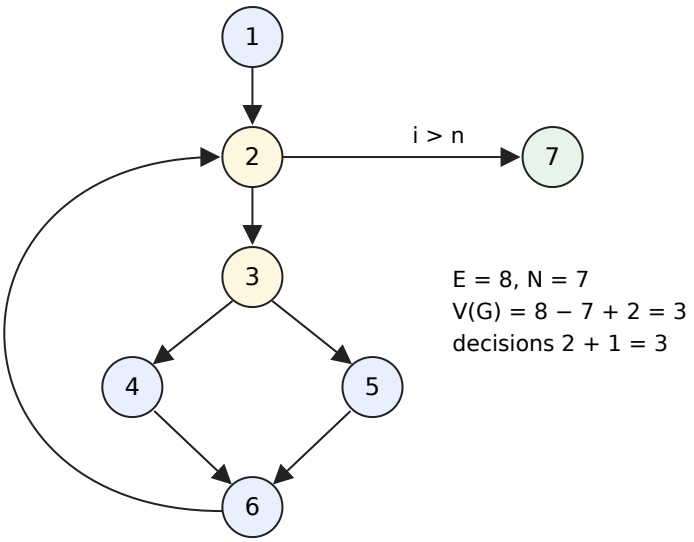


Fig: Control-flow graph of f(); edge 6→2 is the loop back

Edges: 1→2, 2→3, 2→7, 3→4, 3→5, 4→6, 5→6, 6→2 → E = 8, N = 7, **V(G) = 8 − 7 + 2 = 3** (verified with python3). Check: 2 decision nodes + 1 = 3. Basis paths: (i) 1-2-7, (ii) 1-2-3-4-6-2-7, (iii) 1-2-3-5-6-2-7 → 3 test cases (n=0, n=2 path via even, n=1 via odd). V(G) ≤ 10 is the usual guideline for a maintainable module.

Exam tip: A 15-mark "software testing strategies" answer = V&V table + levels diagram + integration approaches + alpha/beta + black vs white table + one BVA example + cyclomatic complexity example + regression.

2.10 Software Quality Assurance (SQA)

Importance 9/10 · very likely · "Quality Assurance" ends the syllabus phrase and is unasked; QA vs QC and CMM are favourites

Software quality = conformance to explicitly stated functional and performance requirements, documented development standards, and implicit characteristics expected of professionally developed software (Pressman). **SQA** is the planned and systematic set of activities that ensures the *process* and *product* conform to requirements and standards.

SQA activities: prepare SQA plan; participate in defining the process; formal technical reviews (FTR), walkthroughs and inspections; audits of work products; ensure deviations are documented and handled; record non-compliance and report to management; testing strategy oversight; software configuration management (change control); measurement/metrics collection.

QA vs QC

Quality Assurance (QA)	Quality Control (QC)
Process-oriented — prevents defects	**Product-oriented** — detects defects
Proactive	Reactive
Audits, process definition, standards, training	Testing, inspections of the product
Whole team responsible	Testing team responsible
e.g. adopting coding standards, CMMI	e.g. executing test cases

ISO 9001: international standard for a quality management system (QMS), applicable to any industry; software guidance in **ISO 9000-3 / ISO/IEC 90003**. Requires documented processes, management responsibility, internal audits, corrective/preventive action, continual improvement. Certification says *a documented quality system exists*; it does not grade maturity levels.

CMM / CMMI (Capability Maturity Model, SEI Carnegie Mellon) — five maturity levels of an organisation's software process:

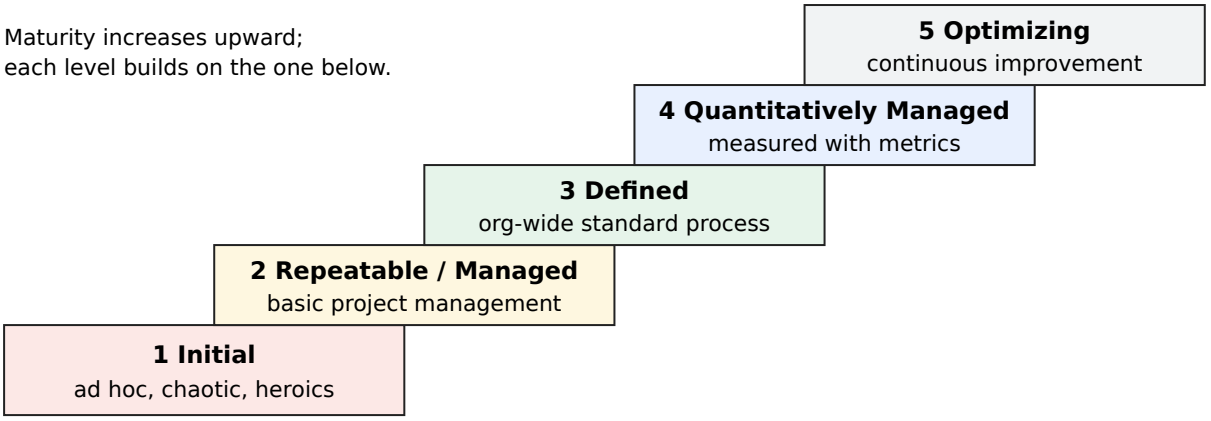


Fig: CMM/CMMI maturity levels (CMM names level 2 "Repeatable", CMMI calls it "Managed" and level 4 "Quantitatively Managed")

Software quality attributes — McCall's quality model (1977): 11 factors in three groups.

Perspective	Factors
Product operation	Correctness, Reliability, Efficiency, Integrity, Usability
Product revision	Maintainability, Flexibility, Testability
Product transition	Portability, Reusability, Interoperability

(ISO/IEC 9126 / 25010 uses: functionality, reliability, usability, efficiency, maintainability, portability; 25010 adds security and compatibility.)

Exam tip: QA-vs-QC table + list of SQA activities + CMM staircase is a complete 10-mark answer.

2.11 Software Maintenance

Importance 5/10 · possible · part of SDLC; frequent MCQ on the four types

Maintenance is modification of software after delivery; it typically consumes the largest share of total life-cycle cost.

Type	Purpose	Example
Corrective	Fix bugs found after release	Fixing a crash
Adaptive	Adapt to a changed environment	New OS, new tax law
Perfective	Improve performance or add features requested by users	Faster report, new filter
Preventive	Improve future maintainability (refactoring, documentation)	Restructuring code

Related: **reverse engineering** (recover design from code), **re-engineering** (reverse engineer + restructure + forward engineer).

2.12 Risk Management

Importance 6/10 · moderate · part of project management; links to spiral model

A **risk** is a potential future problem that may or may not happen. Types: **project risks** (schedule, budget, staff), **technical risks** (design, technology), **business risks** (market, management). Also known / predictable / unpredictable risks.

Process: **risk identification → risk analysis/projection** (probability × impact = **risk exposure**) → **risk planning** (avoid, transfer, mitigate, accept) → **risk monitoring**. Pressman's **RMMM** plan =

Risk Mitigation, Monitoring and Management. Reactive strategy waits for problems ("fire-fighting"); proactive strategy plans ahead.

3. Database Management Systems

Syllabus phrase (Paper-II): *Data Abstraction, Data models, Data independence, DDL, Attributes, Keys, Query processing, Structure of relational Databases, Example of SQL; Distributed Databases, Security and integrity, Network Model.* 2025 asked: RDBMS characteristics (Q5), primary & foreign key (Q6), query processing + optimization (Q18, 15 m). Every other phrase is still unasked.

3.1 Data, Database and DBMS vs File System

Importance 7/10 · good chance · foundation for every DBMS answer; "DBMS vs file system" is an evergreen 5-marker

In simple words: **data** are raw facts (e.g. "Kohima", 52); **information** is processed, meaningful data ("52 students from Kohima passed"). A **database** is an organised, related collection of data; a **DBMS** is the software that lets you define, store, query and protect it (MySQL, Oracle, PostgreSQL).

Disadvantages of file-processing systems that a DBMS removes:

File system problem	DBMS solution
Data redundancy and inconsistency	Centralised control, normalisation
Difficulty in accessing data (new program per query)	Declarative query language (SQL)
Data isolation (many formats/files)	Single integrated schema
Integrity problems (constraints buried in code)	Declarative constraints (CHECK, keys)
Atomicity problems (partial updates on crash)	Transactions + recovery
Concurrent-access anomalies	Concurrency control (locking)
Security problems	Authorisation, views, GRANT/REVOKE

Database Administrator (DBA): defines schema, storage structure and access methods, grants authorisation, routine maintenance (backup, space, tuning).

3.2 Data Abstraction and the Three-Schema Architecture

Importance 9/10 · very likely · "Data Abstraction" is an unasked syllabus phrase — very likely 5- or 10-mark

In simple words: users should not have to know how data is physically stored on disk. The DBMS hides complexity in three layers — each layer shows only what that audience needs.

Level	What it describes	Who uses it
Physical (internal)	*How* data is actually stored: files, blocks, indexes, B+-trees, hashing	DBMS developers, DBA for tuning
Logical (conceptual)	*What* data is stored and the relationships among them (tables, attributes, constraints) for the whole database	DBA, designers
View (external)	Part of the database relevant to a particular user group; hides other data (e.g. salary hidden from clerks)	End users, application programs — many views possible

ANSI/SPARC three-schema architecture: one **internal schema**, one **conceptual schema**, many **external schemas**, connected by **mappings** (external/conceptual and conceptual/internal). The mappings are what provide **data independence**.

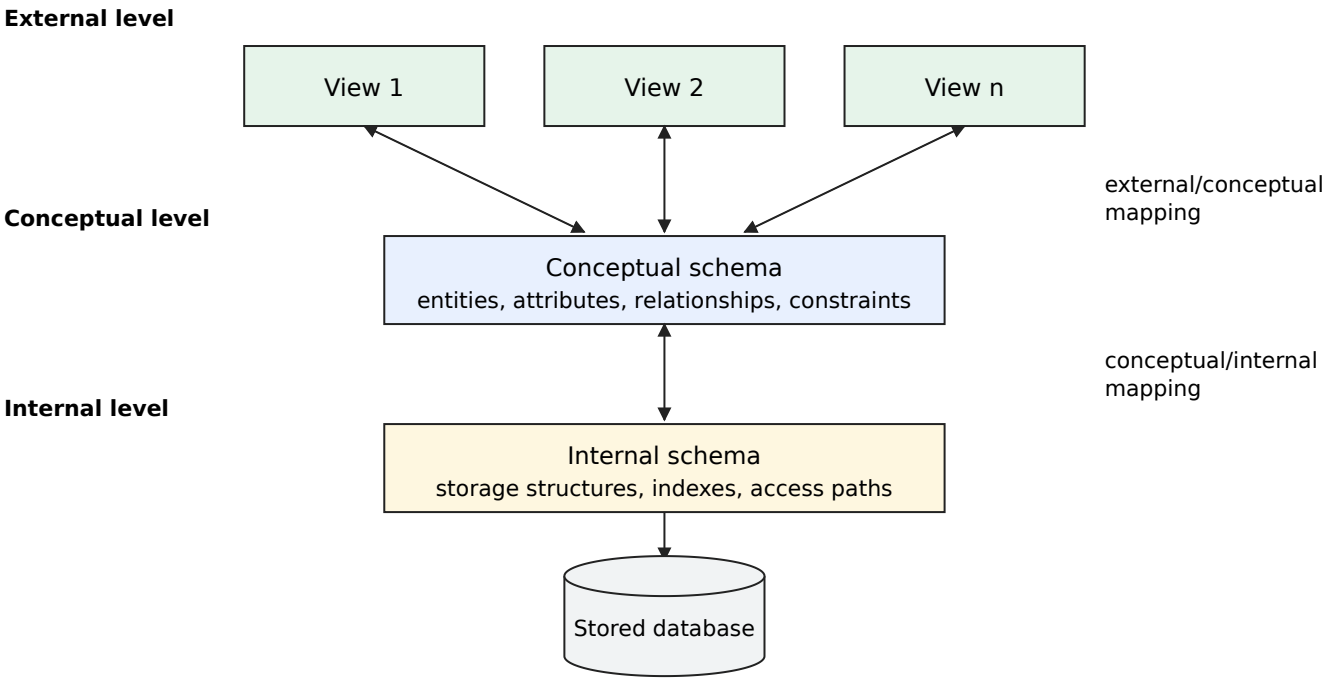


Fig: ANSI/SPARC three-schema architecture — many external views, one conceptual and one internal schema

Exam tip: Use a "student record" example at each level — physical: B+-tree on roll_no; logical: STUDENT(roll_no, name, dept, marks); view: a clerk sees (roll_no, name) only.

3.3 Data Independence

Importance 9/10 · very likely · unasked syllabus phrase that pairs with data abstraction — very likely

In simple words: the ability to change the schema at one level **without having to change** the schema (or programs) at the next higher level.

Logical data independence	Physical data independence
Change the **conceptual** schema without changing **external** schemas / application programs	Change the **internal** schema without changing the **conceptual** schema
e.g. add a new attribute `email` to STUDENT, split a table — old views still work	e.g. add an index, move to a new disk, switch from heap file to B+-tree
Provided by **external/conceptual mapping**	Provided by **conceptual/internal mapping**
Harder to achieve (programs depend on logical structure)	Easier, and more commonly achieved

Exam tip: One sentence each + one example each + "logical is harder" = full 5 marks.

3.4 Schema vs Instance

Importance 6/10 · moderate · common MCQ; small part of longer answers

In simple words: a **schema** is the design/blueprint of the database (like a class or variable type); an **instance** is the actual data stored at a particular moment (like the variable's current value).

Schema	Instance (state / snapshot)
Overall logical design	Collection of data at a moment in time
Changes rarely (by DDL)	Changes frequently (by DML)
e.g. `STUDENT(roll_no, name, dept)`	e.g. 3 rows currently in STUDENT

3.5 Data Models

Importance 8/10 · likely · "Data models" is an unasked syllabus phrase — strong 10-mark candidate

In simple words: a data model is a set of concepts used to describe the structure of a database — data, relationships, semantics and constraints.

Model	Structure	Relationship representation	Example / notes
Hierarchical	**Tree** of records; each child has exactly **one parent**	Parent–child links (1:N only)	IBM IMS; M:N needs duplication
Network	**Graph**; record types linked by **set types** (owner–member); a member may have many owners	Pointers / links	CODASYL DBTG, IDMS; handles M:N
Relational	**Tables** (relations) of rows and columns	Common values (foreign keys)	Codd 1970; Oracle, MySQL — most widely used
Entity–Relationship	Entities, attributes, relationships (diagrammatic)	Relationship sets with cardinality	Chen 1976; used for conceptual design
Object-based (OO / object-relational)	Objects with attributes + methods, classes, inheritance	Object references (OIDs)	db4o, PostgreSQL (object-relational)
Semi-structured	Self-describing (tags)	Nesting	XML, JSON

Categories: **conceptual (high-level)** — ER, object; **representational (implementation)** — relational, network, hierarchical; **physical (low-level)** — storage details.

Exam tip: For "compare data models" draw one small example in each: a tree (hierarchical), a graph (network), a table (relational).

3.6 Network Model

Importance 8/10 · likely · named separately in the syllabus and never asked yet — strong Sec A/B candidate

In simple words: data is stored as **records** connected by **links (pointers)**, forming a graph. It is like the hierarchical model, but a record can have more than one parent, so many-to-many relationships are possible.

- Standardised by **CODASYL DBTG** (Data Base Task Group, 1971). Implementations: **IDMS**, IDS, Raima.
- **Record type** — a collection of fields (like an entity type / table).
- **Set type** — a named **1:N relationship** between one **owner** record type and one **member** record type. Each set occurrence = one owner record + its member records, usually linked in a **circular pointer chain** (ring).
- A record type can be **member in several set types** (different owners) → graph structure; M:N is modelled via a **link (junction) record**.
- Data manipulated by **navigational** commands (FIND, GET, STORE, CONNECT, DISCONNECT) — programmer follows pointers record-at-a-time.
- Diagram used: **Bachman diagram** (boxes = record types, arrows owner → member = set types).

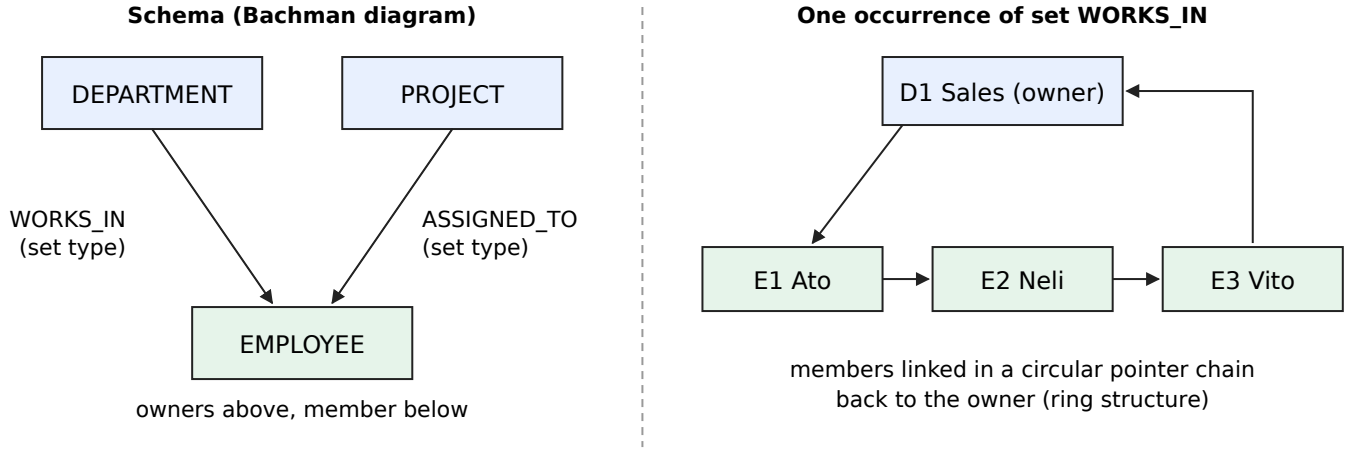


Fig: Network model — EMPLOYEE is a member of two set types (two owners), and a set occurrence as a pointer ring

Advantages	Disadvantages
Models M:N and complex relationships naturally	**Navigational** access — programmer must know the pointer structure
Fast access via direct pointers	Structural changes need program changes (**poor data independence**)
Data integrity: member cannot exist without owner (depending on set options)	Complex to design and maintain; replaced by relational model

Exam tip: Hierarchical = tree (one parent); network = graph (many parents via set types); relational = tables. Write "CODASYL, owner–member, set type, IDMS" — these keywords score.

3.7 Database Languages — DDL, DML, DCL, TCL

Importance 8/10 · likely · "DDL" is an unasked syllabus phrase; easy 5 marks with examples

In simple words: SQL commands are grouped by what they do — define structure (DDL), work with the data (DML), control permissions (DCL), and control transactions (TCL).

Language	Purpose	Commands
DDL (Data Definition)	Define/modify schema; output stored in the **data dictionary** (metadata)	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`
DML (Data Manipulation)	Retrieve and change data. **Procedural** (what + how, e.g. relational algebra) vs **declarative** (only what, e.g. SQL)	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
DCL (Data Control)	Grant/withdraw privileges	`GRANT`, `REVOKE`
TCL (Transaction Control)	Manage transactions	`COMMIT`, `ROLLBACK`, `SAVEPOINT`

```
-- DDL
CREATE TABLE dept (dept_id INT PRIMARY KEY, dname VARCHAR(30) NOT NULL);
ALTER TABLE dept ADD location VARCHAR(30);
DROP TABLE old_logs;
-- DML
INSERT INTO dept VALUES (10, 'Computer Science', 'Kohima');
UPDATE dept SET location = 'Dimapur' WHERE dept_id = 10;
-- DCL
GRANT SELECT, INSERT ON dept TO clerk;
REVOKE INSERT ON dept FROM clerk;
-- TCL
UPDATE account SET bal = bal - 500 WHERE acc_no = 1;
SAVEPOINT s1;
UPDATE account SET bal = bal + 500 WHERE acc_no = 2;
COMMIT;
```

DDL also specifies **integrity constraints** (domain, NOT NULL, PRIMARY KEY, FOREIGN KEY, CHECK) and authorisation.

Exam tip: `TRUNCATE` is DDL (removes all rows, cannot be rolled back in most DBMSs, no WHERE); `DELETE` is DML (row-wise, WHERE allowed, can be rolled back).

3.8 Attributes

Importance 8/10 · likely · "Attributes" is an unasked syllabus phrase — classify-with-examples 5-marker

In simple words: an attribute is a property describing an entity — e.g. a STUDENT has roll number, name, phone, date of birth. Each attribute takes values from a **domain** (value set).

Type	Meaning	Example	ER symbol
Simple (atomic)	Cannot be divided	`roll_no`, `gender`	Oval
Composite	Can be divided into sub-parts	`name` → first, middle, last; `address` → street, city, PIN	Oval with child ovals
Single-valued	One value per entity	`DOB`	Oval
Multi-valued	Several values per entity	`phone_numbers`, `email_ids`	**Double oval**
Stored	Physically stored	`DOB`	Oval
Derived	Computed from other attributes	`age` (from DOB), `total_marks`	**Dashed oval**
Key	Uniquely identifies an entity	`roll_no`	Oval with **underlined** name
Null-valued	Value unknown or not applicable	`apartment_no` for a house; unknown phone	—
Complex	Composite + multi-valued nesting	several addresses each with parts	—

3.9 Keys

Importance 8/10 · likely · primary/foreign key asked 2025 Q6; super/candidate/alternate likely in a different form

In simple words: keys are the attribute(s) used to tell rows apart and to link tables.

Example relation **STUDENT(roll_no, aadhaar_no, email, name, dept_id)** and **DEPT(dept_id, dname)**:

roll_no	aadhaar_no	email	name	dept_id
101	4321 8765 1111	ato@x.in	Ato	CSE
102	4321 8765 2222	neli@x.in	Neli	ECE
103	4321 8765 3333	vito@x.in	Vito	CSE

Key	Definition	In the example
Super key	Any set of attributes that uniquely identifies a tuple (may contain extra attributes)	{roll_no}, {roll_no, name}, {aadhaar_no, email}, …
Candidate key	A **minimal** super key (no proper subset is a super key)	{roll_no}, {aadhaar_no}, {email}
Primary key	The candidate key **chosen** by the designer; **unique and NOT NULL** (entity integrity)	roll_no
Alternate (secondary) key	Candidate keys not chosen as primary	aadhaar_no, email
Foreign key	Attribute(s) in one relation that refer to the primary key of another (or same) relation; value must match an existing key or be NULL (referential integrity)	STUDENT.dept_id → DEPT.dept_id
Composite key	A key made of **two or more** attributes	ENROLLS(roll_no, course_id) — the pair is the key

Relationship: Primary ⊂ Candidate ⊆ Super keys. Every relation has at least one candidate key (the whole tuple set is a super key since relations contain no duplicates).

Exam tip: Draw two tables with an arrow from foreign key to primary key, then define each key against that example.

3.10 ER Model and ER Diagram

Importance 8/10 · likely · ER diagrams appear in all three CS syllabi; a drawing question is likely

In simple words: the ER model describes the real world as **entities** (things), their **attributes** (properties) and **relationships** (associations) — drawn as a diagram before building tables.

- **Entity / entity set** — rectangle (STUDENT). **Weak entity** (no key of its own, depends on owner, e.g. DEPENDENT of EMPLOYEE) — double rectangle with **identifying relationship** as double diamond; partial key dashed-underlined.
- **Relationship set** — diamond. **Degree:** unary, binary, ternary.
- **Mapping cardinalities:** 1:1, 1:N, N:1, M:N. **Participation:** total (double line — every entity participates) vs partial (single line).
- **Extended ER:** specialisation/generalisation (ISA triangle), aggregation.

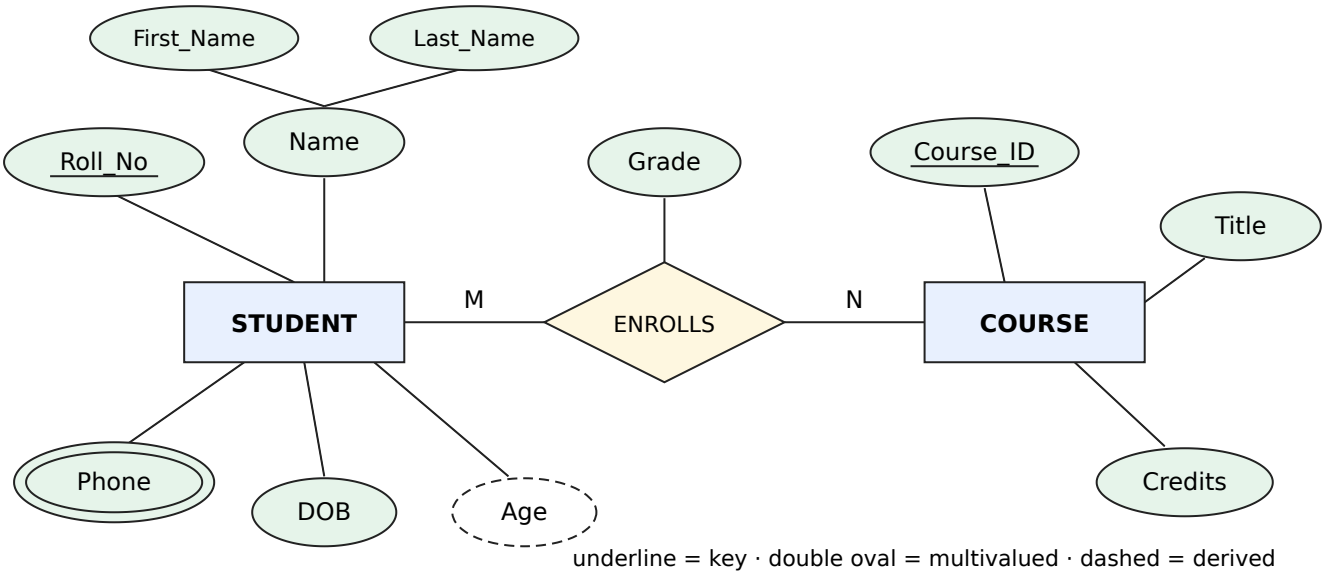


Fig: ER diagram (Chen notation) — STUDENT M:N COURSE via ENROLLS, with key, composite, multivalued and derived attributes

ER → relational mapping (brief): strong entity → table (key = PK); multivalued attribute → separate table (STUDENT_PHONE(roll_no, phone)); composite → its simple components; derived →

usually not stored; 1:N → FK on the N side; M:N → separate table with both PKs as composite key: **ENROLLS(roll_no, course_id, grade)**; weak entity → table with owner's PK + partial key.

3.11 Structure of Relational Databases and RDBMS Characteristics

Importance 7/10 · good chance · RDBMS characteristics asked 2025 Q5; "Structure of relational databases" phrase still unasked

In simple words: in the relational model everything is stored in **tables**. Each table (relation) has named columns (attributes) and rows (tuples).

Term (formal)	Informal	Example
Relation	Table	STUDENT
Tuple	Row / record	(101, 'Ato', 'CSE')
Attribute	Column / field	name
Domain	Set of allowed atomic values for an attribute	dept ∈ {CSE, ECE, ME, CE}
Degree (arity)	Number of attributes	STUDENT(roll_no, name, dept) → degree 3
Cardinality	Number of tuples	3 rows → cardinality 3
Relation schema	Name + attribute list	STUDENT(roll_no, name, dept)
Relation instance	Current set of tuples	the rows today

Properties of a relation: values are **atomic** (1NF); no two tuples identical; **order of tuples and of attributes is immaterial**; each attribute name distinct; values from the attribute's domain; NULL for unknown.

Characteristics of an RDBMS (2025 Q5 answer): data in tables; relationships via keys (PK/FK); SQL as a standard declarative language; integrity constraints (entity, referential, domain); normalisation to reduce redundancy; ACID transactions; concurrency control and recovery; data independence; security (users, roles, GRANT/REVOKE); views; data dictionary (catalogue) also stored as tables (Codd's rules).

3.12 Relational Algebra (basics)

Importance 6/10 · moderate · underpins query processing; MCQs on symbols and joins

In simple words: relational algebra is a **procedural** query language — a set of operations that take relations as input and produce a new relation.

Operation	Symbol	Meaning	Example
Select	σ	Rows satisfying a predicate	σ dept='CSE' (STUDENT)
Project	π	Chosen columns (duplicates removed)	π name (STUDENT)
Union	∪	Tuples in either (union-compatible)	R ∪ S
Set difference	−	In R but not in S	R − S
Cartesian product	×	Every tuple of R paired with every tuple of S	STUDENT × DEPT
Rename	ρ	Rename relation/attributes	ρ S(STUDENT)
Intersection	∩	In both (derived: R − (R − S))	R ∩ S
Natural join	⋈	× then equate common attributes, keep one copy	STUDENT ⋈ DEPT
Theta join	⋈θ	σθ(R × S)	
Division	÷	Tuples related to **all** tuples of the divisor	students who took all courses

Fundamental operations: σ, π, ∪, −, ×, ρ (the others are derived). Example: names of CSE students = π name (σ dept='CSE' (STUDENT)).

3.13 SQL Examples

Importance 9/10 · very likely · "Example of SQL" is in the syllabus verbatim and unasked — expect "write SQL for…" questions

In simple words: SQL (Structured Query Language) is the standard declarative language for relational databases — you say *what* you want, the DBMS works out *how*.

Common data types: `INT`, `SMALLINT`, `NUMERIC(p,d)/DECIMAL`, `FLOAT`, `CHAR(n)`, `VARCHAR(n)`, `DATE`, `TIME`, `BOOLEAN`.

```
-- 1. Create tables with constraints
CREATE TABLE dept (
  dept_id  CHAR(4) PRIMARY KEY,
  dname    VARCHAR(30) NOT NULL UNIQUE
);
CREATE TABLE employee (
  emp_id   INT PRIMARY KEY,
  name     VARCHAR(40) NOT NULL,
  salary   NUMERIC(10,2) CHECK (salary > 0),
  dept_id  CHAR(4) REFERENCES dept(dept_id) ON DELETE SET NULL
);

-- 2. Insert
INSERT INTO dept VALUES ('CSE', 'Computer Science'), ('ECE', 'Electronics');
INSERT INTO employee (emp_id, name, salary, dept_id) VALUES (1, 'Ato', 55000, 'CSE');

-- 3. Select with WHERE and ORDER BY
SELECT name, salary FROM employee
WHERE salary BETWEEN 40000 AND 80000 AND name LIKE 'A%'
ORDER BY salary DESC;

-- 4. Aggregate with GROUP BY and HAVING
SELECT dept_id, COUNT(*) AS n, AVG(salary) AS avg_sal
FROM employee
GROUP BY dept_id
HAVING COUNT(*) > 2;

-- 5. Join
SELECT e.name, d.dname
FROM employee e JOIN dept d ON e.dept_id = d.dept_id;      -- inner join
SELECT d.dname, e.name
FROM dept d LEFT OUTER JOIN employee e ON e.dept_id = d.dept_id;   -- all depts, even empty

-- 6. Subquery: employees earning more than the company average
SELECT name FROM employee WHERE salary > (SELECT AVG(salary) FROM employee);

-- 7. Update and delete
UPDATE employee SET salary = salary * 1.10 WHERE dept_id = 'CSE';
DELETE FROM employee WHERE emp_id = 1;

-- 8. View
CREATE VIEW cse_staff AS
  SELECT emp_id, name FROM employee WHERE dept_id = 'CSE';
SELECT * FROM cse_staff;
```

Logical order of evaluation of a SELECT: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Hence **WHERE filters rows before grouping; HAVING filters groups after**

aggregation (aggregates allowed in HAVING, not in WHERE).

Other SQL features in the Paper-II(B)/TP syllabi: **stored procedures/functions** (named SQL blocks stored in the DB), **triggers** (code that runs automatically on INSERT/UPDATE/DELETE), **embedded SQL** (SQL inside a host language such as C, with `EXEC SQL` and cursors).

Exam tip: Write SQL keywords in capitals, one clause per line, and add a one-line comment saying what each query returns.

3.14 Query Processing

Importance 8/10 · likely · asked 2025 Q18 (15 m) — may return reworded (e.g. "steps of query evaluation" or "role of optimizer")

In simple words: when you submit an SQL query, the DBMS first checks and translates it, then picks the cheapest way to run it, then runs it.

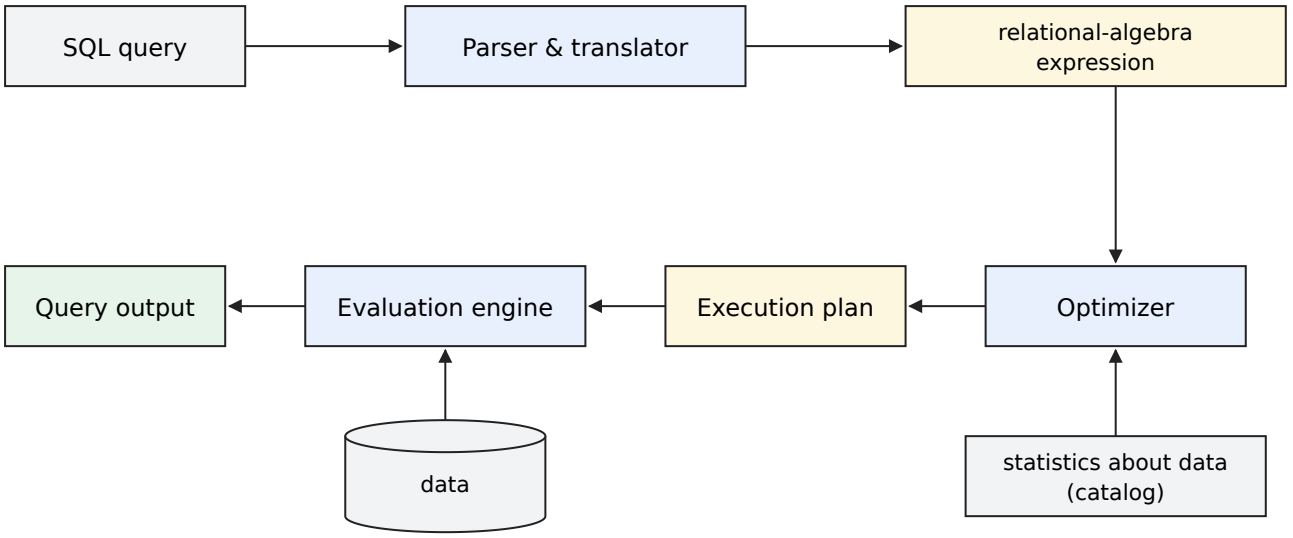


Fig: Steps in query processing (Silberschatz) — parsing & translation, optimization, evaluation

1. **Parsing and translation** — scanner/parser checks **syntax**; verifies relation and attribute names against the **data dictionary** (semantic check) and privileges; translates the query into an internal form — a **relational-algebra expression / query tree**. Views are expanded here.
2. **Optimization** — among many equivalent expressions and many algorithms per operation, choose the **evaluation plan** with the lowest estimated cost (cost measured mainly in **disk block transfers and seeks**; also CPU).
3. **Evaluation** — the **query-execution engine** takes the plan, executes it (using **materialization** — write intermediate results to disk — or **pipelining** — pass tuples directly to the next operation), and returns the answer.

Evaluation plan = relational-algebra expression annotated with the algorithm for each operation (e.g. "use index on salary for σ salary<75000; hash join for ⋈").

Algorithms: selection — linear scan, binary search, index scan; join — nested-loop, block nested-loop, indexed nested-loop, sort-merge join, hash join; sorting — external merge sort.

3.15 Query Optimization

Importance 8/10 · likely · second half of 2025 Q18; heuristic vs cost-based comparison is a natural 5/10-marker

In simple words: the same question can be answered many ways; some take seconds, others hours. The optimizer picks a cheap way automatically so the user need not care.

Why important: cost of different plans can differ by orders of magnitude; users write declarative SQL and cannot be expected to choose plans; reduces I/O, response time and resource use; adapts as data/statistics change.

Heuristic (rule-based) optimization	Cost-based optimization
Applies **transformation rules** that usually improve the query	Generates alternative plans and **estimates the cost** of each using catalog statistics
Rules: perform **selection early** (push σ down), **projection early**, replace × + σ by ⋈, do most restrictive joins first, break conjunctive selections	Statistics: number of tuples n_r, blocks b_r, distinct values V(A, r), histograms, index height
Fast, cheap to run; not always optimal	Better plans; optimization itself costs time (search space of join orders — dynamic programming)
Example: σ dept='CSE'(STUDENT ⋈ DEPT) → (σ dept='CSE'(STUDENT)) ⋈ DEPT	Example: choose hash join vs index nested-loop join by estimated block transfers

Equivalence rules used: σ is commutative and cascades; π cascades; joins are commutative and associative; σ distributes over ⋈ when the predicate involves one relation only.

Exam tip: For a 15-mark question, give the 3-step diagram, explain each step, then a before/after query-tree example of "push selection down".

3.16 Normalisation (1NF to BCNF, brief)

Importance 6/10 · moderate · not in the Paper-II(A) phrase list, but in two other syllabi and a regular MCQ topic

In simple words: normalisation splits badly designed tables into smaller ones to remove redundancy and **update, insertion and deletion anomalies**, using **functional dependencies** (X → Y: X determines Y).

Normal form	Condition	Removes
1NF	All attribute values atomic; no repeating groups	Multi-valued / composite cells
2NF	1NF + no **partial dependency** (non-prime attribute depends on part of a composite candidate key)	Partial dependency
3NF	2NF + no **transitive dependency** (non-prime → non-prime). Formally: for every X → A, X is a super key or A is prime	Transitive dependency
BCNF	For every non-trivial X → A, **X is a super key**	All FD-based anomalies (stricter than 3NF)

Example: ENROLL(roll_no, course_id, sname, cname, grade) with key (roll_no, course_id); roll_no → sname is partial → split into STUDENT(roll_no, sname), COURSE(course_id, cname), ENROLL(roll_no, course_id, grade).

Decompositions must be **lossless-join**; 3NF always preserves dependencies, BCNF may not.

3.17 Transactions and ACID

Importance 7/10 · good chance · transaction processing in TP syllabus; ACID is a standard 5-marker

In simple words: a transaction is a group of operations that must be treated as one unit — like transferring ₹500 between two accounts: debit and credit both happen, or neither does.

Property	Meaning	Ensured by
Atomicity	All operations or none	Recovery manager (undo log, shadow paging)
Consistency	Takes DB from one consistent state to another	Programmer + integrity constraints
Isolation	Concurrent transactions do not see each other's intermediate results; result = some serial order	Concurrency-control manager
Durability	Once committed, changes survive failures	Recovery manager (redo log, stable storage)

Transaction states: active → partially committed → committed; active/partially committed → failed → aborted (rollback, then restart or kill).

3.18 Concurrency Control (basics)

Importance 6/10 · moderate · TP-syllabus phrase; MCQs on 2PL and anomalies

In simple words: running many transactions at the same time improves throughput, but uncontrolled interleaving gives wrong answers. Concurrency control allows only safe interleavings.

- **Problems:** lost update (two writes, one overwritten), dirty read (reading uncommitted data — temporary update problem), unrepeatable read, incorrect summary/phantom.
- **Schedule** = order of operations from several transactions. **Serial** schedule is always correct; a **serializable** schedule is equivalent to some serial one (**conflict serializability** — test with a **precedence graph**: acyclic ⇒ conflict serializable).
- **Lock-based protocols:** shared (S) lock for read, exclusive (X) lock for write. **Two-Phase Locking (2PL)**: growing phase (acquire only) then shrinking phase (release only) ⇒ guarantees conflict serializability; can deadlock. **Strict 2PL** holds X locks till commit (avoids cascading rollback).
- **Timestamp ordering:** each transaction gets a timestamp; conflicting operations executed in timestamp order, else rollback.
- **Optimistic (validation)** protocols, **multiversion** concurrency control (MVCC).
- **Recovery:** log-based (write-ahead logging, undo/redo), checkpoints, shadow paging.

3.19 Distributed Databases

Importance 9/10 · very likely · "Distributed Databases" is an unasked syllabus phrase — very likely 10- or 15-mark

In simple words: a distributed database is one logical database whose data is physically spread over several computers (sites) connected by a network, but users see it as a single database.

Distributed DBMS (DDBMS): software that manages the distributed database and makes the distribution **transparent** to users. Each site can process local transactions and also take part in global transactions.

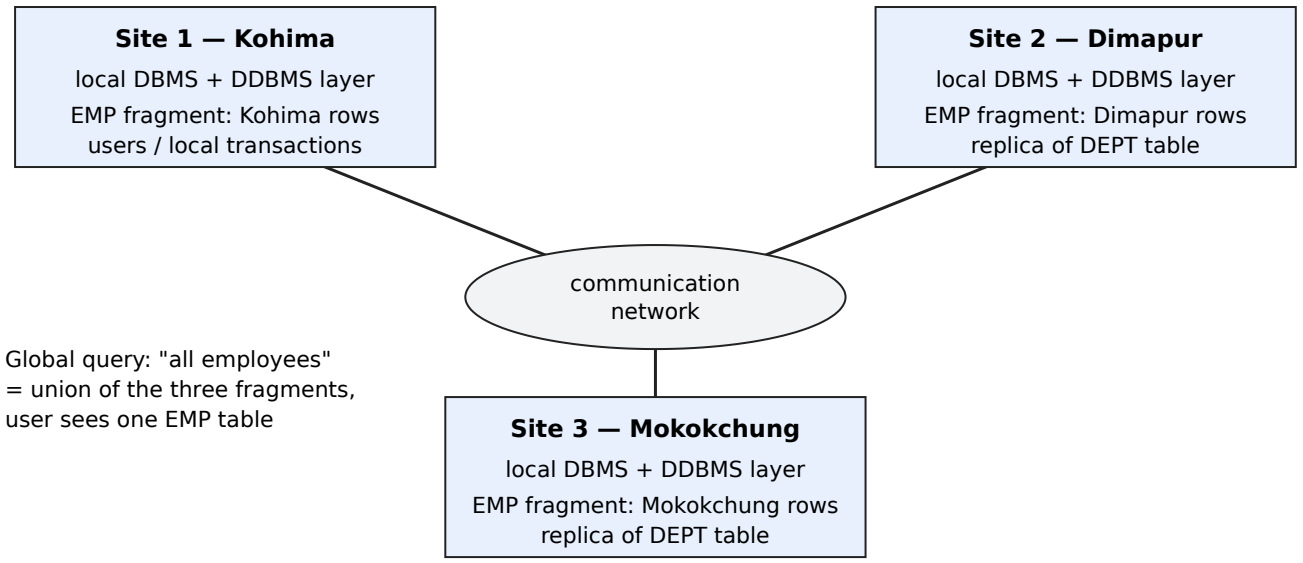


Fig: Distributed database — horizontally fragmented EMP table across three sites, DEPT replicated, linked by a network

Types:

Homogeneous DDB	Heterogeneous DDB
All sites use the **same DBMS software** and schema; aware of each other	Sites may use **different DBMSs/schemas/data models** (Oracle + MySQL + legacy)
Easier to design and manage	Needs translation, **gateways / middleware**; limited cooperation

Storing data:

- **Fragmentation** — split a relation into fragments stored at different sites.
- **Horizontal**: subsets of **rows** (σ), e.g. EMP rows by branch city. Reconstruct with **UNION**.
- **Vertical**: subsets of **columns** (π), each fragment must include the primary key. Reconstruct with **natural JOIN**.
- **Mixed (hybrid)**: both.
- **Replication** — keep copies of a relation/fragment at several sites. **Full replication** (copy everywhere), **partial**, **none**. Increases availability and read speed; costs on update (all copies must be kept consistent).

Transparency (the user should not need to know):

Transparency	Hides
Location	Where data is stored
Fragmentation	That a relation is split
Replication	That copies exist
Naming	Data items have unique global names usable from any site
Transaction / failure	Distribution of a transaction's work; site failures

Advantages: matches decentralised organisations (e.g. district offices); improved **availability and reliability** (one site down ≠ whole system down); faster local access; modular growth (add sites); sharing of data across sites.
Disadvantages: complex design and software; higher development cost; harder **concurrency control and recovery**; security across many sites; communication overhead; integrity enforcement is costly.

Distributed transactions — Two-Phase Commit (2PC): a **coordinator** ensures all participating sites commit or all abort.

1. **Phase 1 (prepare / voting):** coordinator writes `<prepare T>` to log and sends PREPARE to all sites; each site that can commit force-writes `<ready T>` and replies READY (vote yes), else ABORT (vote no).
2. **Phase 2 (commit / decision):** if **all** voted ready → coordinator logs `<commit T>`, sends COMMIT; if **any** voted no or timed out → logs `<abort T>`, sends ABORT. Sites act and send acknowledgement.

Weakness: **blocking** — if the coordinator fails after sites voted ready, they must wait (3PC reduces this).

Exam tip: A 15-mark answer = definition + diagram + homogeneous/heterogeneous + fragmentation (with SQL-like example) + replication + transparency + pros/cons + 2PC.

3.20 Database Security and Integrity

Importance 9/10 · very likely · "Security and integrity" is an unasked syllabus phrase (also in two other syllabi) — very likely

In simple words: **integrity** = the data is *correct and consistent* (rules prevent bad data getting in). **Security** = the data is *protected from unauthorised access* (only the right people can see or change it).

Integrity constraints:

Constraint	Rule	SQL
Domain	Each value must come from the attribute's domain (type, range)	data type, `CHECK (marks BETWEEN 0 AND 100)`, `NOT NULL`, `DEFAULT`
Entity integrity	Primary-key attributes cannot be NULL (and must be unique)	`PRIMARY KEY`
Referential integrity	A foreign-key value must match an existing primary-key value in the referenced table, or be NULL	`FOREIGN KEY … REFERENCES … ON DELETE CASCADE / SET NULL`
Key (uniqueness)	No two tuples share a candidate-key value	`UNIQUE`
General / assertions, triggers	Arbitrary business rules	`CREATE ASSERTION`, triggers

Security mechanisms:

- **Authentication** — verify identity: user name + password, multi-factor, certificates.
- **Authorization / access control** — what an authenticated user may do. **Discretionary access control (DAC)** — privileges granted by owners via SQL; **Mandatory access control (MAC)** — security classes (Top Secret > Secret > Confidential > Unclassified, Bell–LaPadula: no read up, no write down); **Role-based (RBAC)** — privileges granted to roles, roles to users.
- **Views** — show only permitted rows/columns (hide salary column).
- **Encryption** — data at rest and in transit (TLS); stored passwords hashed.
- **Auditing** — audit trail/log of who did what.
- **Statistical database security** — prevent inference of individual data from aggregate queries.

```
GRANT SELECT, UPDATE (salary) ON employee TO hr_clerk;
GRANT SELECT ON cse_staff TO public;              -- through a view only
GRANT INSERT ON employee TO manager WITH GRANT OPTION;
REVOKE UPDATE ON employee FROM hr_clerk CASCADE;
CREATE ROLE auditor;  GRANT SELECT ON employee TO auditor;  GRANT auditor TO neli;
```

SQL injection: an attack where user input is pasted directly into an SQL string so the attacker changes the query.

```
-- Vulnerable code builds: SELECT * FROM users WHERE name = '<input>' AND pwd = '<input2>';
-- Attacker types as name:   ' OR '1'='1' --
-- Resulting query:          SELECT * FROM users WHERE name = '' OR '1'='1' -- ' AND pwd =
'';
-- '1'='1' is always true and "--" comments out the password check -> login bypassed
```

Prevention: **parameterised queries / prepared statements** (input bound as data, never as SQL), stored procedures, input validation/escaping, least-privilege DB accounts, hide detailed error messages.

Exam tip: Structure a 10-mark answer as two halves — "Integrity" (3 constraint types with SQL) and "Security" (authentication, authorization with GRANT/REVOKE, views, encryption, SQL injection).

4. Data Communication & Computer Networks

Syllabus phrase (Paper-II §4): *Basic Network Concepts, Network Topologies, Networking devices, Transmission media, OSI Model, TCP/IP fundamentals, Knowledge of Internet, Internet routing, IP address.* 2025 asked OSI layers (5), TCP vs UDP (5) and topologies (10). IP addressing, devices, media and routing are the strongest unasked phrases.

4.1 Basic Network Concepts

Importance 8/10 · likely · first syllabus phrase; LAN/MAN/WAN and transmission modes are classic 5-markers and MCQs

A **computer network** is a collection of autonomous computers and devices interconnected by a communication medium so that they can exchange data and share resources (files, printers, Internet connection). **Goals:** resource sharing, reliability (alternate sources), cost reduction, communication (e-mail, video), scalability.

Components of a network: hosts/nodes (end devices), NIC, transmission media (cables/wireless), networking devices (switch, router…), protocols, and network software (NOS).

Data communication model (Forouzan) — five components: message, sender, receiver, transmission medium, protocol. Effectiveness depends on **delivery, accuracy, timeliness, jitter**.

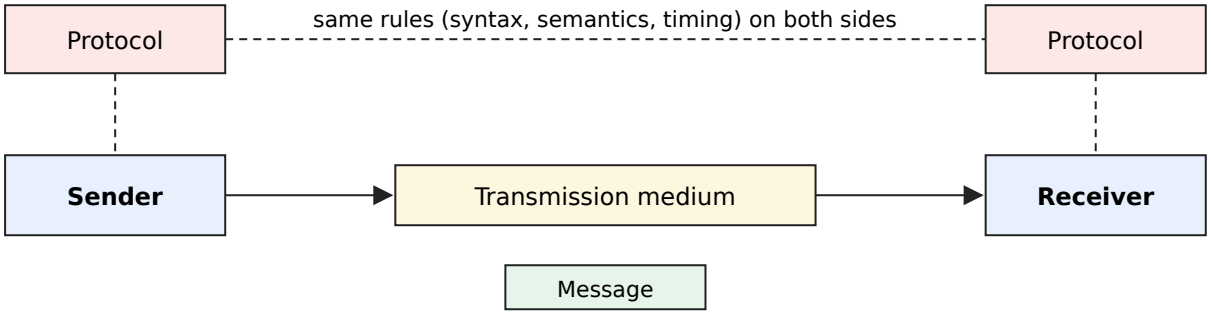


Fig: Five components of data communication — sender, receiver, medium, message, protocol

Types of networks by area

Type	Coverage	Owner	Speed / error	Example	Typical technology
PAN	~10 m, one person	Individual	—	Phone–earbuds	Bluetooth, USB
LAN	Room/building/campus (up to ~1–2 km)	Private	High (100 Mbps–10 Gbps), low error	Office, college lab	Ethernet (IEEE 802.3), Wi-Fi (802.11)
MAN	A city (up to ~50–100 km)	Private or public	Medium	Cable TV network, city-wide bank links	Metro Ethernet, DQDB (802.6), WiMAX
WAN	Country/continent/world	Usually public/telecom	Lower, higher delay and error	Internet, bank network across India	Leased lines, MPLS, satellite

Transmission modes (direction of data flow)

Mode	Direction	Example
Simplex	One way only	Keyboard → CPU, TV broadcast
Half-duplex	Both ways, but **one at a time**	Walkie-talkie
Full-duplex	Both ways **simultaneously**	Telephone, modern Ethernet

Network architectures

Client–server	Peer-to-peer (P2P)
Dedicated server provides services; clients request	Every node is both client and server
Centralised control, security, backup	Decentralised; each user manages own resources
Server is a single point of failure; costly	No dedicated server; cheap, simple
Scales well with powerful servers	Suitable for small networks (≤10 PCs) or file sharing (BitTorrent)
e.g. web, e-mail, banking	e.g. home workgroup, BitTorrent

Also: **wireless networks** (Wi-Fi, cellular), **internetwork (internet)** = network of networks joined by routers; **intranet** = private network using Internet protocols; **extranet** = intranet partly opened to partners.

4.2 Network Topologies

Importance 7/10 · good chance · asked as 10-marker in 2025; still a common MCQ (mesh link count)

Topology is the physical or logical arrangement of nodes and links in a network.

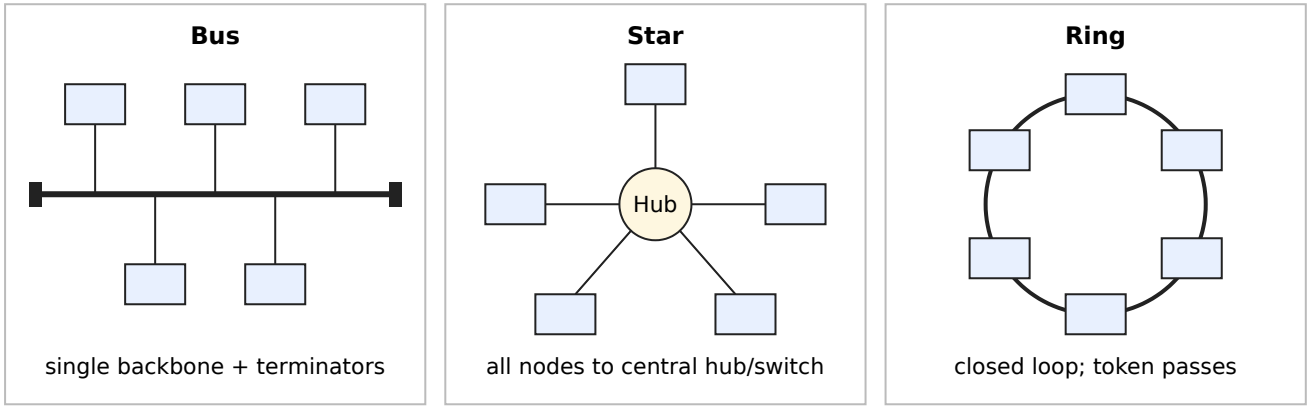


Fig: Bus, star and ring topologies (boxes = computers)

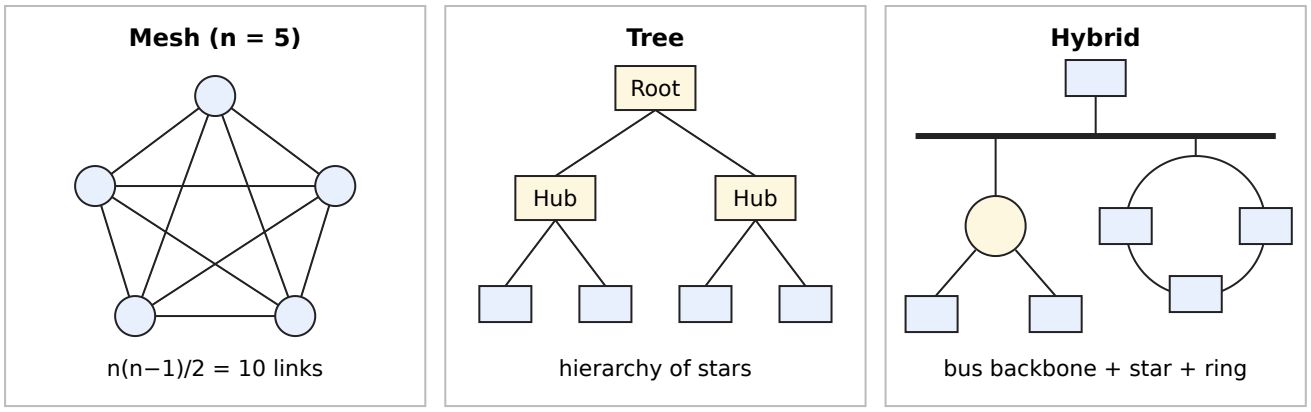


Fig: Mesh (fully connected), tree and hybrid topologies

Topology	Key feature	Advantages	Disadvantages
Bus	All nodes on one backbone cable with terminators	Cheap, little cable, easy to add nodes	Backbone failure kills network; collisions; hard to troubleshoot; limited length
Star	Every node to a central hub/switch	Easy install and fault isolation; one link failure affects one node	Hub is single point of failure; more cable than bus
Ring	Each node connected to two neighbours in a loop; token passing	No collisions; equal access; predictable	One break stops ring (unless dual ring); adding nodes disturbs network
Mesh	Every node linked to every other	Robust, fault-tolerant, private, no traffic problems	Very costly, much cabling and I/O ports
Tree	Hierarchy of star networks from a root	Scalable, easy expansion, segment management	Root/backbone failure affects branches; complex cabling
Hybrid	Combination of two or more topologies	Flexible, reliable, fits large organisations	Complex design and costly

Mesh formulas (fully connected, n nodes): links = **n(n−1)/2** (duplex); I/O ports per device = **n − 1**. For n = 5: 10 links; n = 10: 45 links (verified).

Exam tip: For the 10-mark version, draw all six, then the table. Mention star is the most common LAN topology today (Ethernet with switches).

4.3 Networking Devices

Importance 9/10 · very likely · unasked syllabus phrase; "hub vs switch vs router" is a classic 10-marker and MCQ

Device	OSI layer	Function	Key point
NIC (network interface card)	1 & 2	Connects a host to the network; holds the 48-bit **MAC address**	Every networked device has one
Repeater	1 Physical	Regenerates (not just amplifies) a weakened signal to extend cable length	No filtering; 2 ports
Hub	1 Physical	Multiport repeater; sends incoming bits out of **all** ports	One collision domain, one broadcast domain; half-duplex
Bridge	2 Data link	Connects two LAN segments; filters frames by MAC address table	Separates collision domains
Switch	2 Data link (L3 switches also route)	Multiport bridge; forwards frame only to destination port using MAC table	Each port = own collision domain; full-duplex; one broadcast domain (without VLANs)
Router	3 Network	Connects different networks; forwards packets using **IP address** and routing table; picks best path	Each interface = separate broadcast domain
Gateway	All layers / 4–7	Protocol converter between dissimilar networks/architectures	e.g. e-mail gateway, IoT gateway; "default gateway" = router
Modem	1 Physical	MOdulator–DEModulator: converts digital ↔ analog for phone/cable lines	DSL, cable, dial-up
Access point (AP)	2 Data link	Connects wireless devices to a wired LAN (Wi-Fi 802.11)	Acts as a wireless bridge/hub
Brouter	2 & 3	Bridge + router	—
Firewall	3–7	Filters traffic by rules	Security device

Hub vs Switch vs Router (quick)

	Hub	Switch	Router
Layer	1	2	3
Address used	None	MAC	IP
Sends data to	All ports	Destination port only	Best next hop network
Collision domains	1	One per port	One per port
Broadcast domains	1	1	One per interface
Intelligence	None	Medium	High

Exam tip: Always write the layer in brackets after each device name — examiners mark that first.

4.4 Transmission Media

Importance 9/10 · very likely · unasked phrase; guided vs unguided with comparison table is an easy 10-marker

Transmission media are the physical paths that carry signals between sender and receiver (layer below the physical layer). Two classes: **guided (wired)** — signal confined to a physical conductor; **unguided (wireless)** — signal propagates through air/space.

Guided media

- **Twisted pair:** two insulated copper wires twisted together to cancel electromagnetic interference (crosstalk). **UTP** (unshielded) — cheap, used in Ethernet LANs (Cat5e/6, RJ-45 connector, 100 m limit). **STP** (shielded) — metal foil/braid shield, better noise immunity, costlier, bulkier.
- **Coaxial cable:** central copper core, insulator, braided metal shield, outer jacket. Higher bandwidth and better shielding than twisted pair. Used in cable TV, older Ethernet (10Base2/10Base5). BNC connector.

- **Optical fibre:** glass/plastic core surrounded by cladding of lower refractive index; light travels by **total internal reflection**. Very high bandwidth, low attenuation, immune to EMI, secure, light; costly and fragile, hard to splice. Types: **single-mode** (narrow core, long distance, laser) and **multi-mode** (step-index / graded-index, LED, shorter distance).

Unguided media

- **Radio waves** (3 kHz–1 GHz): omnidirectional, penetrate walls; AM/FM radio, cordless phones, Wi-Fi bands.
- **Microwaves** (1–300 GHz): unidirectional, **line-of-sight**, towers with dish antennas; terrestrial links, cellular backhaul.
- **Infrared** (300 GHz–400 THz): short range, cannot pass walls; TV remotes, short-range device links.
- **Satellite:** microwave relay via a satellite (geostationary at ~36,000 km); wide coverage, high propagation delay (~0.25 s one-way up and down); VSAT, DTH TV.
- **Lightwave / laser:** unguided optical link between buildings (line-of-sight; affected by fog).

Feature	Twisted pair	Coaxial	Optical fibre	Wireless (radio/microwave)
Signal	Electrical	Electrical	Light	Electromagnetic waves
Bandwidth	Low–medium	Medium–high	Very high	Varies, shared
Attenuation	High	Medium	Very low	Weather/distance dependent
EMI immunity	Low (STP better)	Good	Immune	Susceptible
Cost	Cheapest	Moderate	Highest	No cable; equipment cost
Security	Low (easy tap)	Medium	High (hard to tap)	Low (needs encryption)
Typical use	LAN, telephone	Cable TV	Backbone, undersea links	Mobility, remote areas

4.5 OSI Reference Model

Importance 7/10 · good chance · asked as 5-marker in 2025 (list layers); functions of each layer can still come as 10-marker

The **Open Systems Interconnection** model (ISO, 1984) is a 7-layer conceptual framework that standardises how heterogeneous systems communicate. Each layer serves the layer above and uses the layer below. Mnemonic top-down: **"All People Seem To Need Data Processing"**.

Layer	Main function	Protocols / devices	PDU
7 Application	Network services to user applications	HTTP, FTP, SMTP, DNS	Data
6 Presentation	Translation, encryption, compression	SSL/TLS, JPEG, ASCII	Data
5 Session	Dialog control, synchronisation	NetBIOS, RPC	Data
4 Transport	Process-to-process delivery, ports	TCP, UDP	Segment
3 Network	Logical (IP) addressing, routing	IP, ICMP, OSPF · router	Packet
2 Data link	Framing, MAC addressing, error control	Ethernet, PPP · switch	Frame
1 Physical	Bits over medium: signals, connectors	RS-232, cables · hub	Bit

Sender encapsulates downwards 7→1 (adds headers); receiver decapsulates upwards 1→7

Fig: The seven OSI layers (top-down 7→1) with function, protocols and protocol data unit

Layer details worth a line each in an answer:

- **Physical:** bit synchronisation, data rate, line configuration, topology, transmission mode.
- **Data link:** framing, physical (MAC) addressing, flow control, error control (CRC), medium access control; sub-layers **LLC** and **MAC**.
- **Network:** logical addressing, routing, fragmentation; source-to-destination delivery across networks.
- **Transport:** port addressing, segmentation/reassembly, connection control, end-to-end flow and error control.
- **Session:** establishes/maintains/terminates sessions, dialog control (half/full duplex), synchronisation points.
- **Presentation:** syntax/semantics of data — translation (character codes), encryption, compression.
- **Application:** user interface to network services — e-mail, file transfer, directory services, web.

4.6 TCP/IP Model and Comparison with OSI

Importance 9/10 · very likely · syllabus says "TCP/IP fundamentals"; OSI vs TCP/IP comparison unasked

TCP/IP (Internet protocol suite, DoD model) is the practical model on which the Internet runs. Original version has **4 layers**: Application, Transport, Internet, Network Access (Link). Forouzan/Tanenbaum teaching versions use **5 layers** by splitting Network Access into Data Link and Physical.

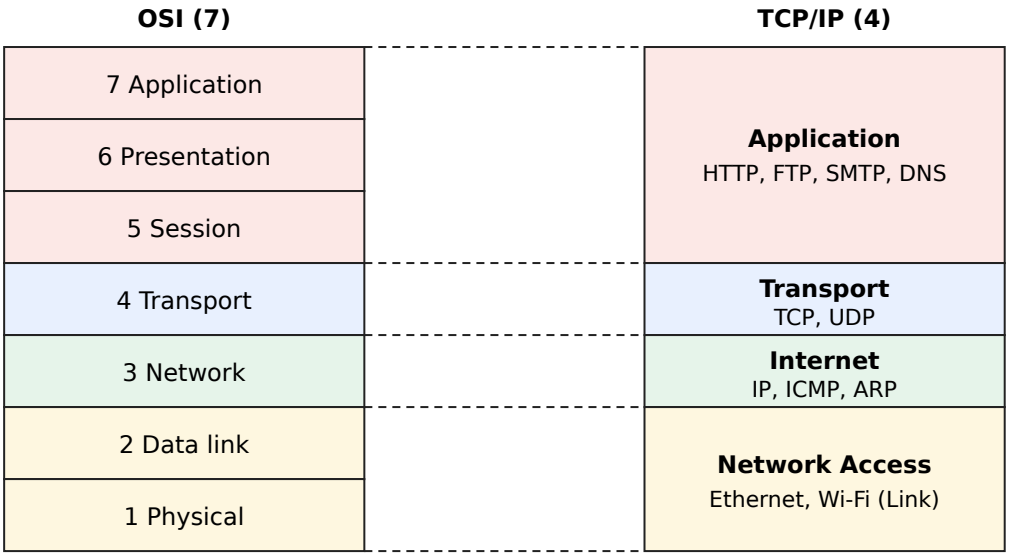


Fig: Mapping of OSI layers onto the TCP/IP model

Basis	OSI	TCP/IP
Developed by	ISO (1984)	US DoD / ARPANET (1970s)
Layers	7	4 (or 5 in textbooks)
Nature	Reference/theoretical model; model came before protocols	Practical; protocols came first, model described them
Session & presentation	Separate layers	Merged into Application
Transport	Connection-oriented only (in OSI design)	Both TCP (connection-oriented) and UDP (connectionless)
Network layer	Connection-oriented and connectionless	Connectionless only (IP)
Distinction service/interface/protocol	Clear	Not clear
Usage	Teaching, troubleshooting reference	Basis of the Internet

TCP vs UDP

Importance 7/10 · good chance · asked in 2025 (5 marks); still frequent in MCQs

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Reliable — ACKs, retransmission	Unreliable — no ACK, best effort
Ordering	In-order delivery (sequence numbers)	No ordering
Flow / congestion control	Yes (sliding window)	No
Header size	20–60 bytes	8 bytes fixed
Speed / overhead	Slower, more overhead	Faster, light
Data unit	Byte stream → segments	Message → user datagram
Broadcast/multicast	No	Yes
Used by	HTTP/HTTPS, FTP, SMTP, SSH, Telnet	DNS queries, DHCP, SNMP, TFTP, VoIP, video streaming, online games

TCP 3-way handshake (connection establishment):

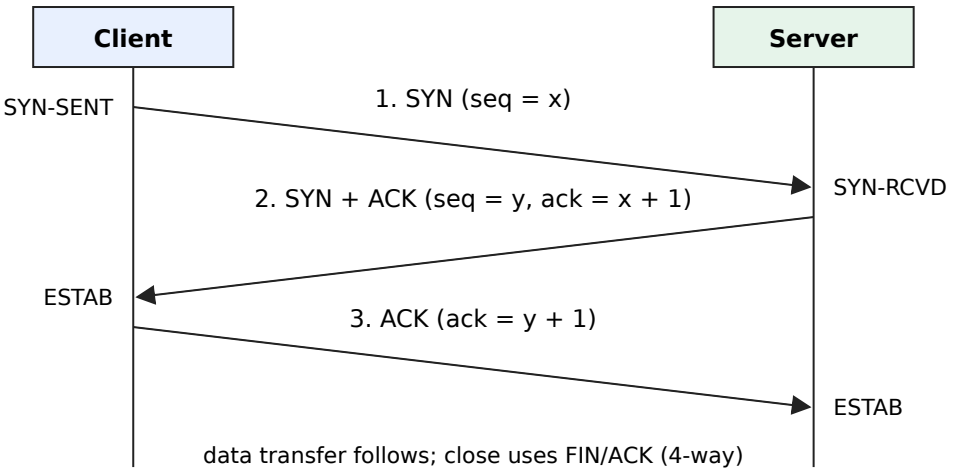


Fig: TCP three-way handshake — SYN, SYN-ACK, ACK

Key application-layer protocols and ports

Protocol	Port	Transport	Purpose
HTTP	80	TCP	Web pages (stateless)
HTTPS	443	TCP	HTTP over TLS (encrypted)
FTP	20 (data), 21 (control)	TCP	File transfer
SSH	22	TCP	Secure remote login/shell
Telnet	23	TCP	Remote login (plain text, insecure)
SMTP	25 (587 submission)	TCP	**Sending** e-mail (push)
DNS	53	UDP (TCP for zone transfer/large)	Name → IP resolution
DHCP	67 server / 68 client	UDP	Automatic IP configuration
TFTP	69	UDP	Trivial file transfer
POP3	110	TCP	**Retrieve** mail, downloads and usually deletes from server
IMAP	143	TCP	**Retrieve** mail, keeps mail on server, folders, multi-device sync
SNMP	161	UDP	Network management

- **ARP** (Address Resolution Protocol): finds the **MAC address for a known IP** on the local network (broadcast request, unicast reply). RARP does the reverse.
- **ICMP** (Internet Control Message Protocol): error reporting and diagnostics at network layer — used by **ping** (echo request/reply) and **traceroute** (TTL exceeded). No port numbers.
- **DHCP** process: **DORA** — Discover, Offer, Request, Acknowledge.

4.7 Knowledge of the Internet

Importance 7/10 · good chance · syllabus phrase "Knowledge of Internet" unasked; DNS resolution is a strong 10-marker

The **Internet** is a global network of networks using the TCP/IP protocol suite.

- **History: ARPANET** (1969, US DoD ARPA) linked 4 nodes (UCLA, SRI, UCSB, Utah); TCP/IP adopted on 1 Jan 1983; NSFNET (1986) backbone; commercialisation in the early 1990s. Tim Berners-Lee invented the **World Wide Web** at CERN (1989–91): HTML, HTTP, URL. Mosaic (1993) was the first popular graphical browser.

- **Internet vs WWW:** the Internet is the **infrastructure** (networks, routers, TCP/IP); the WWW is a **service** running on it (hyperlinked documents via HTTP). E-mail, FTP and VoIP are other Internet services.
- **ISP (Internet Service Provider):** company that provides Internet access (e.g. BSNL, Jio, Airtel). Tier-1 (global backbone) → Tier-2 (regional) → Tier-3 (local access). Access types: dial-up, DSL, leased line, cable, fibre (FTTH), ISDN, VSAT, mobile (4G/5G).
- **URL (Uniform Resource Locator):** `scheme://host:port/path?query#fragment` — e.g. `https://www.npsc.co.in:443/notice/list.php?year=2026#top`.
- **DNS (Domain Name System):** distributed, hierarchical database mapping domain names to IP addresses. Hierarchy: root (.) → top-level domains (generic .com .org .edu; country .in) → second-level (npsc.co.in) → hosts. **Resource records:** A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail server), NS (name server), PTR (reverse), SOA.

DNS resolution steps (recursive query from the client to its resolver; iterative queries from resolver to servers):

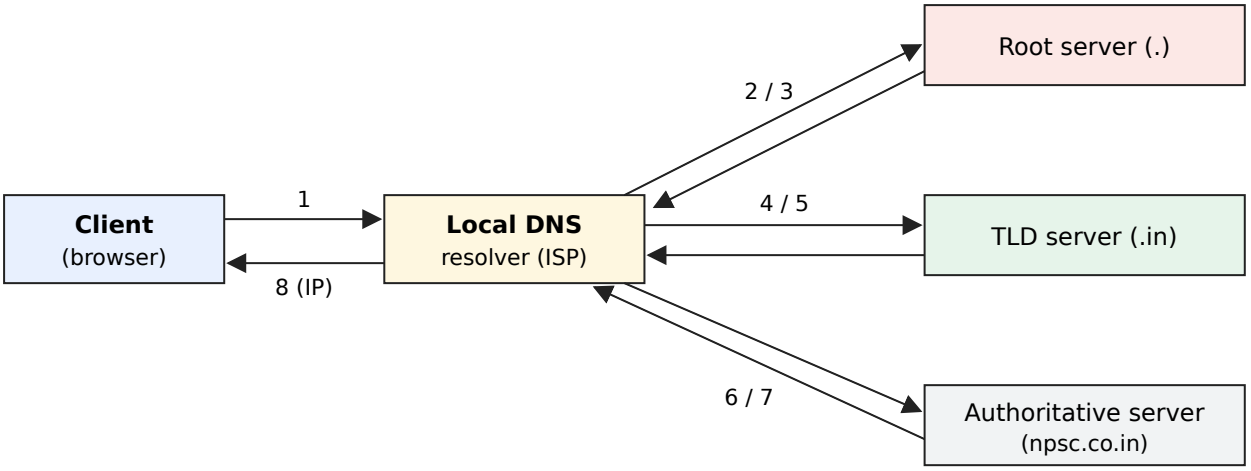


Fig: Resolving www.npsc.co.in — resolver queries root, TLD and authoritative servers in turn

1. Browser checks its cache, OS cache and hosts file; if not found, asks the **local resolver** (recursive query).
2. Resolver asks a **root server**. 3. Root replies with a referral to the **.in TLD** servers.
3. Resolver asks the **TLD server**. 5. TLD refers it to the **authoritative server** for npsc.co.in.
4. Resolver asks the authoritative server. 7. It returns the **A record (IP address)**.
5. Resolver caches the answer (for its TTL) and returns the IP to the client, which then opens a TCP connection to it.

4.8 Internet Routing

Importance 9/10 · very likely · syllabus phrase "Internet routing" unasked; distance vector vs link state is a classic 10-marker

Routing = the network-wide process of computing the best paths and building **routing tables** (control plane). **Forwarding** = the per-router action of moving an arriving packet to the correct output interface by looking up the table (data plane). A **hop** is one router-to-router link; **hop count** = number of routers a packet passes through.

Static routing	Dynamic routing
Routes entered manually by administrator	Routers learn routes automatically via routing protocols
No overhead, secure, predictable	Adapts to topology changes and failures
Does not adapt; unmanageable in large networks	Uses CPU/bandwidth for updates; more complex
Small/stub networks, default route	Large networks, the Internet

Distance vector vs Link state

Feature	Distance vector (e.g. RIP)	Link state (e.g. OSPF)
Algorithm	**Bellman–Ford**	**Dijkstra** (shortest path first)
What is shared	Whole routing table (distance vector)	Link-state advertisements (state of own links)
Shared with	Only **neighbours**	**All routers** in the area (flooding)
Knowledge	"Routing by rumour" — knows next hop and distance only	Full topology map of the area
Updates	Periodic (RIP every 30 s)	Triggered on change
Metric	**Hop count** (RIP max 15; 16 = unreachable)	**Cost** (based on bandwidth)
Convergence	Slow; **count-to-infinity** problem (fixes: split horizon, poison reverse)	Fast
Resources	Low CPU/memory	Higher CPU/memory
Scale	Small networks	Large enterprise networks, hierarchical areas

- **RIP** — Routing Information Protocol: distance vector, UDP port 520, hop-count metric, max 15 hops.
- **OSPF** — Open Shortest Path First: link state, runs directly over IP (protocol 89), areas with backbone area 0.
- **BGP** — Border Gateway Protocol (BGP-4): **path-vector** protocol used **between autonomous systems** (inter-domain, EGP); runs over TCP port 179; chooses routes by policy and AS-path. It is the routing protocol that glues the Internet together.
- **Autonomous System (AS):** a network under one administration. **IGP** (interior: RIP, OSPF, EIGRP) inside an AS; **EGP** (exterior: BGP) between ASes.

Exam tip: Mention "RIP = hop count, Bellman–Ford, 15 max, count-to-infinity" and "OSPF = Dijkstra, LSAs flooded, fast convergence" — those are the scoring keywords.

4.9 IP Address

Importance 10/10 · sure to come · "IP address" is the last syllabus phrase and unasked; classes + subnetting numerical very likely in descriptive and MCQ

An **IP address** is a logical, hierarchical address that identifies a host interface on a network (layer 3). **IPv4** is **32 bits**, written in **dotted-decimal** as four octets (e.g. 192.168.1.10). It has two parts: **network ID** + **host ID**. Total IPv4 space = 2^32 ≈ 4.3 billion addresses.

Classful addressing

Class	First-octet bits	First-octet range	Default mask	Network / host bits	No. of networks	Hosts per network	Use
A	0	1–126 (0, 127 reserved)	255.0.0.0 (/8)	8 / 24	126	2^24 − 2 = 16,777,214	Very large networks
B	10	128–191	255.255.0.0 (/16)	16 / 16	2^14 = 16,384	2^16 − 2 = 65,534	Medium
C	110	192–223	255.255.255.0 (/24)	24 / 8	2^21 = 2,097,152	2^8 − 2 = 254	Small
D	1110	224–239	—	—	—	—	Multicast
E	1111	240–255	—	—	—	—	Experimental/reserved

Special and private addresses

- **Private (RFC 1918, not routed on the Internet, used with NAT):** 10.0.0.0/8 (10.0.0.0–10.255.255.255), 172.16.0.0/12 (172.16.0.0–172.31.255.255), 192.168.0.0/16 (192.168.0.0–192.168.255.255).
- **127.0.0.0/8** loopback (127.0.0.1 = localhost); **0.0.0.0** "this host"; **255.255.255.255** limited broadcast; **169.254.0.0/16** link-local (APIPA when DHCP fails).
- In every network: host bits all 0 = **network address**; host bits all 1 = **directed broadcast** → so usable hosts = **2^h − 2**.

Subnet mask: 32-bit value with 1s for network+subnet bits and 0s for host bits. **Network address = IP AND mask.**
Subnetting: borrowing host bits to divide one network into smaller subnets — reduces broadcast traffic, improves security and address utilisation. With s borrowed bits: subnets = 2^s; hosts per subnet = 2^h − 2.
CIDR (Classless Inter-Domain Routing, 1993): drops classes; prefix length written as /n (e.g. 192.168.10.0/26). Allows **supernetting/route aggregation** and variable-size blocks (VLSM).

Worked example 1 (verified with python3 `ipaddress`): Divide 192.168.10.0/24 into 4 equal subnets.
Need 4 subnets → borrow s = 2 bits (2^2 = 4) → new prefix /26 → mask **255.255.255.192**; host bits h = 6 → 2^6 − 2 = **62 usable hosts** each; block size = 256 − 192 = 64.

Subnet	Network address	First host	Last host	Broadcast
1	192.168.10.0/26	192.168.10.1	192.168.10.62	192.168.10.63
2	192.168.10.64/26	192.168.10.65	192.168.10.126	192.168.10.127
3	192.168.10.128/26	192.168.10.129	192.168.10.190	192.168.10.191
4	192.168.10.192/26	192.168.10.193	192.168.10.254	192.168.10.255

Worked example 2 (verified): Host 172.16.45.130/20. Mask /20 = **255.255.240.0**; block size in 3rd octet = 256 − 240 = 16; 45 lies in block 32–47 → **network 172.16.32.0**, **broadcast 172.16.47.255**, usable hosts = 2^12 − 2 = **4094**.

IPv6 basics: 128-bit address (2^128), written as **8 groups of 4 hex digits** separated by colons, e.g. `2001:0db8:0000:0000:0000:ff00:0042:8329` → compressed `2001:db8::ff00:42:8329` (leading zeros dropped; one run of zero groups replaced by `::`, used only once). Features: huge address space (no NAT needed), simplified fixed 40-byte header, no broadcast (uses multicast/anycast), built-in IPsec support, stateless auto-configuration (SLAAC), no router fragmentation. Loopback `::1`. Types: unicast, multicast, anycast.

MAC address vs IP address

MAC (physical) address	IP (logical) address
48 bits, hexadecimal, e.g. 00:1A:2B:3C:4D:5E	IPv4 32 bits / IPv6 128 bits
Burned into NIC by manufacturer (first 24 bits = OUI vendor ID)	Assigned by admin or DHCP; changes with network
Data link layer (layer 2)	Network layer (layer 3)
Used for delivery within the local network (hop to hop)	Used for end-to-end delivery across networks
Flat (no hierarchy)	Hierarchical (network + host)
Found from IP by **ARP**	—

Exam tip: Class questions usually ask "identify class, default mask, network address" — check the first octet, AND with the mask, and always subtract 2 for usable hosts.

5. Web Technologies

Syllabus phrase (Paper-II §5): *The Phases of web site development: Implementation, Maintenance, Testing. Basic HTML Concepts, HTML, HEAD, TITLE, BODY, Paragraphs, Lists, Formatted and Unformatted Text, Hyperlink, Font (Size, Colour), image. PHP: Server-side web scripting, Installing PHP, Adding PHP to HTML, Syntax and Variables, Passing information between pages, Basic PHP error/problems.* 2025 asked GET vs POST (5), PHP syntax & variables (5), phases of website development (10).

5.1 Phases of Website Development

Importance 6/10 · moderate · asked as 10-marker in 2025; may reappear as "testing/maintenance of a website" (the syllabus names those words)

Website development follows a life cycle like the SDLC.

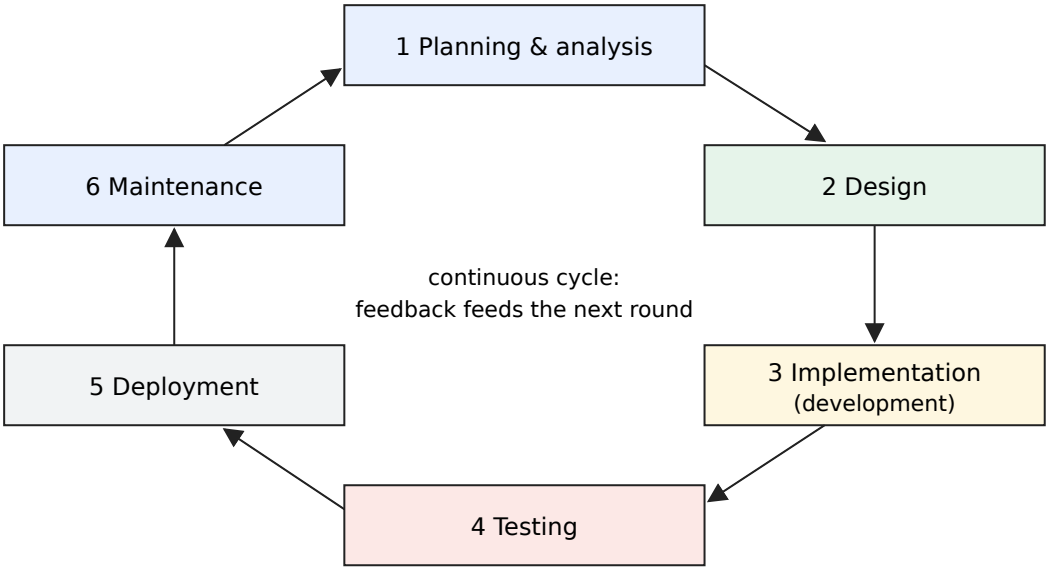


Fig: Website development life cycle

Phase	Key activities	Output
Planning & analysis	Purpose, target audience, goals, content requirements, budget, domain name, hosting, technology choice	Requirements / project plan
Design	**Site map**, navigation structure, **wireframes**, mock-ups, colour scheme, layout, responsive design, database design	Design mock-ups, site map
Implementation (development)	Front-end coding (HTML, CSS, JavaScript), back-end (PHP, database), content creation and integration, CMS set-up	Working website on test server
Testing	Functional tests (links, forms), **cross-browser** and device/responsive testing, usability, performance/load, security (SQL injection, XSS), HTML/CSS validation, accessibility	Bug-free, validated site
Deployment (launch)	Upload files via FTP/Git to web host, configure domain/DNS, SSL certificate, submit to search engines (SEO)	Live website
Maintenance	Update content, fix bugs, security patches, backups, monitor traffic/analytics, add features, renew domain/hosting	Up-to-date, secure site

Exam tip: The syllabus singles out **implementation, testing and maintenance** — give those three extra lines each (e.g. list five website tests).

5.2 Basic HTML Concepts and Document Structure

Importance 9/10 · very likely · syllabus lists HTML, HEAD, TITLE, BODY by name; unasked in descriptive

HTML (HyperText Markup Language) is the standard markup language for web pages. It uses **tags** in angle brackets to describe the structure of content; browsers interpret the tags and render the page. Most tags are paired (`<p> … </p>`); some are empty/void (`<br>`, `<img>`, `<hr>`). Tags can have **attributes** (`name="value"`). HTML is not case-sensitive and is not a programming language.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>NPSC Prep</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph.</p>
</body>
</html>
```

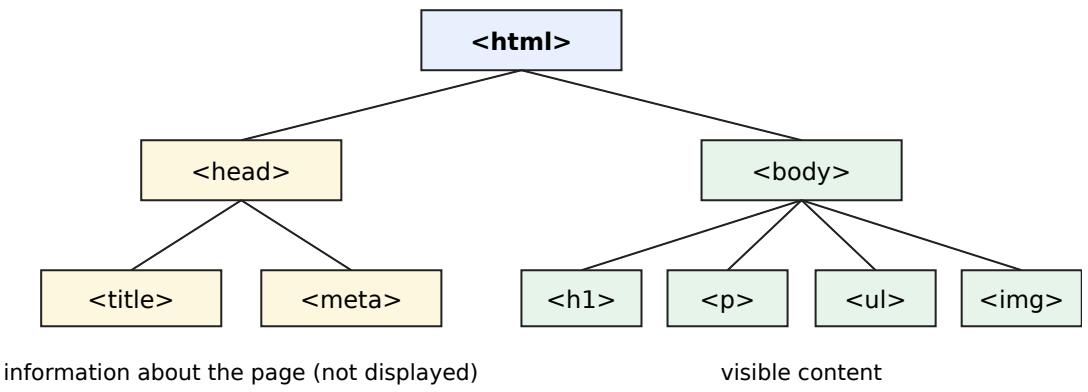


Fig: HTML document tree — head holds metadata, body holds visible content

Tag	Purpose
`<!DOCTYPE html>`	Declaration (not a tag) telling the browser the document is HTML5
`<html>`	Root element enclosing the whole document
`<head>`	Container for metadata: title, meta, link (CSS), script, style — not shown in page body
`<title>`	Text shown in the browser tab/title bar and used by search engines and bookmarks; only inside head
`<body>`	All visible content. Old attributes: `bgcolor`, `text`, `background` (now CSS)

5.3 Paragraphs, Headings, Line Breaks and Lists

Importance 8/10 · likely · named in syllabus ("Paragraphs, Lists"); easy 5-marker with code

- `<p>` — paragraph; browser adds space before/after; extra spaces and newlines in source collapse into one space.
- `<h1>` … `<h6>` — six heading levels, h1 largest/most important.
- `<br>` — line break (empty tag); `<hr>` — horizontal rule.

Three kinds of lists

```
<!-- Unordered list (bullets); type: disc | circle | square (old attribute; CSS list-style-type now) -->
<ul>
    <li>DBMS</li>
    <li>Networks</li>
</ul>

<!-- Ordered list (numbered); type="1|A|a|I|i", start="5" -->
<ol type="I" start="1">
    <li>Planning</li>
    <li>Design</li>
</ol>

<!-- Description (definition) list: term + description -->
<dl>
    <dt>HTTP</dt>
    <dd>Protocol for transferring web pages</dd>
</dl>
```

Lists can be **nested** (a `<ul>` inside an `<li>`).

5.4 Formatted and Unformatted (Preformatted) Text

Importance 7/10 · good chance · syllabus phrase "Formatted and Unformatted Text"; likely 5-marker or MCQ

Formatted text uses tags to change the appearance or meaning of text. The browser ignores the source's own spacing.

Tag	Effect	Note
`<b>`	**Bold**	Presentational
`<strong>`	Bold + *strong importance*	Semantic (screen readers stress it)
`<i>`	*Italic*	Presentational
`<em>`	Italic + *emphasis*	Semantic
`<u>`	Underline	
`<sub>`	Subscript: H_2O	
`<sup>`	Superscript: x^2	
`<mark>`	Highlighted text	HTML5
`<small>`, `<del>`, `<ins>`	Smaller, struck-through, inserted text	

Unformatted / preformatted text — `<pre>`: text is shown **exactly as typed** in the source — spaces, tabs and line breaks are preserved, usually in a monospace font. Used for code, poems, ASCII tables.

```
<pre>
Name      Marks
Asha        92
Vini        88
</pre>
```

(Related: `<code>` for inline code, `<tt>` obsolete teletype text.)

5.5 Hyperlinks

Importance 8/10 · likely · syllabus word "Hyperlink"; absolute vs relative and target attribute are common

A **hyperlink** is created with the **anchor tag `<a>`**; the `href` attribute gives the destination. Unvisited links are blue and underlined by default, visited links purple.

```
<a href="https://www.npsc.nagaland.gov.in/">NPSC site</a>          <!-- absolute URL -->
<a href="syllabus/cse.html">CSE syllabus</a>                        <!-- relative URL -->
<a href="https://example.com" target="_blank">Open in new tab</a>
<a href="mailto:office@example.com?subject=Query">E-mail us</a>
<a href="#section2">Jump to Section 2</a>                            <!-- internal link -->
<h2 id="section2">Section 2</h2>                                     <!-- link target -->
<a href="notes.pdf" download>Download notes</a>
```

- **Absolute URL:** full address including scheme and host — used for external sites.
- **Relative URL:** path relative to the current page — used within the same site; survives moving the site to another domain.
- **target:** `_self` (default, same tab), `_blank` (new tab/window), `_parent`, `_top`.
- **Internal anchors:** link to an `id` (older: `<a name="x">`) on the same or another page (`page.html#x`).
- An image can be a link: `<a href="home.html"><img src="logo.png" alt="Home"></a>`.

5.6 Font (Size, Colour) and Images

Importance 8/10 · likely · syllabus says "Font (Size, Colour), image" explicitly — expect the `<font>` attributes

The **`<font>`** tag (HTML 3.2/4) set text appearance. It is **deprecated in HTML 4.01 and removed in HTML5** — CSS is used instead — but the exam syllabus still names it, so know both.

```
<!-- Old way: size 1 to 7 (default 3), or relative +1 / -2 -->
<font size="5" color="red" face="Arial, Verdana">Important notice</font>
<basefont size="4">   <!-- default font for page (also obsolete) -->

<!-- Modern CSS equivalent -->
<span style="font-size: 24px; color: #ff0000; font-family: Arial, Verdana;">Important
notice</span>
```

`<font>` attribute	Values	CSS equivalent
`size`	1–7 (3 default), or +n / −n	`font-size` (px, em, %, keywords)
`color`	Colour name (`red`) or hex `#RRGGBB`	`color`
`face`	Font family list	`font-family`

Images — `<img>` (empty tag):

```
<img src="images/kohima.jpg" alt="View of Kohima" width="400" height="250" title="Kohima">
```

Attribute	Purpose
`src`	Path/URL of the image file (required)
`alt`	Alternative text shown if image fails; read by screen readers; helps SEO (required for valid HTML)
`width`, `height`	Display size in pixels (keeps layout stable)
`title`	Tooltip text
`border`, `align`	Old presentational attributes (now CSS)

Common formats: JPEG (photos, lossy), PNG (lossless, transparency), GIF (256 colours, animation), SVG (vector), WebP.

5.7 Tables, Forms and GET vs POST

Importance 7/10 · good chance · GET vs POST asked in 2025; forms are needed for "passing information between pages"

Tables (briefly): `<table border="1">` with rows `<tr>`, header cells `<th>`, data cells `<td>`; `colspan`/`rowspan` merge cells; `<caption>` gives a title.

Forms: collect user input and send it to a server-side script.

```
<form action="welcome.php" method="post">
    Name: <input type="text" name="uname" required>
    Password: <input type="password" name="pwd">
    Gender: <input type="radio" name="g" value="M"> M <input type="radio" name="g"
value="F"> F
    <select name="dept"><option>CSE</option><option>ECE</option></select>
    <textarea name="msg" rows="3" cols="30"></textarea>
    <input type="submit" value="Send">
</form>
```

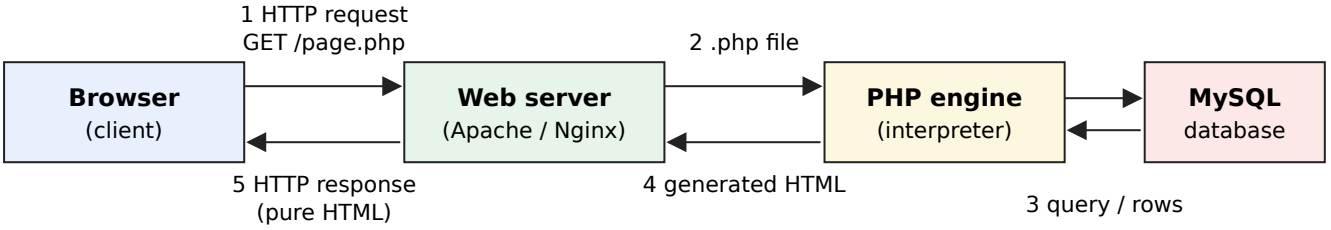
Basis	GET	POST
Where data goes	Appended to the URL as a **query string** (`?a=1&b=2`)	In the HTTP **request body**
Visibility	Visible in address bar, history, server logs	Not visible in URL
Data size	Limited by URL length (~2 KB practical)	No practical limit (server-configured)
Data type	ASCII text only	Text and binary (file upload with `enctype="multipart/form-data"`)
Bookmark / cache	Can be bookmarked and cached	Cannot be bookmarked; not cached
Security	Less secure — never for passwords	Better (still needs HTTPS for real security)
Idempotent	Yes — safe to repeat (retrieval)	No — may change server state; browser warns on resubmit
Use	Searches, filters, links	Login, registration, payments, uploads
PHP array	`$_GET`	`$_POST`

5.8 PHP and Server-side Web Scripting

Importance 7/10 · good chance · syllabus phrase "Server-side web scripting"; definition + flow diagram answer

PHP (PHP: Hypertext Preprocessor) is an open-source, **server-side scripting language** embedded in HTML, created by Rasmus Lerdorf (1994). PHP code runs on the **web server**; the browser receives only the generated HTML. It is interpreted, loosely (dynamically) typed, cross-platform, and works with many databases (MySQL, PostgreSQL). Files have the `.php` extension.

Client-side vs server-side scripting: client-side (JavaScript) runs in the browser, code visible to user, used for interactivity/validation; server-side (PHP, ASP.NET, JSP, Python) runs on the server, code hidden, can access databases and files, generates dynamic pages.



The user never sees PHP source — only the HTML output it produced.

Fig: Server-side scripting flow for a PHP page

Steps: (1) browser requests `page.php`; (2) web server sees `.php` and passes the file to the PHP interpreter; (3) PHP executes, possibly querying a database; (4) output (HTML) is returned to the web server; (5) server sends plain HTML to the browser, which renders it.

5.9 Installing PHP

Importance 7/10 · good chance · syllabus phrase "Installing PHP"; unasked

PHP needs a **web server + PHP interpreter (+ database)**. The easiest way is a bundled stack:

Stack	OS	Components
XAMPP	Cross-platform (X)	Apache, MariaDB/MySQL, PHP, Perl
WAMP	Windows	Apache, MySQL, PHP
LAMP	Linux	Apache, MySQL, PHP (e.g. `sudo apt install apache2 php libapache2-mod-php mysql-server`)
MAMP	macOS	Apache, MySQL, PHP

Steps (XAMPP on Windows): download installer from apachefriends.org → install (e.g. `C:\xampp`) → open XAMPP Control Panel and **start Apache** (and MySQL) → save scripts in the document root **`C:\xampp\htdocs\`** (Linux: `/var/www/html/`) → open `http://localhost/test.php` in the browser (not by double-clicking the file).

Test the installation with `phpinfo()`:

```
<?php
phpinfo();   // shows PHP version, loaded php.ini, extensions, server variables
?>
```

php.ini is PHP's main configuration file — e.g. `display_errors`, `error_reporting`, `upload_max_filesize`, `post_max_size`, `max_execution_time`, `memory_limit`, `extension=mysqli`. Restart Apache after editing it. PHP also has a built-in development server: `php -S localhost:8000`.

5.10 Adding PHP to HTML

Importance 8/10 · likely · syllabus phrase; unasked; 5-marker with a short code example

PHP code is placed inside **PHP tags** anywhere in an HTML page; the file must be saved with a `.php` extension. Everything outside the tags is sent to the browser unchanged.

Tag style	Status
`<?php ... ?>`	Standard, always works — use this
`<?= $x ?>`	Short echo tag = `<?php echo $x; ?>` (always enabled since PHP 5.4)
`<? ... ?>`	Short open tag — works only if `short_open_tag` is on; avoid
`<script language="php">, <% %>`	Removed in PHP 7

```
<!DOCTYPE html>
<html>
<head><title>PHP in HTML</title></head>
<body>
    <h1>Today</h1>
    <p>Date: <?php echo date("d-m-Y"); ?></p>
    <?php
        $name = "Asha";
        print "<p>Hello, $name</p>";      // double quotes expand variables
        echo 'Single: $name', "<br>";     // single quotes do not expand
    ?>
    <p>Short echo: <?= strtoupper($name) ?></p>
</body>
</html>
```

echo vs print: both output text. `echo` is a language construct with no return value and can take **multiple comma-separated arguments**; `print` takes **one argument and returns 1**, so it can be used in expressions. `echo` is marginally faster.

5.11 PHP Syntax and Variables

Importance 7/10 · good chance · asked in 2025 (5 marks); MCQ favourite ($ rules, case-sensitivity)

Basic syntax rules

- Code inside `<?php ... ?>`; every statement ends with a **semicolon** (the one before `?>` is optional).
- Comments: `// single`, `# single`, `/* multi-line */`.
- **Variable names are case-sensitive** (`$age` ≠ `$Age`); **keywords, function and class names are case-insensitive** (`ECHO`, `Echo` both work).
- Whitespace and line breaks are ignored.

Variables

- Start with `$`, followed by a letter or underscore, then letters, digits, underscores (`$_x`, `$total2` valid; `$2x`, `$my-var` invalid).
- No declaration needed; created on first assignment. PHP is **loosely typed** — type is decided by the value and can change.
- Scope: local (inside function), global (use `global $x;` or `$GLOBALS['x']` in functions), static (`static $count = 0;` keeps value between calls).
- `var_dump($x)` shows type and value; `isset()`, `unset()`, `empty()`, `gettype()`.

Data types: scalar — `int`, `float`, `string`, `bool`; compound — `array`, `object` (plus `callable`, `iterable`); special — `NULL`, `resource`.

Constants: `define("PI", 3.14159);` or `const MAX = 100;` — no `$`, cannot be changed, global scope. Magic constants: `__LINE__`, `__FILE__`, `__DIR__`, `__FUNCTION__`.

Superglobals (built-in arrays available everywhere): `$_GET`, `$_POST`, `$_REQUEST`, `$_SESSION`, `$_COOKIE`, `$_FILES`, `$_SERVER`, `$_ENV`, `$GLOBALS`.

Operators (briefly): arithmetic `+ - * / % **`; assignment `= += .=`; string concatenation `.`; comparison `== != < > <= >=` and **`===` (equal value and same type)**, `!==`, spaceship `<=>`; logical `&& || ! and or xor`; increment `++ --`; null-coalescing `??`.

Control structures (briefly): `if / elseif / else`, `switch`, `while`, `do…while`, `for`, `foreach ($arr as $key => $value)`, `break`, `continue`; functions with `function name($a, $b = 2) { return $a + $b; }`.

```
<?php
$name  = "Vini";            // string
$marks = [78, 92, 85];      // indexed array
$info  = ["dept" => "CSE", "year" => 2026];   // associative array
define("PASS", 40);
$total = 0;
foreach ($marks as $m) { $total += $m; }
echo $name . " scored " . $total . " in " . $info["dept"];   // Vini scored 255 in CSE
var_dump("5" == 5, "5" === 5);   // bool(true) bool(false)
?>
```

5.12 Passing Information Between Pages

Importance 9/10 · very likely · syllabus phrase unasked; HTTP is stateless so this is the core PHP 10-marker

HTTP is **stateless**: each request is independent, so the server does not remember previous requests. PHP offers several ways to carry data from one page to another:

Method	How	Stored where	Typical use
GET / query string	`page2.php?id=5&name=Asha` (link or `method="get"` form); read with `$_GET['id']`	URL	Search, IDs in links
POST	Form with `method="post"`; read with `$_POST['field']`	Request body	Login, registration
`$_REQUEST`	Contains GET, POST and (by default config) COOKIE data together	—	Convenience; avoid for security clarity
Hidden form fields	`<input type="hidden" name="step" value="2">`	In the page HTML (user can view/modify)	Multi-step forms
Sessions	`session_start();` then `$_SESSION['user'] = 'asha';`	**Server**; only session ID (cookie `PHPSESSID`) on client	Login state, shopping cart
Cookies	`setcookie("lang", "en", time() + 86400);` read via `$_COOKIE['lang']`	**Client browser** (max ~4 KB each)	Preferences, "remember me"

Session vs Cookie: sessions are stored on the server (more secure, larger data, end when browser closes or timeout); cookies are stored in the browser (user can see/alter, size-limited, can persist until an expiry date). `session_start()` and `setcookie()` must be called **before any output** is sent, because they send HTTP headers. End a session with `session_unset(); session_destroy();`. Delete a cookie by setting an expiry in the past.

Working example — form page + handler page

`form.html`

```
<!DOCTYPE html>
<html>
<head><title>Register</title></head>
<body>
<form action="handler.php" method="post">
    Name: <input type="text" name="uname" required><br>
    Course: <select name="course"><option>CSE</option><option>ECE</option></select><br>
    <input type="hidden" name="source" value="regform">
    <input type="submit" value="Register">
</form>
<a href="handler.php?uname=Guest&course=CSE">Quick link using GET</a>
</body>
</html>
```

`handler.php`

```
<?php
session_start();                                  // before any output
$name   = $_POST['uname']  ?? $_GET['uname']  ?? 'unknown';
$course = $_POST['course'] ?? $_GET['course'] ?? '';
$_SESSION['uname'] = $name;                       // remembered on later pages
setcookie("last_course", $course, time() + 7 * 24 * 3600);   // 7 days
$safe = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');        // prevent XSS
?>
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
    <p>Welcome, <?= $safe ?>! You chose <?= htmlspecialchars($course) ?>.</p>
    <p>Method used: <?= $_SERVER['REQUEST_METHOD'] ?></p>
    <a href="profile.php">Go to profile</a>   <!-- profile.php: session_start(); echo
$_SESSION['uname']; -->
</body>
</html>
```

Exam tip: Structure the 10-mark answer as: "HTTP is stateless" → table of the six methods → GET vs POST → session vs cookie → the code above.

5.13 Basic PHP Errors and Problems

Importance 8/10 · likely · syllabus phrase "Basic PHP error/problems" unasked; good 5/10-marker

Error type	Cause	Effect	Example
Parse (syntax) error	Code breaks PHP grammar	Script does not run at all	Missing `;`, unmatched brace or quote
Fatal error	Serious runtime problem	Script stops at that point	Calling undefined function, `require` of missing file, memory exhausted
Warning	Non-fatal problem	Script continues	`include` of missing file, wrong argument count to built-in, division by zero (PHP 7)
Notice (PHP 8: many became warnings)	Minor issue, possible bug	Script continues	Using an undefined variable or array index
Deprecated	Feature will be removed	Continues	Old function use

Common problems and fixes

- **Missing semicolon** → `Parse error: syntax error, unexpected ...` reported on the *next* line. Check the line above.
- **Undefined variable / index** → misspelt or case-different variable (`$Name` vs `$name`), or form field not sent; use `isset()` or `??`.
- **"Headers already sent"** → output (even a blank line or space before `<?php`, or a UTF-8 BOM) was sent before `header()`, `setcookie()` or `session_start()`. Fix: call them before any output, remove whitespace, or use output buffering `ob_start()`.
- **Blank white page** → a fatal/parse error with `display_errors` off. Turn on error display in development or read the server error log.
- **PHP code shown as text in browser** → file not saved as `.php`, opened via file path instead of `http://localhost/`, or server not running/PHP not configured.
- `=` **instead of** `==` in conditions; **mismatched quotes**; calling functions before including the file that defines them.
- **Call to undefined function mysql_connect()** → old `mysql_*` API removed in PHP 7; use `mysqli` or PDO.

Controlling error display

```
<?php
error_reporting(E_ALL);          // report every error level
ini_set('display_errors', '1');  // show on screen (development only)
// Production: display_errors = Off, log_errors = On in php.ini (errors go to log file)
?>
```

Levels: `E_ERROR`, `E_WARNING`, `E_PARSE`, `E_NOTICE`, `E_DEPRECATED`, `E_ALL`. The `@` operator suppresses errors for one expression (bad practice). PHP 7+ throws `Error`/`Exception` objects that can be caught with `try { … } catch (Throwable $e) { … }`.

5.14 PHP Security Basics

Importance 5/10 · possible · not a syllabus phrase by itself, but adds marks to any PHP form answer

- **Never trust user input** — validate (type, length, format, e.g. `filter_var($email, FILTER_VALIDATE_EMAIL)`) and sanitise.

- **XSS (cross-site scripting):** attacker injects `<script>` into a page. Escape output with **`htmlspecialchars()`** (converts `< > & " '` to entities).
- **SQL injection:** attacker inserts SQL through input (e.g. `' OR '1'='1`). Use **prepared statements** with bound parameters:

```
<?php
$pdo  = new PDO("mysql:host=localhost;dbname=college", "user", "secret");
$stmt = $pdo->prepare("SELECT name FROM students WHERE roll = ?");
$stmt->execute([$_GET['roll']]);          // value sent separately from the SQL text
$row  = $stmt->fetch();
?>
```

- Store passwords with **`password_hash()`** / verify with `password_verify()` (never plain text or MD5).
- Use **POST + HTTPS** for sensitive data; call `session_regenerate_id()` after login; hide errors in production.

6. Artificial Intelligence

Syllabus (verbatim): *Knowledge and Reasoning: Building a Knowledge Base. Learning from Observations: Inductive Learning, Explanation Based Learning. Pattern Recognition: Structured Description, Symbolic Description, Object Identification, Speech Recognition. Programming Language: LISP, PROLOG.*

2025 asked: **Q19 (15 marks) building a knowledge base + challenges**, **Q14 (10 marks) knowledge-level vs symbol-level learning**, plus 8 MCQs (rules as KR, ontology, decision trees, EBL, structured description, symbolic description, HMM, PROLOG). Every phrase in this syllabus is fair game in 2026.

6.1 What is AI? Agents in one page

Importance 5/10 · possible · background for every AI answer; rarely asked alone, but a 2-line intro earns marks

Plain language: AI is the branch of CS that builds programs which do things that would need "intelligence" if a human did them — reasoning, learning, perceiving, understanding language. Russell & Norvig frame AI as building **rational agents**: things that perceive and act to achieve the best expected outcome.

- **Definitions (4 views, Russell–Norvig):** thinking humanly (cognitive modelling) · acting humanly (Turing Test, 1950) · thinking rationally (laws of thought / logic) · acting rationally (rational agent — the modern view).
- **Rich & Knight:** "AI is the study of how to make computers do things which, at the moment, people do better."
- **Agent** = anything that perceives its environment through **sensors** and acts on it through **actuators**. Agent = architecture + program.
- **PEAS** description of a task: Performance measure, Environment, Actuators, Sensors (e.g. taxi driver: safety/speed; roads; steering/brake; camera/GPS).
- **Agent types (increasing capability):** simple reflex → model-based reflex → goal-based → utility-based → **learning agent** (any of the above + learning element).
- **Environment properties:** fully/partially observable · deterministic/stochastic · episodic/sequential · static/dynamic · discrete/continuous · single/multi-agent.

Exam tip: Open any AI long answer with one line on rational agents, then go straight to the asked topic. Don't spend more than 3 lines here.

6.2 Knowledge, knowledge base and inference engine

Importance 8/10 · likely · core of "Knowledge and Reasoning"; frames the 2025 Q19 type of answer

Plain language: A knowledge-based system separates **what it knows** (the knowledge base, KB) from **how it reasons** (the inference engine). You "teach" it by TELLing sentences into the KB; you "use" it by ASKing questions; the inference engine derives answers that follow from the KB.

- **Data → information → knowledge:** data are raw facts (38.5); information is data with context (patient temperature 38.5 °C); knowledge is understanding that supports action ("temperature > 38 °C ⇒ fever ⇒ suspect infection").
- **Types of knowledge:** declarative (facts: "Kohima is capital of Nagaland") · procedural (how-to: steps of an algorithm) · heuristic (rules of thumb from experts) · meta-knowledge (knowledge about knowledge, e.g. which rule to try first) · structural (relations among concepts: is-a, part-of).
- **Knowledge base (KB):** a set of **sentences** written in a **knowledge representation language**. Initial sentences = **background knowledge** (axioms).
- **Inference engine:** the reasoning mechanism that derives new sentences from old ones (Modus Ponens, resolution, forward/backward chaining). Must be **sound** (derives only entailed sentences) and ideally **complete** (derives every entailed sentence).
- **TELL / ASK interface (Russell–Norvig):** each time step the agent TELLs the KB what it perceives, ASKs the KB what action to take, then TELLs the KB which action it took.
- **Declarative vs procedural approach:** declarative = TELL the agent what it needs to know, let inference work it out; procedural = encode behaviour directly as code. KB systems favour declarative.
- **Expert system** = KB + inference engine + user interface + explanation facility + knowledge-acquisition facility + working memory.

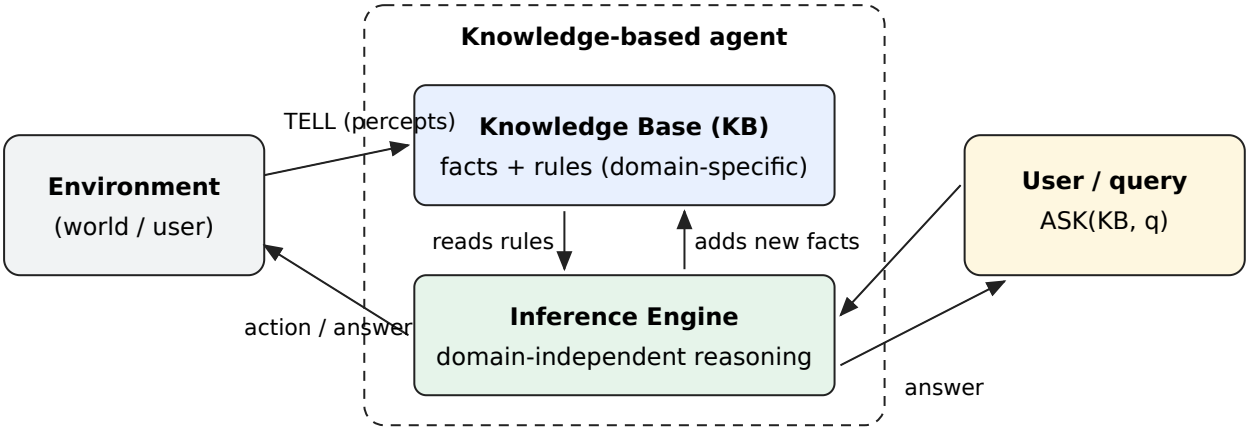


Fig: Knowledge-based agent — KB (what it knows) is separate from the inference engine (how it reasons); interaction via TELL and ASK

Knowledge base	Inference engine
Domain-specific content (facts, rules)	Domain-independent reasoning procedure
Changes as knowledge is added (TELL)	Same engine can be reused across domains
Written by knowledge engineer + expert	Written once by system builder
e.g. "IF fever AND rash THEN measles"	e.g. forward chaining, backward chaining, resolution

Exam tip: The separation "KB vs inference engine" is the single most important idea — it is why an expert-system *shell* (e.g. CLIPS) can be reused for a new domain by only swapping the KB.

6.3 Knowledge representation (KR) methods

Importance 8/10 · likely · 2025 MCQs on rules and ontology; a natural 10-mark "explain KR techniques" question

Plain language: KR is about choosing a *language* in which facts about the world can be written so that a program can reason with them. Different methods trade off expressiveness against efficiency of reasoning.

(a) Propositional logic — atoms (P, Q) are whole statements, true or false; connectives ¬, ∧, ∨, →, ↔.

- Example: R = "it is raining", W = "ground is wet". KB: R → W, R. By **Modus Ponens** infer W.
- Limitation: cannot talk about objects or generalise ("all men are mortal" needs one proposition per man).

(b) First-order (predicate) logic, FOL — adds objects, predicates, functions, variables and quantifiers ∀, ∃.

- ∀x Man(x) → Mortal(x) · Man(Socrates) ⊢ Mortal(Socrates) (universal instantiation + Modus Ponens).
- ∀x Student(x) → ∃y Likes(x, y) — "every student likes something".
- ∀x ∀y Parent(x, y) ∧ Female(x) → Mother(x, y).
- Pros: very expressive, precise semantics, sound & complete proof procedures (resolution). Cons: semi-decidable, poor at uncertainty/defaults.

(c) Semantic networks (Quillian, 1968) — a labelled directed graph: nodes = objects/concepts, arcs = relations (**is-a**, **instance-of**, **has-part**, property links). **Inheritance**: a node inherits properties of its ancestors along is-a links unless overridden.

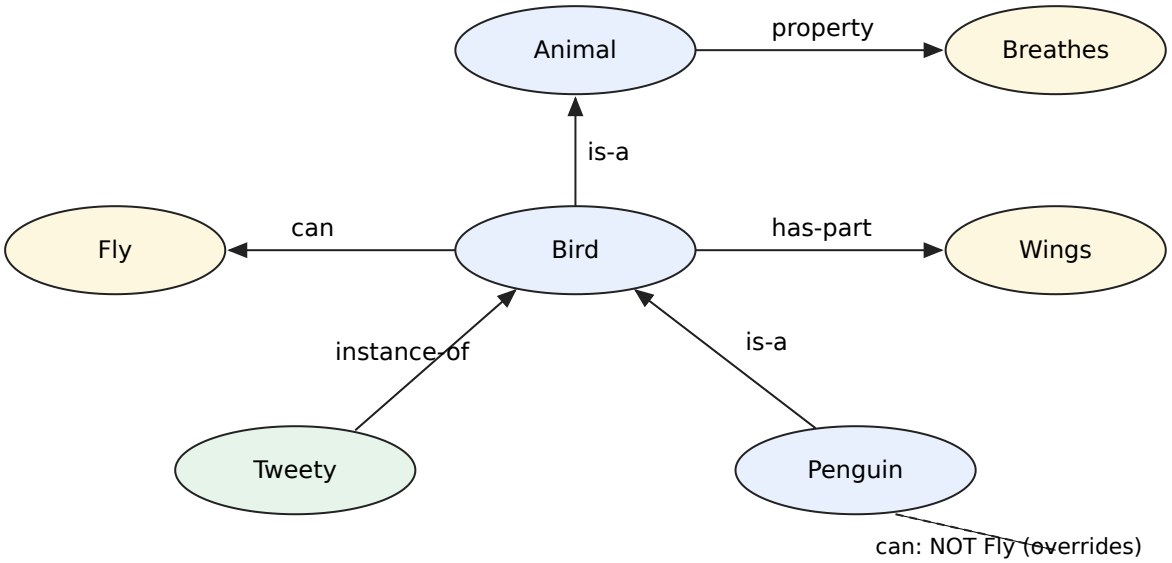


Fig: Semantic network — Tweety inherits "can Fly", "has Wings", "Breathes"; Penguin overrides "can Fly" (default inheritance with exception)

(d) Frames (Minsky, 1975) — a record-like structure for a stereotyped object/situation: **slots** (attributes) with **fillers** (values), **default values**, and **procedural attachments** (*if-needed*, *if-added* demons). Frames are linked by is-a for inheritance.

```
Frame: Hotel-Room                  Frame: Room-101
  is-a     : Room                    instance-of : Hotel-Room
  location : Hotel                   beds        : 2
  contains : (bed, chair, phone)     rate        : if-needed -> compute from season
  beds     : default 1
```

(e) Production rules — **IF condition THEN action** pairs; a **production system** = rule base + working memory (facts) + control strategy (rule interpreter with conflict resolution). The most common KR in expert systems (MYCIN, ~600 rules; this is why the 2025 MCQ answer was "Rules").

- Example: `IF animal has feathers AND animal lays eggs THEN animal is bird.`
- Pros: modular (add a rule without touching others), natural to experts, explainable. Cons: large rule sets hard to maintain; conflicts between rules; inefficient without RETE-style matching.

(f) Ontologies — a **formal, explicit specification of a shared conceptualization** (Gruber 1993; "shared" added by Studer et al.). Defines the **vocabulary** of a domain: classes (concepts), instances, properties/relations, and axioms/constraints.

- Example: Class *Vehicle* ⊃ *Car*, *Bus*; property *hasWheels*; axiom "every Car has exactly 4 wheels".
- Languages/tools: RDF, RDFS, **OWL** (Web Ontology Language), Protégé. Used in the Semantic Web, medicine (SNOMED CT), Gene Ontology, WordNet-style lexical resources.
- Upper ontology = general concepts (time, event, physical object) shared across domains.

(g) Others (one line each): scripts (Schank — stereotyped event sequences, e.g. restaurant script), conceptual dependency, Bayesian networks (uncertain knowledge).

Method	Unit of knowledge	Reasoning	Strength	Weakness
Propositional logic	proposition	truth tables, resolution	simple, decidable	no objects/variables
First-order logic	predicates + quantifiers	unification, resolution	expressive, precise	semi-decidable, no uncertainty
Semantic network	nodes + labelled arcs	inheritance, spreading	visual, inheritance	weak formal semantics
Frames	slots + fillers	inheritance, defaults, demons	structured, defaults	exceptions messy
Production rules	IF–THEN	forward/backward chaining	modular, explainable	conflicts, scale
Ontology	classes, relations, axioms	DL reasoners	shared vocabulary, reuse	costly to build/agree

Exam tip (MCQ): "Ontology primarily focuses on" → *formal, explicit specification of a shared conceptualization*. "Common way to represent knowledge in a KB" → *rules*. "Inheritance" → semantic nets/frames.

6.4 Properties of a good knowledge representation

Importance 6/10 · moderate · short 5-mark candidate; also a strong opening paragraph for any KR/KB answer

Rich & Knight list four properties a KR system should have:

Property	Meaning
Representational adequacy	can represent all kinds of knowledge needed in the domain
Inferential adequacy	can manipulate structures to derive new knowledge from old
Inferential efficiency	can add extra information (e.g. pointers, indexes) to focus inference in promising directions
Acquisitional efficiency	new knowledge can be acquired easily (ideally automatically)

Also desirable: **clarity/naturalness** (readable by experts), **modularity** (change one piece without breaking others), **well-defined semantics**, **ability to handle uncertainty and defaults**.

6.5 Building a knowledge base — the knowledge-engineering process

Importance 8/10 · likely · asked as 15-mark Q19 in 2025; may return reworded as "steps / knowledge engineering"

Plain language: A knowledge engineer studies a domain, decides which concepts matter, writes them down formally, and keeps testing the KB until it gives right answers. Russell & Norvig give seven steps (their running example: a digital-circuit KB that can answer "what does this adder output for inputs 1,0,1?").

1. **Identify the task** — what questions will the KB answer, what facts will be available per problem? (Determines what must be represented.) *Circuit: should it reason about timing/power, or only logical function?*
2. **Assemble the relevant knowledge** — **knowledge acquisition**: interview domain experts, read manuals/literature; understand the scope informally. *How gates work, how wires connect.*
3. **Decide on a vocabulary** of predicates, functions and constants — i.e. build the domain **ontology**. *Type(X1) = XOR, Connected(Out(1,X1), In(1,X2)), Signal(t).*
4. **Encode general knowledge about the domain** — axioms/rules true for all problem instances. *∀t1,t2 Connected(t1,t2) → Signal(t1) = Signal(t2); ∀g Type(g)=AND → (Signal(Out(1,g)) = 0 ⇔ some input is 0).*
5. **Encode a description of the specific problem instance** — facts for this case. *Type(X1)=XOR, Type(A1)=AND, the wiring of this adder.*
6. **Pose queries to the inference procedure and get answers** — ASK the KB. *What are the outputs for inputs 1, 0, 1? Which input combinations give sum = 0?*
7. **Debug the knowledge base** — wrong/missing answers reveal missing axioms or wrong facts; fix and re-test. *E.g. forgot the axiom 1 ≠ 0 → engine could not prove an output.*

Alternative "expert-system lifecycle" wording (also accepted): identification → conceptualization → formalization → implementation → testing/validation → maintenance.

Participants: domain expert (has the knowledge) · knowledge engineer (elicits & encodes it) · end user. **Acquisition methods:** interviews, protocol analysis (expert thinks aloud), observation, case studies, repertory grids, machine learning from data.

Challenges in building a KB

Challenge	What it means	Mitigation
Knowledge-acquisition bottleneck	experts' knowledge is tacit, hard to articulate; slow and costly (Feigenbaum)	structured interviews, ML/rule induction, reuse ontologies
Consistency	contradictory facts/rules (A and ¬A) → anything can be derived	consistency checking, truth-maintenance systems
Incompleteness	KB can never cover every case; missing rule = no answer	closed-world/default assumptions, fallback to human
Uncertainty & vagueness	real knowledge is probabilistic ("usually", "likely")	certainty factors, Bayesian networks, fuzzy logic
Representation choice	wrong KR ⇒ inefficient or impossible reasoning	pick KR by task (rules vs frames vs FOL)
Scalability & efficiency	thousands of rules ⇒ slow inference	indexing, RETE algorithm, modular KBs
Maintenance / knowledge evolution	domain changes; rules interact in unexpected ways	modular design, versioning, regular validation
Validation & verification	hard to prove the KB is correct	test cases, expert review, debugging step
Common-sense knowledge	huge, implicit (e.g. Cyc project, decades-long effort)	reuse upper ontologies

Exam tip (15-mark answer plan): definition of KB + KB/inference-engine figure (6.2) → 7 steps each with a one-line example → challenges table → 2-line conclusion. That is a full-marks structure.

6.6 Reasoning: forward chaining, backward chaining, resolution

Importance 7/10 · good chance · classic 5/10-mark "compare forward and backward chaining"; MCQs on data-driven vs goal-driven

Plain language: With IF–THEN rules there are two ways to reason. **Forward chaining** starts from known facts and keeps firing rules to produce new facts until the goal appears ("what can I conclude?"). **Backward chaining** starts from the goal and asks "which rule could prove this?", turning its conditions into sub-goals until everything rests on known facts ("is this hypothesis true?").

Worked example. Facts: A, B, E. Rules: R1: A ∧ B → C · R2: C → D · R3: D ∧ E → G. Goal: G.

- *Forward:* {A,B,E} → R1 fires → C → R2 fires → D → R3 fires → **G** ✓.
- *Backward:* G? R3 needs D and E. E is a fact. D? R2 needs C. C? R1 needs A and B — both facts ✓ ⇒ **G proved**.

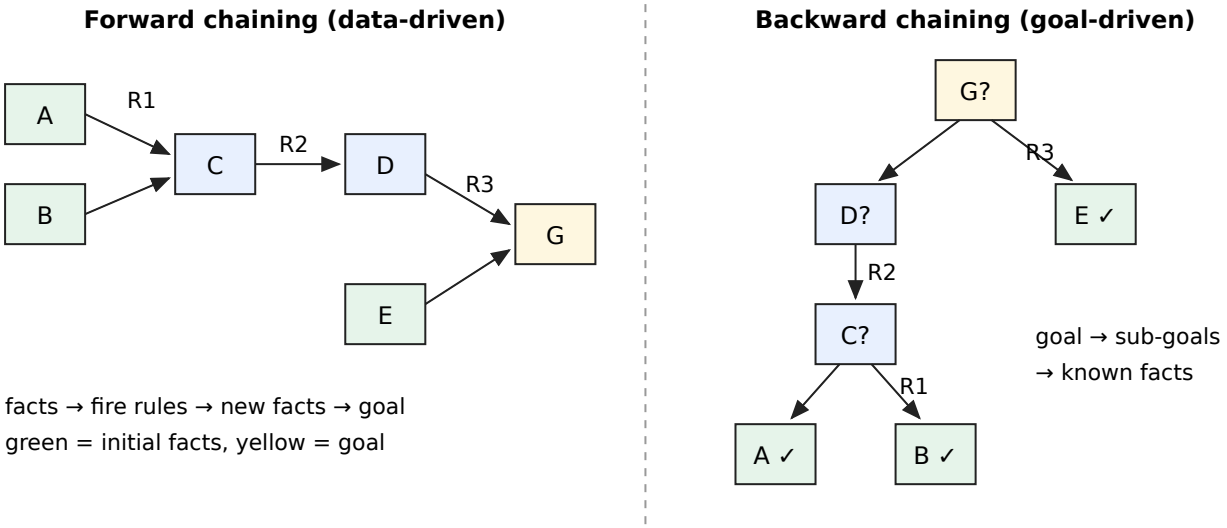


Fig: Same rule base, two directions — forward chaining builds up from facts; backward chaining decomposes the goal into sub-goals

Point	Forward chaining	Backward chaining
Direction	facts → goal (bottom-up)	goal → facts (top-down)
Also called	data-driven	goal-driven
Search	usually breadth-first	usually depth-first
Good when	many possible conclusions, few facts; monitoring, planning, configuration	specific hypothesis to verify; diagnosis
Risk	derives many irrelevant facts	may loop on recursive rules
Examples	XCON/R1, CLIPS, OPS5	MYCIN, PROLOG

Resolution (in brief): a single, **refutation-complete** inference rule for logic in CNF. To prove KB ⊨ α: convert KB ∧ ¬α to clauses, repeatedly resolve pairs of clauses containing complementary literals; deriving the **empty clause** □ means contradiction ⇒ α is proved. Example: KB = {P → Q, P}, prove Q. Clauses: (¬P ∨ Q), (P), negated goal (¬Q). Resolve (¬P ∨ Q) with (P) → (Q); resolve (Q) with (¬Q) → □. ∴ Q. In FOL, resolution uses **unification** (find substitution making literals identical, e.g. {x/Socrates}). PROLOG runs on SLD resolution of Horn clauses.

Exam tip: "PROLOG uses which chaining?" → backward (goal-driven, depth-first). "MYCIN" → backward. "XCON/R1" → forward.

6.7 Learning from observations — the learning agent and forms of learning

Importance 6/10 · moderate · gives the frame for inductive & EBL answers; learning-agent figure is a quick diagram for marks

Plain language: An agent *learns* if it improves its performance on future tasks after observing the world. Russell–Norvig's learning agent has four parts: the **performance element** chooses actions (the "old" agent); the **critic** tells it how well it is doing against a fixed **performance standard**; the **learning element** uses that feedback to improve the performance element; the **problem generator** suggests exploratory actions that lead to informative experiences.

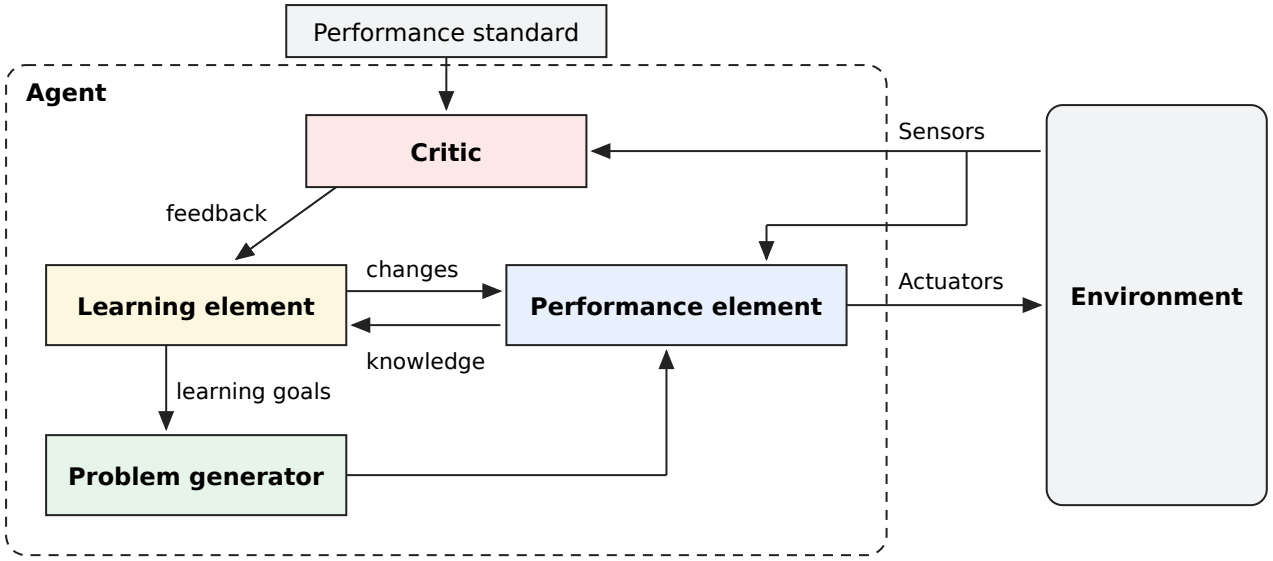


Fig: General learning agent (Russell–Norvig) — critic, learning element, performance element, problem generator

Forms of learning (by feedback available):

Form	Feedback	Example
Supervised	correct output (label) given for each input	spam / not-spam e-mails; inductive learning, decision trees, k-NN
Unsupervised	no labels; find structure	clustering customers (K-means)
Reinforcement	only a reward/penalty, often delayed	game playing, robot navigation (Q-learning)
(Semi-supervised)	few labels + many unlabelled	photo tagging

Other classification by *how much prior knowledge* is used: **inductive** (from examples, little prior knowledge) vs **analytical/deductive** such as **EBL** (from prior domain theory + one example).

6.8 Inductive learning

Importance 9/10 · very likely · syllabus phrase, MCQ in 2025 (decision trees); 5/10-mark definition + example very likely

Plain language: Induction = generalising from specific examples to a general rule. We are given pairs (x, f(x)) of an unknown function f; we must find a **hypothesis h** that approximates f well enough to predict f on *new* x. Example: seeing 100 swans, all white → hypothesise "all swans are white". It is useful but not guaranteed correct (one black swan breaks it) — induction is **not truth-preserving**, unlike deduction.

- **Formal (Russell–Norvig):** given a training set of N examples $(x_1,y_1)\ldots(x_N,y_N)$ where each $y_j = f(x_j)$, find h from the **hypothesis space H** such that $h \approx f$.

- **Consistent hypothesis:** agrees with all training examples. **Generalisation:** performs well on unseen test data — the real goal.
- **Hypothesis space H:** set of all hypotheses the learner may consider (e.g. all straight lines, all polynomials of degree ≤ k, all decision trees). If f ∉ H the problem is **unrealisable**.
- **Ockham's razor:** prefer the **simplest** hypothesis consistent with the data (a straight line that fits 7 points beats a degree-6 polynomial that also fits them).
- **Inductive bias:** the assumptions that make the learner prefer some hypotheses (restriction bias = limit H; preference bias = prefer shorter trees).
- **Trade-off:** expressive H ⇒ more likely to contain f, but harder to search and more prone to **overfitting**.
- **Classification vs regression:** f discrete (classes) vs continuous (numbers).
- **Algorithms:** decision trees (ID3, C4.5), Find-S and candidate-elimination/version spaces (concept learning), neural networks, k-NN, Naive Bayes.

Concept learning example (Mitchell's EnjoySport style): examples ⟨Sunny, Warm, High⟩ → Yes, ⟨Sunny, Warm, Normal⟩ → Yes, ⟨Rainy, Cold, High⟩ → No. Find-S starts with the most specific hypothesis and generalises minimally over positives: h = ⟨Sunny, Warm, ?⟩ ("enjoy sport whenever sunny and warm; humidity irrelevant").

Exam tip: Define inductive learning, write the formal (x, f(x)) → h statement, mention H, Ockham's razor, then give a decision-tree example (6.9). That is a complete 10-mark answer.

6.9 Decision-tree learning (the standard inductive algorithm)

Importance 8/10 · likely · "common algorithm used in inductive learning" was a 2025 MCQ; entropy numericals are easy marks

Plain language: A decision tree tests one attribute at each internal node, branches on its values, and gives the class at the leaves. ID3 (Quinlan, 1986) builds it top-down, greedily choosing at each node the attribute that best separates the classes — the one with the highest **information gain** (largest drop in **entropy**).

- **Entropy** of a set S with proportions p_+, p_-: $\mathbf{H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-}$ (0 for pure, 1 for 50/50).
- **Information gain:** $\mathbf{Gain(S, A) = H(S) - \Sigma_v (|S_v|/|S|) \cdot H(S_v)}$.
- Stop when a node is pure or no attributes remain (majority class). Prune to avoid overfitting. C4.5 uses **gain ratio**; CART uses **Gini index** $1 - \Sigma p_i^2$.

Worked example (8 days; verified with python3):

Day	Outlook	Wind	Play
1	Sunny	Weak	Yes
2	Sunny	Strong	No
3	Sunny	Strong	No
4	Overcast	Weak	Yes
5	Overcast	Strong	Yes
6	Rain	Weak	Yes
7	Rain	Weak	Yes
8	Rain	Strong	No

1. Whole set: 5 Yes, 3 No → $H(S) = -(5/8)\log_2(5/8) - (3/8)\log_2(3/8) = \mathbf{0.9544}$.
2. **Outlook:** Sunny (1Y,2N) H = 0.9183; Overcast (2Y,0N) H = 0; Rain (2Y,1N) H = 0.9183. Weighted = (3/8)(0.9183) + (2/8)(0) + (3/8)(0.9183) = 0.6887 → **Gain = 0.9544 − 0.6887 = 0.2657**.

3. **Wind:** Weak (4Y,0N) H = 0; Strong (1Y,3N) H = 0.8113. Weighted = (4/8)(0) + (4/8)(0.8113) = 0.4056 → **Gain = 0.9544 − 0.4056 = 0.5488**.
4. Wind has the higher gain → **root = Wind**. Weak branch is pure → **Yes**.
5. Strong branch {2,3,5,8}: H = 0.8113; splitting on Outlook gives pure subsets (Overcast → Yes; Sunny → No; Rain → No) → Gain = 0.8113. Done.

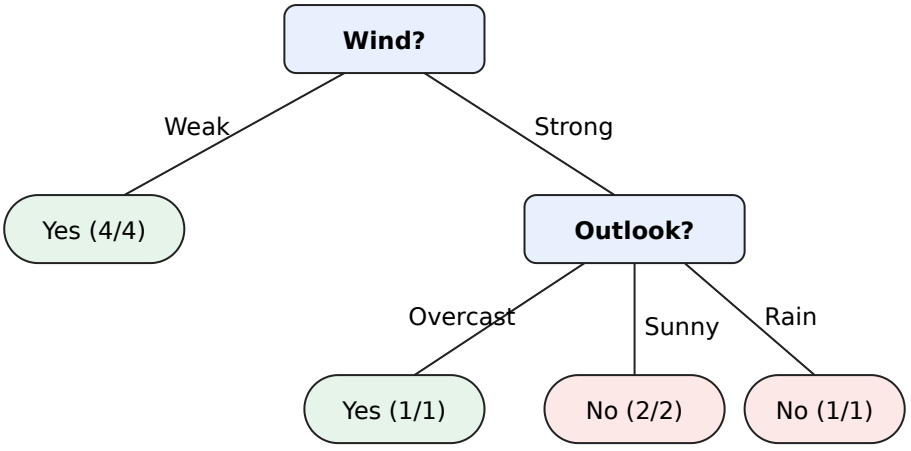


Fig: Decision tree learned by ID3 from the 8-day table (root chosen by information gain 0.5488 > 0.2657)

Rule form: `IF Wind = Weak THEN Yes; IF Wind = Strong AND Outlook = Overcast THEN Yes; ELSE No.`

Exam tip: Memorise H(1/3, 2/3) = 0.918, H(1/4, 3/4) = 0.811, H(1/2, 1/2) = 1 — they appear in most textbook numericals.

6.10 Explanation-based learning (EBL)

Importance 9/10 · very likely · syllabus phrase, 2025 MCQ (advantage = needs far less data); 10-mark "explain EBL with example" likely

Plain language: Inductive learning needs many examples and no theory. EBL is the opposite: the learner already has a **domain theory** (background rules). Shown **one** example, it *explains* (proves) why the example is an instance of the concept, then **generalises the explanation** into an efficient rule it can reuse. It learns "why", not just "what", so it needs only a single example. (Mitchell, Keller & Kedar-Cabelli 1986; DeJong & Mooney 1986.)

Inputs to EBL (Mitchell / Rich & Knight):

1. **Goal concept** — a high-level, non-operational description of what is to be learned (e.g. *Cup*, *SafeToStack(x, y)*).
2. **Training example** — one positive instance described by low-level facts.
3. **Domain theory** — background rules/axioms that can explain why the example satisfies the goal concept.
4. **Operationality criterion** — says which predicates the final rule may use (those cheap to evaluate/observable, e.g. structural features).

Process:

1. **Explain** — use the domain theory to build a proof (explanation tree) that the training example satisfies the goal concept; facts not used in the proof (colour, owner) are thereby identified as **irrelevant**.
2. **Generalise** — regress the goal concept through the proof, replacing constants by variables (goal regression), keeping only conditions the proof needed.
3. **Output** — a new rule whose body is a conjunction of operational predicates: a **sufficient condition** for the goal concept. Added to KB as a "macro-rule"/chunk.

Classic Cup example (Winston et al.)

```
Goal concept : Cup(x) <- Liftable(x) AND Stable(x) AND OpenVessel(x)
Domain theory: Is(x,Light) AND PartOf(y,x) AND Isa(y,Handle)            -> Liftable(x)
               PartOf(y,x) AND Isa(y,Bottom) AND Is(y,Flat)             -> Stable(x)
               PartOf(y,x) AND Isa(y,Concavity) AND Is(y,UpwardPointing)-> OpenVessel(x)
Training ex. : Owner(Obj1,Ravi), Colour(Obj1,Red), Is(Obj1,Light), PartOf(H1,Obj1),
               Isa(H1,Handle), PartOf(B1,Obj1), Isa(B1,Bottom), Is(B1,Flat),
               PartOf(C1,Obj1), Isa(C1,Concavity), Is(C1,UpwardPointing)
Operational  : concept must be described with structural features (Light, PartOf, Isa, Flat
...)
Learned rule : Cup(x) <- Is(x,Light) AND PartOf(y,x) AND Isa(y,Handle)
                         AND PartOf(z,x) AND Isa(z,Bottom) AND Is(z,Flat)
                         AND PartOf(w,x) AND Isa(w,Concavity) AND Is(w,UpwardPointing)
               (Owner and Colour dropped: not used in the explanation)
```

Safe-to-stack example (Mitchell, ML ch. 11) — domain theory: SafeToStack(x,y) ← Lighter(x,y); Lighter(x,y) ← Weight(x,wx) ∧ Weight(y,wy) ∧ LessThan(wx,wy); Weight(x,w) ← Volume(x,v) ∧ Density(x,d) ∧ Equal(w, v×d); Weight(x,5) ← Type(x,Endtable). Training example: a box on an end-table. Learned rule: **SafeToStack(x,y) ← Volume(x,vx) ∧ Density(x,dx) ∧ Type(y,Endtable) ∧ LessThan(vx×dx, 5)**.

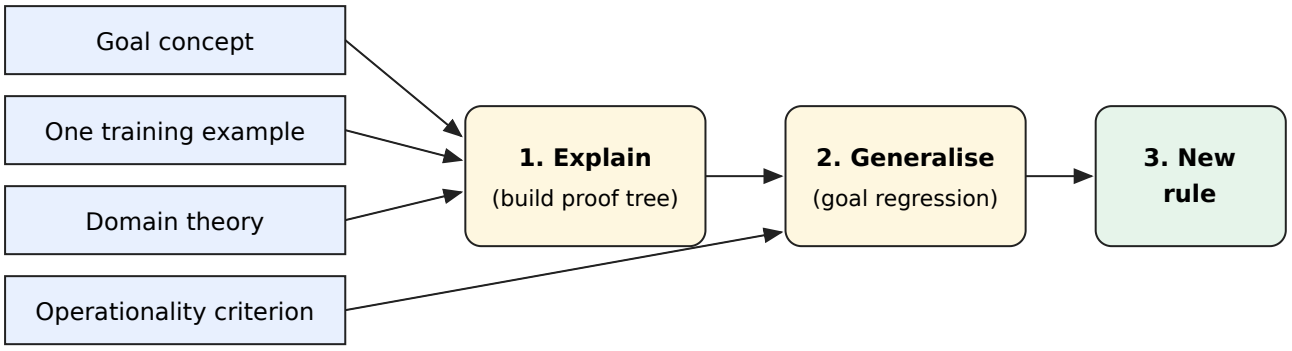


Fig: Explanation-based learning — four inputs, explain → generalise → operational rule

Point	Inductive learning	Explanation-based learning
Starts from	many examples, little prior knowledge	one (or few) example + strong domain theory
Type of inference	induction (generalisation from data)	deduction (proof) + generalisation
Output correctness	may be wrong (not truth-preserving)	correct *if* domain theory is correct
Learns	new knowledge (knowledge-level)	more efficient form of existing knowledge (symbol-level / speed-up)
Irrelevant features	needs many examples to discover them	eliminated by the explanation
Noise sensitivity	statistical methods tolerate noise	sensitive to an incorrect/incomplete theory
Examples	ID3, version spaces, neural nets	Cup, Safe-to-stack, PRODIGY, SOAR chunking

Limitations of EBL: needs a complete, correct domain theory (rare); "imperfect theory problem"; learned rules may accumulate and slow the system (**utility problem**, Minton).

Exam tip (MCQ): EBL's main advantage over induction → *needs far less training data (even one example)*. EBL learns nothing deductively new → *symbol-level learning*.

6.11 Knowledge-level vs symbol-level learning (cross-reference)

Importance 7/10 · good chance · asked as 10-mark Q14 in 2025; full treatment in §7.2

In one line: **symbol-level learning** improves *how efficiently* an agent uses knowledge it already has (EBL, caching, chunking) — its deductive closure does not change; **knowledge-level learning** adds *new knowledge* that was not entailed before (induction, being told new facts). See **§7.2** for definitions, the Dietterich (1986) framing, comparison table and examples.

6.12 Pattern recognition — definition, pipeline, approaches

Importance 7/10 · good chance · frames the four syllabus sub-phrases; pipeline figure is a ready 5-mark answer

Plain language: Pattern recognition (PR) is assigning an input (an image, a sound, a signal) to a category, or describing it, based on regularities in its features. Humans do it effortlessly (recognising a face or a word); machines do it with a pipeline of stages.

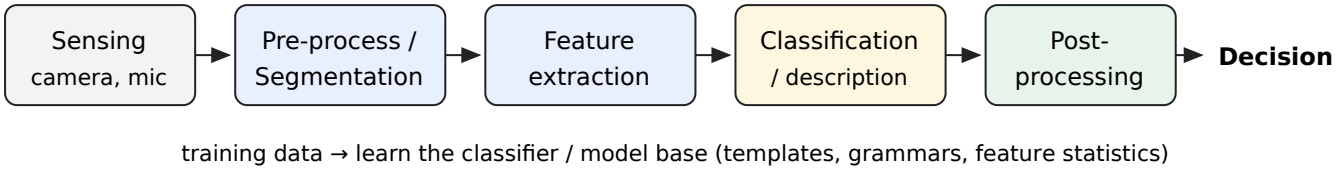


Fig: Pattern-recognition system (Duda–Hart style) — sensing → segmentation → feature extraction → classification → post-processing

- **Sensing:** transducer converts the physical pattern to a signal (pixels, waveform).
- **Segmentation:** separate the object from background / one word from the next — often the hardest step.
- **Feature extraction:** compute distinguishing, invariant measurements (length, area, edges, colour histogram, MFCCs). Good features: same within a class, different across classes, invariant to translation/rotation/scale.
- **Classification:** assign a label using the features (Bayes classifier, k-NN, decision tree, neural net) — or, in structural PR, parse the description.
- **Post-processing:** use context and costs to refine the decision (e.g. spelling context in OCR), reject uncertain cases.

Point	Statistical (decision-theoretic) PR	Structural / syntactic PR
Pattern represented as	feature **vector** of numbers	**primitives + relations** (graph, string, tree)
Recognition by	decision boundaries, probabilities (Bayes, k-NN)	matching / **parsing** with a pattern grammar
Captures structure?	no — only quantitative features	yes — how parts are arranged
Best for	noisy, numeric data; when classes are statistically separable	patterns with rich structure: characters, chromosomes, scenes, circuit diagrams
Learning	estimate parameters from data	grammatical inference / learning descriptions

Third approach: **template matching** (compare with stored prototypes by correlation); modern: **neural/deep** (learn features automatically).

6.13 Structured description

Importance 9/10 · very likely · syllabus phrase; 2025 MCQ answer was "arrangement of shapes and their spatial relationships"

Plain language: Instead of describing an object by a list of numbers, a **structured description** says *what parts it has* and *how those parts are related*. A house drawing is "a triangle **above** a square, **touching** it"; an arch is "two standing blocks that **support** a third, and do **not touch** each other". The description is usually a **graph**: nodes = parts (primitives) with their attributes, edges = relations (above, left-of, supports, touches, part-of).

- Built from **primitives** (lines, arcs, corners, blocks, strokes) plus **relations** (spatial, part-whole, adjacency).
- Representation: **relational graph / semantic network**, attributed graph, tree, or string grammar.
- Recognition = **graph matching** (is the scene's graph isomorphic to a model graph?) or **parsing** with a grammar.
- Learning structural descriptions: **Winston's arch-learning program (1970)** — from positive examples and **near misses** (negative examples differing in one feature) it learned that an arch *must* have support relations and the posts *must not* touch.
- Advantages: captures shape/arrangement, handles hierarchical composition, explainable, can describe never-seen combinations. Disadvantages: primitive extraction is sensitive to noise; graph matching is computationally expensive.

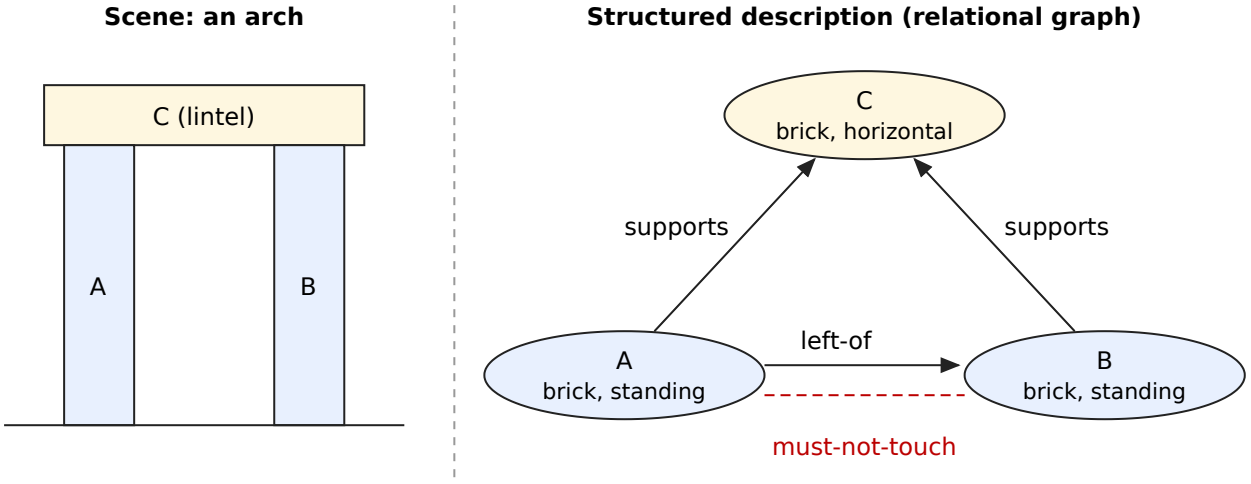


Fig: Structured description of Winston's arch — parts (nodes with attributes) and relations (labelled edges)

Same idea for a **house shape**: nodes {triangle T, square S, rectangle D (door)}; relations above(T,S), touches(T,S), inside(D,S), at-bottom(D,S).

6.14 Symbolic description

Importance 8/10 · likely · syllabus phrase; 2025 MCQ "phoneme sequence is a symbolic description"

Plain language: A **symbolic description** represents a pattern by a set or sequence of **discrete symbols/labels** drawn from a fixed vocabulary, rather than by raw numbers. Once a signal has been turned into symbols, AI's symbolic reasoning (rules, logic, grammars) can work on it. Speech becomes a string of **phonemes** /k/ /æ/ /t/; a handwritten digit's contour becomes a **chain code** 0-0-6-6-4-4-2-2; an image region becomes predicates like `colour(r1, red), shape(r1, round).`

- Produced by **quantising/labelling** features: continuous measurements → symbolic tokens (e.g. "edge", "corner", "vowel").
- Forms: strings (chain codes, phoneme strings), attribute–value lists, predicate-logic facts, labels of nodes in a structured description.
- Used with: string matching / edit distance, formal grammars (syntactic PR), rule-based inference, symbolic learning (decision trees, EBL).
- Advantages: compact, human-readable, supports reasoning and explanation, uses AI KR methods directly. Disadvantages: information lost in quantisation; errors in early labelling propagate.

Point	Numeric (feature-vector) description	Symbolic description	Structured description
Content	real numbers (area = 12.4, mean intensity = 0.37)	discrete symbols/labels (round, red, /a/)	parts + relations among them
Typical form	vector $x \in \mathbb{R}^n$	string, set of predicates	graph / tree
Recognition by	statistical classifier	symbol matching, rules, grammars	graph matching, parsing
Example	colour histogram of an image	phoneme string of a word	"triangle above square"

Exam tip: Structured description ⊂ symbolic in many books (structured = symbolic description *with relations*). In MCQs: *arrangement/spatial relationships* → structured; *sequence of phonemes/labels* → symbolic; *average intensity, number of edges* → numeric/statistical features.

6.15 Object identification (recognition)

Importance 7/10 · good chance · syllabus phrase, unasked; 5-mark "steps of object recognition" plausible

Plain language: Object identification answers "*what* is this object, and where?" in an image. The system finds candidate regions, describes them, and matches the description against a database of known object **models**.

Steps:

1. **Image acquisition** — camera/scanner produces a digital image.
2. **Pre-processing** — noise removal (filtering), contrast enhancement, normalisation.
3. **Segmentation** — separate objects from background: thresholding, **edge detection** (Sobel, Canny), region growing.
4. **Feature extraction / description** — size, area, perimeter, moments, corners, colour, texture; or a structured/symbolic description of parts.
5. **Matching against models** — compare with stored models: template matching, feature matching, graph matching, or a trained classifier.
6. **Verification & labelling** — check hypothesis (pose, consistency), output object identity (+ location).

Approach	Idea
Template matching	slide a stored image of the object; maximum correlation = match. Simple; fails with rotation/scale change
Feature-based	match distinctive local features (corners, SIFT-type descriptors); robust to partial occlusion
Model-based (3-D)	hypothesise object + pose, project the model, verify against image edges
Structural	build relational graph of parts, match with model graphs (links to 6.13)
Learning-based	trained classifier/CNN maps image (patch) → object class

Challenges: viewpoint and pose variation, scale, illumination, **occlusion**, cluttered background, intra-class variation (many kinds of chair), real-time constraints.

6.16 Speech recognition and HMMs

Importance 8/10 · likely · syllabus phrase, unasked as descriptive; 2025 MCQ on HMMs — 10-mark pipeline answer likely

Plain language: Automatic speech recognition (ASR) converts spoken audio into text. The audio is cut into short frames, each frame is summarised by acoustic features (MFCCs); an **acoustic model** gives the probability of those features for each phoneme; a **pronunciation lexicon** says which phonemes make each word; a **language model** says which word sequences are likely; a **decoder** searches for the word sequence W that maximises **P(W | O) ∝ P(O | W) · P(W)** (acoustic × language model).

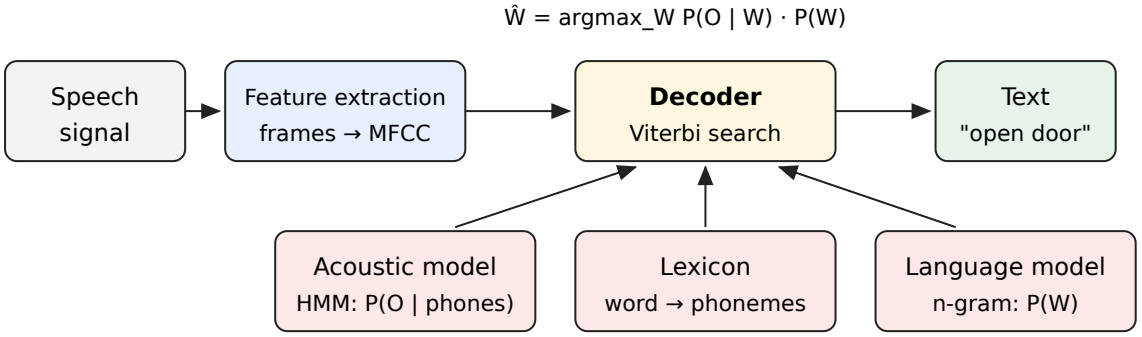
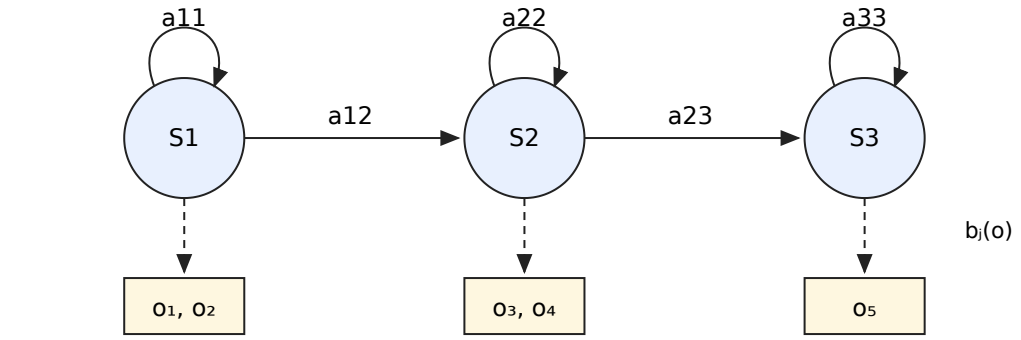


Fig: Classic (HMM-based) speech-recognition pipeline — features, acoustic model, lexicon, language model, decoder

- **Signal processing / feature extraction:** sample (e.g. 16 kHz), split into ~25 ms frames every 10 ms, compute **MFCCs** (Mel-Frequency Cepstral Coefficients — spectrum on the Mel scale that mimics human hearing; ~13 coefficients + deltas per frame).
- **Acoustic model:** P(features | phoneme); classically an **HMM per phoneme** (3 left-to-right states) with Gaussian-mixture emissions (GMM-HMM); modern systems use DNNs.
- **Pronunciation lexicon:** "cat" → /k æ t/.
- **Language model:** P(word sequence), e.g. bigram/trigram: P(door | open the) is high, learned from text corpora.
- **Decoder:** search (Viterbi, beam search) for the best word sequence.
- **Types:** isolated-word vs continuous speech; speaker-dependent vs speaker-independent; small vs large vocabulary.
- **Evaluation: Word Error Rate** WER = (S + D + I)/N (substitutions, deletions, insertions over reference words).
- **Challenges:** speaker variability (accent, gender, speed), **co-articulation** (sounds blend into neighbours), continuous speech has no pauses between words (segmentation), background noise & channel, homophones ("right/write"), large vocabularies, out-of-vocabulary words, spontaneous speech disfluencies.

Hidden Markov Model (HMM) basics. A Markov process whose states are **hidden**; we only see **observations** emitted by states. Defined by **λ = (A, B, π)**: A = state-transition probabilities a_{ij}, B = emission probabilities $b_j(o)$, π = initial-state distribution. Markov assumption: next state depends only on current state.

Three classic problems (Rabiner): **(1) Evaluation** P(O | λ) — **forward algorithm**; **(2) Decoding** most likely hidden state sequence — **Viterbi algorithm**; **(3) Learning** estimate λ from data — **Baum–Welch (EM)** algorithm. In ASR: hidden states = phoneme sub-states, observations = MFCC frames.



hidden states (phone beginning / middle / end) emit observable MFCC frames

Fig: 3-state left-to-right phoneme HMM — transitions a_{ij} (self-loops model duration), emissions $b_j(o)$

Exam tip (MCQ): HMMs → historically central to **speech recognition**; Viterbi → decoding; Baum–Welch → training; MFCC → speech features.

6.17 LISP

Importance 8/10 · likely · syllabus phrase; 5-mark "features of LISP" or MCQ "car/cdr output" likely

Plain language: LISP (**LIS**t **P**rocessing) was invented by **John McCarthy in 1958** at MIT (published 1960) — the second-oldest high-level language still in use (after FORTRAN). Everything — data *and* programs — is a **list** written in parentheses, and functions are applied in **prefix** form: `(+ 2 3)` means 2 + 3. Because programs are lists, a LISP program can build and run other programs, which made it the dominant AI language for decades.

- **Atoms:** numbers (42), symbols (APPLE, X). **Lists:** `(A B C)`, `(1 (2 3) 4)`, empty list `()` = `NIL` (also means false; `T` = true).
- **S-expression** (symbolic expression) = an atom or a list of S-expressions. Program = S-expressions evaluated by the **REPL** (read–eval–print loop).
- **Prefix (Polish) notation:** `(* (+ 2 3) 4)` → 20.
- **Quote** stops evaluation: `'(A B C)` is data, not a function call.
- **Core list functions:**

```
(car '(a b c))          ; => A          first element   (Contents of Address part of
Register)
(cdr '(a b c))          ; => (B C)      rest of list    (Contents of Decrement part of
Register)
(cons 'a '(b c))        ; => (A B C)    build a new cell
(car (cdr '(a b c)))    ; => B          also written (cadr '(a b c))
(list 1 2 3)            ; => (1 2 3)
(append '(1 2) '(3 4))  ; => (1 2 3 4)
(atom 'x)  ; => T       (null '())  ; => T       (eq 'a 'a) ; => T
(setq x 10)             ; assign
(cond ((> x 0) 'positive) ((< x 0) 'negative) (t 'zero))   ; => POSITIVE
```

- **Functions with `defun`, recursion is the natural style:**

```
(defun factorial (n)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))
(factorial 5)            ; => 120   (5*4*3*2*1)

(defun my-length (lst)                ; length of a list, recursive on cdr
  (if (null lst) 0 (+ 1 (my-length (cdr lst)))))
(my-length '(a b c d))   ; => 4
```

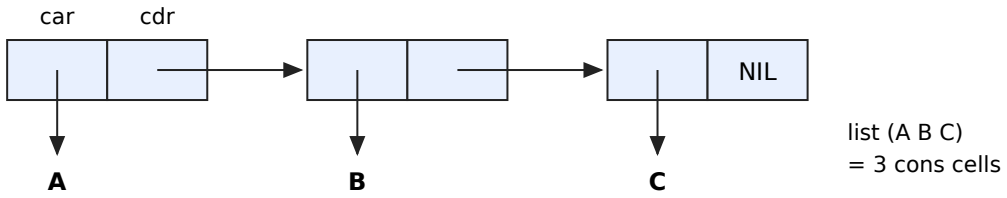


Fig: Box-and-pointer view of (A B C) — each cons cell holds car (element) and cdr (rest); the last cdr is NIL

Features: symbolic computation · lists as the single data structure · **homoiconic** (code = data) → powerful **macros** · recursion · **automatic garbage collection** (LISP introduced it) · dynamic typing · interactive REPL / interpreted (and compiled) · higher-order functions (`mapcar`, `lambda`) · conditional `cond` (LISP introduced the if-then-else expression).

Dialects: Common LISP, Scheme, Emacs Lisp, Clojure. **Applications:** early AI programs and expert systems, natural-language processing, symbolic algebra (Macsyma), theorem provers, AutoCAD (AutoLISP), Emacs.

6.18 PROLOG

Importance 9/10 · very likely · syllabus phrase, 2025 MCQ (application = expert systems/KR); family-tree query output is a favourite

Plain language: PROLOG (**PRO**gramming in **LOG**ic) was created by **Alain Colmerauer and Philippe Roussel at Marseille in 1972**, building on **Robert Kowalski's** procedural interpretation of logic. It is **declarative**: you state **facts** and **rules** (Horn clauses) about a problem, then ask **queries**; PROLOG's built-in inference engine (backward chaining with **unification** and **backtracking**) finds the answers. You describe *what* is true, not *how* to compute it.

- **Facts:** `parent(tom, bob).` — constants start lowercase; every clause ends with a full stop.
- **Rules:** `head :- body.` read "head is true **if** body is true"; `,` = AND, `;` = OR. Variables start with an uppercase letter or `_`.
- **Queries:** `?- goal.` → PROLOG answers `true/false` or variable bindings; type `;` for more answers.
- **Unification:** pattern-matching that makes two terms identical by binding variables: `parent(X, ann)` unifies with `parent(bob, ann)` with X = bob.
- **Backtracking:** if a sub-goal fails, PROLOG goes back to the most recent choice point and tries the next matching clause — depth-first search, clauses tried top-to-bottom, goals left-to-right.
- **Closed-world assumption / negation as failure:** `\+ G` succeeds if G cannot be proved.
- Lists `[H|T]`, recursion, arithmetic with `is` (`X is 2+3`), cut `!` to prune backtracking.

Family-tree example

```
% facts
parent(tom, bob).   parent(tom, liz).
parent(bob, ann).   parent(bob, pat).
parent(pat, jim).
female(liz). female(ann). female(pat).

% rules
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
mother(X, Y)      :- parent(X, Y), female(X).
sibling(X, Y)     :- parent(Z, X), parent(Z, Y), X \= Y.
ancestor(X, Y)    :- parent(X, Y).
ancestor(X, Y)    :- parent(X, Z), ancestor(Z, Y).     % recursion

% queries and answers
?- parent(X, ann).          % X = bob.
?- grandparent(tom, W).     % W = ann ;  W = pat.     (tom->bob->ann, tom->bob->pat; tom-
>liz->none)
?- grandparent(bob, jim).   % true.                  (bob->pat->jim)
?- sibling(ann, S).         % S = pat.
?- mother(M, jim).          % M = pat.
?- ancestor(tom, jim).      % true.                  (tom->bob->pat->jim)
```

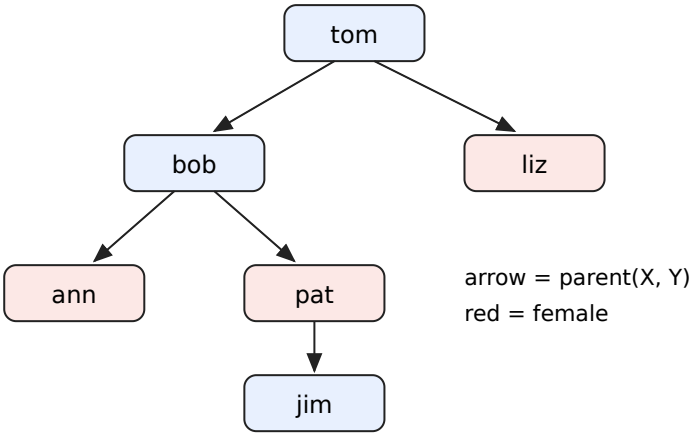


Fig: Family tree encoded by the parent/2 facts above

Features: declarative/logic programming · Horn clauses · built-in unification + backtracking (depth-first SLD resolution) · pattern matching · recursion · symbolic processing · relations are reversible (same `parent/2` answers "who is bob's child?" and "who is ann's parent?").

Applications: expert systems & knowledge representation, natural-language processing (Colmerauer's original aim), theorem proving, deductive databases (Datalog), rule engines, Japanese Fifth Generation Computer project (1982), parts of IBM Watson's NLP.

Point	LISP	PROLOG
Inventor / year	John McCarthy, MIT, 1958	Colmerauer & Roussel, Marseille, 1972 (Kowalski)
Paradigm	**functional** (applicative)	**logic / declarative**
Basic unit	function, list (S-expression)	fact, rule (Horn clause), query
Control	programmer specifies how (recursion, cond)	built-in backward chaining, unification, backtracking
Syntax	prefix, fully parenthesised `(+ 2 3)`	`head :- body.`
Data structure	lists (cons cells)	terms, lists (head + tail notation)
Strength	symbolic manipulation, macros, code-as-data	pattern matching, search, rule-based reasoning
Popular in	USA (MIT, Stanford)	Europe & Japan
Typical uses	AI research, NLP, symbolic maths, Emacs/AutoCAD	expert systems, KR, NLP parsing, theorem proving

Exam tip (MCQ): PROLOG's inference = **backward chaining + depth-first search with backtracking**; `:-` means "if"; variables uppercase. LISP `(car (cdr '(a b c)))` = **B**; `(cdr '(a))` = **NIL**.

7. Machine Learning

Syllabus (verbatim): *Knowledge-level vs symbol-level learning. Supervised Learning: Distance-based methods, Nearest-Neighbours. Unsupervised Learning Clustering: K-means / Kernel K-means. Statistical Learning Theory: Ensemble Methods (Boosting, Bagging, Random Forests).*

2025 asked: **Q14 (10 marks) knowledge-level vs symbol-level learning** (counted under AI in the paper) and **Q15 (10 marks) k-NN + factors affecting performance**, plus MCQs on knowledge-level learning, curse of dimensionality and k-NN as instance-based. Still unasked: **K-means / kernel K-means** and **ensembles (bagging, boosting, random forest)** — the strongest 2026 predictions.

7.1 What is machine learning? Types

Importance 7/10 · good chance · intro for every ML answer; "supervised vs unsupervised" is a ready 5-mark question

Plain language: ML is building programs that improve automatically from data/experience instead of being explicitly programmed with rules. You show a spam filter thousands of labelled e-mails; it works out the rule itself.

- **Mitchell (1997):** "A computer program is said to **learn** from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in T, as measured by P, improves with experience E."
- Example: T = classify e-mail as spam/not; P = % correctly classified; E = a corpus of labelled e-mails. Checkers: T = play checkers, P = % games won, E = games played against itself.
- **Arthur Samuel (1959):** "field of study that gives computers the ability to learn without being explicitly programmed".

Type	Data	Goal	Examples
Supervised	labelled pairs (x, y)	learn mapping x → y	k-NN, decision tree, SVM, linear/logistic regression
Unsupervised	unlabelled x only	find structure/groups	K-means, kernel K-means, hierarchical clustering, PCA
Semi-supervised	few labels + many unlabelled	use both	label propagation
Reinforcement	rewards from interaction	learn a policy maximising reward	Q-learning, game playing

Point	Supervised	Unsupervised
Labels	required (teacher)	not required
Task	classification, regression	clustering, association, dimensionality reduction
Evaluation	accuracy, precision, recall, RMSE vs known labels	harder: SSE, silhouette, domain judgement
Example	predict loan default from history	segment customers by buying pattern

7.2 Knowledge-level vs symbol-level learning

Importance 7/10 · good chance · asked as 10-mark Q14 in 2025 + MCQ 98; repeats are less likely but it is a pure syllabus phrase

Plain language: Ask: *after learning, does the system know anything it did not already (logically) know?* If **no** — it just became **faster** or more efficient at using what it knew — that is **symbol-level learning**. If **yes** — it can now derive conclusions it could not derive before — that is **knowledge-level learning**.

Background: Allen **Newell (1982)** separated the **knowledge level** (what an agent knows, its goals — independent of representation) from the **symbol level** (the actual data structures and algorithms that encode that knowledge). **Thomas Dietterich (1986), "Learning at the Knowledge Level"**, used this to classify learning systems:

- **Symbol-level learning (SLL):** changes the **symbol-level representation** to improve performance (speed, memory, search efficiency) **without changing the knowledge-level content** — the **deductive closure** (set of everything derivable) of the KB stays the same. Anything learned was already *entailed*.
- **Knowledge-level learning (KLL):** changes the **knowledge content** — the deductive closure **grows** (or changes). The system acquires new facts/hypotheses not entailed by what it had.

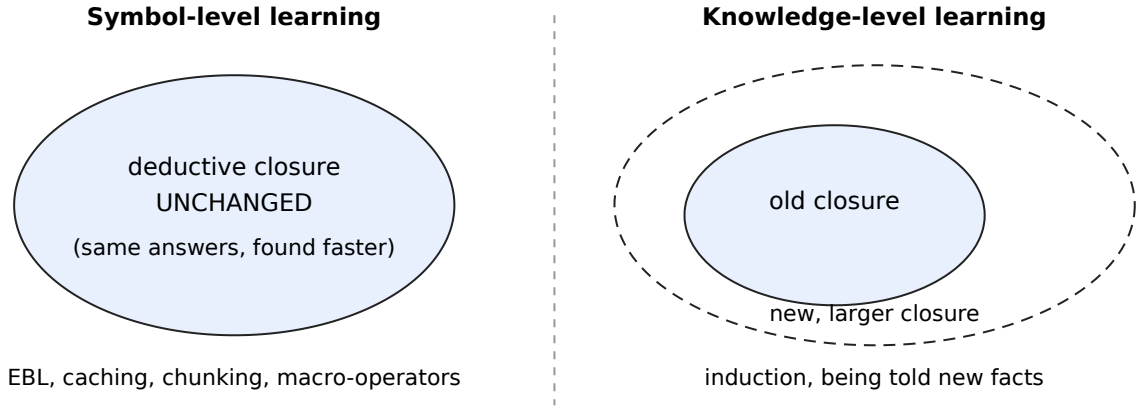


Fig: Symbol-level learning re-encodes the same knowledge more efficiently; knowledge-level learning enlarges what can be derived

Point	Symbol-level learning	Knowledge-level learning
What changes	representation / efficiency	knowledge content (what is believed)
Deductive closure	unchanged	grows / changes
Inference type	**deductive** (analytical)	mainly **inductive** (also learning by being told, analogy)
Effect	speed-up, less search, less memory	new conclusions, better coverage of new situations
Correctness	preserved (if original KB correct)	may be wrong (induction is not truth-preserving)
Needs	strong domain theory	data / examples / new information
Examples	**EBL** (Cup rule), **caching/memoisation** of derived results, **chunking** in SOAR, macro-operators (STRIPS MACROPS), compiling rules, indexing	**inductive learning** (decision trees, k-NN, concept learning), learning from a teacher/being told, rote learning of new facts, knowledge acquisition from experts
Also called	speed-up learning	"true" learning / knowledge acquisition

Worked illustration: a chess program that knows the rules can in principle deduce the best move in any position (closure contains it), but it is too slow. If it stores ("caches") the solved endgame positions or learns macro-rules via EBL, it plays faster but knows nothing new → **symbol-level**. A medical system that, from 1000 patient records, induces "fever ∧ rash ∧ recent travel → dengue likely" now believes something it could not deduce before → **knowledge-level**.

Notes: Dietterich observed that most "analytical" learning of the 1980s (EBL) was symbol-level, while empirical learning (induction) is knowledge-level. EBL *can* become knowledge-level if the domain theory is incomplete or if generalisation goes beyond strict deduction.

Exam tip: In 2025 MCQ 98 the keyed answer for "best represents knowledge-level learning" was *encoding expert medical knowledge into a rule-based system to reason logically* — i.e. **adding new knowledge** to the KB. Remember the test: *new knowledge = knowledge level; faster use of old knowledge = symbol level*.

7.3 Supervised learning: training, testing, classification vs regression

Importance 6/10 · moderate · background for k-NN; short-note material

Plain language: In supervised learning a "teacher" provides the right answer for every training example. The algorithm fits a model to the **training set**, and we check how well it generalises on a separate **test set** it never saw.

- **Training set:** data used to fit the model. **Validation set:** used to tune hyper-parameters (e.g. k in k-NN). **Test set:** used once at the end to estimate real-world error. Typical split 70/15/15 or 80/20.
- **k-fold cross-validation:** split data into k parts; train on k−1, test on 1; rotate; average — better estimate when data are scarce.
- **Classification:** output is a discrete class (spam/ham, disease A/B/C). **Regression:** output is a continuous value (house price, temperature).
- Workflow: collect data → pre-process (clean, normalise) → choose features → choose model → train → validate/tune → test → deploy.

7.4 Distance-based methods and distance measures

Importance 8/10 · likely · syllabus phrase; formulas + small numericals are easy marks (5-mark or MCQ)

Plain language: Distance-based methods assume **similar things are close together** in feature space. To classify or cluster, they measure how far a point is from others. The choice of distance measure changes what "close" means. Used in k-NN, K-means, hierarchical clustering, case-based reasoning.

For points $p = (p_1 \dots p_n)$ and $q = (q_1 \dots q_n)$:

Measure	Formula	Use
Euclidean (L2)	$d = \sqrt{\Sigma(p_i - q_i)^2}$	default for continuous features; straight-line distance
Manhattan / city-block (L1)	$d = \Sigma\|p_i - q_i\|$	grid-like movement; less sensitive to outliers
Minkowski (Lp)	$d = (\Sigma\|p_i - q_i\|^p)^{\wedge}(1/p)$	general family: p = 1 Manhattan, p = 2 Euclidean
Chebyshev ($L\infty$)	$d = \max_i \|p_i - q_i\|$	limit $p \to \infty$; chess king moves
Hamming	number of positions where symbols differ	binary/categorical strings
Cosine similarity	$\cos\theta = (p \cdot q)/(\|p\|\|q\|)$; distance = $1 - \cos\theta$	text/documents (direction matters, not magnitude)

Worked example (verified with python3): p = (1, 2, 3), q = (4, 6, 3).

- Euclidean = $\sqrt{(3^2 + 4^2 + 0^2)} = \sqrt{25} =$ **5**.
- Manhattan = 3 + 4 + 0 = **7**. Chebyshev = max(3, 4, 0) = **4**.
- Cosine: $p \cdot q = 4 + 12 + 9 = 25$; $\|p\| = \sqrt{14}$, $\|q\| = \sqrt{61}$; $\cos\theta = 25/\sqrt{854} =$ **0.8555**.
- Hamming("10110", "11100") = positions 2 and 4 differ = **2**.

A valid **metric** satisfies: $d(p,q) \geq 0$, $d(p,q) = 0$ iff $p = q$, symmetry $d(p,q) = d(q,p)$, triangle inequality $d(p,r) \leq d(p,q) + d(q,r)$. (Cosine distance is not a true metric.)

Feature scaling is essential: features on large scales (salary in ₹) swamp small ones (age). **Min–max:** $x' = (x - \min)/(\max - \min)$ → e.g. age 30 with range 20–60 → $(30 - 20)/40 =$ **0.25**. **Z-score:** $x' = (x - \mu)/\sigma$.

7.5 k-Nearest Neighbours (k-NN)

Importance 8/10 · likely · 10-mark Q15 in 2025 + 2 MCQs; core algorithm, may come again as 5-mark or a numerical

Plain language: To classify a new point, find the **k training points closest to it** and let them **vote**; the majority class wins. For regression, average their values. There is no training phase — the algorithm simply stores the data and does all the work at query time. ("Tell me who your neighbours are and I'll tell you who you are.") Proposed by Fix & Hodges (1951), analysed by Cover & Hart (1967).

Algorithm:

1. Choose k and a distance measure; normalise features.
2. For the query point x_q, compute the distance to every training example.
3. Sort and select the **k nearest** neighbours.
4. **Classification:** return the majority class among the k (ties: choose odd k, or weight by distance). **Regression:** return the mean (or distance-weighted mean) of their values.

Worked example (verified with python3). Training points: A(1,1) Red, B(2,4) Red, C(3,0) Red, E(5,3) Blue, F(4,5) Blue, G(4,7) Blue, D(6,6) Blue. Query Q = (3, 3). Euclidean distance squared $d^2 = (x-3)^2 + (y-3)^2$:

Rank	Point	Class	d^2	d
1	B(2,4)	Red	1 + 1 = 2	1.414
2	E(5,3)	Blue	4 + 0 = 4	2.000
3	F(4,5)	Blue	1 + 4 = 5	2.236
4	A(1,1)	Red	4 + 4 = 8	2.828
5	C(3,0)	Red	0 + 9 = 9	3.000
6	G(4,7)	Blue	1 + 16 = 17	4.123
7	D(6,6)	Blue	9 + 9 = 18	4.243

- k = 1 → {B} → **Red**.
- k = 3 → {B, E, F} = 1 Red, 2 Blue → **Blue**.
- k = 5 → {B, E, F, A, C} = 3 Red, 2 Blue → **Red**.
- **Distance-weighted ($w = 1/d^2$), k = 3:** Red = 1/2 = 0.50; Blue = 1/4 + 1/5 = 0.45 → **Red** (the very close B outweighs two farther Blues).

The answer changes with k — exactly why "choice of k" is the #1 factor in exam answers.

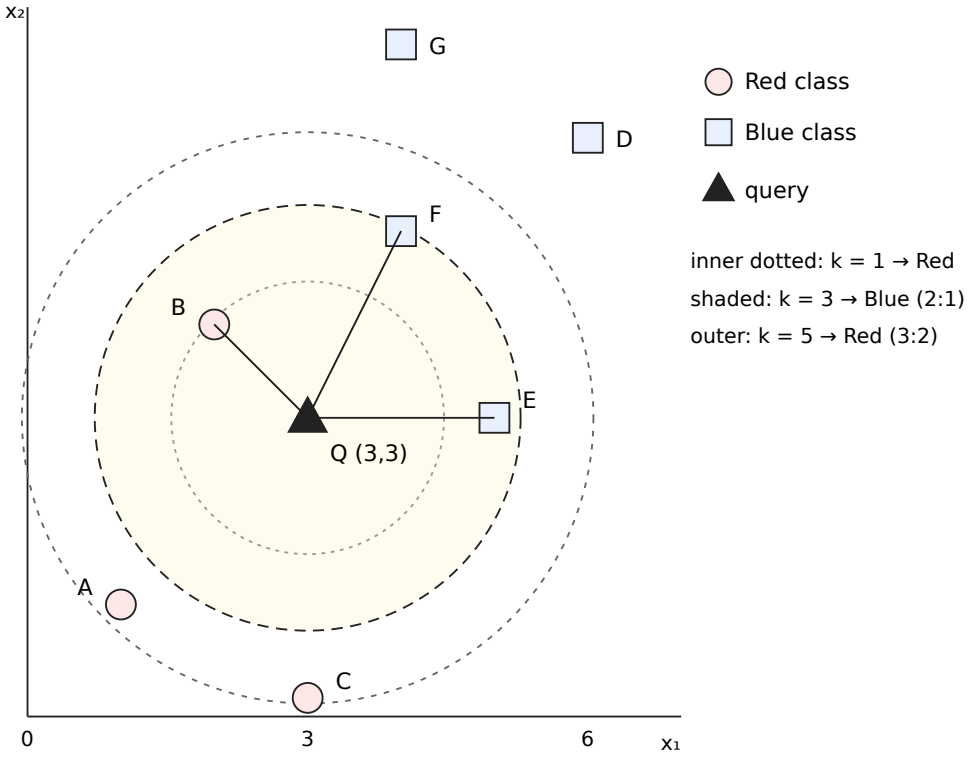


Fig: k-NN — the predicted class of Q depends on k (circles enclose the 1, 3 and 5 nearest neighbours)

Factors affecting k-NN performance (2025 Q15 asked exactly this):

Factor	Effect / rule of thumb
Value of k	small k (k = 1) → sensitive to noise, overfits (high variance); large k → over-smooth, majority class dominates (high bias). Choose by cross-validation; heuristic k ≈ √N
Odd k	for two classes, odd k avoids tied votes
Distance metric	Euclidean for continuous, Hamming for categorical, cosine for text; wrong metric → wrong neighbours
Feature scaling	unscaled large-range features dominate the distance → always normalise (min–max / z-score)
Irrelevant/redundant features	add noise to distance → feature selection / weighting
Curse of dimensionality	in high dimensions points become sparse and almost equidistant, so "nearest" loses meaning; need exponentially more data → use dimensionality reduction (PCA)
Class imbalance	majority class wins votes → distance weighting, resampling
Noisy / mislabelled data & outliers	hurt small k especially
Dataset size	prediction cost O(N·d) per query → KD-trees, ball trees, approximate NN

- **Weighted k-NN:** each neighbour votes with weight $w_i = 1/d(x_q, x_i)^2$ (or 1/d), so nearer points count more; reduces sensitivity to k.
- **Nature: lazy learner** (no model built at training time), **instance-based / memory-based**, **non-parametric**, decision boundary is piecewise linear (Voronoi cells for k = 1).

Pros	Cons
Simple, intuitive, no training time	slow prediction, high memory (stores all data)
Naturally multi-class; works for regression too	sensitive to k, scaling, irrelevant features
Non-linear boundaries; adapts as new data arrive	curse of dimensionality
Few assumptions about data distribution	no interpretable model; imbalanced data issues

Applications: recommender systems (similar users), handwriting/digit recognition, medical diagnosis by similar cases, anomaly detection, imputing missing values.

Exam tip (MCQ): k-NN is **instance-based (memory-based, lazy)** learning; key high-dimensional problem = **curse of dimensionality**; k = 1 → overfitting; very large k → underfitting.

7.6 Unsupervised learning and clustering

Importance 7/10 · good chance · lead-in to K-means; "what is clustering, applications, types" is a 5-mark candidate

Plain language: With no labels, the algorithm must discover structure on its own. **Clustering** groups data so that points in the same cluster are **similar** (high intra-cluster similarity) and points in different clusters are **dissimilar** (low inter-cluster similarity).

- **Types:** partitional (K-means, K-medoids) · hierarchical (agglomerative bottom-up / divisive top-down; dendrogram) · density-based (DBSCAN — arbitrary shapes, finds noise) · model-based (Gaussian mixture via EM) · kernel/spectral (kernel K-means).
- **Hard vs soft:** each point in exactly one cluster (K-means) vs membership probabilities (GMM, fuzzy c-means).
- **Applications:** customer/market segmentation, document/news grouping, image segmentation & colour quantisation, anomaly/fraud detection, gene-expression grouping, grouping cell-tower usage.
- **Evaluation:** within-cluster SSE (inertia), silhouette coefficient, Davies–Bouldin index; external indices if true labels exist.

7.7 K-means clustering

Importance 10/10 · sure to come · syllabus phrase never asked yet; algorithm + worked example is the most predictable 10-mark ML question

Plain language: Pick k "centres". Assign every point to its nearest centre. Move each centre to the average of the points assigned to it. Repeat until nothing changes. Each cluster is represented by its mean (**centroid**). It minimises the **within-cluster sum of squared errors**: $\mathbf{J = \Sigma_j \Sigma_{x \in C_j} \|x - \mu_j\|^2}$. (Lloyd's algorithm; MacQueen 1967 named it k-means.)

Algorithm:

1. Choose k; initialise k centroids (random points, or k-means++).
2. **Assignment step:** assign each point to the cluster with the nearest centroid (Euclidean distance).
3. **Update step:** recompute each centroid as the mean of its assigned points.
4. Repeat 2–3 until assignments (centroids) no longer change, or J stops decreasing, or max iterations reached.

Each step never increases J ⇒ guaranteed to converge, but only to a **local** minimum. Complexity O(n·k·d·t) per run (t iterations).

Worked example (1-D, k = 2; verified with python3). Data {2, 4, 10, 12, 3, 20, 30, 11, 25}; initial centroids $m_1 = 2$, $m_2 = 4$ (ties go to m_1).

Iter	Centroids (m_1, m_2)	Cluster 1	Cluster 2	New centroids
1	2, 4	{2, 3}	{4, 10, 12, 20, 30, 11, 25}	2.5, 16
2	2.5, 16	{2, 4, 3}	{10, 12, 20, 30, 11, 25}	3, 18
3	3, 18	{2, 4, 10, 3}	{12, 20, 30, 11, 25}	4.75, 19.6
4	4.75, 19.6	{2, 4, 10, 12, 3, 11}	{20, 30, 25}	7, 25
5	7, 25	{2, 4, 10, 12, 3, 11}	{20, 30, 25}	7, 25 — **no change, stop**

Check of iteration 1: point 3 is equidistant from $m_1 = 2$ and $m_2 = 4$ (distance 1 each) → tie goes to m_1 (stated rule); point 4 is at distance 0 from m_2. Mean of {4,10,12,20,30,11,25} = 112/7 = 16. Final clusters **{2, 3, 4, 10, 11, 12}** (μ = 7) and **{20, 25, 30}** (μ = 25). Final SSE J = (25 + 16 + 9 + 9 + 16 + 25) + (25 + 0 + 25) = 100 + 50 = **150**.

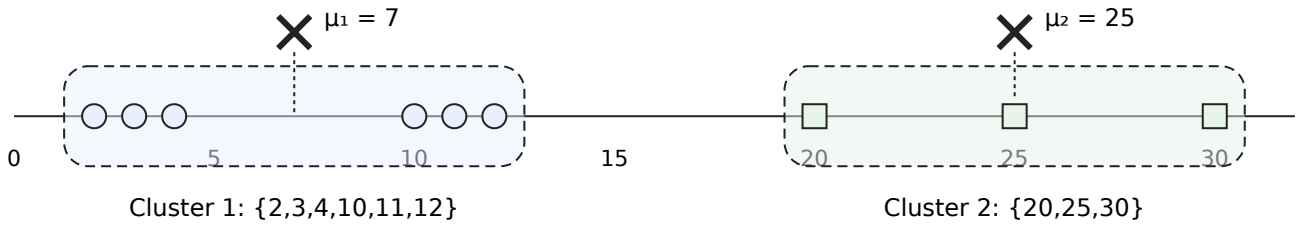


Fig: Converged K-means (k = 2) on the 1-D data — × marks the centroids

Choosing k — elbow method: run K-means for k = 1, 2, 3, … and plot SSE; pick the "elbow" where adding clusters stops giving a big drop. For the data above the best SSE values (python3, optimal partitions) are k=1: 798, k=2: 150, k=3: 54, k=4: 16.5, k=5: 4 — the sharp bend is at **k = 2** (798 → 150; later drops are small). Other methods: silhouette score, gap statistic, domain knowledge.

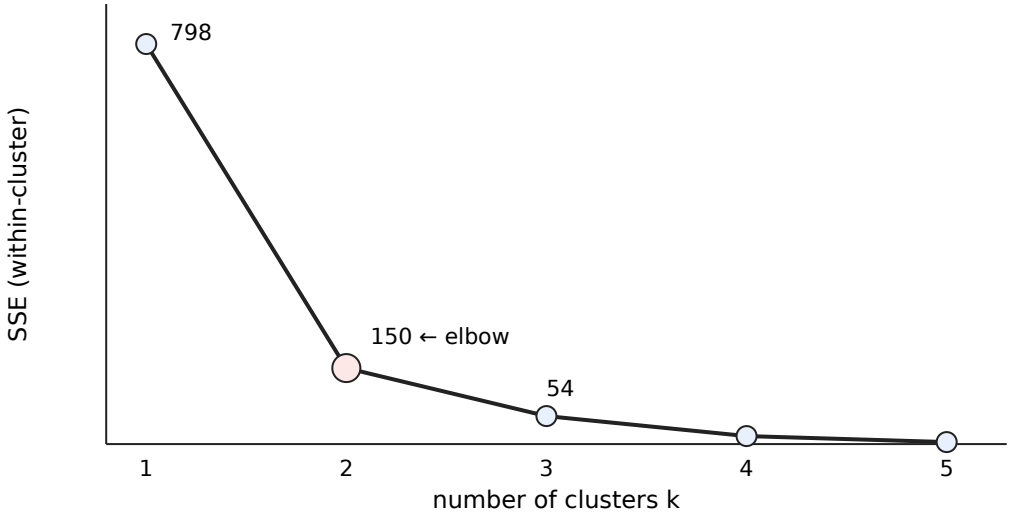


Fig: Elbow plot for the worked data — SSE falls sharply up to k = 2, then flattens

Advantages	Limitations
Simple, fast, scales to large data: O(n·k·d·t)	k must be chosen in advance
Converges in few iterations	result depends on initial centroids (local minima) → run several times / k-means++
Easy to interpret (centroids)	assumes spherical, similar-size, similar-density clusters
Works well for compact, well-separated groups	sensitive to outliers (mean is pulled) → K-medoids
	only numeric data with meaningful mean; needs scaling
	cannot find non-convex clusters (rings, moons) → kernel K-means / DBSCAN

Exam tip: Always write the objective J, the two steps (assignment, update), the stopping criterion, a small worked table and one figure. Then 4 limitations — the last one leads naturally to kernel K-means.

7.8 Kernel K-means

Importance 8/10 · likely · named in the syllabus; usually asked together with K-means ("explain K-means and kernel K-means")

Plain language: Ordinary K-means draws straight (linear) boundaries between clusters, so it fails when one cluster surrounds another, like two **concentric rings**. Kernel K-means first (implicitly) maps the data into a higher-dimensional feature space with a function $\boldsymbol{\varphi}$ where the clusters become linearly separable, then runs K-means there. Example: map each 2-D point (x_1, x_2) to its squared radius $r^2 = x_1^2 + x_2^2$ — inner-ring points get small values, outer-ring points large values, and a simple threshold separates them.

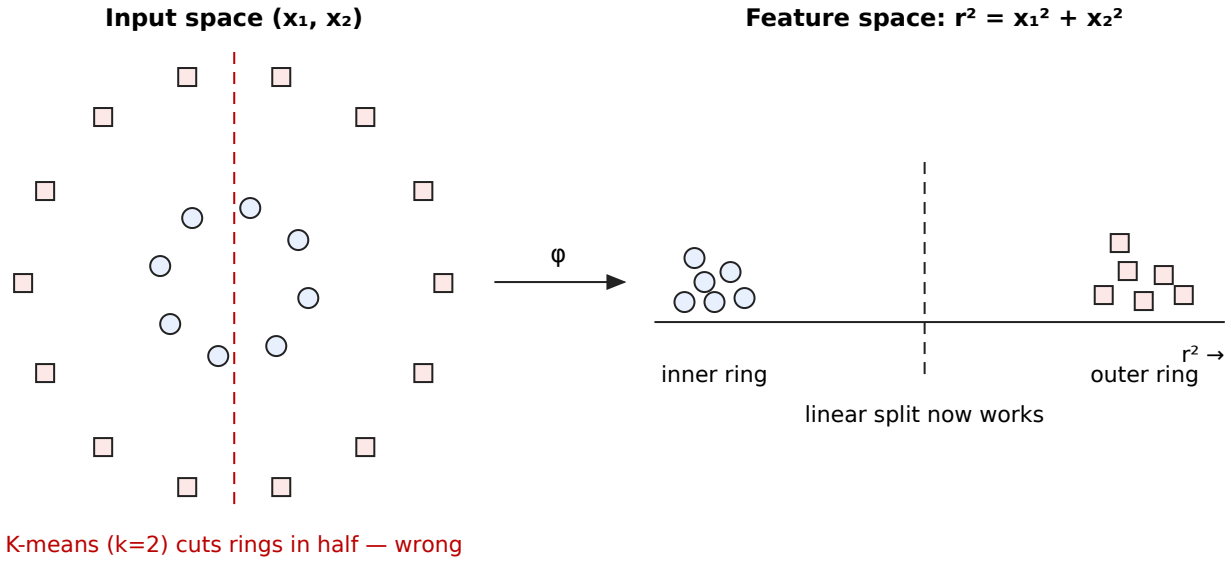


Fig: Concentric rings — not linearly separable in input space; after mapping φ they separate, which kernel K-means exploits

- **Kernel trick:** we never compute φ(x) explicitly. K-means only needs distances, and $\|\varphi(x) - \varphi(y)\|^2$ can be written entirely with dot products $\varphi(x)\cdot\varphi(y) =$ **K(x, y)**, a **kernel function**.
- **Distance of a point to a cluster centre in feature space** (cluster C with centre μ_C = mean of $\varphi(x_j)$, $x_j \in C$):
  $\mathbf{\|\varphi(x) - \mu_C\|^2 = K(x,x) - (2/|C|)\,\Sigma_{j\in C} K(x, x_j) + (1/|C|^2)\,\Sigma_{j\in C}\Sigma_{l\in C} K(x_j, x_l)}$.
- **Common kernels:** linear K(x,y) = x·y (gives ordinary K-means) · polynomial $K(x,y) = (x\cdot y + c)^d$ · **RBF / Gaussian $K(x,y) = \exp(-\|x - y\|^2 / (2\sigma^2))$**. Example: x = (1,2), y = (2,3), σ = 1 → $\|x-y\|^2 = 2 \to K = e^{-1} =$ **0.3679** (python3).
- **Algorithm:** compute the n×n kernel (Gram) matrix → initialise cluster assignments → assign each point to the cluster minimising the feature-space distance above → repeat until assignments stop changing. (Centroids are not explicit points; clusters are tracked by membership.)
- **Pros:** finds non-convex, non-linearly-separable clusters; same simple iterative idea. **Cons:** $O(n^2)$ memory/time for the kernel matrix; must choose kernel and its parameters (σ); still needs k; local minima. Closely related to **spectral clustering**.

Point	K-means	Kernel K-means
Space	original input space	implicit high-dimensional feature space φ
Boundaries	linear (Voronoi), convex clusters	non-linear in input space
Distance	$\|x - \mu\|^2$	computed from kernel values K(x, y)
Cost	O(n·k·d) per iteration	$O(n^2)$ kernel matrix
Centroid	explicit mean vector	implicit (membership-based)
Concentric rings	fails	succeeds (e.g. RBF kernel)

7.9 Statistical learning theory: generalisation, bias–variance, PAC/VC

Importance 7/10 · good chance · syllabus heading "Statistical Learning Theory"; bias–variance is the reason ensembles work

Plain language: Statistical learning theory asks: *when can a model learned from a finite sample be trusted on new data?* A model that memorises training data (including its noise) performs badly on new data (**overfitting**); a model too simple to capture the pattern performs badly on both (**underfitting**). The goal is low **generalisation error** (expected error on unseen data).

- **Training error vs test (generalisation) error.** Overfitting = low training error, high test error. Underfitting = high error on both.

- **Bias–variance decomposition** (squared loss): **Expected error = Bias² + Variance + Irreducible noise (σ^2)**.
  - **Bias:** error from wrong/too-simple assumptions (linear model for a curved pattern) → underfitting.
  - **Variance:** error from sensitivity to the particular training sample (deep unpruned tree, 1-NN) → overfitting.
  - Increasing model complexity lowers bias but raises variance → choose the sweet spot (regularisation, pruning, cross-validation, more data).

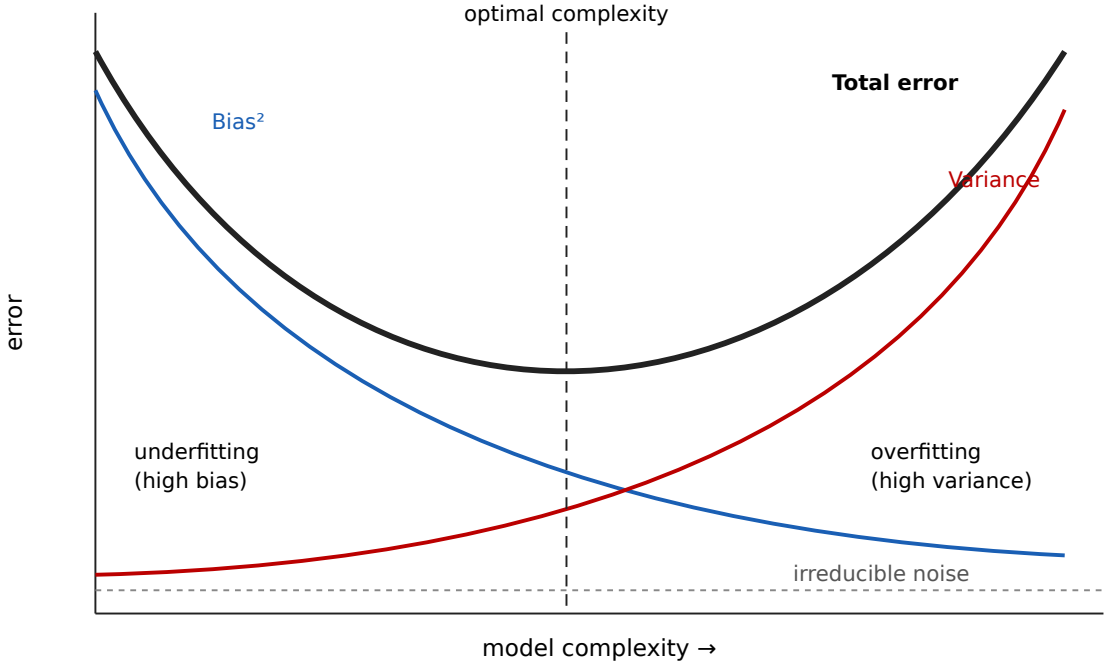


Fig: Bias–variance trade-off — total error is U-shaped; bagging attacks variance, boosting attacks bias

- **PAC learning (Valiant, 1984):** a concept class is *Probably Approximately Correct*-learnable if, for any ε (accuracy) and δ (confidence), an algorithm can, with probability ≥ 1 − δ, output a hypothesis with error ≤ ε using a number of examples polynomial in 1/ε, 1/δ and problem size. For a finite hypothesis space and a consistent learner: **$m \geq (1/\varepsilon)(\ln|H| + \ln(1/\delta))$** examples suffice.
- **VC dimension (Vapnik–Chervonenkis):** the size of the largest set of points that hypothesis class H can **shatter** (classify correctly under every possible labelling). A straight line in 2-D has VC dimension **3**. Higher VC dimension = more capacity = more data needed to generalise; it bounds the gap between training and test error.

7.10 Ensemble methods — why they work

Importance 9/10 · very likely · syllabus phrase never asked; any of bagging/boosting/RF or "compare them" is a top 2026 pick

Plain language: An ensemble combines many models ("weak learners", often decision trees) so that their individual mistakes cancel out — the "wisdom of crowds". It works only if the base models are **better than random** and make **different (not perfectly correlated) errors**.

- **Numerical intuition** (python3): 3 independent classifiers each 70% accurate; majority vote is correct when ≥ 2 are right: $0.7^3 + 3(0.7^2)(0.3) = 0.343 + 0.441 =$ **0.784**. With 5 such classifiers: **0.837**; with 21: **0.974**. (Real models are correlated, so gains are smaller — hence the effort to make them diverse.)

- **Variance reduction:** averaging B independent models each with variance σ^2 gives variance σ^2/B; with pairwise correlation ρ it is $\rho\sigma^2 + (1-\rho)\sigma^2/B$ → reduce ρ to gain more (this is the idea behind random forests).
- **Ways to create diversity:** different training samples (bagging), reweighted samples (boosting), different feature subsets (random subspace / RF), different algorithms (stacking).
- **Combining:** majority/plurality vote (classification), averaging (regression), weighted vote (boosting), meta-learner (stacking).

7.11 Bagging (Bootstrap AGGregatING)

Importance 9/10 · very likely · syllabus phrase; "explain bagging" is a clean 5/10-mark question

Plain language: Make B different training sets by **sampling the original data with replacement** (bootstrap samples, each the same size as the original); train one model on each **in parallel**; combine by **voting** (classification) or **averaging** (regression). Proposed by **Leo Breiman (1996)**.

Algorithm:

1. For b = 1 … B: draw a bootstrap sample D_b of size N from D with replacement; train model h_b on D_b.
2. Predict: classification **$H(x)$ = majority vote of $h_1(x) \dots h_B(x)$**; regression **$H(x) = (1/B)\,\Sigma\, h_b(x)$**.
3. Each bootstrap sample contains about **63.2%** of the distinct original examples $(1-(1-1/N)^N \to 1-1/e \approx 0.632)$; the remaining **≈ 36.8% are "out-of-bag" (OOB)** and can be used as a free validation set.
4. **Reduces variance** (does little for bias) → works best with **unstable, high-variance, low-bias** learners: fully grown decision trees, neural nets. Little help for stable learners like k-NN or linear regression.
5. Models are independent → **parallelisable**; robust to noise/outliers (no reweighting).

7.12 Boosting and AdaBoost

Importance 9/10 · very likely · syllabus phrase; AdaBoost steps + α formula is a likely 10-mark answer

Plain language: Train weak learners **one after another**; each new learner concentrates on the examples the previous ones got **wrong** (by increasing their weights). The final model is a **weighted vote** in which more accurate learners get bigger say. It turns many weak learners (slightly better than chance, e.g. one-level "decision stumps") into a strong one. (Schapire 1990; **AdaBoost — Freund & Schapire, 1995/97**.)

AdaBoost (binary labels $y_i \in \{-1, +1\}$):

1. Initialise weights **$w_i = 1/N$** for all N examples.
2. For t = 1 … T:
   - Train weak learner h_t on the weighted data.
   - Weighted error **$\varepsilon_t = \Sigma_i\, w_i \cdot [h_t(x_i) \neq y_i]$** (must be < 0.5).
   - Learner weight **$\alpha_t = \tfrac{1}{2} \ln((1-\varepsilon_t)/\varepsilon_t)$**.
   - Update **$w_i \leftarrow w_i \cdot \exp(-\alpha_t\, y_i\, h_t(x_i))$** — i.e. multiply by e^(+α) if misclassified, e^(−α) if correct.
   - **Normalise** weights to sum 1 (divide by Z_t).
3. Final classifier **$H(x) = \text{sign}(\ \Sigma_t\, \alpha_t\, h_t(x)\ )$**.

Worked round (verified with python3): N = 10, all $w_i = 0.1$; stump misclassifies 3 → $\varepsilon = 0.3$.

- α = ½ ln(0.7/0.3) = ½ ln(2.333) = **0.4236**.
- Correct: 0.1 × e^(−0.4236) = 0.0655; wrong: 0.1 × e^(0.4236) = 0.1528.
- Z = 7(0.0655) + 3(0.1528) = 0.9165. Normalised: correct **0.0714** (= 1/14), wrong **0.1667** (= 1/6).
- The 3 misclassified now hold 3 × 1/6 = **50%** of total weight → next stump is forced to focus on them.
- Sanity values: ε = 0.1 → α = 1.0986 (confident learner, big vote); ε = 0.5 → α = 0 (random learner, no vote).
- **Reduces bias** (and also variance somewhat); uses simple base learners (stumps).
- **Sensitive to noisy data and outliers** (mislabelled points keep gaining weight); sequential → not parallel; can overfit if T too large on noisy data.
- Variants: **Gradient Boosting** (fit each new tree to the residuals / negative gradient of the loss), **XGBoost**, LightGBM, CatBoost.

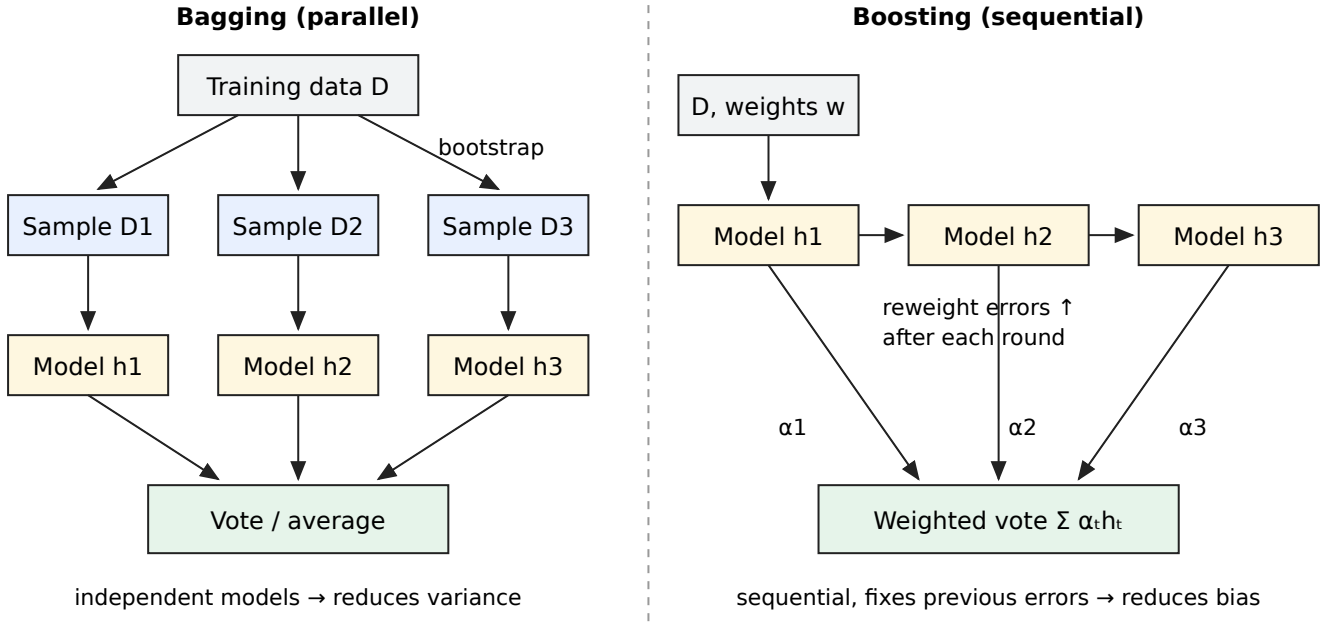


Fig: Bagging trains models in parallel on bootstrap samples; boosting trains them in sequence on reweighted data

7.13 Random forests

Importance 9/10 · very likely · syllabus phrase; very common short-answer and MCQ topic

Plain language: A random forest (**Breiman, 2001**) is **bagging of decision trees plus extra randomness**: at every split, each tree may only choose among a **random subset of m features** (not all p). This makes the trees **less correlated**, so averaging them cuts variance more than plain bagging.

Algorithm:

1. For b = 1 … B (e.g. 100–500 trees): draw a bootstrap sample of the training data.
2. Grow a decision tree on it; at each node, pick **m random features** out of p and choose the best split among only those. Grow deep, usually no pruning.
3. Predict by **majority vote** (classification) or **average** (regression) of all trees.
4. **Typical m:** √p for classification, p/3 for regression (m = p gives plain bagged trees).
5. **OOB (out-of-bag) error:** each tree is tested on the ≈ 36.8% of examples not in its bootstrap sample; aggregating these gives an unbiased error estimate **without a separate validation set**.

6. **Feature importance:** (i) mean decrease in impurity (Gini) from splits on that feature, averaged over trees; (ii) **permutation importance** — shuffle a feature's values in OOB data and measure the rise in error.
7. **Pros:** high accuracy with little tuning, handles many features & missing values reasonably, resistant to overfitting as trees are added, gives feature importance, parallelisable. **Cons:** less interpretable than one tree, larger memory and slower prediction, may be biased toward features with many levels.
8. **Applications:** credit scoring, medical diagnosis, remote-sensing land-cover classification, fraud detection, feature selection.

Point	Bagging	Boosting (AdaBoost)	Random forest
Training order	parallel, independent	**sequential**, each depends on previous	parallel, independent
Data for each model	bootstrap sample	full data with **reweighted** examples	bootstrap sample **+ random feature subset per split**
Base learner	any unstable learner (deep trees)	weak learners (stumps)	decision trees
Combination	equal-weight vote/average	**weighted** vote (α_t)	equal-weight vote/average
Mainly reduces	**variance**	**bias** (and some variance)	**variance** (more than bagging — de-correlated trees)
Overfitting risk	low	can overfit noisy data	low
Noise/outliers	robust	**sensitive**	robust
Built-in validation	OOB error	no	OOB error, feature importance
Proposed by	Breiman 1996	Schapire 1990; Freund & Schapire (AdaBoost)	Breiman 2001

Exam tip: One-liners — *Bagging: parallel, bootstrap, vote, ↓variance. Boosting: sequential, reweight errors, weighted vote, ↓bias. RF: bagging + random feature subsets, OOB, feature importance.* Draw the parallel-vs-sequential figure in any ensemble answer.

7.14 Evaluation metrics (brief)

Importance 5/10 · possible · useful in any classification answer; occasional MCQ numerical

Confusion matrix (binary):

	Predicted Positive	Predicted Negative
Actual Positive	TP (true positive)	FN (false negative, Type II)
Actual Negative	FP (false positive, Type I)	TN (true negative)

Metric	Formula	Meaning
Accuracy	(TP + TN)/(TP + TN + FP + FN)	fraction correct overall (misleading for imbalanced data)
Precision	TP/(TP + FP)	of predicted positives, how many are right
Recall (sensitivity, TPR)	TP/(TP + FN)	of actual positives, how many we caught
Specificity	TN/(TN + FP)	of actual negatives, how many correctly rejected
F1 score	2·P·R/(P + R)	harmonic mean of precision and recall

Example (python3): TP = 40, FP = 10, FN = 5, TN = 45 → Accuracy = 85/100 = **0.85**; Precision = 40/50 = **0.80**; Recall = 40/45 = **0.889**; F1 = 2(0.8)(0.889)/(1.689) = **0.842**.

Regression metrics: MAE = mean $|y - \hat{y}|$, MSE = mean $(y - \hat{y})^2$, RMSE = √MSE.

Exam tip: Disease screening → maximise **recall** (do not miss sick patients); spam filter → favour **precision** (do not lose genuine mail).

Part D — Descriptive Questions with Model Answers

Part D2 — Predicted 2026 questions with model answers

Each prediction targets a Paper-II syllabus phrase that the 2025 paper did **not** use, placed in the section and subject slot it occupied in 2025. Likelihood is per question. Write answers in the same shape: definition first, then numbered points or a table, a diagram where it helps, and a one-line conclusion.

Section A type (5 marks)

Importance 9/10 · very likely · 10 slots (OS×2, SE×2, DBMS×2, CN×2, Web×2), you answer 8; every item below is an unasked syllabus phrase

P1. Trace the evolution of operating systems through their generations. *(5 marks · OS · Likelihood 8/10)*

An operating system's design has followed the hardware of each period. Tanenbaum divides its history into five generations:

Generation	Hardware	OS stage	Key ideas
1st (1945–55)	Vacuum tubes, plugboards	**No OS**	Programmer ran the machine by hand; one job at a time, by sign-up sheet
2nd (1955–65)	Transistors, mainframes	**Batch systems**	Jobs grouped on tape; a resident monitor (e.g. FMS, IBSYS) loads the next job automatically; JCL cards
3rd (1965–80)	Integrated circuits	**Multiprogramming, time-sharing**	Several jobs in memory; CPU switches while one waits for I/O; spooling; interactive terminals (CTSS, MULTICS); OS/360; UNIX (1969)
4th (1980–present)	LSI/VLSI, personal computers	**PC, network and distributed OS**	GUI (Macintosh, Windows), MS-DOS, Linux; network OS, distributed OS, multiprocessor OS
5th (1990–present)	Mobile devices	**Mobile OS**	Smartphones; Symbian, then Android and iOS; touch UI, power management

Each generation tried to make better use of an expensive CPU (batch → multiprogramming) and then to make the user more productive (time-sharing → GUI → mobile). The trend also gave us the main OS types: batch, multiprogrammed, time-sharing, real-time, distributed and embedded/mobile.

P2. What are the services provided by an operating system? Explain system calls and their types. *(5 marks · OS · Likelihood 8/10)*

OS services (Silberschatz). The services that help the user are: (1) **user interface**, which can be a CLI, GUI or touch screen; (2) **program execution**, meaning loading, running and ending programs; (3) **I/O operations**; (4) **file-system manipulation**, i.e. creating, deleting, searching and setting permissions; (5) **communication** through shared memory or message passing; (6) **error detection**. The services that keep the system efficient are: (7) **resource allocation**, (8) **accounting/logging**, (9) **protection and security**.

A **system call** is the programmatic interface through which a user program asks the kernel for a service. It causes a trap that switches the CPU from user mode to kernel mode. Programs usually

reach system calls through an API such as POSIX or Win32. Parameters are passed in registers, in a memory block whose address goes in a register, or on the stack.

Type	Purpose	UNIX examples
Process control	create, end, wait, load/execute	`fork()`, `exec()`, `exit()`, `wait()`
File management	create, open, read, write, close	`open()`, `read()`, `write()`, `close()`
Device management	request/release, read/write device	`ioctl()`, `read()`, `write()`
Information maintenance	time, date, process attributes	`getpid()`, `alarm()`, `sleep()`
Communication	pipes, shared memory, messages	`pipe()`, `shm_open()`, `mmap()`
Protection	permissions	`chmod()`, `umask()`, `chown()`

P3. Explain the process state diagram. What is a Process Control Block (PCB)? *(5 marks · OS · Likelihood 9/10)*

A **process** is a program in execution. As it runs it moves through five states: **New** (being created), **Ready** (waiting for the CPU), **Running** (executing instructions), **Waiting** (blocked until an event such as I/O completion occurs) and **Terminated** (finished).

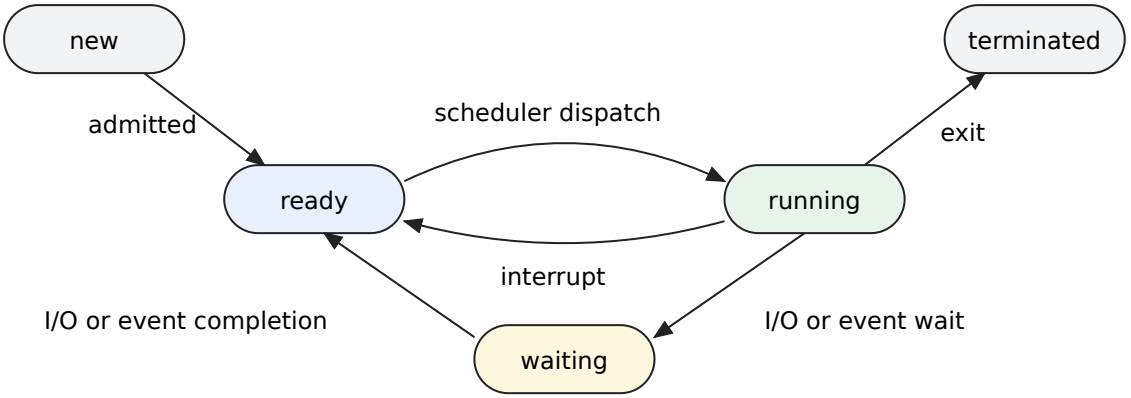


Fig: Five-state process model (Silberschatz)

Process Control Block (PCB). The PCB, also called a task control block, is the kernel data structure that represents one process. On a context switch the OS saves the CPU context into the PCB of the outgoing process and reloads it from the PCB of the incoming one. It holds:

1. **Process state**: new, ready, running and so on.
2. **Process ID** and parent ID.
3. **Program counter**: address of the next instruction.
4. **CPU registers**: accumulators, stack pointer, general-purpose registers.
5. **CPU-scheduling information**: priority and pointers to scheduling queues.
6. **Memory-management information**: base/limit registers, page or segment tables.
7. **Accounting information**: CPU time used, time limits.
8. **I/O status information**: list of open files and allocated devices.

P4. What is a deadlock? State the necessary conditions for deadlock. *(5 marks · OS · Likelihood 9/10)*

A **deadlock** is a situation in which every process in a set is waiting for an event, usually the release of a resource, that only another process in the same set can cause. None of them can ever proceed. Example: P1 holds the printer and wants the scanner, while P2 holds the scanner and wants the printer.

Coffman's four necessary conditions. A deadlock can arise only if all four hold at the same time:

#	Condition	Meaning
1	**Mutual exclusion**	At least one resource is non-sharable; only one process can use it at a time.
2	**Hold and wait**	A process holds at least one resource while waiting to acquire more resources held by others.
3	**No preemption**	A resource cannot be taken away forcibly; the holder releases it voluntarily.
4	**Circular wait**	There is a chain P0 → P1 → … → Pn → P0 in which each process waits for a resource held by the next.

Notes: (i) Circular wait implies hold-and-wait, so the conditions are not fully independent. (ii) In a resource-allocation graph with **one instance per resource type**, a cycle is both necessary and sufficient for deadlock. With **multiple instances**, a cycle is necessary but not sufficient. (iii) **Prevention** works by making sure at least one condition can never hold, for example by imposing a total order on resource requests to break circular wait.

P5. Explain the concept of a system and its characteristics. *(5 marks · SE · Likelihood 8/10)*

A **system** is an organised set of interrelated components that work together towards a common objective. It accepts **inputs**, **processes** them and produces **outputs**, and it uses **feedback** and **control** to stay on target. Examples: a payroll system, a university admission system, the human body.

Characteristics of a system:

1. **Organisation**: the components follow a structure and order, e.g. an organisation chart or layered software.
2. **Interaction**: components work with one another, e.g. the CPU with memory, or sales with inventory.
3. **Interdependence**: each part depends on the others; the output of one is the input of another.
4. **Integration**: the parts are tied together so the system works as a whole.
5. **Central objective**: one common goal that all subsystems serve.

Elements of a system: input, output, processor, control, feedback, environment, boundaries and interface.

Types of system: physical vs abstract; **open** (interacts with its environment, e.g. a business) vs **closed** (isolated); deterministic vs probabilistic; man-made information systems (TPS, MIS, DSS).

In software engineering, a software product is one component of a larger computer-based system made up of hardware, software, people, data and procedures. System engineering therefore comes before software requirements analysis.

P6. Differentiate between functional and non-functional requirements. What are the characteristics of a good SRS? *(5 marks · SE · Likelihood 8/10)*

Basis	Functional requirements	Non-functional requirements
Describes	**What** the system must do	**How well** the system must do it (quality attributes, constraints)
Examples	"User can reset password by OTP"; "Generate monthly salary slip"	Response time < 2 s; 99.9% availability; encryption; usability; portability
Source	Use cases, user stories	Quality standards, environment, regulations
Testing	Functional (black-box) tests	Performance, load, security, usability tests
If missing	Feature absent	System works but is slow, insecure or unusable

The **SRS (Software Requirements Specification)** is the baseline document that records the agreed requirements. It acts as the contract between client and developer and is the input to design and testing.

Characteristics of a good SRS (IEEE 830):

1. **Correct**: every stated requirement is one the software must meet.
2. **Unambiguous**: each requirement has only one interpretation.
3. **Complete**: covers all functions, inputs/outputs and constraints.
4. **Consistent**: no requirements conflict.
5. **Ranked** for importance and/or stability.
6. **Verifiable**: each requirement can be tested; "fast" is not verifiable, "< 2 s" is.
7. **Modifiable**: well structured and not redundant.
8. **Traceable**: each requirement can be traced back to its origin and forward to design and tests.

P7. Differentiate between verification and validation. Compare black-box and white-box testing. *(5 marks · SE · Likelihood 8/10)*

Verification vs validation (Boehm):

Verification	Validation
"Are we building the product **right**?"	"Are we building the **right** product?"
Checks each work product against its specification	Checks the final software against the user's real needs
Mostly **static**: reviews, walkthroughs, inspections	Mostly **dynamic**: executing the software, acceptance testing
Done during each phase	Done at the end of development / on the product

Black-box vs white-box testing:

Basis	Black-box (functional)	White-box (structural / glass-box)
Based on	Requirements/specification only	Internal code, logic and paths
Knowledge of code	Not needed	Required
Techniques	Equivalence partitioning, boundary value analysis, cause-effect graphs, decision tables	Statement, branch and path coverage; basis-path testing (cyclomatic complexity); loop and condition testing
Done by	Independent testers, users	Developers
Level	System and acceptance testing	Unit and integration testing
Finds	Missing or incorrect functions, interface errors	Logic errors, dead code, unreachable paths

The two methods complement each other. White-box testing shows the code is exercised, and black-box testing shows the behaviour meets the specification.

P8. What is data abstraction? Explain the three levels of data abstraction. *(5 marks · DBMS · Likelihood 9/10)*

Data abstraction means hiding the complex details of how data is stored and maintained, and showing users only what they need. In the ANSI/SPARC three-schema architecture there are three levels:

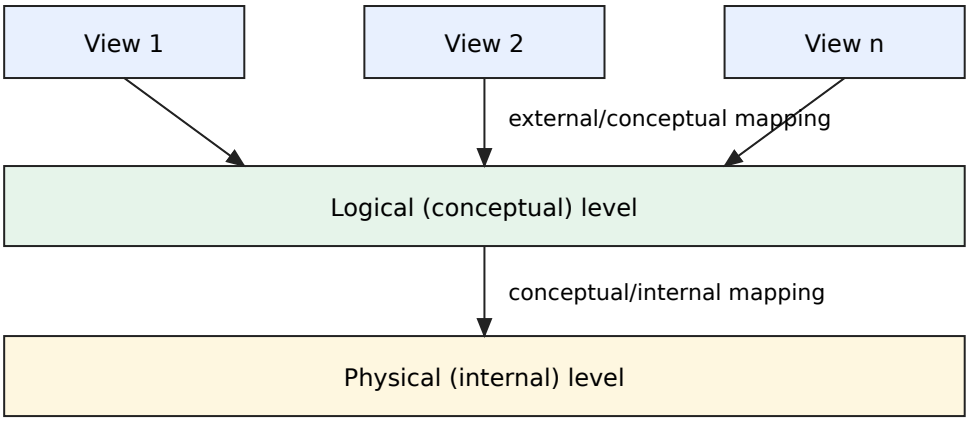


Fig: Three levels of data abstraction (view, logical, physical)

1. **Physical (internal) level**: the lowest level. It describes **how** data is actually stored: file organisation, indexes, B+ trees, hashing and record layout. The DBA and the storage manager work here.
2. **Logical (conceptual) level**: describes **what** data is stored and how the items relate, i.e. all relations, attributes, keys and constraints, e.g. `instructor(ID, name, dept_name, salary)`. Database designers and DBAs work here.
3. **View (external) level**: the highest level. Each user group sees only the part of the database it needs, e.g. a clerk's view without `salary`. It also adds security.

Schema vs instance: the schema is the overall design and changes rarely. An instance is the data held at a given moment. The **mappings** between levels give data independence (see P9).

P9. Differentiate between logical and physical data independence. *(5 marks · DBMS · Likelihood 9/10)*

Data independence is the ability to change the schema at one level without changing the schema at the next higher level. It follows from the three-level architecture and the mappings between levels.

Basis	Physical data independence	Logical data independence
Definition	Change the **physical (internal)** schema without changing the conceptual schema	Change the **conceptual (logical)** schema without changing external views/application programs
Mapping that absorbs the change	Conceptual–internal mapping	External–conceptual mapping
Example changes	Add an index; move from heap to hashed file; new storage device; compress data	Add a new attribute/table; split a table into two; merge relations
Ease of achieving	**Easier**; most DBMSs give it	**Harder**, because application programs depend heavily on the logical structure
Who benefits	Logical schema and applications unaffected	Users' views and programs unaffected

Example. Adding a B+ tree index on `student.roll_no` is a physical change, and existing queries keep working. Adding an `email` column to `student` is a logical change, and the clerk's view `(roll_no, name)` still works unchanged.

Importance. Data independence lowers maintenance cost, lets the DBA tune storage freely, and lets the database evolve without rewriting applications. This was a major advantage over file-processing systems.

P10. Differentiate between DDL and DML with SQL examples. *(5 marks · DBMS · Likelihood 8/10)*

Basis	DDL (Data Definition Language)	DML (Data Manipulation Language)
Purpose	Defines/changes the **structure** (schema)	Accesses/changes the **data** (instances)
Commands	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`
Effect	Updates the **data dictionary** (metadata)	Changes table rows
Transaction	Usually auto-committed (cannot be rolled back in most DBMSs)	Can be rolled back before `COMMIT`
Types	—	Procedural (how) vs declarative (what; SQL)

```
-- DDL
CREATE TABLE student (
  roll_no  INT PRIMARY KEY,
  name     VARCHAR(40) NOT NULL,
  dept     VARCHAR(10),
  cgpa     DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 10)
);
ALTER TABLE student ADD email VARCHAR(50);
DROP TABLE old_marks;

-- DML
INSERT INTO student VALUES (101, 'Ateno', 'CSE', 8.40, 'ateno@x.in');
UPDATE student SET cgpa = 8.60 WHERE roll_no = 101;
SELECT name, cgpa FROM student WHERE dept = 'CSE' ORDER BY cgpa DESC;
DELETE FROM student WHERE roll_no = 101;
```

There are two related sub-languages: **DCL**, which controls access with `GRANT` and `REVOKE`, and **TCL**, which controls transactions with `COMMIT`, `ROLLBACK` and `SAVEPOINT`. DDL also covers integrity constraints (`PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `NOT NULL`, `UNIQUE`) and authorisation.

P11. Explain the different types of attributes in a database (ER model) with examples. *(5 marks · DBMS · Likelihood 8/10)*

An **attribute** is a property that describes an entity. Each attribute takes its values from a **domain**, the set of permitted values.

Type	Meaning	Example	ER notation
Simple (atomic)	Cannot be divided further	`roll_no`, `gender`	Ellipse
Composite	Made of sub-parts	`name` → first, middle, last; `address` → street, city, PIN	Ellipse with child ellipses
Single-valued	One value per entity	`date_of_birth`	Ellipse
Multivalued	Several values per entity	`phone_numbers`, `email_ids`	**Double ellipse**
Stored	Value is physically stored	`date_of_birth`	Ellipse
Derived	Computed from other attributes	`age` (from DOB), `total_marks`	**Dashed ellipse**
Key	Uniquely identifies an entity	`roll_no`, `Aadhaar_no`	**Underlined** name
Null-valued	Value may be unknown or not applicable	`apartment_no`	—
Complex	Composite + multivalued nested	`{address_phone({phone},address)}`	—

When an ER model is mapped to tables: a composite attribute is flattened into its component columns; a multivalued attribute gets a **separate table** (entity key + value); a derived attribute is

usually not stored and is computed when queried.

P12. Compare LAN, MAN and WAN. *(5 marks · CN · Likelihood 8/10)*

Networks are classified by their geographical **span**:

Basis	LAN (Local Area Network)	MAN (Metropolitan Area Network)	WAN (Wide Area Network)
Coverage	A room, building or campus (up to a few km)	A city or town (tens of km)	Country, continent, world
Ownership	Private (one organisation)	Private or public (city, cable or telecom operator)	Usually public/leased (telecom carriers, ISPs)
Data rate	High (100 Mbps to 10 Gbps typical)	Medium to high	Lower per link, varies widely
Delay / errors	Low delay, very low error rate	Moderate	Higher propagation delay and error rate
Cost	Low	Moderate	High
Media	UTP, Wi-Fi, fibre	Fibre, cable TV network, WiMAX	Fibre backbones, satellite, leased lines, microwave
Typical topology	Star (switched Ethernet)	Ring/mesh backbone	Mesh (routers)
Standards / examples	IEEE 802.3 Ethernet, 802.11 Wi-Fi; office/college network	Cable TV network, IEEE 802.16 WiMAX; city government network	The Internet, bank networks (MPLS), NICNET
Maintenance	Easy	Moderate	Difficult

Other classes are the **PAN**, which spans about 10 m (e.g. Bluetooth), and the **CAN** (campus area network). A WAN usually connects many LANs through routers, and the Internet is the largest WAN.

P13. Explain the classes of IPv4 addresses with their ranges. *(5 marks · CN · Likelihood 9/10)*

An **IPv4 address** is 32 bits long and written in dotted-decimal form, e.g. `192.168.1.10`. In **classful** addressing the leading bits fix the class and therefore how the address splits into **net-id** and **host-id**:

Class	Leading bits	First-octet range	Default mask	Networks	Hosts per network	Use
A	`0`	0–127 (usable 1–126)	255.0.0.0 (/8)	$2^7 - 2 = 126$	$2^{24} - 2 = 16{,}777{,}214$	Very large networks
B	`10`	128–191	255.255.0.0 (/16)	$2^{14} = 16{,}384$	$2^{16} - 2 = 65{,}534$	Medium networks
C	`110`	192–223	255.255.255.0 (/24)	$2^{21} = 2{,}097{,}152$	$2^8 - 2 = 254$	Small networks
D	`1110`	224–239	—	—	—	**Multicast**
E	`1111`	240–255	—	—	—	**Reserved** (experimental)

Special addresses: `0.0.0.0` means "this host". `127.x.x.x` is **loopback** (e.g. 127.0.0.1), which is why Class A loses networks 0 and 127. In every network, the address with an all-0s host-id is the network address and the one with an all-1s host-id is the directed broadcast; `255.255.255.255` is the limited broadcast. This is why "−2" appears in host counts.

Private ranges (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12` (172.16–172.31), `192.168.0.0/16`. They are used inside LANs behind NAT.

Drawback. Classful addressing wastes addresses: a firm needing 300 hosts must take a whole Class B of 65,534. **CIDR** (classless addressing, `a.b.c.d/n`) and subnetting replaced it.

P14. Explain the basic structure of an HTML document. Write HTML code for ordered, unordered and definition lists. *(5 marks · Web · Likelihood 9/10)*

An HTML document is a tree of elements. `<!DOCTYPE html>` declares HTML5. The `<html>` root has two children: `<head>` holds metadata such as `<title>` (shown in the browser tab), `<meta>`, `<link>` and `<style>`; `<body>` holds the visible content, such as headings, paragraphs (`<p>`), lists, links (`<a href>`) and images (`<img src alt>`).

```
<!DOCTYPE html>
<html>
<head>
  <title>Lists Demo</title>
</head>
<body>
  <h2>Ordered list (numbered)</h2>
  <ol type="1" start="1">          <!-- type: 1, A, a, I, i -->
    <li>Plan</li>
    <li>Design</li>
    <li>Develop</li>
  </ol>

  <h2>Unordered list (bulleted)</h2>
  <ul type="square">                <!-- disc, circle, square (CSS: list-style-type) -->
    <li>HTML</li>
    <li>CSS</li>
    <li>PHP</li>
  </ul>

  <h2>Definition (description) list</h2>
  <dl>
    <dt>HTML</dt> <dd>HyperText Markup Language - structure of web pages.</dd>
    <dt>PHP</dt>  <dd>Server-side scripting language.</dd>
  </dl>
</body>
</html>
```

List	Container tag	Item tag	Rendered as
Ordered	`<ol>`	`<li>`	1, 2, 3 / A, B / i, ii
Unordered	`<ul>`	`<li>`	Bullets
Definition	`<dl>`	`<dt>` term, `<dd>` description	Term with indented description

Lists can be nested by placing an `<ol>` or `<ul>` inside an `<li>`. The `type` attribute is deprecated in favour of CSS `list-style-type` but is still widely taught.

P15. How is PHP added to HTML? Explain how information is passed between pages using GET, POST and sessions. *(5 marks · Web · Likelihood 9/10)*

Adding PHP to HTML. The file is saved with a `.php` extension. PHP code goes inside `<?php ... ?>` tags anywhere in the HTML. The web server (e.g. Apache with the PHP module) runs the code and sends **only the resulting HTML** to the browser. `<?= $x ?>` is shorthand for `<?php echo $x; ?>`.

```
<html><body>
  <h1>Welcome</h1>
  <p>Today is <?php echo date("d-m-Y"); ?></p>
</body></html>
```

Passing information between pages:

1. **GET (query string)**: data is appended to the URL, e.g. `result.php?roll=101&sem=5`, and read with `$_GET['roll']`. It is visible, can be bookmarked and is limited in length. Use it for searches and links, never for passwords.
2. **POST**: form data travels in the HTTP request body and is read with `$_POST['name']`. It is not shown in the URL and suits larger data and login forms.

3. **Sessions**: data is kept **on the server** across many pages and linked to the user by a session-ID cookie (`PHPSESSID`).
4. Others: **cookies** (`setcookie()` / `$_COOKIE`, stored on the client), hidden form fields, and `$_REQUEST`.

```
<!-- form.html -->
<form action="welcome.php" method="post">
  Name: <input type="text" name="uname"> <input type="submit">
</form>

<?php // welcome.php
session_start();                          // must come before any output
$name = htmlspecialchars($_POST['uname'] ?? '');
$_SESSION['user'] = $name;                // available on later pages
echo "Hello, $name. <a href='dash.php'>Dashboard</a>";
?>

<?php // dash.php
session_start();
echo "Logged in as " . ($_SESSION['user'] ?? 'guest');
?>
```

Always validate and escape incoming data, e.g. `htmlspecialchars()` for output and prepared statements for SQL. `session_destroy()` ends the session at logout.

Section B type (10 marks)

Importance 9/10 · very likely · 5 slots (OS, CN, Web, AI, ML), you answer 3; each item covers a syllabus phrase or algorithm not yet asked

P16. Compare FCFS, SJF and Priority CPU scheduling algorithms with a suitable example. Compute average waiting time and turnaround time. *(10 marks · OS · Likelihood 10/10)*

1. Definitions

- **FCFS (First-Come, First-Served):** the CPU goes to processes in order of arrival. It uses a FIFO queue and is non-preemptive.
- **SJF (Shortest-Job-First):** the CPU goes to the process with the smallest next CPU burst. It is **optimal**, giving the minimum average waiting time. The preemptive version is called SRTF.
- **Priority:** the CPU goes to the highest-priority process (here, a **smaller number means higher priority**). It can be preemptive or non-preemptive. SJF is a special case in which priority = 1/burst.

Formulas: **TAT = Completion − Arrival**; **WT = TAT − Burst**.

2. Worked example (all processes arrive at t = 0, non-preemptive)

Process	Burst	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

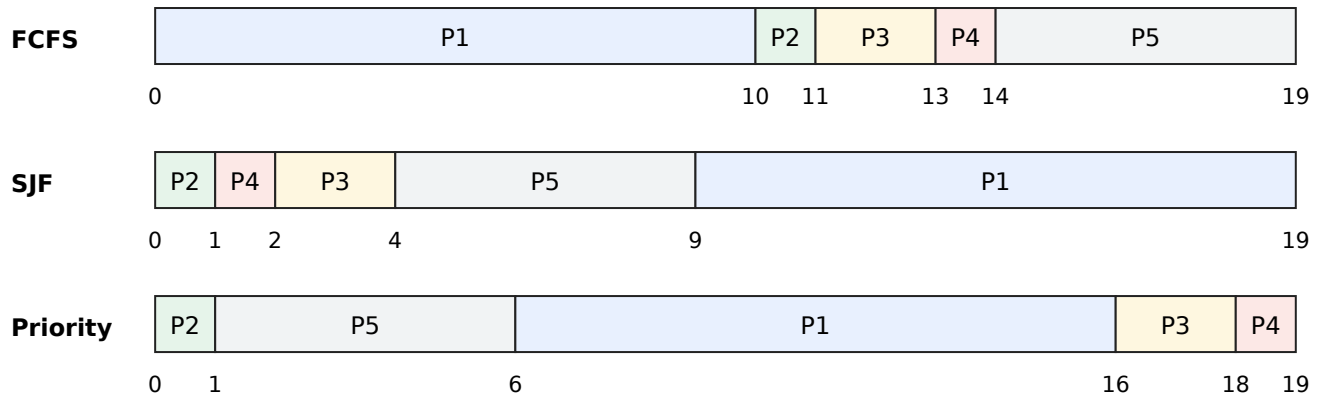


Fig: Gantt charts for FCFS, SJF and non-preemptive Priority (time units)

Process	FCFS WT / TAT	SJF WT / TAT	Priority WT / TAT
P1	0 / 10	9 / 19	6 / 16
P2	10 / 11	0 / 1	0 / 1
P3	11 / 13	2 / 4	16 / 18
P4	13 / 14	1 / 2	18 / 19
P5	14 / 19	4 / 9	1 / 6
Average	**9.6 / 13.4**	**3.2 / 7.0**	**8.2 / 12.0**

Check for FCFS: WT = (0+10+11+13+14)/5 = 48/5 = 9.6. For SJF: (9+0+2+1+4)/5 = 16/5 = 3.2. For Priority: (6+0+16+18+1)/5 = 41/5 = 8.2.

3. Comparison

Basis	FCFS	SJF	Priority
Selection	Earliest arrival	Shortest next burst	Highest priority
Preemption	No	Either (SRTF if preemptive)	Either
Avg WT	Often high	**Minimum (optimal)**	Depends on priorities
Starvation	No	Yes (long jobs)	Yes (low priority); cure: **aging**
Problem	**Convoy effect**: short jobs stuck behind a long one	Next burst length unknown; predicted by exponential averaging $\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$	Indefinite blocking
Overhead	Lowest	Needs burst prediction	Needs a priority policy
Suited to	Batch	Batch / short-term with estimates	Real-time, OS-level tasks

Conclusion. SJF gives the lowest average waiting time (3.2 here, against 9.6 for FCFS) but needs burst estimates and can starve long jobs. FCFS is simple and fair but suffers the convoy effect. Priority scheduling is flexible and needs aging to prevent starvation.

P17. Explain FIFO, LRU and Optimal page replacement algorithms with an example. What is Belady's anomaly? *(10 marks · OS · Likelihood 9/10)*

Page replacement. In demand paging, a **page fault** occurs when a referenced page is not in memory. If no frame is free, the OS chooses a **victim** page, writes it to disk if it is dirty, and loads the new page into that frame. The goal is the **lowest page-fault rate**.

1. **FIFO**: replace the page that has been in memory **longest**. It is simple (a queue) but may evict heavily used pages, and it suffers from Belady's anomaly.
2. **Optimal (OPT/MIN)**: replace the page that will **not be used for the longest time in the future**. It gives the minimum possible faults but needs future knowledge, so it is used only as a

benchmark.

3. **LRU (Least Recently Used)**: replace the page **not used for the longest time in the past**, using the recent past to approximate the future. It is implemented with counters/timestamps or a stack, and approximated with a reference bit (second-chance/clock).

Worked example. Reference string `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1`, **3 frames**, all initially empty (F = fault, · = hit):

Ref	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**	**1**	**2**	**0**	**1**	**7**	**0**	**1**	**Faults**
FIFO	F	F	F	F	·	F	F	F	F	F	F	·	·	F	F	·	·	F	F	F	**15**
LRU	F	F	F	F	·	F	·	F	F	F	F	·	·	F	·	F	·	F	·	·	**12**
OPT	F	F	F	F	·	F	·	F	·	·	F	·	·	F	·	·	·	F	·	·	**9**

Sample steps:

- **FIFO**, at ref 2: the frames {7,0,1} are full, so 7 (the oldest) is evicted, giving {0,1,2}. At ref 3, **0** (now the oldest) is evicted, giving {1,2,3}, so the very next reference to 0 faults under FIFO. LRU instead evicts 1 at that point, because 0 was used just before, and so gets a hit on 0.
- **LRU**, at ref 4 (frames {0,2,3} after earlier steps): the last uses were 2 at position 4, 3 at position 6 and 0 at position 7, so **2** is least recent and is replaced.
- **OPT**, at ref 2 (frames {7,0,1}): 7 is next used at position 18, the farthest of the three, so 7 is replaced.

Result: **OPT 9 < LRU 12 < FIFO 15** (counts verified by simulation).

Belady's anomaly. For some algorithms, **adding frames increases the number of page faults**. FIFO, string `1 2 3 4 1 2 5 1 2 3 4 5`:

Frames	**3**	**4**
FIFO faults	**9**	**10**

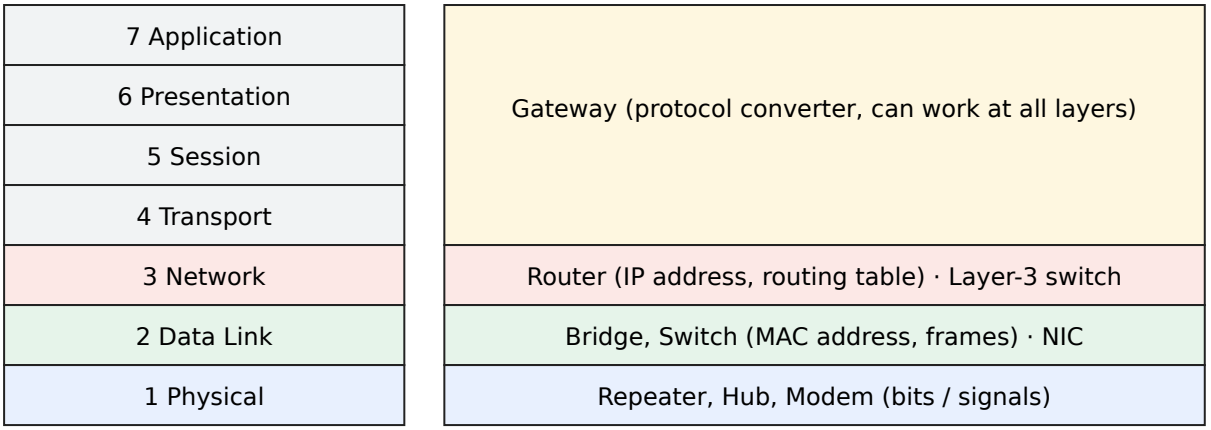
Cause: FIFO is not a **stack algorithm**. With a stack algorithm, the set of pages in memory with n frames is always a subset of the set with n+1 frames. **LRU and OPT are stack algorithms**, so they never show Belady's anomaly.

Basis	**FIFO**	**Optimal**	**LRU**
Victim	Oldest loaded	Used farthest in future	Least recently used
Implementable	Yes, easy	No (needs future)	Yes, but needs hardware support
Belady's anomaly	**Yes**	No	No
Performance	Worst	Best (benchmark)	Close to OPT

Conclusion. OPT is the theoretical lower bound and LRU is the practical near-optimal choice, usually implemented through clock approximations. FIFO is simplest but performs worst and shows Belady's anomaly.

P18. Explain the networking devices hub, switch, bridge, router, gateway, repeater and modem, stating the OSI layer of each. *(10 marks · CN · Likelihood 9/10)*

Networking devices connect hosts and networks. Each works up to a particular OSI layer, and the higher that layer, the more "intelligent" the device.



Each device also implements every layer below the one it is placed at.

Fig: Networking devices mapped to the OSI layer at which they operate

1. **Repeater (Layer 1).** Regenerates (not merely amplifies) a weakened signal to extend cable length. It does not filter or examine addresses and joins two segments of the **same** LAN.
2. **Hub (Layer 1).** A multiport repeater. It **broadcasts every incoming bit to all ports**, so all ports share a single collision domain and bandwidth. Passive hubs only connect; active hubs regenerate. Hubs are now obsolete.
3. **Bridge (Layer 2).** Joins two LAN segments and **filters frames by MAC address** using a learned table, forwarding a frame only if its destination is on the other side. Each port is a separate collision domain. A transparent bridge learns addresses by itself and uses the Spanning Tree Protocol to avoid loops.
4. **Switch (Layer 2).** A multiport bridge. It keeps a **MAC address table (CAM)** and forwards each frame only to the destination port, giving full-duplex links with **one collision domain per port** but one broadcast domain overall unless VLANs are used. A Layer-3 switch can also route.
5. **Router (Layer 3).** Connects **different networks** and forwards **packets by IP address** using a **routing table** built statically or by routing protocols (RIP, OSPF, BGP). It separates broadcast domains, chooses the best path, and can apply NAT and ACLs.
6. **Gateway (Layers 4–7, potentially all layers).** Connects networks that use **different protocols** and **translates** between them, e.g. an email gateway or VoIP-to-PSTN gateway. A host's "default gateway" is in practice the router.
7. **Modem (Layer 1).** A **mo**dulator–**dem**odulator. It converts digital data to an analogue signal for telephone, cable or DSL lines and back again, using modulation such as ASK, FSK, PSK or QAM.

Device	OSI layer	Addressing used	Collision domains	Broadcast domains	Intelligence
Repeater	1	None	1	1	None
Hub	1	None	1 (all ports)	1	None
Modem	1	None	—	—	Signal conversion
Bridge	2	MAC	1 per port	1	Filters frames
Switch	2	MAC	1 per port	1 (without VLANs)	Filters/forwards frames
Router	3	IP	1 per port	1 per port	Routing decisions
Gateway	4–7 (all)	Protocol-specific	—	—	Protocol translation

Conclusion. Choose the lowest-layer device that solves the problem. Use repeaters or hubs to extend a segment, switches to segment a LAN, routers to join different networks, and gateways where the protocols themselves differ.

P19. Explain guided and unguided transmission media with their types, advantages and disadvantages. *(10 marks · CN · Likelihood 9/10)*

Transmission media form the physical path between sender and receiver and belong below the physical layer. **Guided (wired)** media confine the signal inside a physical conductor. **Unguided (wireless)** media send electromagnetic waves through air, water or vacuum.

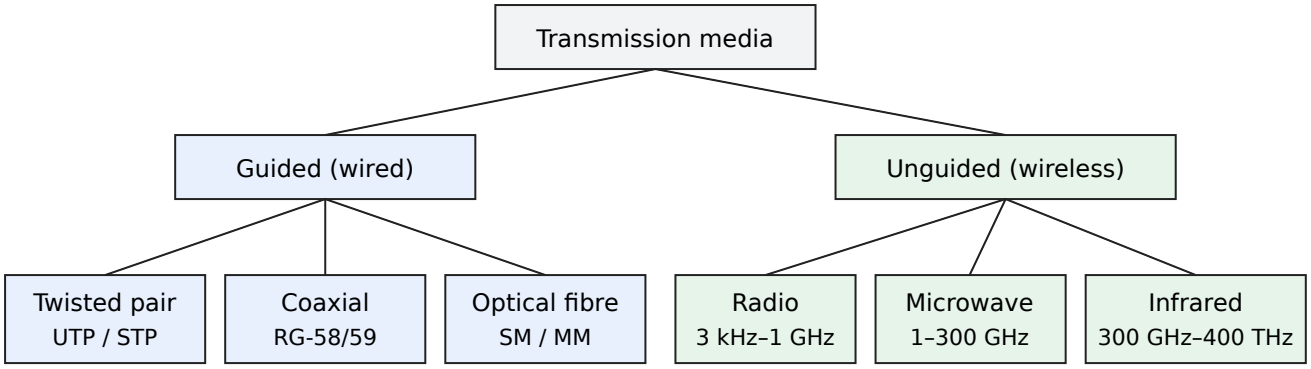


Fig: Classification of transmission media (frequency bands as in Forouzan)

A. Guided media

1. **Twisted-pair cable.** Two insulated copper wires twisted together. The twisting cancels electromagnetic interference and crosstalk. **UTP** (unshielded) is cheap and used in Ethernet LANs (Cat 5e, Cat 6) with RJ-45 connectors. **STP** adds a metal foil shield, giving better noise immunity at higher cost. Range is about 100 m per segment.
2. **Coaxial cable.** A central copper core, insulator, braided metal shield and outer jacket. It offers higher bandwidth and better shielding than twisted pair and is used in cable TV (RG-59) and old thin Ethernet (RG-58) with BNC connectors.
3. **Optical fibre.** Carries **light** through a glass core surrounded by cladding of lower refractive index, guided by **total internal reflection**. Modes: **multimode** (step-index or graded-index, for short distances) and **single-mode** (narrow core, for long-haul links). It has very high bandwidth, low attenuation, immunity to EMI and good security, but is costly and fragile, and installation and splicing are harder.

B. Unguided media

1. **Radio waves (3 kHz–1 GHz).** Omnidirectional and able to pass through walls. Used for AM/FM radio, cordless phones and paging. Suffer interference.
2. **Microwaves (1–300 GHz).** Unidirectional and need **line of sight**. Terrestrial microwave uses towers and repeaters; satellite links and cellular networks also use microwaves.
3. **Infrared (300 GHz–400 THz).** Short range, blocked by walls, so no interference between rooms. Used in TV remotes and IrDA.

C. Comparison

Basis	Guided	Unguided
Signal path	Confined to cable	Free space
Examples	Twisted pair, coax, fibre	Radio, microwave, infrared, satellite
Security	Higher (needs physical tap)	Lower (can be intercepted)
Mobility	None	Supports mobile users
Installation	Cabling cost, rights of way	Easier across terrain such as the hills of Nagaland
Interference	Low (fibre is immune)	Weather, obstacles, multipath
Capacity	Fibre highest of all	Limited by spectrum

Conclusion. Fibre forms backbones, UTP serves LAN access, and wireless gives mobility and reach where cabling is impractical. Most real networks combine all three.

P20. Explain Internet routing. Compare distance-vector and link-state routing. *(10 marks · CN · Likelihood 8/10)*

Routing is the process of finding the best path for packets from source to destination across interconnected networks. Each **router** keeps a **routing table** of (destination network, next hop, cost/metric, interface) entries and forwards each packet by **longest-prefix match** on its destination IP address. Routes can be **static** (configured by hand) or **dynamic** (learnt by routing protocols).

Hierarchy on the Internet. The Internet is divided into **Autonomous Systems (AS)**, which are networks under one administration, such as an ISP.

- **Intra-domain (interior) routing** inside an AS uses **RIP** (distance vector) or **OSPF** (link state).
- **Inter-domain (exterior) routing** between ASes uses **BGP** (path vector), which works with policies rather than shortest paths.

1. Distance-vector routing (Bellman–Ford). Each router keeps a vector of its best-known distance to every destination and **periodically sends its whole table to its neighbours only**. On receiving a neighbour's vector it applies:
D(x→y) = min over neighbours v of { c(x,v) + D(v→y) }.
Example: A reaches B at cost 1 and B reports "C at 2", so A records C at 1 + 2 = 3 via B. **Problem:** bad news spreads slowly, giving the **count-to-infinity** problem. Remedies include split horizon, poison reverse and triggered updates. **RIP** uses hop count as its metric with a maximum of 15 (16 = unreachable) and sends updates every 30 s.

2. Link-state routing (Dijkstra). Each router learns the state of its directly attached links and **floods link-state packets (LSPs) to all routers** in the area. Every router therefore builds an identical **map of the whole topology** and runs **Dijkstra's shortest-path algorithm** to compute its own routing table. Updates are sent only when something changes. **OSPF** (and IS-IS) work this way, with cost based on bandwidth and with areas for scalability.

Basis	Distance vector	Link state
Algorithm	Bellman–Ford (distributed)	Dijkstra (each router, centrally)
What is shared	Entire routing table (distances)	Only the state of its own links
Shared with	Neighbours only	All routers (flooding)
Topology knowledge	Only distances via neighbours ("routing by rumour")	Full map of the network
Updates	Periodic	Event-triggered
Convergence	Slow; count-to-infinity	Fast; no count-to-infinity
Resources	Low CPU and memory	Higher CPU and memory; LSP flooding
Scalability	Small networks	Large networks (with areas)
Protocol	RIP	OSPF, IS-IS

Path-vector (BGP) is related to distance vector, but it advertises the full AS path. This prevents loops and lets each AS apply its own routing policy.

Conclusion. Distance vector is simple but converges slowly, so it suits small networks. Link state converges fast and scales with areas, which is why OSPF dominates inside large organisations. BGP ties the ASes of the Internet together.

P21. What are the common errors in PHP? How are they detected and debugged? *(10 marks · Web · Likelihood 8/10)*

PHP reports problems at run time through the web server. If `display_errors` is off and a fatal error occurs, the user sees only a **blank page**, often called the "white screen of death". Knowing the error types speeds up debugging.

1. Types of PHP errors

Type (constant)	When it occurs	Script stops?	Example
Parse / syntax error (`E_PARSE`)	Code violates syntax before execution	Yes, nothing runs	Missing `;`, unmatched `{ }` or quotes
Fatal error (`E_ERROR`)	A runtime operation cannot continue	Yes	Calling an undefined function; `require` of a missing file; memory limit exceeded
Warning (`E_WARNING`)	Serious but non-fatal	No	`include` of a missing file; wrong argument type; undefined variable (PHP 8)
Notice (`E_NOTICE`)	Possible bug, minor	No	Minor runtime notices (an undefined variable was a notice before PHP 8)
Deprecated (`E_DEPRECATED`)	Feature will be removed	No	Using a removed or old function

2. Common problems and their cause or fix

1. **Missing semicolon or brace**: "Parse error: syntax error, unexpected ...". The reported line is often the one **after** the real mistake.
2. **Undefined variable or array index**: initialise variables first; use `isset()` or the null-coalescing `??`, e.g. `$_GET['id'] ?? 0`.
3. **"Headers already sent"**: `header()`, `setcookie()` or `session_start()` was called after output (even a space before `<?php`). Move these calls to the top.
4. **include vs require**: `include` gives a warning and continues; `require` gives a fatal error. Check the file paths.
5. **PHP code shown as plain text in the browser**: the file has a `.html` extension, the page was opened directly as a file instead of through `http://localhost`, or PHP is not installed.
6. **Database errors**: wrong credentials or an SQL syntax error. Check with `mysqli_connect_error()` or a `PDOException`.
7. **Logic errors**: `=` used instead of `==`, or a loop off by one. No message appears, so trace the values.
8. **Form data missing**: the `method` does not match (`$_POST` read while the form uses GET) or the input `name` attribute is missing.

3. Debugging techniques

```
<?php
error_reporting(E_ALL);          // report every error type
ini_set('display_errors', 1);    // show on screen (development only)
var_dump($user);                 // type + value
print_r($_POST);                 // readable array dump
error_log("cart total = $total");// write to server log instead of screen
try { $pdo = new PDO($dsn, $u, $p); }
catch (PDOException $e) { echo "DB error: " . $e->getMessage(); }
?>
```

- Run `php -l file.php` to lint, which catches syntax errors without executing the script.
- Read the server error log, e.g. Apache `error.log`.
- Use a debugger (**Xdebug**) with breakpoints in an IDE.
- Use custom handlers: `set_error_handler()` and `try/catch` for exceptions.
- In **production**, set `display_errors = Off` and `log_errors = On` so that paths and SQL are never revealed to attackers.

Conclusion. Most PHP problems are syntax slips, undefined values, header/output ordering and path issues. Turn on full error reporting during development, dump the values, read the logs, and hide errors in production.

P22. Explain inductive learning and explanation-based learning (EBL). Compare them. *(10 marks · AI · Likelihood 9/10)*

Both are forms of **learning from observations**: the agent improves its future performance by studying examples.

1. Inductive learning

Inductive learning infers a **general rule (hypothesis) h** from specific **examples** of an unknown target function f, so that h ≈ f, in the form of (x, f(x)) pairs. It is reasoning **from specific to general**.

- Input: many labelled training examples (positive and negative).
- Process: search a hypothesis space for an h **consistent** with the examples. Prefer the simplest (Ockham's razor) so that it generalises to unseen cases.
- Methods: **decision trees** (ID3), version spaces / candidate elimination, Winston's arch learning, neural networks.
- Example: from labelled cases of past loan applicants, learn "if income is high and there is no default history then approve".
- Nature: **knowledge-poor and data-rich**. The result is not guaranteed correct, because a new example may contradict it.

2. Explanation-based learning (EBL)

EBL learns a general, operational rule from a **single example** by **explaining** why that example is an instance of the concept using existing **domain knowledge**, and then generalising the explanation. It is analytical (deductive) learning.

Inputs (Mitchell; Rich & Knight):

1. **Training example**: one positive instance.
2. **Goal concept**: a high-level description of the concept to learn.
3. **Domain theory**: rules that can explain the example.
4. **Operationality criterion**: the terms in which the learnt rule must be expressed so it can be used efficiently.

Steps: (i) **Explain**: build a proof that the example satisfies the goal concept using the domain theory. (ii) **Generalise**: replace constants by variables while keeping the proof valid (goal regression). (iii) Store the result as a new rule or macro.

Example (Cup): Goal concept: Cup(x). Domain theory: `liftable ∧ stable ∧ open-vessel → cup`; `light ∧ has-handle → liftable`; `flat-bottom → stable`; `concavity-up → open-vessel`. From one red ceramic cup the system learns

`Cup(x) ← light(x) ∧ has-handle(x) ∧ flat-bottom(x) ∧ concavity-up(x).`

The irrelevant feature (colour = red) is dropped automatically because it does not appear in the explanation.

3. Comparison

Basis	Inductive learning	Explanation-based learning
Reasoning	Specific → general (induction)	Deductive (explain, then generalise)
Examples needed	**Many**	**One** (or very few)
Prior knowledge	Little or none	**Complete domain theory required**
Output	New knowledge (hypothesis)	Reformulates existing knowledge into operational form
Correctness	Not guaranteed (may over/under-generalise)	Correct if the domain theory is correct
Learning type	Empirical/statistical, knowledge-level	Speed-up learning, **symbol-level**
Weakness	Needs much data; sensitive to noise	Needs a perfect domain theory; does not learn genuinely new facts
Examples	ID3 decision trees, version spaces	PRODIGY, chess-pattern learning, SOAR chunking

Conclusion. Induction discovers new generalisations from data. EBL turns knowledge the system already has into efficient rules from one example. Hybrid systems use induction where the theory is weak and EBL where it is strong.

P23. Explain the K-means clustering algorithm with an example. What is kernel K-means? *(10 marks · ML · Likelihood 10/10)*

Clustering is **unsupervised** learning that groups unlabelled points so that points in the same cluster are similar to each other and dissimilar to points in other clusters. **K-means** is a partitional, centroid-based algorithm. It minimises the **within-cluster sum of squared errors**:
$J = \Sigma_k \Sigma_{x \in C_k} \|x - \mu_k\|^2$.

Algorithm

1. Choose **k** and initialise k centroids, e.g. as k random points.
2. **Assignment step**: assign each point to its **nearest** centroid by Euclidean distance.
3. **Update step**: recompute each centroid as the **mean** of the points assigned to it.
4. Repeat steps 2–3 until the assignments no longer change (convergence).

Worked example. Points A(1,1), B(2,1), C(4,3), D(5,4), E(1,2), F(2,3); **k = 2**; initial centroids μ_1 = A(1,1), μ_2 = B(2,1).

Point	Iter 1: d(μ_1), d(μ_2) → cluster	Iter 2: μ_1=(1,1.5), μ_2= (3.25,2.75)	Iter 3: μ_1=(1.33,1.33), μ_2=(3.67,3.33)
A(1,1)	0.00, 1.00 → **C1**	0.50, 2.85 → **C1**	0.47, 3.54 → C1
B(2,1)	1.00, 0.00 → **C2**	1.12, 2.15 → **C1**	0.75, 2.87 → C1
C(4,3)	3.61, 2.83 → C2	3.35, 0.79 → C2	3.15, 0.47 → C2
D(5,4)	5.00, 4.24 → C2	4.72, 2.15 → C2	4.53, 1.49 → C2
E(1,2)	1.00, 1.41 → C1	0.50, 2.37 → C1	0.75, 2.98 → C1
F(2,3)	2.24, 2.00 → C2	1.80, 1.27 → C2	1.80, 1.70 → C2
New centroids	C1={A,E} → (1, 1.5); C2= {B,C,D,F} → (3.25, 2.75)	C1={A,B,E} → (1.33, 1.33); C2= {C,D,F} → (3.67, 3.33)	**No change → stop**

Final clusters: {A, B, E} and {C, D, F}, with centroids (4/3, 4/3) and (11/3, 10/3). B moved from C2 to C1 in iteration 2, and the algorithm converged in iteration 3. The SSE fell from 12.0 after iteration 1 to 6.67.

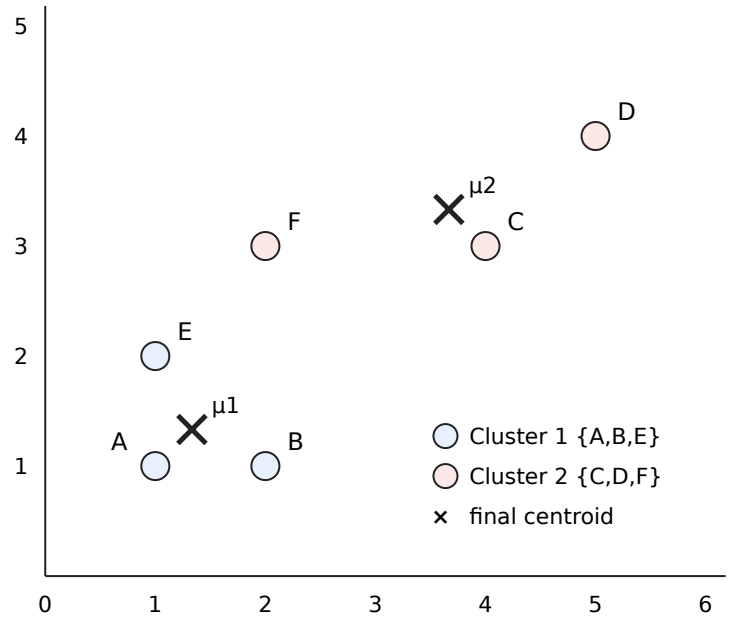


Fig: Final K-means clusters (k = 2) with centroids μ1 = (1.33, 1.33), μ2 = (3.67, 3.33)

Properties. Each iteration costs O(n·k·d). The algorithm always converges, but only to a **local** optimum, and the result depends on the initial centroids, so use k-means++ initialisation or several restarts. **k** must be chosen in advance, using the elbow method or silhouette score. It is sensitive to outliers and feature scale, so normalise the features. It assumes spherical, similar-sized clusters.

Kernel K-means. Plain K-means draws only **linear** boundaries, so it fails on, for example, two concentric rings. Kernel K-means maps the data implicitly into a high-dimensional feature space $\varphi(x)$ using a **kernel function** $K(x, y) = \varphi(x)\cdot\varphi(y)$, such as the RBF kernel $\exp(-\|x-y\|^2/2\sigma^2)$ or a polynomial kernel, and runs K-means there. Distances are computed with the **kernel trick**:
$\|\varphi(x) - \mu_k\|^2 = K(x,x) - (2/|C_k|)\ \Sigma_{j\in C_k} K(x,x_j) + (1/|C_k|^2)\ \Sigma_{i,j\in C_k} K(x_i,x_j)$.
It finds **non-linearly separable** clusters at a cost of $O(n^2)$ for the kernel matrix. It is closely related to spectral clustering.

Conclusion. K-means is simple, fast and widely used for clustering, e.g. customer segmentation and image colour quantisation. Kernel K-means extends it to non-linear cluster shapes.

P24. Explain ensemble methods: bagging, boosting and random forests. *(10 marks · ML · Likelihood 10/10)*

An **ensemble** combines the predictions of several **base learners**, often weak learners such as small decision trees, to get a model that is more accurate and more stable than any single one. It works when the base models are **accurate (better than random) and diverse** (make different errors). Ensembles reduce **variance** (bagging) or **bias** (boosting). Combination is by **majority voting** for classification or **averaging** for regression.

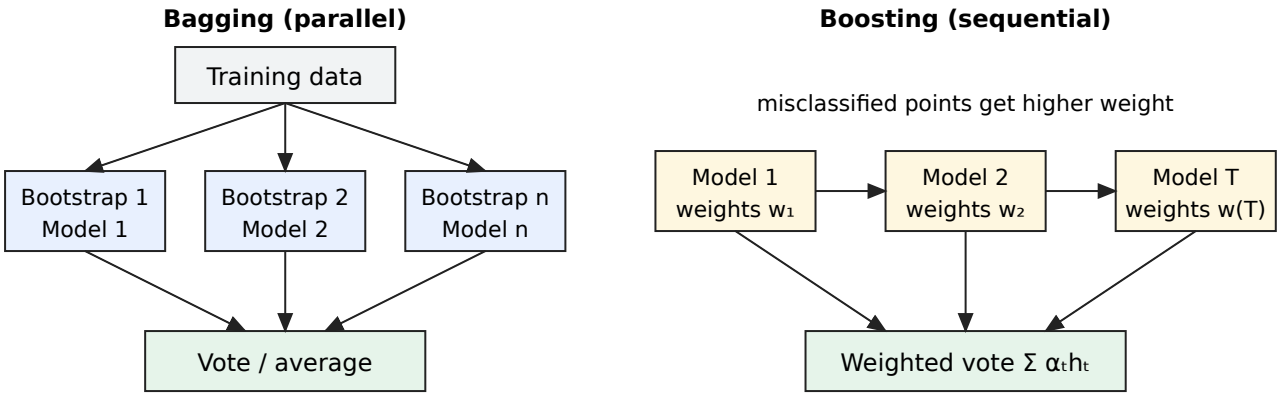


Fig: Bagging trains models in parallel on bootstrap samples; boosting trains them in sequence on re-weighted data

1. Bagging (Bootstrap AGGregatING, Breiman 1996)

- Create n **bootstrap samples**, each formed by drawing N records **with replacement** from the training set of N.
- Train one base model on each sample **independently (in parallel)**.
- Combine by majority vote or averaging.
- Each sample omits about $(1 - 1/N)^N \approx 1/e \approx$ **36.8%** of the records. These **out-of-bag (OOB)** records give a free validation estimate.
- It **reduces variance** and works best with unstable, high-variance learners such as deep decision trees.

2. Boosting (e.g. AdaBoost, Freund & Schapire)

- Train weak learners **sequentially**. Each new learner concentrates on the examples that earlier learners **misclassified**.
- AdaBoost: start with equal weights $w_i = 1/N$. After learner t with weighted error ε_t, compute $\alpha_t = \frac{1}{2} \ln((1-\varepsilon_t)/\varepsilon_t)$. Increase the weights of misclassified points, decrease the weights of correct ones, then normalise.
- Final classifier: $H(x) = \text{sign}(\Sigma\ \alpha_t\ h_t(x))$. More accurate learners get larger α; for example, $\varepsilon = 0.3$ gives $\alpha \approx 0.42$.
- It **reduces bias** (and variance), turning weak learners into a strong one. It is sensitive to noisy data and outliers and can overfit. Variants: Gradient Boosting, XGBoost.

3. Random Forest (Breiman 2001)

- An ensemble of **decision trees** that combines bagging with **random feature selection**: at each split, only a random subset of m features is considered (typically $m \approx \sqrt{p}$ for classification).
- This **decorrelates the trees**, so averaging reduces variance more than plain bagging does.
- Prediction is by majority vote or the mean.
- Benefits: high accuracy, handles many features, robust to overfitting, gives an OOB error estimate and **feature importance**. Drawbacks: less interpretable than a single tree, and needs more memory and time.

4. Comparison

Basis	Bagging	Boosting	Random forest
Training	Parallel, independent	Sequential, dependent	Parallel
Sampling	Bootstrap (uniform)	Re-weighting of examples	Bootstrap + random feature subsets
Mainly reduces	Variance	Bias (and variance)	Variance (more than bagging)
Combination	Equal vote	Weighted vote (α_t)	Equal vote
Overfitting / noise	Robust	Can overfit noisy data	Robust
Base learner	Unstable (deep trees)	Weak (stumps)	Decision trees

Conclusion. Bagging and random forests stabilise high-variance models, while boosting builds a strong model from weak ones. Random forests and gradient boosting are among the most accurate methods for tabular data.

P25. Explain the features of LISP and PROLOG with example programs. Compare the two languages. *(10 marks · AI · Likelihood 9/10)*

1. LISP (LISt Processing), John McCarthy, 1958

LISP is a **functional** language and the classic language of symbolic AI. Its features:

1. **Everything is a list or an atom.** Programs and data share the same form (S-expressions), so programs can build and run code (homoiconicity).
2. **Prefix notation**: `(+ 2 3)` gives 5.
3. **Recursion** is the main control structure, and functions are first-class (`lambda`, `mapcar`).
4. **Dynamic typing** and **automatic garbage collection**, which LISP introduced.
5. Basic list functions: `car` (first element), `cdr` (the rest), `cons` (build a list), along with `atom`, `eq`, `cond`, `setq` and `defun`.
6. It is interpreted and interactive (the REPL), which suits rapid prototyping. Dialects: Common Lisp, Scheme, Clojure.

```
(defun factorial (n)
  (if (<= n 1) 1
      (* n (factorial (- n 1)))))
(factorial 5)                ; => 120
(car '(a b c))               ; => A
(cdr '(a b c))               ; => (B C)
(cons 'x '(y z))             ; => (X Y Z)
```

2. PROLOG (PROgramming in LOGic), Colmerauer & Roussel, 1972

PROLOG is a **declarative, logic-programming** language based on first-order predicate logic restricted to **Horn clauses**. The programmer states **what** is true, and the built-in inference engine works out **how** to answer.

1. A program is a set of **facts**, **rules** (`head :- body.`) and **queries** (`?- goal.`).
2. Its inference engine uses **resolution with depth-first search and automatic backtracking**.
3. **Unification** matches terms and binds variables. Variables start with an uppercase letter.
4. **Closed-world assumption**: anything not provable is false (negation as failure).
5. It has built-in lists `[H|T]` and suits expert systems, NLP parsing and theorem proving.

```
parent(tom, bob).
parent(bob, ann).
parent(bob, pat).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

factorial(0, 1).
factorial(N, F) :- N > 0, N1 is N - 1, factorial(N1, F1), F is N * F1.

?- grandparent(tom, Who).     % Who = ann ; Who = pat
?- factorial(5, F).           % F = 120
```

3. Comparison

Basis	LISP	PROLOG
Paradigm	Functional (applicative)	Logic / declarative
Basis	Lambda calculus	First-order predicate logic (Horn clauses)
Program	Function definitions	Facts + rules
Control	Programmer-specified (recursion, conditionals)	Built-in backtracking search
Key mechanism	Function application, list manipulation	Unification and resolution
Data structure	Lists (S-expressions)	Terms, lists, relations
Variables	Bound by assignment/parameters	Logical variables bound by unification
Popular in	USA; symbolic AI, MACSYMA, early expert systems	Europe and Japan (Fifth-Generation project); expert systems, NLP
Strength	Flexible, extensible, code-as-data	Concise for search, rule-based and relational problems
Weakness	Many parentheses, less readable	Less efficient numerically; depth-first search can loop

Conclusion. LISP excels at symbolic manipulation and building AI tools. PROLOG excels at knowledge representation and automatic reasoning. Both remain the two classic AI programming languages named in the syllabus.

Section C type (15 marks)

Importance 9/10 · very likely · 4 slots (OS, SE, DBMS, AI in 2025), you answer 2; every item is a broad syllabus phrase not yet asked

P26. What is deadlock? Explain the necessary conditions, and the methods of deadlock prevention, avoidance (Banker's algorithm with an example), detection and recovery. *(15 marks · OS · Likelihood 9/10)*

1. Introduction. In a multiprogrammed system, processes compete for a finite number of resources. A **deadlock** is a state in which each process in a set is waiting for an event (the release of a resource) that only another process in the same set can cause, so none of them ever proceeds.

2. System model and Resource-Allocation Graph (RAG). A process uses a resource in three steps: request → use → release. In a RAG, a **request edge** runs $P_i \rightarrow R_j$ and an **assignment edge** runs $R_j \rightarrow P_i$. With one instance per resource type, a **cycle means deadlock**.

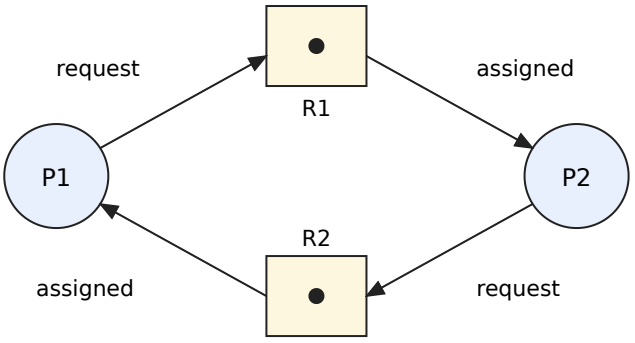


Fig: RAG with cycle P1 → R1 → P2 → R2 → P1 (single instances, so this is a deadlock)

3. Necessary conditions (Coffman). All four must hold at once: (i) **mutual exclusion**; (ii) **hold and wait**; (iii) **no preemption**; (iv) **circular wait**.

4. Methods of handling deadlock. The OS can **prevent** or **avoid** deadlock so the system never enters it, **detect and recover** after it occurs, or **ignore** the problem (the Ostrich approach used by UNIX and Windows).

A. Prevention makes sure at least one condition can never hold:

Condition attacked	Technique	Drawback
Mutual exclusion	Make resources sharable (read-only files, spooling a printer)	Not possible for inherently non-sharable resources
Hold and wait	Request all resources at start, or request only when holding none	Low utilisation, starvation
No preemption	If a request fails, release held resources; preempt from waiting processes	Only for resources whose state can be saved (CPU, memory)
Circular wait	**Total ordering** of resource types; request only in increasing order	Ordering is hard to choose; may be inconvenient

B. Avoidance uses advance information about each process's **maximum** needs and grants a request only if the system stays in a **safe state**. A state is safe if some sequence $\langle P_1 \ldots P_n \rangle$ exists in which every process can finish with the currently available resources plus those released by the processes before it. An unsafe state is not necessarily deadlocked but may lead to deadlock. For single-instance resources the **claim-edge RAG** method is used; for multiple instances, **Banker's algorithm** (Dijkstra).

Data structures: `Available[m], Max[n][m], Allocation[n][m], Need = Max − Allocation.`
Safety algorithm: Work = Available, Finish[i] = false. Find an i with Finish[i] = false and $Need_i \le$ Work; set Work = Work + $Allocation_i$ and Finish[i] = true; repeat. If every Finish[i] becomes true, the state is safe.
Resource-request algorithm: if $Request_i \le Need_i$ and $Request_i \le$ Available, pretend to allocate and run the safety algorithm. Grant the request only if the result is safe; otherwise P_i waits.

Worked example. 5 processes, resources A = 10, B = 5, C = 7; Available = (3, 3, 2).

Process	Allocation (A B C)	Max (A B C)	Need = Max − Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Step	Pick (Need ≤ Work)	Work after release
start	—	(3, 3, 2)
1	P1: (1,2,2) ≤ (3,3,2)	(5, 3, 2)
2	P3: (0,1,1) ≤ (5,3,2)	(7, 4, 3)
3	P4: (4,3,1) ≤ (7,4,3)	(7, 4, 5)
4	P0: (7,4,3) ≤ (7,4,5)	(7, 5, 5)
5	P2: (6,0,0) ≤ (7,5,5)	(10, 5, 7)

The system is **safe**, with safe sequence **⟨P1, P3, P4, P0, P2⟩**. ⟨P1, P3, P4, P2, P0⟩ is also safe, since a safe sequence need not be unique. *Request check:* P1 requests (1,0,2). This is ≤ Need (1,2,2) and ≤ Available (3,3,2). After the pretend allocation, Available = (2,3,0) and $Need_1$ = (0,2,0), and the sequence ⟨P1, P3, P4, P0, P2⟩ is still safe, so the **request is granted** (verified by simulation).

C. Detection and recovery. Deadlocks are allowed to happen and are then detected. For single instances, search the **wait-for graph** (a RAG with resource nodes collapsed) for a cycle, at $O(n^2)$ cost. For multiple instances, use a Banker-like detection algorithm that uses current **Request** instead of Need. Detection can run on every request, periodically, or when CPU utilisation drops. **Recovery** options: (i) **process termination**: abort all deadlocked processes, or abort one at a time until the cycle breaks, choosing the victim by priority, work done or resources held; (ii) **resource preemption**: select a victim, **roll back** it to a safe state or restart it, and prevent **starvation** by counting rollbacks in the cost factor.

5. Comparison

Basis	Prevention	Avoidance	Detection & recovery
Idea	Deny one condition	Stay in safe states	Let it happen, then fix
Advance info	None	Max needs of every process	None
Utilisation	Low	Medium	High
Overhead	Low (at design time)	Safety check on every request	Periodic detection + rollback cost
Example	Resource ordering	Banker's algorithm	Wait-for graph, process abort

Conclusion. Prevention is simple but wasteful, avoidance is precise but needs maximum demands in advance, and detection gives the highest utilisation at the cost of recovery. General-purpose OSs mostly ignore deadlock and leave it to applications, while databases use detection with transaction rollback.

P27. Explain file system management in an operating system. Describe the disk scheduling algorithms FCFS, SSTF, SCAN and C-SCAN with an example. *(15 marks · OS · Likelihood 8/10)*

1. Introduction. The **file system** gives a logical, uniform view of stored information. It maps **files** (named collections of related data) onto physical secondary storage and provides directories, access methods, allocation and protection. **Disk scheduling** orders pending I/O requests to reduce head movement.

2. File concept. A file has **attributes**: name, identifier, type, location, size, protection, timestamps and owner. Its **operations** are create, open, read, write, reposition (seek), delete, truncate and close. The OS keeps an **open-file table** (file pointer, open count, location). File **types** are indicated by extension (.exe, .c, .txt).

3. Access methods.

1. **Sequential**: records are processed in order, like a tape. Example: an editor or compiler.
2. **Direct (relative)**: any block n can be read or written directly. Example: a database.
3. **Indexed**: an index of pointers to blocks; large files use multi-level indexes, e.g. ISAM.

4. Directory structures. Single-level (name clashes), **two-level** (one directory per user), **tree-structured** (absolute and relative paths), **acyclic-graph** (sharing through links), and **general graph** (cycles allowed, which needs garbage collection).

5. Allocation methods.

Method	How	Pros	Cons
Contiguous	Start block + length	Fast sequential and direct access	External fragmentation; file growth hard
Linked	Each block points to the next (FAT keeps the pointers in a table)	No external fragmentation	Slow direct access; pointer overhead; reliability
Indexed	An index block holds all block addresses (UNIX inode: direct + single/double/triple indirect)	Fast direct access, no external fragmentation	Index-block overhead

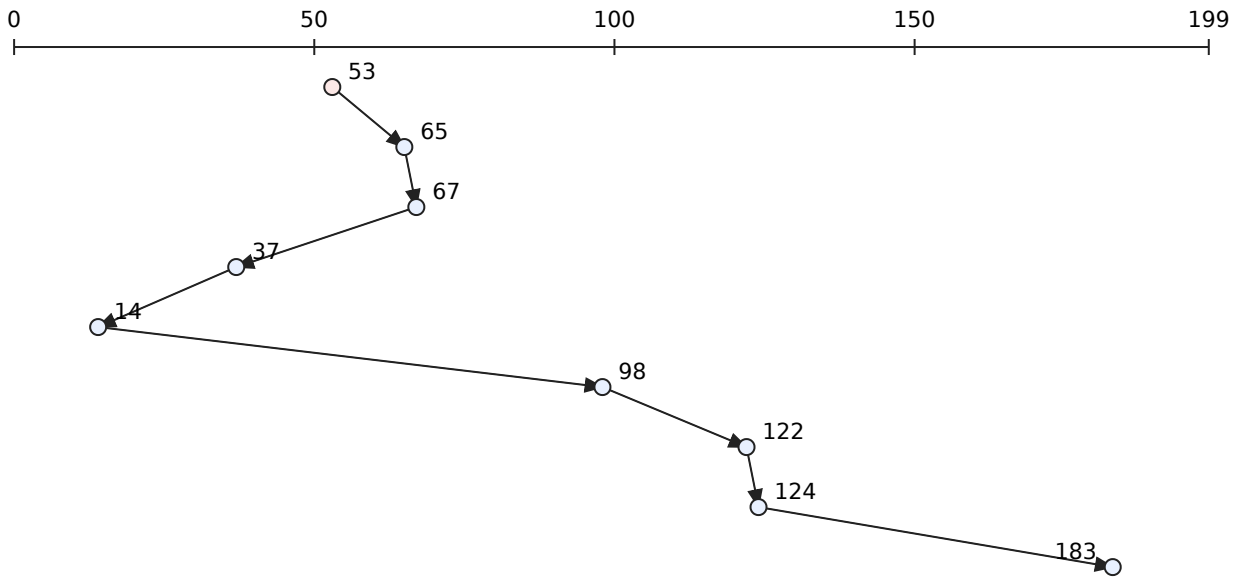
6. Free-space management. Bit vector (1 bit per block), **linked list** of free blocks, **grouping** (the first free block stores addresses of n free blocks), **counting** (address + count of contiguous free blocks).

7. Protection. Access types are read, write, execute, append, delete and list. Mechanisms are **access-control lists** and UNIX **owner/group/others rwx** bits, e.g. `chmod 754`.

8. Disk scheduling. Disk access time = **seek time + rotational latency + transfer time**. Seek time dominates, so the aim is to **minimise total head movement**.

Example. Cylinders 0–199. Queue: **98, 183, 37, 122, 14, 124, 65, 67**. Head at **53**; for SCAN the head is moving towards 0.

Algorithm	Order of service	Total head movement
FCFS: serve in arrival order	53→98→183→37→122→14→124→65→67	45+85+146+85+108+110+59+2 = **640**
SSTF: nearest request next	53→65→67→37→14→98→122→124→183	12+2+30+23+84+24+2+59 = **236**
SCAN (elevator): sweep to one end, reverse	53→37→14→**0**→65→67→98→122→124→183	53 + 183 = **236**
C-SCAN: sweep up to the end, jump back to 0, sweep up again	53→65→67→98→122→124→183→**199**→**0**→14→37	146 + 199 + 37 = **382** (183 if the return jump is not counted)



Cylinder axis on top; time runs downward. SSTF order shown; total = 236 cylinders.

Fig: SSTF head movement for queue 98, 183, 37, 122, 14, 124, 65, 67 with head at 53

Basis	FCFS	SSTF	SCAN	C-SCAN
Idea	Arrival order	Shortest seek first	Elevator, both directions	One direction, circular
Head movement	Highest	Low	Low	Moderate
Starvation	No	**Yes** (far requests)	No	No
Waiting-time uniformity	Fair but slow	Poor	Middle cylinders favoured	**Most uniform**
Best for	Light load	—	Heavy load	Heavy load, uniform response

LOOK and **C-LOOK** are variants that reverse at the last request instead of at the disk end. For this queue, LOOK moving towards 0 (as in SCAN) goes 53→37→14→65→…→183 = 39 + 169 = **208**. Moving towards 199 it would be 130 + 169 = 299, so always state the direction.

Conclusion. File-system management organises, allocates and protects data. Disk scheduling then reduces seek time. SSTF and SCAN cut head movement sharply compared with FCFS (236 against 640), and C-SCAN/C-LOOK give the most uniform waiting time under heavy load.

P28. Explain software testing strategies. Discuss levels of testing, black-box and white-box testing, integration approaches and regression testing. *(15 marks · SE · Likelihood 10/10)*

1. Introduction. Testing is the process of executing a program with the intent of **finding errors** (Myers). A **testing strategy** is a road map that sets out the steps, effort, time and resources of testing. Pressman describes it as a **spiral**: unit → integration → validation → system testing, working outwards from code to the whole system. Its principles are to test early, to have independent testers for the higher levels, and to plan tests from the requirements (verification and validation).

2. Levels of testing (V-model). Each development phase on the left has a matching test level on the right.

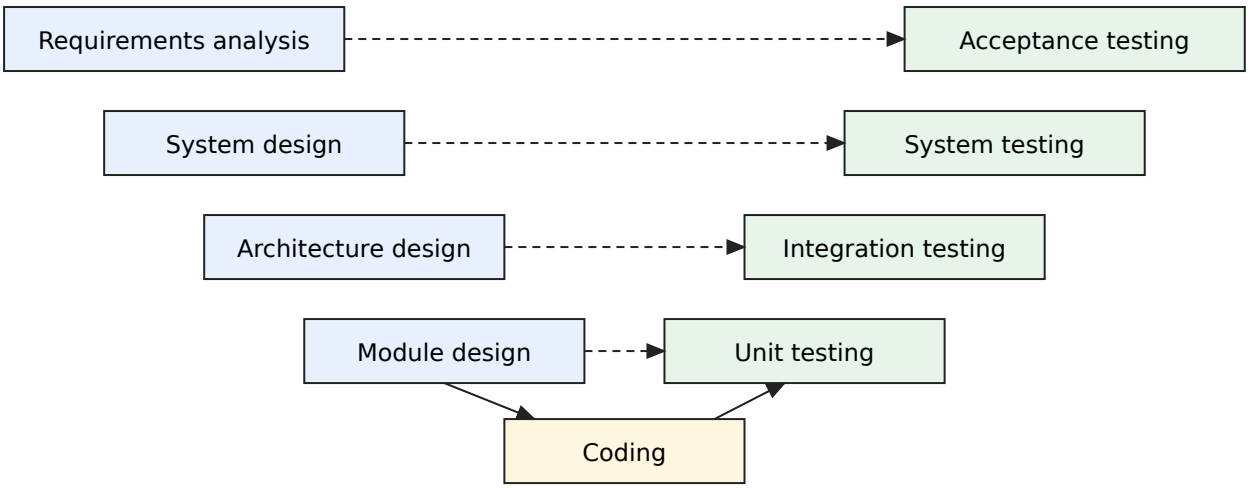


Fig: V-model; dashed arrows show which specification each test level is planned from

Level	Tests what	By	Technique
Unit	Individual module/function	Developer	Mainly white-box; uses drivers and stubs
Integration	Interfaces between modules	Developers/testers	Black + white box
System	Whole system against SRS, including NFRs: performance, load, stress, security, recovery, usability	Independent test team	Black-box
Acceptance	Fitness for the user's needs	Customer/users	**Alpha** (at developer's site) and **beta** (at user's site)

3. Black-box vs white-box testing.

- **Black-box (functional):** derived from the specification only.
  - **Equivalence partitioning**: e.g. for age 18–60, test one value from each class: <18, 18–60, >60.
  - **Boundary value analysis**: test 17, 18, 60, 61.
  - Cause–effect graphing, decision tables, state-transition testing.
- **White-box (structural):** derived from the code.
  - Statement, branch/decision and condition coverage.
  - **Basis-path testing** using **cyclomatic complexity** $V(G) = E - N + 2 = P + 1$ (P = number of predicate nodes). This gives the number of independent paths to test.
  - Loop testing and data-flow testing.

Basis	Black-box	White-box
Knowledge of code	No	Yes
Basis	Requirements	Logic/paths
Finds	Missing functions, interface errors	Logic errors, unreachable code
Level	System, acceptance	Unit, integration

4. Integration testing approaches.

1. **Big-bang**: combine everything at once. It is simple, but faults are hard to localise.
2. **Top-down**: start from the main module and add lower modules depth-first or breadth-first, using **stubs** for missing lower modules. Major control flaws show up early.
3. **Bottom-up**: start with the lowest modules (clusters) and use **drivers** to call them. No stubs are needed, but the program as a whole is not visible until the end.
4. **Sandwich (hybrid)**: top-down for the upper levels and bottom-up for the lower ones.

5. Regression testing. Re-run a selected subset of earlier tests after a **change** (bug fix or enhancement) to make sure no new errors have entered previously working parts. It is usually **automated** (e.g. in CI pipelines). **Smoke testing** is a quick daily build check of critical functions.

6. Other strategies. Validation testing against the SRS; **debugging**, which follows testing and uses brute force, backtracking or cause elimination; and object-oriented testing, which uses class/thread-based testing.

7. Advantages of a defined strategy: errors are found early, when they are cheaper to fix; coverage is systematic; it gives measurable quality and reliability and builds customer confidence.

Conclusion. Effective testing proceeds level by level (unit, integration, system, acceptance), combines black-box and white-box techniques, chooses an integration order, and protects every change with regression testing.

P29. Explain software process models: Waterfall, Prototyping, Spiral and Agile, with diagrams. Compare them. *(15 marks · SE · Likelihood 9/10)*

1. Introduction. A **software process model** is an abstract representation of the ordered set of activities (communication, planning, modelling, construction, deployment) used to develop software. Choosing the right model depends on how stable the requirements are, the size of the project, its risk and how much the customer is involved.

2. Waterfall (classic linear sequential) model, Royce 1970. The phases are done strictly **one after another**. Each phase finishes, with documentation, before the next begins.

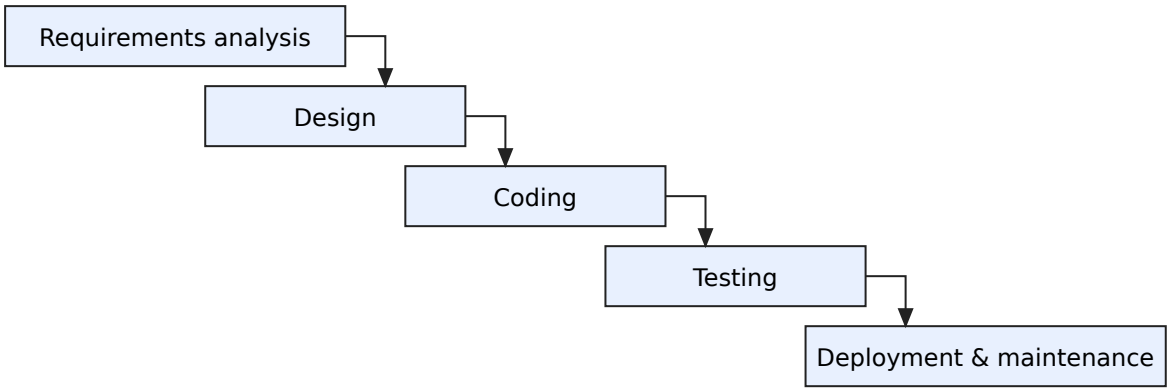


Fig: Waterfall model (each phase flows down into the next)

Pros: simple, easy to manage, well documented, and good when requirements are clear and stable (e.g. well-understood government payroll systems). *Cons:* inflexible to change; working software appears only late; errors are found late, which makes them costly; the customer has to state every requirement up front.

3. Prototyping model. A quick **prototype** (a mock-up of the UI or key features) is built from initial requirements, then evaluated by the customer and refined repeatedly until the requirements are clear. The real system is then engineered properly.
Cycle: *communication → quick plan → quick design → build prototype → customer evaluation/feedback → refine*, repeated as needed.
Pros: clarifies vague requirements, involves the user, and finds misunderstandings early. *Cons:* the customer may mistake the prototype for the product; quick-fix choices can creep into the final system; cost can rise. Types: **throw-away** and **evolutionary**.

4. Spiral model, Boehm 1986. A **risk-driven**, evolutionary model that combines prototyping with the systematic control of the waterfall. Each loop of the spiral is one phase and passes through four quadrants. The radius shows cumulative cost; the angle shows progress.

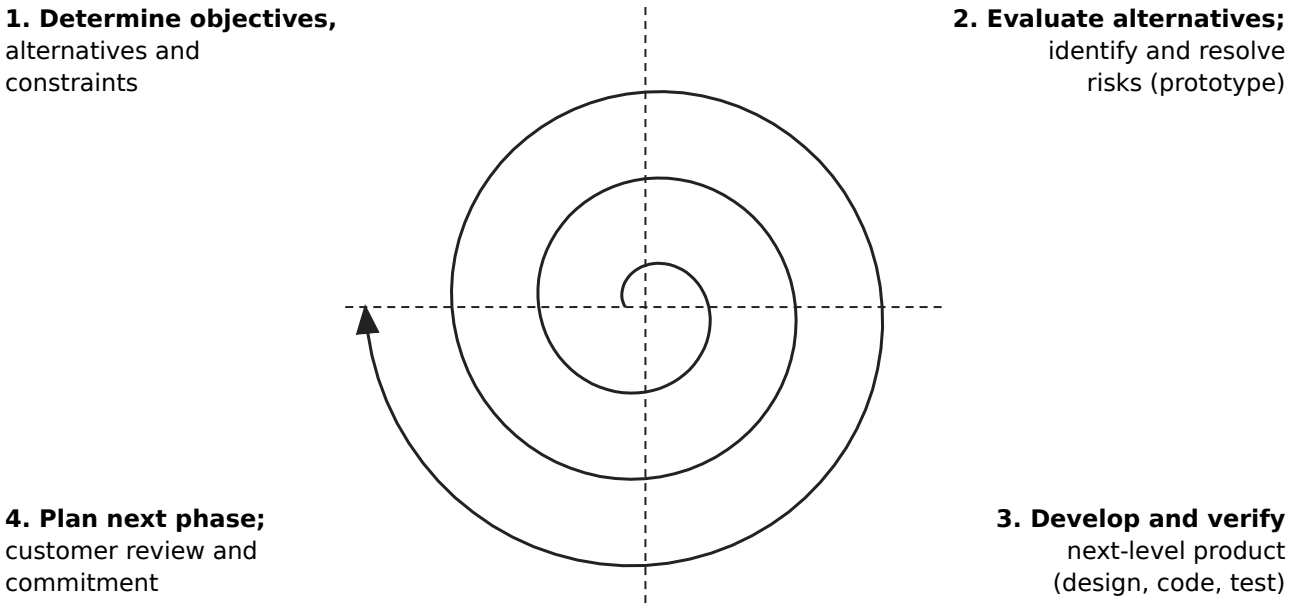


Fig: Boehm's spiral model: each clockwise loop passes the four quadrants; radius = cumulative cost

Pros: explicit **risk analysis**; suits large, complex, high-risk projects; changes can be made in later loops. *Cons:* needs risk-assessment expertise; costly; complex to manage; not suitable for small projects.

5. Agile model. An **iterative and incremental** approach based on the **Agile Manifesto (2001)**: individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan. Working software is delivered in short **iterations/sprints** (1–4 weeks).

- **Scrum:** roles are Product Owner, Scrum Master and Team; artefacts are the product backlog and sprint backlog; events are sprint planning, the daily stand-up, the review and the retrospective.
- **XP (Extreme Programming):** pair programming, test-driven development, continuous integration, refactoring.

Pros: fast, frequent delivery; welcomes change; high customer satisfaction. *Cons:* less documentation; depends on skilled, self-organising teams; hard to estimate fixed cost; scaling is difficult.

6. Comparison

Basis	Waterfall	Prototyping	Spiral	Agile
Approach	Linear, sequential	Iterative prototype → product	Iterative, risk-driven	Iterative + incremental
Requirements	Fixed, known up front	Unclear initially	Evolving	Continuously changing
Risk handling	Poor	Moderate (requirement risk)	**Explicit, best**	Good (short cycles)
Customer involvement	Start and end only	High during prototyping	Each loop	**Continuous**
Delivery of working software	At the end	After prototype acceptance	Incremental per loop	Every sprint
Documentation	Heavy	Moderate	Heavy	Light
Cost of change	Very high	Low early	Moderate	Low
Suitable for	Small/clear, regulated projects	UI-heavy, unclear requirements	Large, high-risk, mission-critical	Dynamic products, start-ups, web/mobile apps

Conclusion. No model is best in every situation. Use waterfall for stable, well-understood requirements, prototyping to discover requirements, spiral when risk dominates, and agile when change is constant and fast delivery matters. Many organisations use hybrids such as an agile-waterfall mix.

P30. What is a distributed database? Explain its architecture, fragmentation, replication, transparency and the two-phase commit protocol. Give advantages and disadvantages. *(15 marks · DBMS · Likelihood 9/10)*

1. Definition. A **distributed database (DDB)** is a collection of logically interrelated databases spread over multiple **sites** connected by a network. A **distributed DBMS (DDBMS)** manages it and makes the distribution **transparent**, so users see a single database. Example: a bank with branch databases in Kohima, Dimapur and Mokokchung.

2. Types.

- **Homogeneous**: every site runs the same DBMS and schema.
- **Heterogeneous**: sites run different DBMSs or schemas and need a gateway or middleware. A **multidatabase / federated** system is one in which the local sites stay autonomous.

3. Architecture.

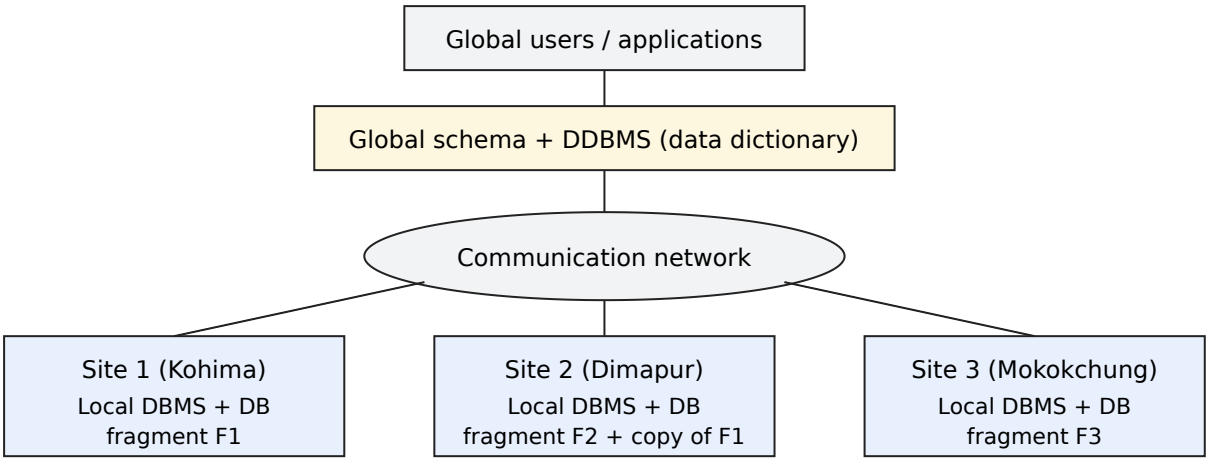


Fig: Distributed database architecture; each site has its own local DBMS, joined by a global layer

Each site has a **local transaction manager** and a **transaction coordinator** that starts and finishes global transactions. Schema levels: global (conceptual) schema → fragmentation schema → allocation schema → local schemas. The overall architecture can be client–server, collaborating servers or peer-to-peer.

4. Data fragmentation. A relation is split into pieces that are stored at different sites.

- **Horizontal fragmentation**: a subset of **rows** chosen by selection, e.g. σ_branch='Kohima'(account). The original is rebuilt by **UNION**.
- **Vertical fragmentation**: a subset of **columns** chosen by projection. Each fragment must keep the **primary key** (or a tuple-id). The original is rebuilt by **natural JOIN**.
- **Mixed (hybrid)**: both kinds combined.

A correct fragmentation must be **complete** (no data lost), **reconstructible** and **disjoint** (apart from the key in vertical fragments).

5. Data replication. Copies of a relation or fragment are kept at several sites, ranging from **full replication** (everywhere) through partial replication to none.

- *Pros:* availability, since a failed site does not stop access; faster local reads and parallelism.
- *Cons:* **update overhead**, because every copy must be kept consistent through synchronous or asynchronous propagation and primary-copy protocols.

6. Transparency. Users should not need to know:

- **Fragmentation transparency**: how a relation is split.
- **Replication transparency**: how many copies exist.
- **Location transparency**: where the data is.
- **Naming transparency**: data has unique global names.

Also failure transparency and transaction transparency.

7. Distributed transactions and Two-Phase Commit (2PC). Atomicity must hold across sites: either all sites commit or all abort.

- **Phase 1 (voting/prepare):** the coordinator logs ⟨prepare T⟩ and sends PREPARE to every participant. Each participant force-writes its log and replies **READY** (vote commit) or **ABORT**.
- **Phase 2 (decision):** if **all** voted READY, the coordinator logs ⟨commit T⟩ and sends COMMIT; otherwise it logs ⟨abort T⟩ and sends ABORT. Participants act and send ACKs.

Weakness: **blocking**. If the coordinator fails after participants have voted READY, they must wait. **Three-phase commit (3PC)** reduces blocking. Concurrency is handled by distributed locking (primary copy or majority protocols) or timestamps, and **distributed deadlock** is detected with a global wait-for graph.

8. Distributed query processing. The cost model includes **data transfer cost**. **Semi-join** strategies reduce what is shipped between sites.

9. Advantages and disadvantages

Advantages	Disadvantages
Data sharing with **local autonomy**	**Complexity** of DDBMS software
Reliability and availability (replication)	Higher cost (hardware, communication, skilled staff)
Faster local access; parallel query processing	Harder concurrency control, recovery, deadlock handling
Modular, incremental growth (add sites)	**Security** harder over a network
Matches distributed organisations	Integrity enforcement across sites is costly; no global standards

Conclusion. A DDB improves availability, performance and autonomy for geographically spread organisations. In return it needs fragmentation and allocation design, replica consistency, distributed concurrency control, and atomic commit through 2PC.

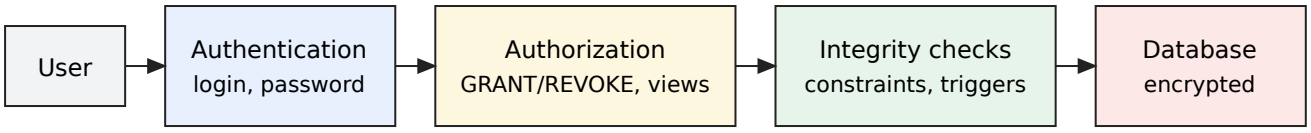
P31. Explain security and integrity in a DBMS. Discuss integrity constraints, authorization (GRANT/REVOKE), views and SQL injection. *(15 marks · DBMS · Likelihood 8/10)*

1. Introduction.

- **Integrity** means the data is **correct and consistent**. It protects against *accidental* errors such as invalid entries or broken references.
- **Security** means protecting the data against **unauthorised access**, which may be *intentional*: reading, changing or destroying it.

Both are enforced by the DBMS together with the DBA.

2. Threats. Loss of **confidentiality** (secrecy), **integrity** and **availability** (the CIA triad), arising from theft, fraud, SQL injection, insider misuse, hardware failure and so on.



Every request passes each layer before reaching the data

Fig: Layers of protection in a DBMS

3. Integrity constraints. These are rules declared in DDL and checked automatically on every insert, update and delete.

Constraint	Ensures	SQL
Domain	Value from a valid type/range	`cgpa DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 10)`
NOT NULL	Value must be present	`name VARCHAR(40) NOT NULL`
UNIQUE	No duplicates (candidate key)	`email VARCHAR(50) UNIQUE`
Entity integrity	Primary key is unique and **never NULL**	`PRIMARY KEY (roll_no)`
Referential integrity	A foreign-key value must match an existing primary key, or be NULL	`FOREIGN KEY (dept_id) REFERENCES dept(dept_id) ON DELETE CASCADE`
Assertions / triggers	Complex rules across tables	`CREATE ASSERTION ...; CREATE TRIGGER ... BEFORE UPDATE ...`

Transactions (the **ACID** properties) preserve integrity when failures or concurrent access occur.

4. Authorization (discretionary access control). Privileges are SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES. The owner or DBA grants them, optionally with the right to pass them on.

```
GRANT SELECT, UPDATE (cgpa) ON student TO clerk1;
GRANT SELECT ON student TO hod WITH GRANT OPTION;   -- hod may grant further
REVOKE UPDATE ON student FROM clerk1;
REVOKE SELECT ON student FROM hod CASCADE;           -- also from those hod granted
CREATE ROLE faculty;  GRANT SELECT ON marks TO faculty;  GRANT faculty TO teacher7;
```

- **Authorization graph**: a privilege survives only if a path back to the DBA still exists.
- **Roles** make it easier to give privileges to groups.
- **Mandatory access control (MAC)**, e.g. the Bell–LaPadula model with "no read up, no write down", labels data and users with levels: top secret, secret, confidential, unclassified.
- **Role-based access control (RBAC)** grants privileges through roles.

5. Views for security. A **view** is a virtual table that shows only selected rows or columns, so sensitive data is hidden:

```
CREATE VIEW cse_students AS
  SELECT roll_no, name, cgpa FROM student WHERE dept = 'CSE';   -- hides salary/phone, other
depts
GRANT SELECT ON cse_students TO cse_coordinator;
```

The user can query the view without having any privilege on the base table. Views therefore provide both **security** and **logical data independence**.

6. SQL injection. An attacker places SQL code in an input field, and the application **concatenates** that input into a query:

```
$q = "SELECT * FROM users WHERE name='$u' AND pwd='$p'";
// input  u = admin' --        → ...WHERE name='admin' --' AND pwd=''   (password check
commented out)
// input  p = ' OR '1'='1      → condition always true → login bypass
```

Consequences: authentication bypass, data theft, and data modification or deletion (`'; DROP TABLE users; --`).

Prevention:

1. **Parameterised queries / prepared statements**, e.g. `$stmt = $pdo->prepare("SELECT * FROM users WHERE name=? AND pwd=?"); $stmt->execute([$u,$p]);`
2. Input validation, whitelisting and escaping.
3. Stored procedures that do not build dynamic SQL.
4. **Least privilege**: the application account cannot DROP tables.
5. Error messages that hide SQL details.
6. A web application firewall (WAF).

7. Other security measures. Encryption (at rest, e.g. TDE, and in transit, e.g. TLS), password **hashing**, an **audit trail** (logs of who did what), backups and recovery, statistical-database protection against inference, and physical security.

8. Integrity vs security

Basis	Integrity	Security
Protects against	Accidental invalid data	Unauthorised (often malicious) access
Concern	Correctness and consistency	Confidentiality, authorised use
Mechanisms	Constraints, triggers, assertions, ACID	Authentication, GRANT/REVOKE, views, encryption, audit
Example	Rejecting cgpa = 12	Clerk cannot read salary

Conclusion. Integrity keeps the data correct and security keeps it in the right hands. A well-designed database combines declarative constraints, least-privilege authorization, views, encryption and injection-safe coding.

P32. Explain pattern recognition in AI. Discuss structured description, symbolic description, object identification and the speech recognition process. *(15 marks · AI · Likelihood 9/10)*

1. Introduction. Pattern recognition (PR) is the process of classifying input data (images, signals, characters, speech) into categories (classes) by extracting and analysing their features. It is the **perception** part of an intelligent agent, turning raw sensory data into symbols the agent can reason with. Applications include OCR, face and fingerprint recognition, medical image diagnosis and speech recognition.

2. General PR process. Sensing/data acquisition → pre-processing (noise removal, normalisation, segmentation) → **feature extraction / description** → **classification / recognition** (matching against stored classes) → post-processing and decision. Approaches:

- **Statistical (decision-theoretic)**: features form a vector, and the classifier is Bayes, k-NN or similar.
- **Structural (syntactic)**: patterns are described by primitives and their relations.
- **Neural**: learned from data.

3. Structured description (structural/syntactic PR). A complex pattern is described as a **composition of simpler sub-patterns (primitives)** together with the **relationships** between them, organised hierarchically, much as a sentence is built from words by grammar rules.

- Primitives: line segments, arcs, corners, strokes.

- Relations: above, left-of, connected-to, inside.
- A **pattern grammar** generates the valid patterns, and recognition is **parsing**: a pattern belongs to the class whose grammar can generate it.
- Example: the letter "A" is two slanted strokes meeting at the top, joined by a horizontal bar in the middle. A chromosome or ECG wave is described as a string of primitives.
- Strengths: captures structure, handles complex patterns, and can describe patterns it has not seen. Weaknesses: primitive extraction is sensitive to noise, and grammar inference is hard.

4. Symbolic description. The pattern is described with **symbols** that carry meaning, such as attribute–value pairs, predicates, semantic networks and frames, so that AI reasoning can operate on the result.

- Example: `object(o1), shape(o1, cylinder), colour(o1, red), has_part(o1, handle), on(o1, table).`
- It bridges low-level features and high-level knowledge. Descriptions are compared by **matching**: exact, partial or with variables through unification.
- Choosing the right symbols (the right level of abstraction) is essential. If they are too detailed, matching becomes costly; if too coarse, discrimination is lost.

5. Object identification. Recognising **what** an object in a scene is, and often **where** it is.

1. **Segmentation**: separate the object from the background using edges and regions.
2. **Feature extraction**: shape, edges, corners, texture, colour, moments; ideally invariant to translation, rotation and scale.
3. **Description**: a structural or symbolic model of the observed object.
4. **Matching**: compare the description with stored **object models** using template matching, feature-vector classification or graph matching.
5. **Verification**: check that the hypothesised object fits all the evidence.

Difficulties: viewpoint changes, occlusion, lighting, deformable objects and cluttered backgrounds. Modern systems use CNNs.

6. Speech recognition. Converting spoken utterances into text or commands.

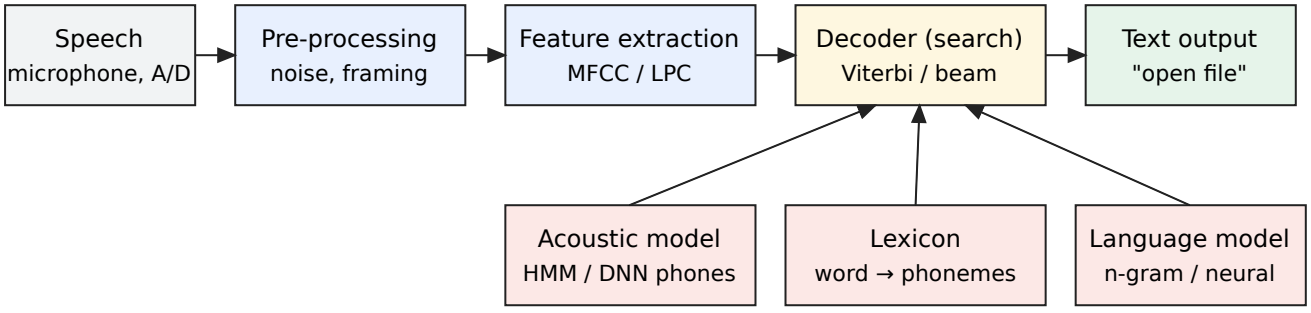


Fig: Speech recognition pipeline; the decoder combines acoustic, lexical and language knowledge

Steps:

1. **Signal acquisition and digitisation** (sampling).
2. **Pre-processing**: noise reduction, pre-emphasis, splitting into 20–30 ms overlapping frames, windowing.
3. **Feature extraction**: spectral features per frame, e.g. **MFCC** (mel-frequency cepstral coefficients) or LPC.
4. **Acoustic matching**: map features to **phonemes** using an **acoustic model**. Classic systems use **Hidden Markov Models**, with template matching by **dynamic time warping** in early systems; modern systems use deep neural networks.
5. **Lexical analysis**: a pronunciation dictionary assembles phonemes into words.

6. **Language model** (syntax/semantics): n-gram probabilities pick likely word sequences, e.g. "recognise speech" rather than "wreck a nice beach".
7. **Decoding**: search for the most probable word sequence W* = argmax P(A|W)·P(W) (Viterbi or beam search).

Difficulties:

- Speaker variability (accent, gender, speed).
- **Continuous speech segmentation**, since there are no pauses between words.
- Coarticulation, background noise, homophones.
- Large vocabulary.

Classification of systems: isolated vs continuous words; speaker-dependent vs speaker-independent; small vs large vocabulary.

7. Comparison of description types

Basis	Structured description	Symbolic description
Represents	Primitives + spatial/structural relations	Meaningful symbols, predicates, frames
Recognition by	Parsing with a pattern grammar	Symbolic matching / inference
Good for	Complex shapes, characters, waveforms	Scene interpretation, reasoning
Weakness	Noise-sensitive primitives	Choosing the right abstraction level

Conclusion. Pattern recognition turns raw percepts into structured and symbolic descriptions. These support object identification and speech recognition, which are the perception front-end of AI systems. Statistical and neural methods now complement the classical structural approach.

P33. Compare the OSI and TCP/IP reference models with a neat diagram. Give the protocols at each layer. Explain IP addressing and subnetting with an example. *(15 marks · CN · Likelihood 9/10)*

1. Introduction. A **reference model** divides network communication into **layers**. Each layer provides services to the layer above and uses the services of the layer below, which makes the design modular and allows different vendors' equipment to work together. **OSI** (ISO, 1984) is a 7-layer *conceptual* model. **TCP/IP** (DoD/ARPANET) is the *practical* model on which the Internet runs.

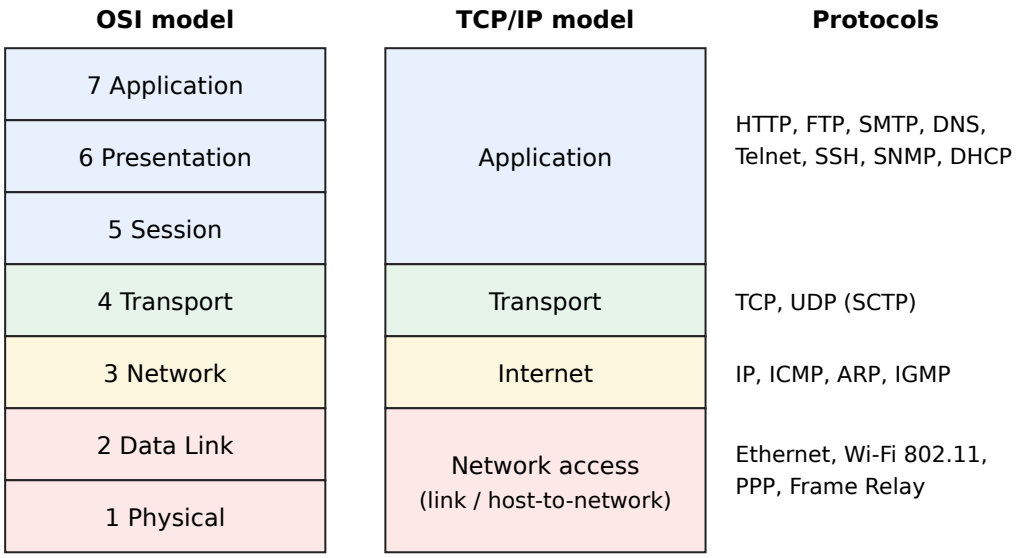


Fig: OSI (7 layers) mapped to TCP/IP (4 layers) with typical protocols

2. OSI layers (7 → 1).

Layer	Function	Data unit	Protocols / devices
7 Application	User services, network access for applications	Data	HTTP, FTP, SMTP, DNS
6 Presentation	Translation, encryption, compression	Data	SSL/TLS, JPEG, ASCII/EBCDIC
5 Session	Dialog control, synchronisation (checkpoints)	Data	RPC, NetBIOS
4 Transport	Process-to-process delivery, segmentation, flow and error control	Segment	TCP, UDP
3 Network	Logical addressing, routing	Packet	IP, ICMP; router
2 Data Link	Framing, MAC addressing, error detection (CRC), access control	Frame	Ethernet, PPP; switch
1 Physical	Bits over the medium, signalling, topology	Bit	RS-232, cables; hub, repeater

3. TCP/IP layers.

- **Application**: combines OSI layers 5–7.
- **Transport**: TCP is reliable and connection-oriented, with a 3-way handshake (SYN, SYN-ACK, ACK); UDP is connectionless and fast.
- **Internet**: IP gives best-effort, connectionless routing.
- **Network access**: hardware and framing.

Forouzan presents TCP/IP as **5 layers**, splitting network access into data-link and physical.

4. Comparison

Basis	OSI	TCP/IP
Layers	7	4 (or 5)
Developed by	ISO, as a generic model	DoD, around existing protocols
Approach	Model first, protocols later	Protocols first, model describes them
Service/interface/protocol distinction	Clear	Not clearly separated
Session and presentation	Separate layers	Merged into application
Network layer	Connectionless + connection-oriented	Connectionless only (IP)
Transport layer	Connection-oriented emphasis	TCP (connection-oriented) and UDP (connectionless)
Usage	Teaching/reference	**Actual Internet**

5. IP addressing. An IPv4 address is 32 bits, written as 4 decimal octets, and consists of **network-id + host-id**.

- **Classful addressing**: A /8, B /16, C /24, D multicast, E reserved.
- **Classless addressing (CIDR)**: notation a.b.c.d/n, where n = number of network bits.
- **Subnet mask**: n ones followed by zeros. **Network address = IP AND mask**; the broadcast address has all host bits set to 1.
- Usable hosts = 2^(32−n) − 2.
- **Private ranges**: 10/8, 172.16/12, 192.168/16, used with NAT.
- **IPv6** is 128 bits (e.g. 2001:db8::1).

6. Subnetting example 1. Divide **192.168.10.0/24** into **4 equal subnets**.
Borrow 2 bits ($2^2 = 4$), giving a new prefix of **/26** and mask **255.255.255.192**. Block size = 256 − 192 = **64**. Hosts per subnet = $2^6 - 2$ = **62**.

Subnet	Network address	First host	Last host	Broadcast
1	192.168.10.0/26	192.168.10.1	192.168.10.62	192.168.10.63
2	192.168.10.64/26	192.168.10.65	192.168.10.126	192.168.10.127
3	192.168.10.128/26	192.168.10.129	192.168.10.190	192.168.10.191
4	192.168.10.192/26	192.168.10.193	192.168.10.254	192.168.10.255

Subnetting example 2. Host **172.16.45.130/20**.

- Mask /20 = **255.255.240.0**, so the third octet has block size 16.
- 45 AND 240 = 32, so the **network is 172.16.32.0/20**.
- The **broadcast** is 172.16.47.255, since 32 + 16 − 1 = 47.
- The usable host range is 172.16.32.1–172.16.47.254, giving $2^{12} - 2$ = **4094 hosts**.

(All values checked with Python's `ipaddress` module.)

Benefits of subnetting: smaller broadcast domains, better security and management, and efficient use of addresses. **VLSM** gives subnets of different sizes.

Conclusion. OSI is the reference framework for understanding networking, while TCP/IP is the working protocol suite of the Internet. IP addressing with CIDR and subnetting lets that suite scale to billions of hosts efficiently.

Part D1 — Real 2025 Paper-II, fully answered

All 19 questions of the actual NPSC CTSE 2025 Descriptive Paper-II (booklet 2537_25 (II)), answered at the length the marks deserve: 5 marks ≈ 150–220 words, 10 marks ≈ 350–500 words, 15 marks ≈ 600–850 words. Numericals were checked with python3.

Section A (5 marks each)

Importance 7/10 · good chance · Section A repeats syllabus basics; any of these could come back almost word for word.

Q1. What are the key components of an operating system? Briefly explain the function of each. *(5 marks · OS)*

An **operating system** is system software that manages the computer's hardware and gives programs a convenient, efficient and protected environment to run in. It acts as the **resource manager** and **control program**.

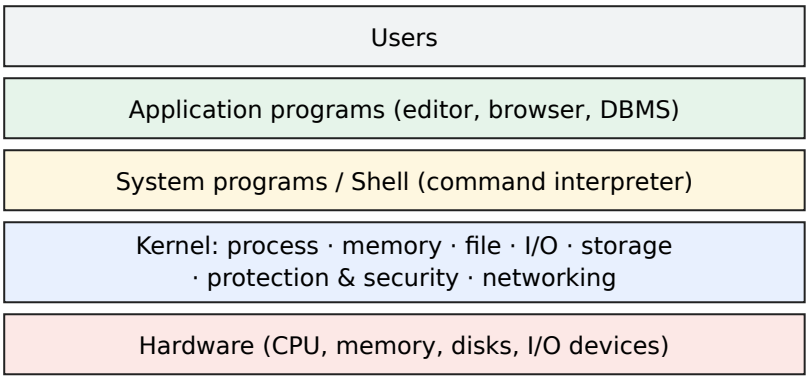


Fig: Layered view of a computer system, with the OS components in the kernel layer

1. **Process management:** creates, schedules, suspends and terminates processes. It also provides synchronisation (semaphores), inter-process communication and deadlock handling.

2. **Main-memory management:** keeps track of which parts of memory are in use and by whom, allocates and frees memory, and maps logical to physical addresses (paging, segmentation, virtual memory).
3. **File management:** creates and deletes files and directories, maps files to secondary storage, and controls access to them.
4. **I/O (device) management:** hides device details using **device drivers**, buffering, caching and spooling.
5. **Secondary-storage management:** manages free space, allocates disk blocks and schedules the disk (FCFS, SSTF, SCAN).
6. **Protection and security:** controls access to resources (user IDs, permissions) and defends against attacks.
7. **Command interpreter / user interface:** a shell (CLI) or GUI that reads user commands and runs them using **system calls**.
8. **Networking:** lets the system share resources across machines through protocol stacks and sockets.

Why it matters for 2026: it can reappear as "Explain the services of an OS" or "Explain system calls", and the same 6–8 components answer both.

Q2. Differentiate between logical and physical addresses in memory management. *(5 marks · OS)*

The **logical (virtual) address** is the address the CPU generates. The **physical address** is the one the memory unit actually sees in the memory-address register. The **Memory-Management Unit (MMU)** maps one to the other at run time.

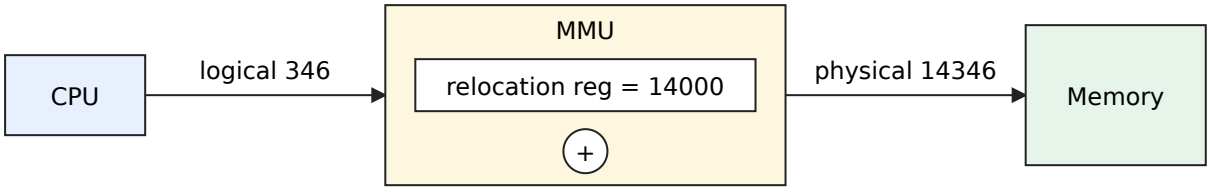


Fig: Dynamic relocation by the MMU (Silberschatz example: 346 + 14000 = 14346)

Basis	Logical address	Physical address
Generated by	CPU, while a program runs	MMU, after translation
Visibility	Seen by the user program	Never seen by the user program
Address space	Logical address space (0 to max)	Physical space (R+0 to R+max)
Existence	Virtual; may not be in RAM	Actual location in RAM
Variation	Same each time the program runs	Can change on swapping or relocation
Binding	Differs from physical with execution-time binding	Same as logical with compile- or load-time binding

Key point: with compile-time or load-time binding the two addresses are identical. With **execution-time binding**, which paging and segmentation use, they differ and the MMU translates every reference.

Why it matters for 2026: it is the base for paging and TLB numericals, so expect "explain address binding" or "role of the MMU".

Q3. What are the different phases in a typical Software Development Life Cycle (SDLC)? Write a brief note on each of them. *(5 marks · SE)*

The **SDLC** is the structured sequence of phases that takes software from an idea to a working, maintained product. It is the basis of process models such as Waterfall, Spiral and Agile.

1. **Feasibility study / planning:** decide whether the project is technically, economically and operationally worth doing. Output: a feasibility report, scope and rough estimates.
2. **Requirement analysis and specification:** gather functional and non-functional requirements from stakeholders. Output: the **SRS** (Software Requirements Specification).
3. **Design:** turn the SRS into architecture (high-level design) and module logic, data structures and interfaces (low-level design). Output: the design document (DFDs, UML).
4. **Coding / implementation:** write code module by module to the design and coding standards, and unit-test each module.
5. **Testing:** integration, system and acceptance testing (alpha/beta) to find defects and confirm the product meets the SRS.
6. **Deployment:** install the software in the user's environment, migrate data and train users.
7. **Maintenance: corrective** (fix bugs), **adaptive** (new environment), **perfective** (new features) and **preventive** maintenance. This is usually the longest and costliest phase.

Note: each phase produces a deliverable that feeds the next one. Reviews at the end of each phase catch errors early, when they are cheapest to fix.

Why it matters for 2026: expect a follow-up such as "compare Waterfall, Spiral and Agile" or "explain the SRS", both of which are syllabus phrases.

Q4. Explain the purpose of a Work Breakdown Structure (WBS) in software project management. *(5 marks · SE)*

A **Work Breakdown Structure (WBS)** is a **hierarchical, deliverable-oriented decomposition** of a project's total scope into smaller, manageable parts. The lowest level is the **work package**, which can be estimated, assigned and tracked. Following the **100% rule**, the children of each node together cover all of the parent's work, and no more.

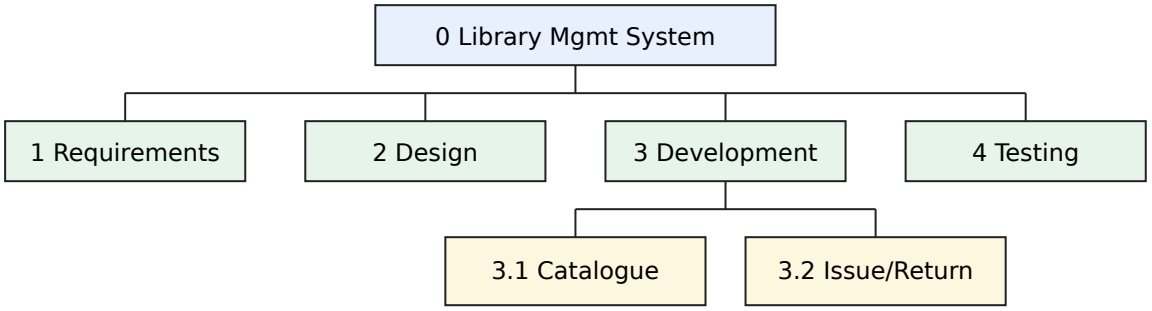


Fig: A WBS tree; the yellow leaves are work packages

Purpose / uses:

1. **Defines the scope completely:** nothing is missed and nothing out of scope is added.
2. **Basis for estimation:** effort and cost are estimated per work package and summed (bottom-up estimation).
3. **Basis for scheduling:** work packages become activities in the **Gantt chart** and **PERT/CPM** network.
4. **Assigning responsibility:** each work package has one owner, which gives clear accountability.
5. **Monitoring and control:** progress and earned value are tracked per package.
6. **Risk identification and communication:** a shared picture of the work for the team and the client.

Why it matters for 2026: it pairs naturally with "Gantt chart vs PERT", "critical path" and "project scheduling", all still unasked syllabus phrases.

Q5. What are the key characteristics of a Relational Database Management System (RDBMS)? *(5 marks · DBMS)*

An **RDBMS** is a DBMS based on E.F. Codd's **relational model** (1970), in which all data is stored as **relations (tables)**. Examples: MySQL, PostgreSQL, Oracle.

1. **Tabular structure:** data sits in tables made of **tuples (rows)** and **attributes (columns)**. Each attribute takes **atomic** values from a **domain** (First Normal Form, 1NF).
2. **Keys and uniqueness:** every row is uniquely identified by a **primary key**. Tables are linked by **foreign keys**.
3. **Integrity constraints: domain**, **entity integrity** (the primary key cannot be NULL) and **referential integrity** (a foreign key must match an existing primary key or be NULL). NOT NULL, UNIQUE and CHECK are also supported.
4. **Declarative query language: SQL** (DDL, DML, DCL), based on relational algebra and calculus. Users say *what* they want, not *how* to get it.
5. **Normalisation:** schemas are decomposed (1NF to BCNF) to reduce redundancy and update anomalies.
6. **ACID transactions:** Atomicity, Consistency, Isolation and Durability, with concurrency control (locking) and recovery (logs).
7. **Data independence:** a three-schema architecture (external, conceptual, internal) gives logical and physical data independence.
8. **Security and multi-user access:** views, GRANT and REVOKE, and authentication.
9. **Order independence:** the order of rows and columns carries no meaning.

Why it matters for 2026: it can return as "Codd's rules", "integrity constraints" or "DBMS vs file system".

Q6. Explain the concept of a primary key and a foreign key in database design. *(5 marks · DBMS)*

Primary key (PK): a **candidate key** chosen by the designer to uniquely identify each tuple in a relation. Its properties:

1. **Unique:** no two rows share the same value.
2. **NOT NULL:** this is the entity-integrity rule.
3. **Minimal:** no proper subset of it is unique.
4. **Stable:** its value rarely changes.
5. **One per table:** a table has only one primary key, which may be **composite** (more than one column).

Foreign key (FK): an attribute, or set of attributes, in one relation (the **referencing / child** table) whose values must match the primary key of another relation (the **referenced / parent** table), or be NULL. This is **referential integrity**, and it is how relationships between tables are represented.

Example. DEPARTMENT is the parent table:

dept_id (PK)	dept_name
10	CSE
20	ECE

STUDENT is the child table:

roll_no (PK)	name	dept_id (FK)
101	Asha	10
102	Vito	20

```
CREATE TABLE student (
  roll_no INT PRIMARY KEY,
  name    VARCHAR(40),
  dept_id INT REFERENCES department(dept_id) ON DELETE SET NULL
);
```

Basis	Primary key	Foreign key
Purpose	Identifies rows of its own table	Links to a row of another table
NULL	Not allowed	Allowed (unless declared NOT NULL)
Duplicates	Not allowed	Allowed (many students in one department)
Number per table	One	Many

Why it matters for 2026: it can come back as "types of keys (super, candidate, alternate)" or "integrity constraints", so learn the full key hierarchy.

Q7. What is the OSI model? Briefly list the seven layers. *(5 marks · CN)*

The **Open Systems Interconnection (OSI) reference model** was published by **ISO** in 1984. It is a **seven-layer conceptual framework** that divides network communication into layers. Each layer provides services to the layer above and uses the services of the layer below, so systems from different vendors can **interoperate**.

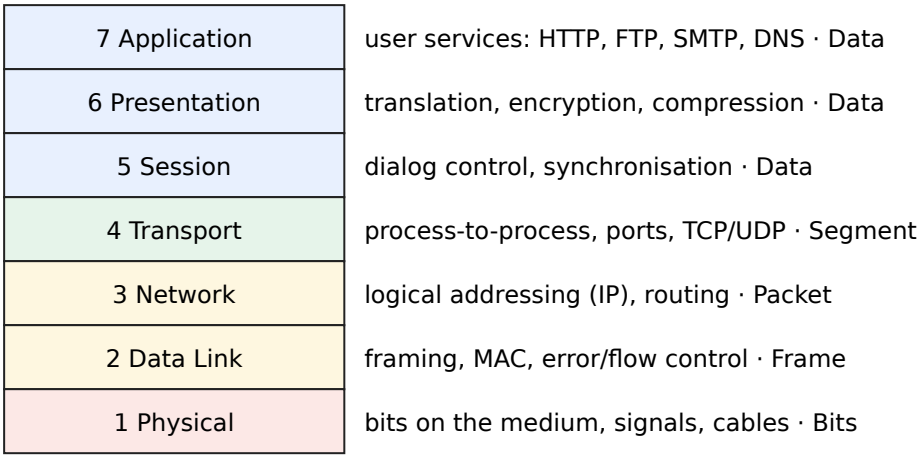


Fig: OSI layers 7 (top) to 1 (bottom), with each layer's main function and PDU

Mnemonic for layers 7 to 1: "**All People Seem To Need Data Processing**".

Key features: layers are independent, and each one adds its own header on the way down (**encapsulation**) and removes it on the way up. **Peer layers** on two hosts talk through **protocols**. Adjacent layers talk through **interfaces**. The TCP/IP model merges layers 5–7 into a single Application layer.

Why it matters for 2026: likely variants are "OSI vs TCP/IP model", "functions of the transport layer" and "PDU at each layer".

Q8. Differentiate between TCP and UDP protocols. *(5 marks · CN)*

TCP (Transmission Control Protocol) and **UDP (User Datagram Protocol)** are the two main **transport-layer** protocols of the TCP/IP suite. Both give process-to-process delivery using **port numbers**.

Basis	TCP	UDP
Connection	**Connection-oriented**: 3-way handshake (SYN, SYN-ACK, ACK)	**Connectionless**: no setup
Reliability	Reliable: acknowledgements, retransmission, checksum	Unreliable: checksum only, no ACK or retransmission
Ordering	Delivers bytes in order (sequence numbers)	No ordering guarantee
Flow / congestion control	Yes: sliding window, slow start	None
Data unit	Byte stream (**segments**)	Message-oriented (**datagrams**)
Header size	20–60 bytes	Fixed **8 bytes**
Speed / overhead	Slower, higher overhead	Faster, low overhead
Broadcast / multicast	Not supported	Supported
Typical uses	HTTP/HTTPS, FTP, SMTP, SSH	DNS queries, DHCP, VoIP, video streaming, online games, SNMP

Conclusion: use **TCP** when correctness matters more than delay, such as file transfer or web pages. Use **UDP** when timeliness matters more than perfect delivery, such as real-time audio and video, or short request/reply exchanges like DNS.

Why it matters for 2026: it may come back as "explain the TCP 3-way handshake" or "TCP header fields". Memorise SYN, SYN-ACK, ACK and the 8-byte UDP header.

Q9. Explain the difference between the GET and POST methods in HTML forms. *(5 marks · Web)*

The `method` attribute of `<form>` decides how form data is sent to the server. **GET** appends the data to the URL as a **query string**. **POST** sends it in the **body** of the HTTP request.

```
<form action="login.php" method="post">
  <input type="text" name="user"> <input type="password" name="pwd">
  <input type="submit" value="Login">
</form>
<!-- GET would produce: login.php?user=asha&pwd=123  (visible!) -->
```

Basis	GET	POST
Where data goes	In the URL (`?name=value&...`)	In the HTTP request body
Visibility	Visible in the address bar, history and server logs	Not visible in the URL
Security	Unsuitable for passwords or sensitive data	Better (but still needs HTTPS for real security)
Size limit	Limited by URL length (browser/server dependent, a few KB)	No practical limit; large data and file uploads
Data type	ASCII text only	Text and binary (`multipart/form-data`)
Caching / bookmarking	Can be cached and bookmarked	Not cached; cannot be bookmarked
Idempotent	Yes: safe to repeat (search, filter)	No: repeating may resubmit (payment, insert)
PHP access	`$_GET['user']`	`$_POST['user']`

Rule of thumb: use **GET** to *retrieve* data (search pages), and **POST** to *send or change* data (login, registration, uploads).

Why it matters for 2026: Web Tech gets about 2 Sec A questions every year, and follow-ups like "form validation in PHP" or "superglobals" are likely.

Q10. What are the fundamental syntax rules and the use of variables in PHP scripting? *(5 marks · Web)*

PHP (PHP: Hypertext Preprocessor) is a server-side scripting language embedded in HTML. The server runs it and sends only the resulting HTML to the browser.

Syntax rules:

1. PHP code goes inside `<?php ... ?>` tags. The file is saved as `.php`.
2. Every statement ends with a **semicolon** `;`.
3. Comments: `//` and `#` for single lines, `/* ... */` for multiple lines.
4. **Case sensitivity:** variable names are case-sensitive (`$age` ≠ `$Age`). Keywords and function names (`echo`, `if`, `strlen`) are not.
5. Output is written with `echo` or `print`. Whitespace is ignored. Blocks use `{ }`.

Variables:

1. Every variable starts with `$`, followed by a letter or underscore, then letters, digits or underscores (`$_x1` is valid, `$1x` is not).
2. **No declaration needed:** a variable is created when it is first assigned.
3. **Loosely (dynamically) typed:** the type follows the value (int, float, string, bool, array, object, NULL).
4. **Scope: local** (inside a function), **global** (reached inside functions with the `global` keyword or `$GLOBALS`), and **static** (keeps its value between calls).
5. **Superglobals** are available everywhere: `$_GET`, `$_POST`, `$_SERVER`, `$_SESSION`, `$_COOKIE`.

```
<?php
  $name = "Kohima";   // string
  $year = 2026;       // integer
  echo "Hello $name, year $year";   // double quotes expand variables
  function counter() { static $c = 0; $c++; return $c; }
?>
```

Why it matters for 2026: likely next asks are "PHP arrays and functions", "sessions vs cookies" and "PHP–MySQL connectivity".

Section B (10 marks each)

Importance 8/10 · likely · these are core syllabus algorithms and models; RR and topologies are perennial favourites.

Q11. Explain the Round Robin CPU scheduling algorithm using an example. What are its advantages and disadvantages? *(10 marks · OS)*

Introduction. Round Robin (RR) is a **preemptive** CPU-scheduling algorithm designed for **time-sharing** systems. Each process gets the CPU for at most one fixed **time quantum (time slice)**, q. If it has not finished, it is **preempted** and placed at the **tail of the ready queue**, which is a circular FIFO. RR is FCFS plus preemption.

Algorithm

1. Keep the ready queue as a FIFO queue. New arrivals join the tail.
2. Dispatch the process at the head and set a timer to interrupt after q.
3. If its CPU burst is ≤ q, it finishes and releases the CPU voluntarily.
4. Otherwise the timer interrupt causes a **context switch**: the process goes to the tail and the next one is dispatched.
5. No process waits more than $(n - 1) \times q$ before its next turn.

Convention used: when a process arrives at the same instant another is preempted, the new arrival is queued first.

Worked example (quantum q = 2 ms)

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	1
P4	3	2

Trace: at t=0 only P1 is ready and runs 0–2. P2 and P3 have arrived, so the queue is P2, P3, P1. P2 runs 2–4 and P4 arrives at 3, so the queue is P3, P1, P4, P2. P3 runs 4–5 and finishes. P1 runs 5–7, P4 runs 7–9 (done), P2 runs 9–10 (done), P1 runs 10–11 (done).

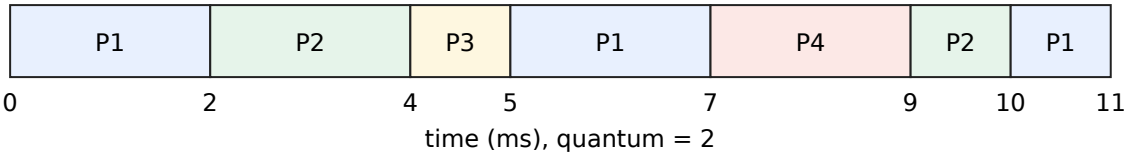


Fig: Gantt chart for Round Robin (q = 2)

Turnaround time (TAT) = Completion − Arrival. Waiting time (WT) = TAT − Burst.

Process	Completion	TAT	WT
P1	11	11 − 0 = 11	11 − 5 = 6
P2	10	10 − 1 = 9	9 − 3 = 6
P3	5	5 − 2 = 3	3 − 1 = 2
P4	9	9 − 3 = 6	6 − 2 = 4
Average		**29 / 4 = 7.25 ms**	**18 / 4 = 4.5 ms**

There were 6 context switches (7 slices).

Effect of the time quantum

- **q very large:** RR becomes **FCFS**.
- **q very small:** the CPU spends most of its time on **context-switch overhead**, which is called processor sharing.
- **Rule of thumb (Silberschatz):** about **80% of CPU bursts should be shorter than q**. The quantum should also be much larger than the context-switch time.

Advantages

1. **Fair:** every process gets an equal share, and there is **no starvation**.
2. **Good response time** for interactive and time-sharing users.
3. Simple to implement with a circular queue.
4. Waiting time is bounded at (n − 1) × q.

Disadvantages

1. **Higher average waiting and turnaround time** than SJF/SRTF.
2. **Context-switch overhead**, especially when q is small.
3. Performance **depends heavily on choosing q** well.
4. No notion of priority, so urgent processes are treated like any other.

Conclusion. RR trades some average turnaround time for **fairness and responsiveness**, which is why it underlies most time-sharing schedulers, often inside multilevel queues.

Why it matters for 2026: expect an RR, SJF/SRTF or Priority Gantt numerical, often "compare with FCFS for the same data". Practise the queue-ordering convention.

Q12. Discuss the different types of network topologies, highlighting their key features, advantages, and disadvantages. *(10 marks · CN)*

Introduction. Network topology is the **physical or logical arrangement** of nodes (devices) and links in a network. It affects cost, performance, reliability and fault tolerance. The main topologies are **bus, star, ring, mesh, tree and hybrid**.

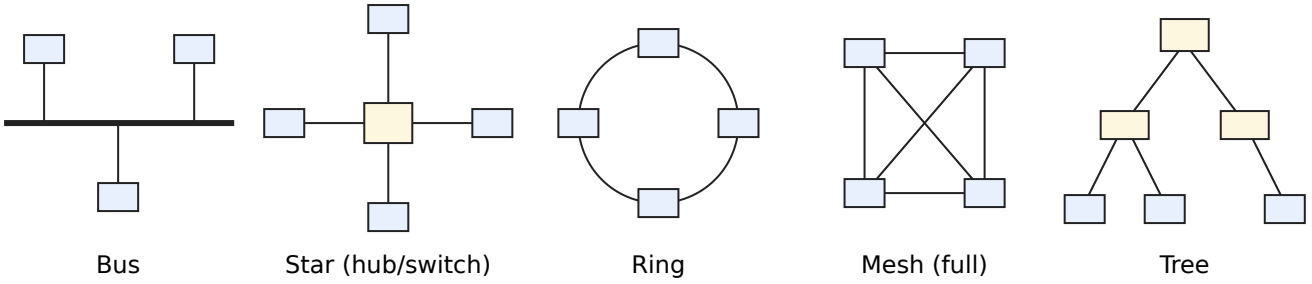


Fig: Basic network topologies (blue = hosts, yellow = hub/switch)

1. Bus. All devices share one backbone cable, with terminators at both ends. A frame broadcast on the bus is seen by every node.
Advantages: cheap, least cabling, easy to install. *Disadvantages:* a fault in the backbone brings the whole network down, collisions increase with load, fault isolation is hard and the number of nodes is limited.

2. Star. Every device has a dedicated link to a central **hub or switch**. This is the most common LAN layout (Ethernet).
Advantages: easy to install, add or remove nodes, one link failing affects only that node, and faults are easy to isolate. *Disadvantages:* the hub is a **single point of failure**, and it needs more cable than a bus.

3. Ring. Each device connects to exactly two neighbours, forming a closed loop. Data travels one way, often using a **token** (Token Ring, FDDI uses a dual ring).
Advantages: orderly access with no collisions, and equal access for all nodes. *Disadvantages:* one break can disable the ring unless it is a dual ring, and adding or removing a node disrupts the network.

4. Mesh. Every device has a dedicated point-to-point link to every other device. For **n nodes**, a full mesh needs **n(n − 1)/2 links** and **n − 1 I/O ports per node**. For example, 5 nodes need 10 links.
Advantages: highly robust with redundant paths, no traffic sharing, privacy and easy fault detection.
Disadvantages: expensive, with a lot of cabling and ports. It is used in WAN backbones.

5. Tree (hierarchical). A star of stars: a root hub connects to secondary hubs, which connect to hosts.
Advantages: scalable and easy to expand, with hierarchical management. *Disadvantages:* a failure at the root or a branch hub cuts off a whole subtree, and there is a lot of cabling.

6. Hybrid. Two or more topologies combined, for example star-bus or star-ring. *Advantages:* flexible and reliable. *Disadvantages:* complex design and higher cost.

Comparison

Topology	Cabling cost	Fault tolerance	Single point of failure	Expansion
Bus	Lowest	Poor	Backbone cable	Limited
Star	Medium	Good (per link)	Central hub	Easy
Ring	Medium	Poor (single ring)	Any link (single ring)	Disruptive
Mesh	Highest	Excellent	None	Costly
Tree	High	Moderate	Root hub	Easy

Conclusion. No topology is best for every case. **Star** dominates LANs because it is easy to manage, and **mesh** (partial) is used where reliability is critical, as in WAN backbones.

Why it matters for 2026: likely variants are "the mesh link formula n(n−1)/2", "LAN vs MAN vs WAN" and "transmission media".

Q13. Describe the phases of website development, from planning to maintenance. *(10 marks · Web)*

Introduction. Website development follows a life cycle like the SDLC, adapted to the web. Each phase produces a deliverable, and the cycle repeats as the site evolves.

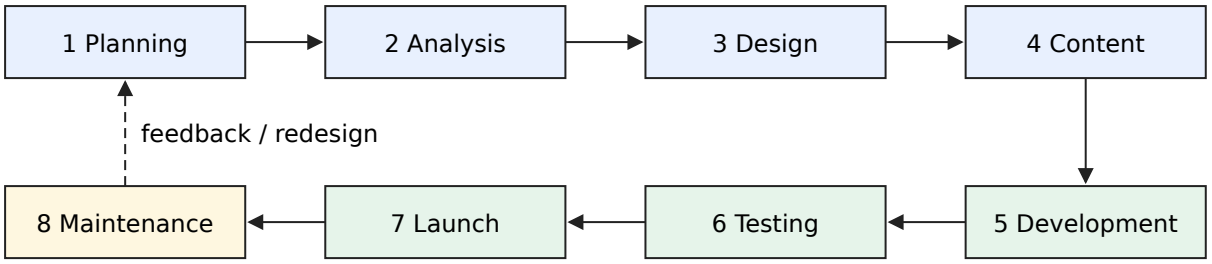


Fig: Website development life cycle; maintenance feeds back into planning

1. Information gathering / planning. Identify the **purpose**, goals, **target audience**, budget and timeline. Output: a project plan and scope.

2. Requirement analysis. List functional needs (pages, forms, login, e-commerce, search), non-functional needs (speed, security, accessibility, SEO), and the technology choice (HTML/CSS/JS, PHP/MySQL, CMS, hosting, domain). Output: a requirements specification and **sitemap**.

3. Design. Information architecture and navigation, **wireframes** and **mock-ups**, colour scheme, typography, branding, and **responsive** layouts for mobile and desktop. The design is approved by the client before coding.

4. Content creation. Write and organise text, images and video. Content should be **SEO-friendly**, with keywords, meta tags and alt text.

5. Development (coding). Front end: HTML structure, CSS presentation, JavaScript behaviour. **Back end:** server-side scripts (PHP), databases (MySQL), forms, sessions and authentication. Work is version-controlled (Git).

6. Testing. Functional tests (links, forms, validation), **cross-browser and cross-device** tests, performance and load tests, security tests (SQL injection, XSS), usability and accessibility tests, and W3C HTML/CSS validation.

7. Deployment / launch. Register the domain, set up hosting, upload by FTP or CI/CD, configure DNS and an SSL (HTTPS) certificate, and submit the sitemap to search engines.

8. Maintenance. Update content, apply security patches and backups, fix bugs, monitor analytics and uptime, and make improvements based on user feedback. This phase feeds back into planning.

Phase	Key deliverable
Planning / Analysis	Scope document, sitemap, requirements
Design	Wireframes, mock-ups, style guide
Development	Working site (front end + back end)
Testing	Test report, bug fixes
Launch	Live site on domain with HTTPS
Maintenance	Updates, backups, analytics reports

Conclusion. Following these phases keeps the site **aligned with user needs, tested, secure and easy to maintain**. Skipping planning or testing is the most common cause of costly rework.

Why it matters for 2026: Web Tech topics not yet asked include "static vs dynamic websites", "client-side vs server-side scripting" and "HTML forms/tables".

Q14. Compare and contrast knowledge-level learning and symbol-level learning in Artificial Intelligence. *(10 marks · AI)*

Introduction. Dietterich (1986), whose ideas Russell and Norvig adopt, classifies learning by **what changes in the agent's knowledge**. The question asked is: after learning, can the agent derive **new conclusions** it could not derive before (its **deductive closure** grows), or does it only derive the **same conclusions faster**?



Fig: Knowledge-level learning grows the deductive closure; symbol-level learning only speeds up reasoning

1. Knowledge-level learning

- **Definition:** learning that **adds genuinely new knowledge**. The deductive closure of the knowledge base after learning is **larger** than before. The agent can now answer questions it previously could not.
- **Typical methods: inductive learning** from examples (decision trees, concept learning), rote learning of new facts, being told new facts, and **relevance-based learning**, which uses background knowledge plus examples to infer new general rules.
- **Example:** after seeing many labelled e-mails, the agent induces "e-mails containing 'lottery' and from unknown senders are spam". It could not deduce this rule before.
- **Nature:** usually **inductive**, so the new knowledge may be wrong and is tested on new data.

2. Symbol-level learning

- **Definition:** learning that **improves efficiency (speed)** without changing the deductive closure. Everything learned was already **logically entailed** by existing knowledge. It is only **re-represented** or **compiled** into a more useful form.
- **Typical methods: Explanation-Based Learning (EBL)** (generalise the proof of one example into a rule), **chunking** (SOAR), **macro-operators**, and **memoisation / caching** of results.
- **Example:** from one worked derivation that "1 × (0 + X) = X", EBL extracts the general rule "ArithmeticUnknown(z) ⇒ Simplify(1 × (0 + z), z)". This rule was already derivable from the algebra axioms, but now applies in one step.
- **Nature: deductive**, so it is always correct if the knowledge base is correct.

Comparison

Basis	Knowledge-level learning	Symbol-level learning
What changes	Content of knowledge (what is known)	Representation / form (how it is used)
Deductive closure	**Increases**	**Unchanged**
Goal	Know more; better coverage and accuracy	Reason **faster**; better efficiency
Inference type	Mostly inductive	Deductive
Correctness	May be wrong; needs validation	Guaranteed if the KB is correct
Examples needed	Usually many	Can learn from one example
Methods	Induction, decision trees, rote learning of new facts	EBL, chunking, macro-operators, caching
Risk	Over-generalisation, noise	**Utility problem**: too many learned rules slow matching

Similarities: both improve the agent's future **performance**, both change the agent's internal representation, and practical systems often combine them. For example, inductive learning finds rules (knowledge level) and EBL then compiles them into fast operational rules (symbol level).

Conclusion. Knowledge-level learning makes an agent **know more**. Symbol-level learning makes it **think faster** with what it already knows.

Why it matters for 2026: AI is about 9 MCQs plus descriptive questions. Explanation-based learning, inductive learning and "learning from observations" are close cousins of this question.

Q15. Explain the k-Nearest Neighbors (k-NN) algorithm for classification. What factors influence its performance? *(10 marks · ML)*

Introduction. k-NN is a **supervised, non-parametric, instance-based ("lazy") learning** algorithm. It builds no explicit model during training and only stores the training examples. To classify a new point, it finds the **k closest training points** and takes a **majority vote** of their classes.

Algorithm

1. Store all training examples (x_i, y_i).
2. For a query point x_q, compute its distance to every training point. Euclidean distance is the usual choice: $d(x, y) = \sqrt{\Sigma(x_j - y_j)^2}$.
3. Select the **k** examples with the smallest distance.
4. Assign the **most frequent class** among them. In **distance-weighted k-NN**, each vote is weighted by $1/d^2$.
5. For regression, return the mean of the k neighbours' values instead.

Worked example. Class A: (1,2), (2,3), (3,1). Class B: (6,5), (7,7), (8,6). Query: (4,3).

Neighbour	Class	Distance to (4,3)	Rank
(2,3)	A	$\sqrt{4} = 2.00$	1
(3,1)	A	$\sqrt{5} = 2.24$	2
(6,5)	B	$\sqrt{8} = 2.83$	3
(1,2)	A	$\sqrt{10} = 3.16$	4
(7,7), (8,6)	B	$\sqrt{25} = 5.00$	5, 6

With **k = 3**, the neighbours are A, A, B, so the query is classified **A**. With k = 1 the answer is also A.

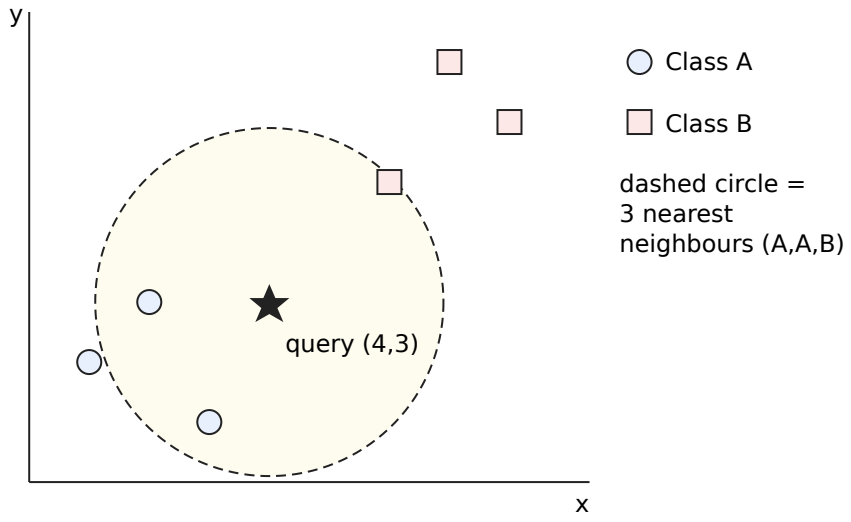


Fig: k-NN with k = 3; two of the three nearest neighbours are class A, so the query is labelled A

Factors influencing performance

1. **Choice of k:** a small k (k = 1) is sensitive to noise and **overfits**. A large k smooths boundaries but **underfits** and lets majority classes dominate. Choose k by **cross-validation**. Use an **odd k** for two classes to avoid ties.
2. **Distance metric:** Euclidean, Manhattan, Minkowski, cosine (text), or Hamming (categorical). The wrong metric gives the wrong neighbours.
3. **Feature scaling:** features with large ranges, such as salary versus age, dominate the distance. **Normalise** the data (min-max or z-score).
4. **Irrelevant or redundant features:** they distort distance, so use feature selection or weighting.
5. **Curse of dimensionality:** in high dimensions all points become nearly equidistant and accuracy falls. Reduce dimensions with PCA.
6. **Noise and outliers:** mislabelled points mislead votes, especially for small k.
7. **Class imbalance:** majority classes win votes, so use distance weighting or resampling.
8. **Dataset size and computational cost:** prediction is O(n·d) per query, with high memory use. Use **KD-trees** or ball trees to speed it up.

Advantages and limitations. k-NN is simple, needs no training, adapts naturally to many classes, and adapts at once to new data. On the other hand, prediction is slow, it uses a lot of memory, it is sensitive to scaling and irrelevant features, and its decisions are hard to interpret.

Conclusion. k-NN performs well when features are **scaled and relevant**, **k is tuned** and the dataset is of moderate size and dimension.

Why it matters for 2026: other ML syllabus items still unasked include "decision trees", "Naive Bayes", "supervised vs unsupervised" and "overfitting/bias-variance".

Section C (15 marks each)

Importance 8/10 · likely · long-answer staples. Paging/segmentation and query processing reappear often, so learn the diagrams.

Q16. Discuss the different memory management techniques used in operating systems, including paging and segmentation. *(15 marks · OS)*

Introduction. Memory management is the OS function that allocates main memory to processes, keeps track of which parts are free or used, **protects** processes from each other, and maps **logical addresses to physical addresses** through the MMU. Its goals are high memory utilisation, a high degree of multiprogramming and low fragmentation.

Classification of techniques

Contiguous allocation	Non-contiguous allocation
Single-partition (monitor + one user)	**Paging**
Multiple fixed partitions (MFT)	**Segmentation**
Multiple variable partitions (MVT)	Segmentation with paging
(supported by swapping and compaction)	**Virtual memory** (demand paging) on top of these

1. Contiguous memory allocation

Each process occupies **one contiguous block**. Protection uses a **relocation (base) register** and a **limit register**: an address is legal only if it is less than the limit, and the physical address is base + logical address.

- **Fixed partitioning:** memory is divided into fixed-size partitions, one process per partition. It is simple but causes **internal fragmentation** (unused space inside a partition), and the degree of multiprogramming is fixed.
- **Variable (dynamic) partitioning:** each process gets exactly what it needs from free **holes**. The placement strategy decides which hole to use:
  - **First-fit:** the first hole that is big enough. This is the fastest.
  - **Best-fit:** the smallest hole that is big enough.
  - **Worst-fit:** the largest hole.
  - First-fit and best-fit beat worst-fit in both speed and storage use (Silberschatz).
  - The problem is **external fragmentation**: enough total free memory exists, but not in one piece. The fix is **compaction**, which needs dynamic relocation and is costly.
- **Swapping:** a process is temporarily moved from memory to the **backing store** and brought back later. This lets the total size of processes exceed physical memory.

2. Paging

Paging removes external fragmentation by allowing a process's physical memory to be **non-contiguous**.

- Physical memory is divided into fixed-size **frames**, and logical memory into **pages** of the same size, a power of 2 such as 4 KB.
- Each logical address is split into a **page number (p)** and a **page offset (d)**. With a logical space of 2^m and page size 2^n, p is the high $m - n$ bits and d is the low n bits.
- The **page table**, one per process, maps p to a **frame number f**. The physical address is f × page size + d.

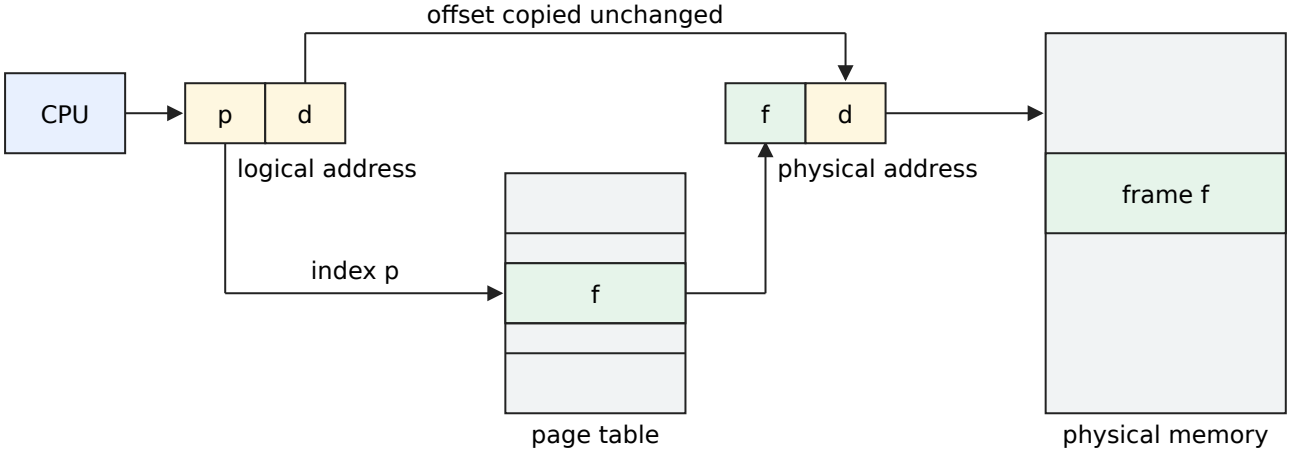


Fig: Paging hardware; p indexes the page table to get frame f, and d is copied unchanged

Example: page size = 1 KB (1024 B) and logical address = 2500. Then p = ⌊2500 / 1024⌋ = **2** and d = 2500 mod 1024 = **452**. If the page table maps page 2 to frame 5, the physical address is 5 × 1024 + 452 = **5572**.

- **TLB (Translation Look-aside Buffer):** a small associative cache of page-table entries. Without it, every access needs two memory accesses. **EAT = h × (TLB + mem) + (1 − h) × (TLB + 2 × mem)**.
- **Features:** no external fragmentation; **internal fragmentation** in the last page only (half a page per process on average); easy sharing of code pages; protection bits (valid/invalid, read/write) in each entry.
- **Large page tables** are handled by hierarchical (multi-level), hashed or inverted page tables.

3. Segmentation

Segmentation supports the **user's view** of a program as a collection of variable-size logical units, or **segments**: main program, functions, stack, symbol table, arrays.

- A logical address is the pair **⟨segment number s, offset d⟩**.
- The **segment table** stores a **base** (starting physical address) and a **limit** (length) for each segment. The **STBR** and **STLR** registers locate the table.
- Translation: if d < limit, the physical address is base + d. Otherwise the hardware **traps** with an addressing error.

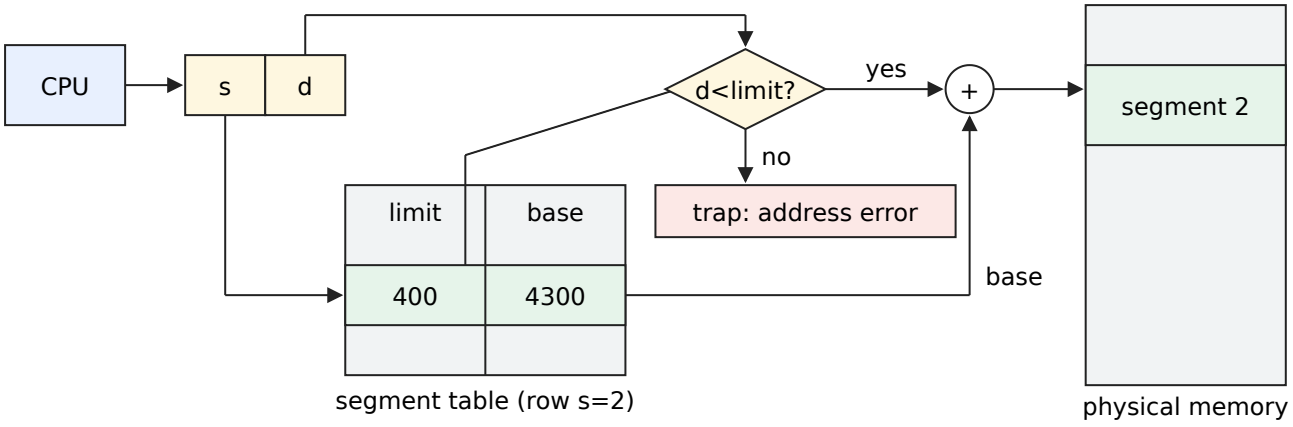


Fig: Segmentation hardware; the offset is checked against the limit, then added to the base

Example (Silberschatz): segment 2 has base 4300 and limit 400. A reference to byte 53 of segment 2 maps to 4300 + 53 = **4353**. A reference to byte 500 of segment 2 traps, because 500 ≥ 400.

- **Features:** matches the programmer's view; protection and sharing per segment (for example a read-only code segment); no internal fragmentation; but it suffers **external fragmentation**, because segments vary in size.

4. Segmentation with paging. Each segment is itself paged, as in Intel x86 and MULTICS. The logical address is first turned into a linear address by segmentation, then into a physical address by paging. This combines the logical view with freedom from external fragmentation.

5. Virtual memory (demand paging). Only the needed pages are loaded. A missing page causes a **page fault**, and a **page-replacement** algorithm (FIFO, Optimal, LRU) chooses a victim frame. This allows programs larger than physical memory and a higher degree of multiprogramming, but too many faults cause **thrashing**.

Paging vs segmentation

Basis	Paging	Segmentation
Unit	Fixed-size pages / frames	Variable-size logical segments
View	System (physical) view; invisible to the user	User (logical) view; visible to the programmer
Address	Page number + offset (one-dimensional)	Segment number + offset (two-dimensional)
Table entry	Frame number (+ protection bits)	Base + limit (+ protection bits)
Division done by	Hardware / OS	Compiler / programmer
Fragmentation	**Internal** (last page)	**External**
Sharing and protection	Possible, page-wise, less natural	Natural, per logical unit
Allocation	Simple (any free frame)	Harder (needs first-fit or best-fit)

Advantages and limitations overall. Contiguous schemes are simple but fragment memory. Paging gives efficient allocation at the cost of page-table memory and translation time, which the TLB reduces. Segmentation gives logical protection and sharing at the cost of external fragmentation. Modern systems use **paged virtual memory**, sometimes on top of segmentation.

Conclusion. Memory management evolved from single contiguous allocation to **paging plus virtual memory**. Paging solves external fragmentation, segmentation reflects program structure, and their combination gives the best of both.

Why it matters for 2026: expect a paging or EAT numerical, "page replacement FIFO/LRU/Optimal with page faults", or "thrashing". "Virtual memory" is a syllabus phrase not yet asked.

Q17. Explain the various cost estimation methods used in software project planning. Which method do you think is most reliable and why? *(15 marks · SE)*

Introduction. Software cost estimation predicts the **effort** (person-months, PM), **duration** (months), **staffing** and therefore the **cost** of a software project, usually from an estimate of its **size** in KLOC or function points. It is done at the start of project planning and refined as the project proceeds. Underestimates lead to overruns. Overestimates lose bids.

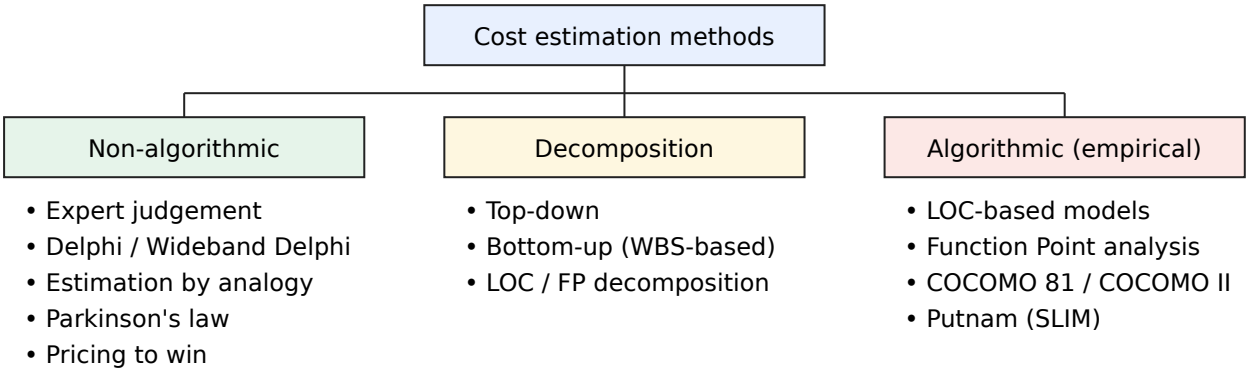


Fig: Classification of software cost estimation methods

A. Non-algorithmic methods

1. **Expert judgement:** one or more experienced people estimate from their experience. It is fast and handles unusual factors, but it is subjective and only as good as the expert.
2. **Delphi / Wideband Delphi:** experts estimate **anonymously**, a coordinator circulates a summary, and the experts revise. This repeats until the estimates **converge**. It reduces bias and the influence of dominant personalities, but takes time.
3. **Estimation by analogy:** compare the project with **completed similar projects** and adjust for differences. It is grounded in real data, but needs a historical database and truly comparable projects.

4. **Parkinson's law:** "work expands to fill the time available", so cost is set by the resources available. **Pricing to win:** cost is set to what the customer will pay. Both are unrealistic and listed only for completeness (Sommerville).

B. Decomposition methods

1. **Top-down:** estimate the overall cost from global properties of the system, then split it among components. It is quick and accounts for system-level costs such as integration, but may miss technical details.
2. **Bottom-up:** estimate each **WBS work package** or component, then **sum** the estimates. It is detailed and accurate when the design is known, but it is time-consuming, needs a detailed design, and may miss system-level costs.

C. Algorithmic (empirical / parametric) models

These use mathematical relationships derived from historical project data: **Effort = f(size, cost drivers)**.

1. LOC-based estimation. Estimate KLOC per module (optimistic, likely and pessimistic: **E = (o + 4m + p) / 6**), then divide by historical productivity (LOC/PM).

2. Function Point (FP) analysis (Albrecht). Size is measured from functionality, independent of language.

- Count the five components: **External Inputs, External Outputs, External Inquiries, Internal Logical Files, External Interface Files**. Weight each as simple, average or complex to get the **UFP**.
- **FP = UFP × VAF**, where **VAF = 0.65 + 0.01 × ΣF_i** over **14 general system characteristics**, each rated 0–5.
- Effort is then FP divided by productivity (FP/PM).

3. COCOMO (COnstructive COst MOdel, Barry Boehm, 1981). Three project classes:

- **Organic:** small team, familiar problem.
- **Semi-detached:** medium size, mixed experience.
- **Embedded:** tight hardware or regulatory constraints.

Basic COCOMO: E = a × (KLOC)^b PM, **D = c × E^d months**, and average **staff = E / D**.

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

- **Intermediate COCOMO: E = a × (KLOC)^b × EAF**, with a = 3.2, 3.0 and 2.8 and the same b values. The **Effort Adjustment Factor (EAF)** is the product of **15 cost drivers**, grouped as product, hardware, personnel and project attributes. Examples: RELY, CPLX, ACAP, TOOL.
- **Detailed COCOMO:** applies the cost drivers phase by phase.
- **COCOMO II (1995 onward)** adds Application Composition, Early Design and Post-Architecture sub-models, with **scale factors** in place of fixed modes.

Worked example (Basic COCOMO, organic, 32 KLOC):

- E = $2.4 \times 32^{1.05} = 2.4 \times 38.05 \approx$ **91.3 PM**
- D = $2.5 \times 91.3^{0.38} \approx$ **13.9 months**
- Staff = 91.3 / 13.9 ≈ **6.6, about 7 persons**
- Productivity = 32,000 / 91.3 ≈ **350 LOC/PM**

At ₹1,00,000 per PM, the cost is about **₹91 lakh**. For comparison, the same 32 KLOC as semi-detached gives E ≈ 145.5 PM and D ≈ 14.3 months, and as embedded gives E ≈ 230.4 PM and D ≈

14.3 months (checked with python3).

4. Putnam model (SLIM). Based on the **Rayleigh–Norden** staffing curve. The software equation is $\mathbf{L = C_k \times K^{1/3} \times t_d^{4/3}}$, where K is total effort and t_d is delivery time. It implies that **compressing the schedule raises effort sharply**: $K \propto 1/t_d^4$.

Comparison of methods

Method	Basis	Strengths	Limitations	Best used when
Expert judgement	Experience	Fast, cheap, flexible	Subjective, biased	Very early, novel factors
Delphi	Group consensus	Reduces individual bias	Slow	High-stakes early estimates
Analogy	Past projects	Based on real data	Needs similar projects	A historical database exists
Bottom-up (WBS)	Component sums	Detailed, traceable	Needs a detailed design	After design
Top-down	Global properties	Quick, includes system costs	Misses detail	Early proposals
Function points	Functionality	Language-independent, available early	Counting is subjective	From requirements
COCOMO	Size + cost drivers	Objective, repeatable, transparent	Needs a KLOC estimate; must be calibrated	Planning, trade-off analysis
Putnam	Staffing curve	Models schedule–effort trade-off	Sensitive to t_d and C_k	Large projects

Which method is most reliable, and why?

No single method is reliable on its own. Every model is only as good as its **size estimate** and its **calibration data**. The most reliable approach is a **combination**:

1. Use an **algorithmic model (COCOMO II or FP-based) calibrated with the organisation's own historical project data**. This makes the estimate objective, repeatable and explainable, and lets the cost drivers be adjusted for "what-if" trade-offs.
2. **Cross-check** it with **expert judgement / Wideband Delphi** and **analogy** with similar past projects. These catch factors the model misses, such as new technology or team changes.
3. Where the design is known, also do a **bottom-up WBS estimate**. If two independent estimates **agree**, confidence is high. If they diverge widely, the assumptions are re-examined.
4. **Re-estimate iteratively** at each milestone, since uncertainty shrinks as the project progresses (the cone of uncertainty).

Among single methods, a **calibrated algorithmic model such as COCOMO** is generally considered most reliable for medium and large projects, because it is grounded in historical data and is **transparent and repeatable**. Even so, Boehm and Pressman both recommend using at least two methods and reconciling them.

Conclusion. Cost estimation should combine **data-driven models** with **human judgement** and be refined throughout the project. Applied this way, COCOMO calibrated with historical data and cross-checked by experts gives the most dependable estimates.

Why it matters for 2026: likely next asks are a COCOMO numerical or "function point calculation", or the unasked syllabus phrases "risk management", "project scheduling (PERT/CPM)" and "software metrics".

Q18. Describe the query processing steps in a relational database management system. Why is query optimization important? *(15 marks · DBMS)*

Introduction. Query processing is the set of activities that turn a high-level **SQL query** into low-level operations that read the database and return the result. In Korth's description there are three

main steps: **(1) parsing and translation, (2) optimisation and (3) evaluation**.

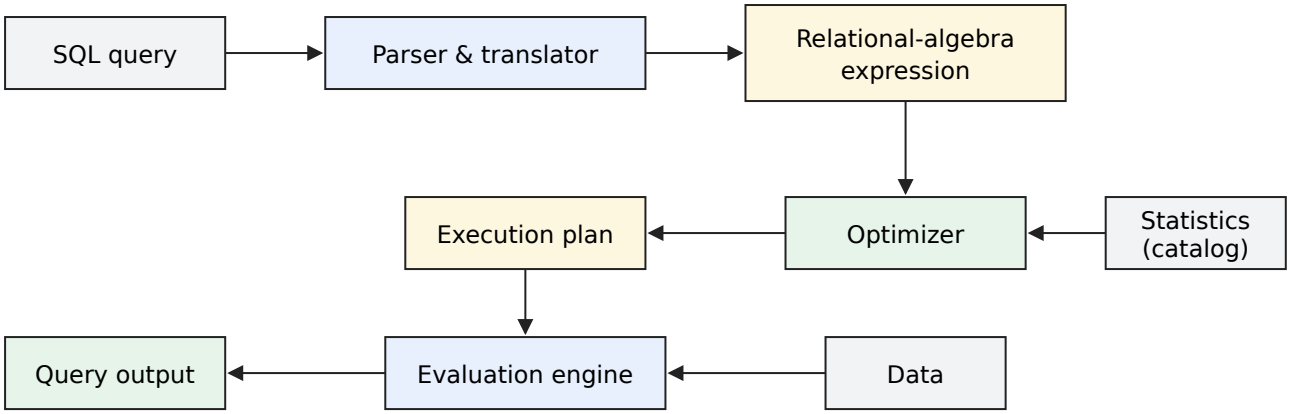


Fig: Steps in query processing (after Korth)

Step 1. Parsing and translation

- **Scanning:** split the SQL text into tokens such as keywords, identifiers and operators.
- **Parsing:** check the **syntax** against the SQL grammar.
- **Semantic check:** use the **data dictionary (catalog)** to confirm that relations and attributes exist, types are compatible and the user has privileges. **Views** are replaced by their definitions.
- **Translation:** produce an internal form, usually a **relational-algebra expression** or **query tree**.

Step 2. Optimisation

- The same query has many **equivalent** relational-algebra expressions, and each operator has several **algorithms**. For example, selection can use a linear scan or an index. Join can use nested-loop, block nested-loop, indexed nested-loop, merge join or hash join.
- An **evaluation primitive** is an operation annotated with the algorithm to use. A sequence of primitives is a **query-execution (evaluation) plan**.
- The **optimiser** chooses the plan with the **lowest estimated cost**.
- Cost is measured mainly as the number of **disk block transfers and seeks**, using catalog **statistics**: n_r (tuples in r), b_r (blocks of r), V(A, r) (distinct values of A) and index heights.

Step 3. Evaluation

- The **query-execution engine** runs the chosen plan. Intermediate results are either **materialised** (written to temporary relations) or **pipelined** (passed tuple by tuple to the next operator, which saves I/O).
- The result is returned to the user.

Example and query trees

```
SELECT name FROM instructor, teaches
WHERE instructor.ID = teaches.ID AND dept_name = 'Music';
```

The initial translation is $\pi_{name}(\sigma_{instructor.ID=teaches.ID \wedge dept_name='Music'}(instructor \times teaches))$. Optimised, it becomes $\pi_{name}(\sigma_{dept_name='Music'}(instructor) \bowtie teaches)$.

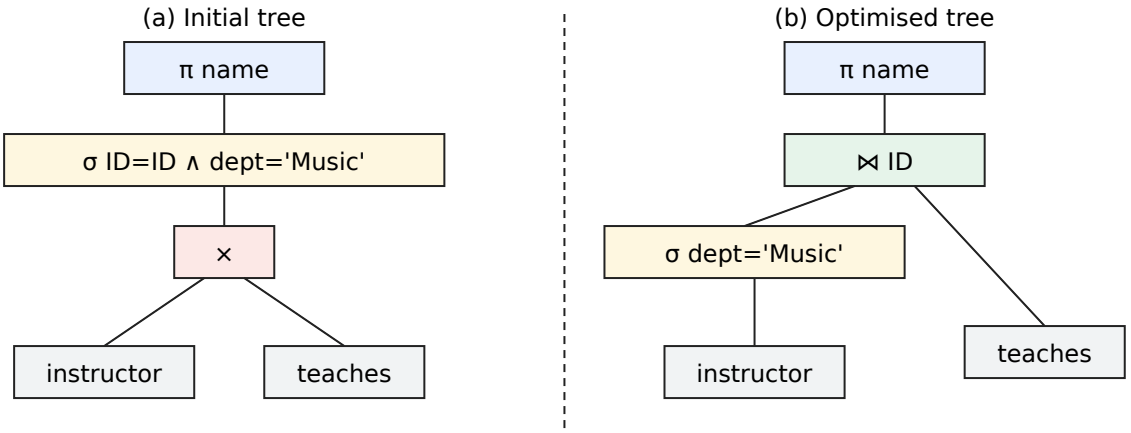


Fig: Query trees; the selection is pushed down and σ over × is replaced by a join

In the initial tree, the Cartesian product builds $n_{instructor} \times n_{teaches}$ tuples before filtering. In the optimised tree, only Music instructors are joined, and through an equi-join, so far fewer tuples and blocks are processed.

Types of optimisation

(a) Heuristic (rule-based) optimisation. This applies **equivalence rules** of relational algebra that almost always reduce cost:

1. **Perform selections as early as possible** (push σ down the tree).
2. **Perform projections early** to drop unneeded attributes.
3. Split conjunctive selections into a cascade of selections, then push each one down.
4. **Replace σ over × with a join (⋈)**.
5. Execute the **most restrictive** selections and joins first, the ones with the smallest results.
6. Use the commutativity and associativity of joins to reorder them.

(b) Cost-based optimisation. This generates alternative plans (join orders and algorithms), **estimates the cost** of each from catalog statistics, and picks the cheapest.

- Example estimate: the size of $\sigma_{A=v}(r)$ is about n_r / V(A, r).
- Join ordering uses **dynamic programming**. System R considers only left-deep trees.

Basis	Heuristic optimisation	Cost-based optimisation
Approach	Fixed transformation rules	Estimate cost of alternative plans
Uses statistics	No	Yes (n_r, b_r, V(A,r), histograms)
Optimality	Good but not guaranteed best	Finds the cheapest of the plans it considers
Optimisation overhead	Low	Higher (search space grows quickly with joins)
Typical use	Pre-processing step	Main optimiser in modern RDBMSs (usually combined with heuristics)

Why is query optimisation important?

1. **Huge cost differences:** equivalent plans can differ in cost by **orders of magnitude**, for example a full Cartesian product versus an indexed join. Choosing well turns hours into seconds.
2. **Disk I/O dominates cost:** reducing block transfers and the size of intermediate results directly cuts response time.
3. **SQL is declarative:** users say *what* they want, not *how*, so the system must pick an efficient strategy. This is what gives users data independence.
4. **Throughput and resources:** efficient plans use less CPU, memory and I/O, so more concurrent users can be served.

5. **Scalability:** as tables grow, an unoptimised query degrades badly while an optimised one stays acceptable.
6. **It adapts to data:** a cost-based optimiser uses current statistics and indexes, so the same query is run in the best way for the current data.

Conclusion. Query processing turns SQL into an efficient execution through **parsing, optimisation and evaluation**. Optimisation is the heart of the process. Without it, the declarative convenience of SQL would come at an unacceptable performance cost.

Why it matters for 2026: closely related unasked syllabus phrases are "indexing (B+ tree)", "transaction / ACID / concurrency control" and "normalisation".

Q19. Discuss the process of building a knowledge base in Artificial Intelligence. What are the key challenges involved? *(15 marks · AI)*

Introduction. A **knowledge base (KB)** is a set of **sentences** written in a **knowledge-representation language**. Each sentence states something about the world, as facts or rules. A **knowledge-based agent** uses the KB with an **inference engine** to decide its actions. The process of building a KB is called **knowledge engineering**. It is carried out by a **knowledge engineer** working with **domain experts**.

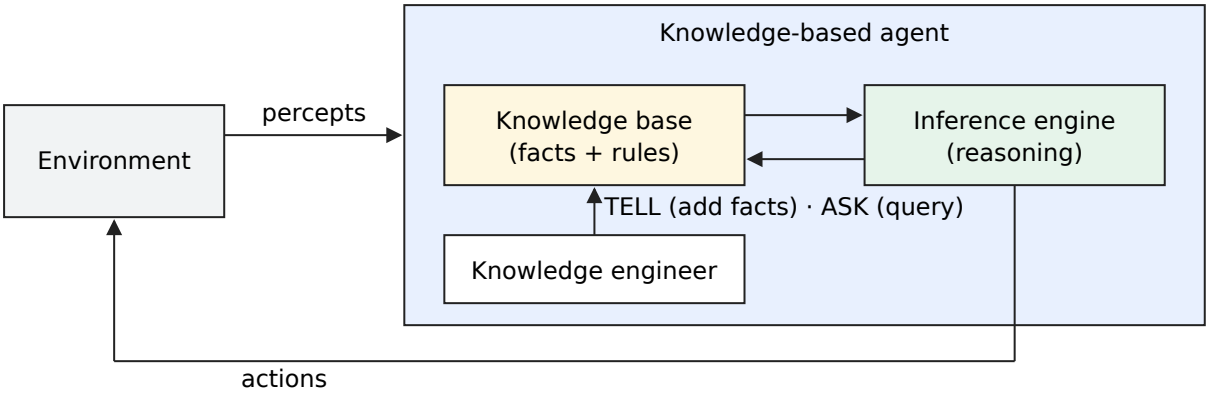


Fig: A knowledge-based agent; percepts are TELLed to the KB, the agent ASKs what to do and acts

The generic knowledge-based agent (Russell–Norvig) does three things on each step. It **TELLs** the KB what it perceives, **ASKs** the KB which action to take, and **TELLs** the KB which action it took. This is the **declarative** approach: knowledge is kept separate from the reasoning procedure.

Process of building a knowledge base (the knowledge-engineering process)

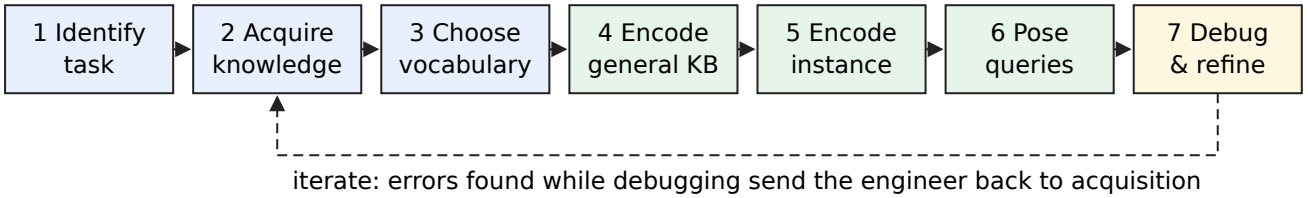


Fig: Knowledge-engineering process (after Russell and Norvig)

1. **Identify the task.** Define the scope: what questions the KB must answer and what information is available for each problem instance. For example, a medical-diagnosis KB answers "which disease, given these symptoms?".
2. **Assemble the relevant knowledge (knowledge acquisition).** Get knowledge from **domain experts** through interviews, observation and protocol analysis, and from books, manuals and databases. Machine learning can also extract rules from data. The aim is to *understand* the domain before formalising it.
3. **Decide on a vocabulary (ontology).** Choose the predicates, functions and constants, for example `Symptom(p, s)` and `HasDisease(p, d)`. The result is the domain's **ontology**: what kinds

of things exist and how they relate. Choices made here are hard to change later.
4. **Encode general knowledge about the domain.** Write axioms and rules that define the vocabulary, such as `∀p Fever(p) ∧ Rash(p) ⇒ Suspect(p, Measles)`. This step often exposes gaps in the vocabulary, which sends the engineer back to step 3.
5. **Encode the specific problem instance.** Add the facts of the current case, such as `Fever(Asha)` and `Rash(Asha)`, usually supplied by sensors or users.
6. **Pose queries to the inference procedure and get answers.** For example, ASK(KB, `Suspect(Asha, x)`). The inference engine uses forward chaining, backward chaining or resolution.
7. **Debug and maintain the KB.** Wrong or missing answers point to **missing axioms** or **incorrect axioms**. Test it, have experts validate it, and update it as the domain changes.

Choices of knowledge representation (KR)

KR scheme	Form	Strength	Weakness
Propositional logic	Facts as true/false symbols	Simple, decidable	Cannot express objects or quantifiers
First-order (predicate) logic	Predicates, quantifiers ∀ ∃	Expressive, sound inference	Semi-decidable, computationally costly
Production rules	IF condition THEN action	Modular, natural for experts	Large rule sets hard to manage and conflicting
Semantic networks	Nodes (concepts) + labelled arcs (is-a, has)	Visual, supports inheritance	Unclear semantics, weak for complex logic
Frames	Slots and fillers with defaults	Structured objects, defaults, inheritance	Exceptions and reasoning are limited
Ontologies (OWL, description logic)	Classes, properties, axioms	Shareable, standardised vocabulary	Effort to build and agree on
Probabilistic (Bayesian networks)	Variables + conditional probabilities	Handles uncertainty	Needs probability estimates

A good KR should have **representational adequacy** (it can express everything needed), **inferential adequacy and efficiency**, and **acquisitional efficiency**, meaning new knowledge is easy to add (Rich and Knight).

Key challenges

1. **Knowledge-acquisition bottleneck:** getting knowledge out of experts is slow and costly, and is often the hardest part of the work.
2. **Tacit knowledge:** experts often cannot state the heuristics they use ("I just know"), and they may disagree with each other.
3. **Choosing representation and ontology:** there is a trade-off between **expressiveness and tractability**. A poor vocabulary forces a costly redesign.
4. **Incompleteness and uncertainty:** real knowledge is incomplete, vague or probabilistic, which needs non-monotonic, fuzzy or probabilistic reasoning.
5. **Consistency and correctness:** contradictory or wrong rules produce false conclusions. Large KBs are hard to verify.
6. **Common-sense knowledge:** large amounts of "obvious" knowledge are hard to encode. The Cyc project is an example of how big this task is.
7. **The frame problem:** representing what does *not* change when an action occurs.
8. **Scalability and efficiency:** inference over large KBs is computationally expensive.
9. **Maintenance:** domain knowledge changes, so rules must be updated without breaking others.
10. **Validation and explanation:** proving the KB is correct and explaining its conclusions to users.

Conclusion. Building a knowledge base is an **iterative knowledge-engineering cycle**: identify the task, acquire knowledge, design the ontology, encode general and specific knowledge, query, and debug. Choosing a suitable representation is central. The **knowledge-acquisition bottleneck**,

uncertainty and maintenance remain the main challenges, which is why modern systems combine hand-built ontologies with **machine learning** that extracts knowledge from data.

Why it matters for 2026: AI syllabus phrases still unasked include "knowledge representation (semantic nets, frames)", "expert systems", "forward vs backward chaining" and "search (A, BFS/DFS)". This answer's KR table covers half of them.*